

NESHBOY SOC PROFESSIONAL LIBRARY

QRADAR SOC OPERATIONS

Offense Management, Rule Engineering, DSM Onboarding,
and Platform Operations — Running QRadar Beyond the
Query Language

Build date: 2026-09-16

Table of Contents

CHAPTERS

Section A — Platform Orientation and the Honesty Constraint

- Part 1 — Why This Book Exists, and What "Operations" Means Here
- Part 2 — QRadar Deployment Architecture: Console, Processors, Collectors, and Data Nodes

Section B — The Offense Model

- Part 3 — Offenses, Magnitude, Credibility, and Relevance

Section C — Getting Data In

- Part 4 — DSMs, QID Mapping, and Log Source Onboarding
- Part 5 — The Universal DSM, DSM Editor, and Custom Log Source Extensions
- Part 6 — Custom Properties: Extraction, Calculated Properties, and Governance
- Part 7 — Flow Data Architecture: QFlow, Flow Sources, and Flow-vs-Event Design

Section D — Building the Correlation Layer

- Part 8 — The Rule Wizard Catalog: Rule Types Beyond the Canonical Example
- Part 9 — Building Blocks: Design Patterns and Governance at Scale
- Part 10 — Reference Data Collections: Sets, Maps, Map of Sets, Map of Maps, and Tables
- Part 11 — Thresholds, Anomaly and Behavioral Rules, and the Limits of Native Aggregation

Section E — Operating the Search and Reporting Layer

- Part 12 — Ariel Search Performance and Tuning
- Part 13 — Dashboards, Pulse, and Executive Reporting

Section F — Running the Queue

- Part 14 — Analyst Triage Workflow for Offenses
- Part 15 — Noisy-Offense Triage and the Tuning Workflow
- Part 16 — Rule Change Management Without a Detection-as-Code Pipeline

Section G — Keeping the Platform Healthy

- Part 17 — Licensing, EPS/FPM Management, and Capacity Planning
- Part 18 — QRadar-Specific Troubleshooting: Log Sources, DSMs, and Offense Flooding
- Part 19 — Apps, Extensions, and the App Framework

Section H — Program, Identity, and Scale

- Part 20 — Asset Model, Network Hierarchy, and Vulnerability Data
- Part 21 — Multi-Tenancy, RBAC, and MSSP Operations
- Part 22 — Staffing and Running a QRadar Practice

CHAPTERS

Section A — Platform Orientation and the Honesty Constraint

neshboy

Part 1 — Why This Book Exists, and What "Operations" Means Here

Why this part exists

[CONCEPT] A reader who already owns *The Detection Engineering Handbook* (DEH) and lands here after finishing Part 27 has a reasonable question before reading another word: didn't that part already cover QRadar? It did — and it said so about itself, plainly, in its own opening section: Part 27 is ~5,800 words that exist to prove one specific claim, that QRadar's correlation model forces a three-object decomposition (a Building Block, a Reference Set, and a Rule) where a KQL or SPL implementation of the same analytic is one deployed artifact. It was never meant to teach anyone how to run a QRadar deployment, and it says that too. This book is the rest of that sentence. It covers the platform-operations experience Part 27 explicitly defers: how an Offense actually gets scored and lived with, the full Rule Wizard catalog beyond one canonical rule, Building Block governance once a library holds hundreds of entries instead of one, the complete reference-data type taxonomy, log source and DSM onboarding as a repeatable process rather than a single failure mode named once, Ariel search performance, dashboards, the noisy-Offense tuning workflow, rule change management with no git pipeline handed to you for free, licensing and capacity planning, and QRadar-specific troubleshooting.

This part does four things before any of that operational material starts. §1 draws the scope boundary against DEH Part 23 and Part 27 precisely, with a table, so nothing in this book quietly re-teaches what those parts already own. §2 defines what "operations" means in this book's own title, because it is a narrower and more specific claim than "everything about QRadar." §3 and §4 name who this book is for, who it isn't, and the one constraint that shapes every figure in it: no real QRadar deployment exists in this author's lab, which is a structurally different evidence position than DEH ever had to work from. §5 and §6 give the reader the two pieces of reading-the-room machinery — the PRODUCT VERSION NOTE convention and the tag/callout system — before Part 2 starts using both without re-explaining them.

1. The gap this book fills: what DEH Part 27 proves, and what it deliberately leaves for this book

[CONCEPT] DEH Part 27's own framing is worth quoting rather than paraphrasing, because it states the boundary this book inherits more precisely than a summary would: QRadar splits "search my data" and "correlate my data into an alert" across two different subsystems. Ariel Query Language (AQL) is the query language for the first; the Custom Rules Engine (CRE) — a separate, largely point-and-click rule-authoring system built from Building Blocks, Rule Tests, and Reference Sets — is the second, and AQL only touches it at the edges. Part 27 teaches AQL's syntax fundamentals (§1: the `events / flows` split, the `QIDNAME()` / `CATEGORYNAME()` / `LOGSOURCETYPENAME()` translation functions, time-window shorthand), introduces the CRE only far enough to make its central point (§2: Building Blocks, Rule Tests, Reference Sets as QRadar's answer to a join), and walks one worked example end to end — DET-27-01, QRadar's re-implementation of DEH Part 23's canonical `lsass.exe` process-access analytic (DET-23-01), built as `BB:LSASS-Access-Candidate`, `RS-Allowlisted-LSASS-Tools`, and `R: Suspicious LSASS Access – Unapproved`

Process working together. That worked example, and the Offense-as-hybrid-of-Alert-and-Case finding Part 27 §6 closes on, are carried forward by reference throughout this book, never redefined — the same discipline STYLE-GUIDE.md §12 requires of every part here.

What Part 27 does not do — because a ~5,800-word part inside a six-platform comparison book has no room to — is anywhere near the depth a team actually running QRadar needs. One worked Building Block is not a governance model for a **BB:** library that has grown past 400 unreviewed entries over three years (Part 9). One Reference Set introduced to prove a structural point is not the full typed taxonomy — Set, Map, Map of Sets, Map of Maps, Table — and the TTL/expiry discipline a real deployment needs across all of them (Part 10). One paragraph naming that an unmapped Sysmon event lands under a generic QID with no custom properties is not a repeatable DSM onboarding checklist (Part 4), nor does it cover what a team does when a source has no native DSM at all (Part 5). And Part 27's closing SOC MANAGEMENT paragraph — that reviewing "one detection" in QRadar means reviewing three separately-changeable objects, not one query file — names a review-cost problem in one sentence that this book's Part 22 has to actually build a headcount model against.

Table 1.1 makes that boundary concrete rather than asserted, topic by topic, so a reader can see exactly where DEH's QRadar coverage ends and this book's begins before committing to read either.

Table 1.1 — What DEH already covers for QRadar vs. what this book adds. Supports the decision "do I need DEH Part 23/27, this book, or both, for the question in front of me right now."

Topic	DEH Coverage	This Book's Coverage
AQL syntax (SELECT / WHERE / GROUP BY , translation functions)	DEH Part 27 §1 — full syntax reference	Not re-taught; every AQL example here links back to DEH Part 27 §1 rather than re- explaining SELECT / FROM / WHERE
Sigma-first vs. native rule authoring	DEH Part 23 §5 (generic tradeoff), DEH Part 27 §2.1/§3.3–3.4 (QRadar instance)	Not re-argued; Part 9 builds past the one worked example into BB: library governance at scale
CRE / Building Block / Reference Set — existence and basic mechanics	DEH Part 27 §2 — introduced only far enough to prove the three- object decomposition	Parts 8–11 — full Rule Wizard catalog, BB: governance, complete reference-data taxonomy
Offense scoring (magnitude, credibility, relevance, severity)	DEH Part 27 §6 — one paragraph, the Offense-as-hybrid finding	Part 3 — full scoring model, lifecycle, retention, entity binding
DSM/QID mapping and log source onboarding	DEH Part 27 §3.1 — one failure mode named once	Parts 4–6 — full onboarding lifecycle, Universal DSM, custom-property governance
Ariel search performance / EPS-FPM licensing	DEH Part 27 §1.3 — one paragraph flag	Parts 12, 17 — full tuning discipline and capacity planning
Analyst triage of a fired Offense	DEH Part 27 §6 — one worked capsule against DET-27-01	Part 14 — day-to-day queue workflow across every Rule type, not one analytic
Noisy-Offense diagnosis, rule change management	Not covered	Parts 15–16 — new material; no DEH equivalent exists
Multi-tenancy, MSSP operations, staffing/TCO	DEH Part 27's closing SOC MANAGEMENT paragraph — the review-cost problem named once	Parts 21–22 — full RBAC/Domain treatment and a headcount model built against that problem

Figure 1.1 gives the same boundary as a diagram, tracing the handoff from DEH's canonical analytic through its one QRadar implementation into this book's own part sequence.

```
flowchart TB
    subgraph DEH["Detection Engineering Handbook – six-platform comparison"]
        P23["DEH Part 23\nQuery Language Strategy\nDET-23-01 canonical analytic,\nSigma-vs-native tradeoff"]
        P27["DEH Part 27\nQRadar AQL\nAQL syntax, CRE/BB/RS intro,\none worked rule: DET-27-01"]
        P23 --> P27
    end
    subgraph BOOK["QRadar SOC Operations – this book"]
        A1["Part 1\nScope, audience,\nno-lab constraint"]
        A2["Part 2\nDeployment architecture"]
        B3["Part 3\nOffense scoring in depth"]
        C["Parts 4-7\nDSM onboarding,\ncustom properties, flows"]
        D["Parts 8-11\nFull Rule Wizard,\nBB governance, reference data,\nthresholds"]
        E["Parts 12-13\nAriel performance,\ndashboards"]
        F["Parts 14-16\nTriage workflow,\ntuning, change mgmt"]
        G["Parts 17-19\nLicensing, troubleshooting,\napps"]
        H["Parts 20-22\nAsset model, multi-tenancy,\nstaffing"]
        A1 --> A2 --> B3 --> C --> D --> E --> F --> G --> H
    end
    P27 -->|"single worked rule handed off\nby reference, not re-taught"| A1
```

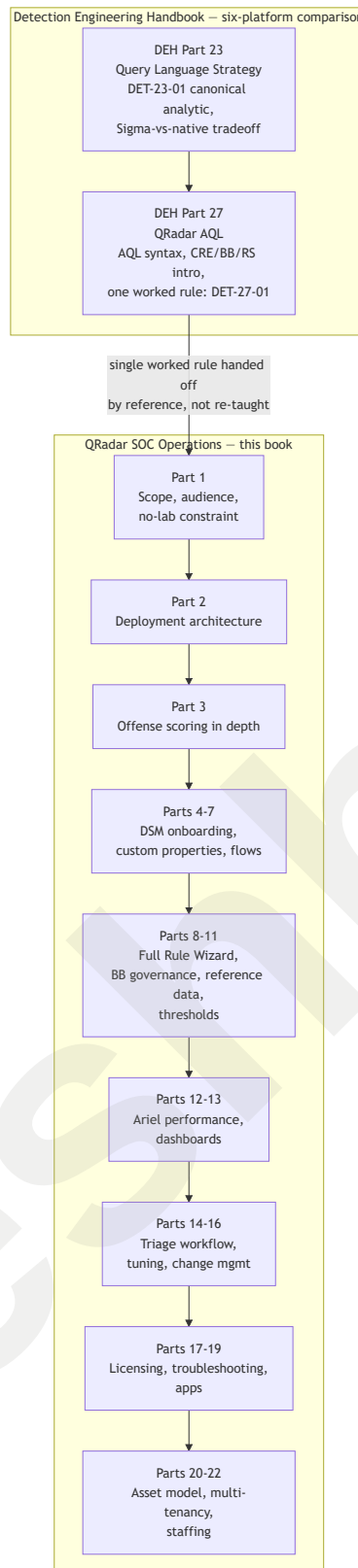


Figure 1.1 — Where DEH's QRadar coverage ends and this book's begins. *CONCEPTUAL.* Illustrates the scope handoff argued in prose above: DEH Part 23 defines one canonical analytic once, DEH Part 27 proves QRadar forces a three-object decomposition of it and stops there, and this book's 22 parts build the rest of the operational platform around that same worked example without re-deriving it. Not a

reproduction of any DEH or IBM diagram, and not a claim about page counts beyond the ~5,800-word figure DEH Part 27 states about itself.

2. What "operations" means in this book's title

[CONCEPT] "Operations" here is a specific, narrower claim than "everything about QRadar," and it is worth being precise about which claim, because the word gets used loosely enough elsewhere that a reader could reasonably guess wrong. This book is about *running* a QRadar deployment that other people (or other parts of this same book, worn as a different hat) have already decided to build: keeping data flowing in correctly, keeping the correlation layer's decisions trustworthy, keeping the queue triageable instead of flooded, keeping the platform itself healthy under its own licensing and resource constraints, and keeping all of that auditable to whoever budgets it. It is not a book about whether to buy QRadar in the first place, and it is not a book about detection logic in the abstract — DEH already owns both of those conversations, the first only glancingly (a SOC Management View can weigh QRadar against alternatives, but the comparison itself belongs to a buyer's evaluation, not this book) and the second directly, across DEH's full 48-part run.

Concretely, "operations" spans five roles this book's own tag system (STYLE-GUIDE.md §7) names directly, because a single console hides more role separation than its unified login screen suggests: the platform engineer keeping Console/Event Processor/Event Collector/Flow Processor/Data Node/App Host topology, DSM onboarding, licensing, and Ariel performance healthy (**[PLATFORM ENGINEER]**); the rule engineer working inside the CRE authoring surface itself — Rule Wizard, Building Blocks, reference data, custom properties (**[RULE ENGINEER]**), renamed from DEH's **[DETECTION ENGINEER]** specifically because this role operates inside one platform's console rather than across six query languages); the analyst reading a fired Offense's magnitude, credibility, and relevance to decide what to do about it right now (**[ANALYST]**); the hunter pivoting from an Offense to its contributing events in AQL, extending DEH Part 27 §5's pattern into full operational practice (**[THREAT HUNTER]**); and whoever budgets, staffs, and answers for all four of the above to someone outside the SOC (**[SOC MANAGEMENT]**). A sixth tag, **[CONCEPT]**, marks the foundational "what and why" every other tag's content assumes — what an Offense is, what Ariel is, what the CRE evaluates and when — before role-specific content builds on it. This part is **[CONCEPT]** almost throughout, for the same reason DEH's own Part 1 is: before any role-specific content makes sense, the reader needs to know what book they're holding.

One vocabulary discipline follows directly from DEH **TERMINOLOGY.md**, and STYLE-GUIDE.md §12 states it as a hard rule rather than a suggestion: this book does not redefine Alert, Case, Incident, Disposition, Tuning, Suppression, or Coverage — they keep their DEH definitions unchanged, including DEH Part 27 §6's own finding that a QRadar Offense is a hybrid of DEH's Alert and Case concepts. Where this book needs genuinely new, QRadar-specific vocabulary — Magnitude, Credibility, Relevance, QID (QRadar's internal event-taxonomy identifier, expanded here on first use per part per STYLE-GUIDE.md §4 and bare thereafter), DSM, the Custom Rules Engine (universally abbreviated CRE after this point), Building Block, Reference Set/Map/Table, Domain, Tenant, EPS/FPM — it lives in this book's own Appendix A1, cross-referencing DEH **TERMINOLOGY.md** rather than duplicating any entry that already exists there.

3. Who this book is for, and who it isn't

[SOC MANAGEMENT] The honest audience for this book is narrower than "anyone who touches QRadar," and naming who it isn't for saves a reader real time. Table 1.2 states both sides plainly.

Table 1.2 — Who this book is for. Supports a reader's decision to keep reading this part, skip ahead to a specific later part, or go read DEH instead.

Reader	What This Book Gives You	If This Isn't Quite You
QRadar platform engineer / administrator	Deployment topology, DSM onboarding, Ariel performance, licensing/capacity, HA/DR, troubleshooting (Parts 2, 4–7, 12, 17–19)	Want AQL syntax taught from zero — DEH Part 27 §1 owns that
Rule/Building Block author working inside QRadar's console specifically	Full Rule Wizard catalog, BB : governance at scale, the complete reference-data taxonomy, custom-property governance (Parts 6, 8–11)	Want a cross-platform Sigma-vs-native argument — DEH Part 23 §5 owns that generically, DEH Part 27 §3.3–3.4 owns it for QRadar
Tier-1/2 analyst triaging the Offense queue day to day	Reading magnitude/credibility/relevance to prioritize, escalation and closing-reason criteria, the tuning-intake process for a noisy Rule (Parts 3, 14–15)	Want general SOC triage practice not specific to one platform — DEH Part 27 §6's own capsule is the QRadar-specific starting point this book's Part 14 expands
SOC manager evaluating or budgeting a QRadar practice	Licensing/TCO, MSSP-vs-in-house tradeoffs, a headcount model built against the three-object review burden (Parts 17, 21–22)	Want a vendor-neutral SIEM-selection comparison — outside both this book's and DEH's scope entirely
A reader who wants AQL syntax, or a six-platform query-language comparison	—	DEH Part 27 §1 (AQL) or DEH Parts 23–29 in full (comparison) — this book assumes both are already read where relevant, per every part's front matter deh_depends_on field

SOC Management View A SOC manager evaluating headcount for a QRadar practice cannot reuse a generic "SIEM analyst" job description unchanged, for the reason DEH Part 27's own closing paragraph names for a single-tenant shop: reviewing "one detection" in QRadar means reviewing a Building Block, a Reference Set's current membership and maintenance owner, and a Rule's chained logic as three separately-changeable artifacts that can drift out of sync with each other — not one query file, the way a KQL or SPL shop's review model assumes. Part 22 is where this book actually builds a headcount model against that fact; this part's job is only to put the fact on the table early enough that a reader evaluating a QRadar hire doesn't discover it twenty parts in.

4. The honesty constraint: no real QRadar deployment in this author's lab

[CONCEPT] DEH could stand up Sysmon, a domain controller, an SSH host — commodity lab infrastructure a solo author can build on a home machine over a weekend. Part 1 of DEH's own current edition documents exactly that kind of real capture: a genuine failed-SSH-authentication burst pulled

from the author's own home lab. QRadar is not commodity lab infrastructure. It is a commercial SIEM platform with licensing, appliance sizing, and — for anything beyond a single all-in-one console — hardware or cloud spend well past what a solo author's lab supports on demand. That means this book's evidence base is structurally different from DEH's, and this is stated here, in the first part, rather than discovered by a reader twelve parts in wondering why every figure carries the same caveat.

Concretely: this book's figures are almost exclusively **OFFICIAL REFERENCE** (sourced and cited from IBM's own public documentation, with the source URL and covered version recorded in **REFERENCES.md**) or **CONCEPTUAL** (this book's own architecture and workflow diagrams — Figure 1.1 above is one — honestly labeled as illustrative, never styled to look like a captured screenshot). **CONTROLLED LAB EXAMPLE** (captured from a lab built specifically to generate the evidence) and **REAL LAB EXAMPLE** (captured from a real, pre-existing deployment) are not used anywhere in this book, on the current premise that neither exists for this project. That is a stronger restriction than DEH itself operates under, and it is a deliberate one, not a placeholder waiting to be quietly upgraded: **STYLE-GUIDE.md §9** is explicit that if lab access is ever secured, that is a scope change to the style guide itself, revised on the record, not a silent per-figure swap-in.

PRODUCT VERSION NOTE *This book's premise — that no low-cost, on-demand QRadar instance is available to this author — assumes no such access currently exists. IBM has, at various points, offered a limited-EPS trial or Community Edition VM for evaluation purposes; verify current availability, licensing terms, and feature completeness against IBM's own current offering before treating that premise as permanent. If that access path opens up, it changes this book's evidence-classification stance (**STYLE-GUIDE.md §9**) as a deliberate, recorded revision — not something a reader should assume has already quietly happened because a later part looks more detailed than this one.*

A second, related fact belongs here rather than buried in a later part: QRadar's own commercial packaging is itself something this book cannot track live. IBM has moved pieces of the QRadar business — in particular the cloud/SaaS-delivered side — through partnership and divestiture arrangements with other vendors in recent release cycles, while continuing to develop and support on-premises QRadar SIEM directly. Exactly which company operates, sells, or supports which piece of "QRadar" by the time a given reader is running this book's advice against a real console is not a fact this book can freeze in place.

PRODUCT VERSION NOTE QRadar's commercial packaging and delivery model have shifted across recent business decisions in ways this book does not have current, verified visibility into — including, at various points, movement of cloud/SaaS-delivered QRadar capability toward other vendors' platforms while IBM continues to sell and support on-premises QRadar SIEM directly. Whether "QRadar" means an IBM-operated deployment, an on-prem license IBM sells and supports directly, or a capability delivered through a different vendor's platform by the time you are reading this is exactly the kind of vendor-side fact *STYLE-GUIDE.md §11* flags as aging fastest and least gracefully. Verify your own organization's actual contract, support path, and delivery model before assuming any specific ownership arrangement is current.

5. The **PRODUCT VERSION NOTE** convention, promoted from footnote to rule

[CONCEPT] DEH Part 27 already hedges version risk in prose, ad hoc, exactly twice: "the exact wording and available test categories in the Rule Wizard have shifted across QRadar releases," and "whether a given QRadar version lets you anchor a Rule Test directly to a saved AQL search... is a version-dependent capability." Both instances above are real quotes from that part, not paraphrases, and both are correct instincts. This book's difference is that it makes that instinct a named, checkable convention instead of leaving it to each writer's individual judgment about when a sentence feels risky. A single-platform operations book makes far more concrete UI, navigation, and wording claims per page than a six-language comparison chapter ever needs to — proportionally, the version-drift risk here is larger, not smaller, which is why the convention gets its own section this early rather than a passing mention.

A **PRODUCT VERSION NOTE** — full callout form, or a lighter inline flag where a callout is too heavy for one clause — is required wherever a claim in this book names: a specific console menu path or click sequence; the exact wording of a Rule Wizard category, dashboard item name, or any other literal UI string; whether a specific feature exists at all in "QRadar" without a version qualifier; or anything about QRadar's commercial packaging, delivery model, or vendor-side support/ownership, exactly as §4 above just did twice. It is deliberately *not* required for the architectural concepts DEH Part 27 already established and this book stays consistent with rather than re-litigates: the existence of the CRE, the Ariel `events / flows` split, the general concept of a DSM parsing raw logs into a QID, or the three-input structure of magnitude (severity, relevance, credibility). Those are stable across the platform's history, not UI specifics, and a note on every one of them would train the reader to skip all of them — the same over-hedging failure the banned-filler rule in *STYLE-GUIDE.md §1* already guards against for prose generally. The test this book uses, matching DEH's own filler-word test in spirit: would a reader on a materially different QRadar release, checking their own console, find a given claim simply wrong, or only differently worded? "Simply wrong" earns a note. "Differently worded, same idea" does not.

6. How to read this book: tags, callouts, and finding the part you actually need

[CONCEPT] Every `## / ###` heading in this book opens with one of the six tags §2 above named — **[CONCEPT]**, **[ANALYST]**, **[RULE ENGINEER]**, **[PLATFORM ENGINEER]**, **[THREAT HUNTER]**, **[SOC MANAGEMENT]** — so a

reader who only cares about one role can scan for that bracket and skip the rest of a section without losing the thread. Nine recurring callout types do the rest of the structural work: **Rule Autopsy** (a Rule/Building Block/Reference Set configuration that shipped broken, dissected in four parts — the rule, why it shipped, how it failed, the fix), **Hunter's Note**, **Platform Reality** (documented behavior vs. what production actually does), **Blind Spot** (a named gap in what QRadar's architecture can see or do), **Noisy Offense Trap** (a specific configuration or operational gap that floods the queue, and its fix), **Validation Test** (Setup/Action/Expected result — deliberately not called "Rule Test," since QRadar's own Rule Wizard already owns that term for one condition inside a Rule), **SOC Management View**, **What Would Change My Mind**, and **PRODUCT VERSION NOTE** (§5 above). Full definitions and worked examples of all nine live in STYLE-GUIDE.md §6; this part only needs to name them so the rest of the book can use them without a reintroduction every time.

Table 1.3 is this part's most directly practical contribution: a wayfinding index by task, for a reader who arrived here from a search engine or a colleague's recommendation with one specific problem, not a plan to read all 22 parts in order.

Table 1.3 — Where to go for a specific problem. Supports jumping directly to the relevant part instead of reading this book front to back.

What You Need	Start At
How magnitude, credibility, and relevance actually combine into one queue-ranking score	Part 3
Getting a new log source parsing correctly for the first time	Part 4
A log source with no native DSM available	Part 5
Why a custom property referenced in a Rule Test keeps returning nothing	Part 6
Deciding whether an analytic belongs on <code>events</code> , <code>flows</code> , or both	Part 7
The full Rule Test catalog beyond a single Event rule	Part 8
A <code>BB:</code> library that has grown past a few hundred entries with no owner	Part 9
Choosing between a Reference Set, Map, Map of Sets, Map of Maps, or Table	Part 10
Why an Ariel search times out or competes with other analysts' searches	Part 12
A Rule flooding the queue overnight	Part 15
Building change control for Rules without a git-based pipeline	Part 16
Diagnosing why licensing dropped or delayed events last month	Part 17
A log source that stopped parsing after a vendor agent upgrade	Part 18
Running QRadar across Domains/Tenants for an MSSP	Part 21
Building a staffing and TCO case for a QRadar practice	Part 22

7. What would change this book's own scope

[CONCEPT] The boundaries this part draws — no AQL re-teaching, no Sigma-vs-native re-argument, no lab-captured evidence — are stated as deliberate, not as the only defensible choice, and it is worth naming up front what would actually move them rather than treating them as beyond question.

What Would Change My Mind This book treats "no AQL re-teaching, ever" as a fixed boundary because DEH Part 27 §1 already does that job well, and duplicating it here would be pure restatement with no added value to the reader. If reviewer or reader feedback across the first several released parts showed people repeatedly unable to follow this book's AQL examples even after being pointed to DEH Part 27 §1 — a demonstrated comprehension gap, not a stylistic preference for having everything in one book — the fix would be a short AQL quick-reference appendix here, cross-linked to DEH rather than duplicating its content, not a reversal of the no-re-teach rule itself. Separately: if a real, authorized QRadar instance becomes available to this project, STYLE-GUIDE.md §9's current stance — that **CONTROLLED LAB EXAMPLE / REAL LAB EXAMPLE** figures are not currently available — changes, but only as a deliberate, recorded revision to STYLE-GUIDE.md §0 and §9 together, never as a quiet swap-in on individual figures where a reviewer might not notice the evidence class had shifted.

Part 2 picks up immediately where this part leaves off, with the first required PRODUCT VERSION NOTE of the operational content proper: the same on-prem-vs-cloud/SaaS packaging risk named in §4 above, applied specifically to deployment topology (Console, Event Processor, Event Collector, Flow Processor, Data Node, and App Host roles) rather than to the platform's commercial existence in the abstract.

Cross-references: DEH Part 23 (Query Language Strategy) §1 (DET-23-01, the canonical analytic this book's own worked examples inherit by reference), §5 (Sigma-first-vs-native tradeoff, not re-argued here); DEH Part 27 (QRadar AQL) §1 (AQL syntax fundamentals, not re-taught), §2 (CRE/Building Block/Reference Set introduction), §3.1–§3.4 (DET-27-01 worked example), §6 (Offense-as-hybrid-of-Alert-and-Case finding); DEH **TERMINOLOGY.md** (Alert, Case, Disposition, Tuning, Suppression, Coverage — inherited unchanged); this book's **STYLE-GUIDE.md** §6 (nine callouts), §7 (six tags), §9 (evidence classification), §11 (PRODUCT VERSION NOTE convention); this book's **BOOK-INDEX.md** (full 22-part table); forward references to Part 2 (deployment architecture), Part 3 (offense scoring), Part 22 (staffing/TCO model).

Part 2 — QRadar Deployment Architecture: Console, Processors, Collectors, and Data Nodes

Why this part exists

[CONCEPT] Every other part in this book assumes an event or a flow has already arrived somewhere QRadar can query or correlate it. Part 3 starts from a fired Offense. Part 4 starts from a raw log a DSM (Device Support Module) hasn't parsed yet. Part 12 starts from an Ariel search that's already slow. None of those parts stop to ask the question this one answers first: what physical or virtual component actually receives that data, what happens to it before it becomes a row in `events` or `flows`, and which component in a distributed deployment is doing which job. DEH Part 27 §1.1 introduces the `events/flows` split as a query-language fact — two tables AQL can search — without needing to explain how a raw log or a raw packet turns into a row in either table, because DEH Part 27's job was proving a point about correlation semantics, not teaching platform topology. This part owes that explanation, and everything from Part 4 onward inherits it.

A single-appliance QRadar deployment — an All-in-One — hides this entirely: one box receives logs, receives flows, parses, indexes, correlates, and serves the console UI, so there's no data-flow question worth asking. A distributed deployment, the shape almost every enterprise-scale QRadar environment actually runs, splits those jobs across named component types that each own one part of the pipeline. Get the roles wrong — assume the Console does correlation, or assume an Event Collector stores what it receives — and every downstream troubleshooting step in Part 18 starts from a wrong mental model of where to even look.

This part covers, in order: the component catalog (Console, Event Collector, Event Processor, Flow Processor, Data Node, App Host), the All-in-One-vs-distributed decision, and the path an event and a flow each physically travel from arrival to the point Part 27 §1.1's two tables pick them up. It closes with the first of this book's required PRODUCT VERSION NOTE callouts on packaging and delivery model, per `STYLE-GUIDE.md` §11 and the honesty constraint in §0 — a topic this book flags explicitly rather than asserting a current state it cannot verify from outside a live deployment.

1. The component catalog

[PLATFORM ENGINEER] A distributed QRadar deployment is built from a small set of named appliance/component types, each installable as a dedicated physical appliance, a virtual appliance, or (where the current licensing and packaging model permits — see §4's PRODUCT VERSION NOTE) a cloud-hosted instance. What follows states each component's job at the level that's stable across releases: what data it's responsible for and what it hands off to which neighbor. Exact sizing figures, model numbers, and appliance naming are the part that drifts and are deliberately not the focus here.

1.1 Console

[PLATFORM ENGINEER] The Console is the management head of the deployment: it's where an administrator logs in, where the Custom Rules Engine (CRE) has its Rule Wizard configured (Part 8), where Reference Sets are administered (Part 10), where the Offense list and Ariel search UI are presented to an analyst (Parts 3, 12, 14), and where the App Framework's apps run unless a dedicated App Host is deployed for that purpose (§1.6, Part 19). In an All-in-One deployment the Console also does everything every other component below does. In a distributed deployment, the Console still holds overall deployment configuration and the CRE's rule-authoring surface, but the actual work of parsing, indexing, and correlating data at scale is delegated to the components below — the Console is the place you look and the place you configure, not necessarily the place doing the heaviest processing.

1.2 Event Collector

[PLATFORM ENGINEER] An Event Collector is the first point of contact for log data: it receives events from configured Log Sources (syslog, WinCollect, JDBC, and every other collection protocol Part 4 covers), applies the relevant DSM to parse and normalize each raw log into a QRadar event with a QID (QRadar's internal event-taxonomy identifier) and extracted fields, and forwards the normalized result onward for processing and storage. An Event Collector by itself does not store events long-term or run the CRE against them — it's a parsing-and-forwarding stage, which matters directly for §3's data-flow walkthrough and for Part 4's onboarding discipline: a Log Source is configured against a specific Event Collector, and DSM mapping happens here, before an event has any chance of being searched or correlated.

1.3 Event Processor

[PLATFORM ENGINEER] An Event Processor receives normalized events (from a co-located Event Collector or from a remote one forwarding to it), runs the CRE's real-time rule evaluation against the incoming stream (Part 27 §2's Custom Rules Engine, generalized past the single worked example there), and writes events to Ariel storage for later search. In many distributed deployments an Event Processor and an Event Collector are combined on one appliance; larger deployments separate them so collection load and correlation/storage load can scale independently. This is the component actually running **BB:**-prefixed Building Block conditions and **R:**-prefixed Rules against live data, which makes it the component Part 15's tuning workflow and Part 17's EPS capacity planning are ultimately budgeting resources against.

1.4 Flow Processor

[PLATFORM ENGINEER] A Flow Processor is the Event Processor's counterpart for network-flow data: it receives flow records (from QRadar's own QFlow collection or from a third-party flow source — Part 7 covers both), applies flow-specific processing (deduplication, superflow creation), runs the CRE against flow-based Rule Tests, and writes to the **flows** side of Ariel storage. Event and flow processing are architecturally parallel paths that converge only where a Rule Test, an AQL query, or an Offense is explicitly written to consider both — a distinction Part 7 builds out in full when it takes on the flow-vs-event design decision DEH Part 27 §1.1 raises but doesn't resolve.

1.5 Data Node

[PLATFORM ENGINEER]

A Data Node adds Ariel storage and search capacity without adding its own event/flow processing responsibility — it's how a deployment scales retention and search performance for a growing data volume independently of scaling correlation throughput. Data Nodes matter directly to Part 12's search-performance tuning: a search that's slow because of storage-tier contention has a different fix than one that's slow because of an underpowered Event Processor, and knowing which component actually holds the data being searched is the first branch in that diagnosis.

1.6 App Host

[PLATFORM ENGINEER]

An App Host is a dedicated component for running App Framework apps (Part 19) — Use Case Manager, Pulse, UBA, and third-party App Exchange apps — off the Console, so a resource-hungry or poorly-behaved app doesn't compete with the Console's own management and UI workload. Not every deployment runs a dedicated App Host; smaller or All-in-One deployments run apps directly on the Console, which is itself a capacity-planning tradeoff Part 19 covers in more depth than this part needs.

[CONCEPT]

Table 2.1 summarizes the six component types by the one question that matters most for troubleshooting and capacity planning: what job does each one own, and what does it hand off.

Table 2.1 — QRadar deployment component roles. Supports deciding which component a given performance, parsing, or correlation problem actually belongs to before troubleshooting it.

Component	Primary job	Hands off to	Does not do
Console	Management, rule/Reference Set authoring, Offense/search UI	— (top of the deployment)	Bulk event/flow parsing or storage at scale
Event Collector	Receives Log Source data, applies DSM, normalizes to a QID	Event Processor	Long-term storage, CRE evaluation
Event Processor	Runs CRE against normalized events, writes to Ariel events	Data Node (storage/search scale-out)	Flow processing
Flow Processor	Processes flow records, runs CRE against flow Rule Tests, writes to Ariel flows	Data Node (storage/search scale-out)	Event (log) processing
Data Node	Adds Ariel storage and search capacity	— (queried by Console/analysts)	Its own independent CRE evaluation
App Host	Runs App Framework apps off the Console	—	Core collection, processing, or correlation

PRODUCT VERSION NOTE *The component list above (Console, Event Collector, Event Processor, Flow Processor, Data Node, App Host) reflects QRadar's long-standing distributed-architecture model and is stable in concept across recent releases. Which specific appliance model numbers map to which role, whether a given role ships only as a combined appliance (an Event Processor bundled with local Event Collector and Data Node functions, sometimes marketed as an "All-in-One Event Processor") versus a standalone one, and the exact terminology IBM's current sales and deployment documentation uses for each, have shifted across release and product-packaging cycles. Verify current appliance/component naming against IBM's own deployment documentation for the release you're planning against before treating this part's names as a literal ordering catalog.*

2. All-in-One vs. distributed: the decision, not just the definition

[PLATFORM ENGINEER] An All-in-One deployment runs every role in §1 on a single appliance. It is the right starting point for a small environment, a proof-of-concept, or an environment whose sustained EPS/FPM (Part 17) comfortably fits one appliance's processing and storage capacity with headroom. It is also, definitionally, a single point of failure and a single scaling ceiling: every additional Log Source, every additional analyst running a concurrent Ariel search, and every additional Rule the CRE evaluates all compete for the same box's CPU, memory, and storage I/O.

A distributed deployment exists to remove that ceiling by letting each role in §1 scale independently: add Event Collectors near where log volume originates (a second data center, a segment with its own compliance-driven data-residency requirement), add Event Processors or Flow Processors to keep pace with growing sustained EPS/FPM, add Data Nodes to keep retention and search performance from degrading as stored volume grows, and keep the Console as the single point of administration and rule authoring across all of it.

The decision isn't binary in practice — most real deployments land somewhere between "everything on one box" and "every role fully separated," adding components incrementally as a specific bottleneck appears rather than fully distributing from day one. Table 2.2 names the signal that justifies each specific addition, which is the more useful framing than a blanket "distributed is better."

Table 2.2 — Signals that justify adding a specific component, rather than distributing everything at once.

Symptom observed	Component most likely to add	Why this one, not another
Sustained EPS approaching license/appliance ceiling, events queuing at ingestion	Event Collector (dedicated)	Collection/parsing load is the bottleneck, not correlation or storage
CRE evaluation lag, Offenses appearing late relative to event arrival	Event Processor (additional or upgraded)	Correlation throughput, not collection, is the constrained resource
Flow volume outpacing event volume disproportionately (heavy NetFlow ingestion)	Flow Processor (dedicated)	Flow and event processing are parallel paths; scale the one under load
Ariel searches slow or timing out as retained data grows, ingestion itself is fine	Data Node	Storage/search capacity, not processing throughput, is the constraint
App Framework app (Part 19) competing with Console UI responsiveness	App Host	Isolates app resource use from Console management workload
Remote site/segment with its own compliance-driven residency requirement	Event Collector (local)	Data needs to be collected and normalized close to origin before crossing a network boundary

Platform Reality A distributed deployment doesn't just add capacity — it adds a network dependency every All-in-One deployment doesn't have. An Event Collector that loses connectivity to its downstream Event Processor doesn't correlate anything until connectivity is restored; depending on configuration and version, it may buffer locally for a bounded period or begin dropping data once its local buffer fills. Before splitting a previously-All-in-One deployment across multiple sites, confirm the buffering behavior and its limits for your specific version, and treat "we added redundancy" and "we added a new single point of failure between two components that used to be one box" as two claims that both need checking, not just the first one.

3. How an event and a flow physically move through the deployment

[CONCEPT] DEH Part 27 §1.1 treats **events** and **flows** as two tables AQL can query, without needing to explain how a row lands in either one. This section is that explanation, walked once for each path, because Part 4 (DSM/QID mapping), Part 7 (flow architecture), and Part 12 (Ariel performance) all build directly on knowing which physical hop a given failure or slowdown belongs to.

3.1 The event path

[PLATFORM ENGINEER] A raw log leaves its source system (a Windows domain controller, a firewall, an application server) and reaches QRadar via whatever collection mechanism Part 4 covers for that Log Source type — syslog forwarding, an agent like WinCollect, a polled protocol like JDBC. It arrives at an Event Collector, which applies the DSM configured for that Log Source: the DSM assigns a QID, extracts any custom properties it's been configured to extract, and produces a normalized QRadar event. That normalized event is forwarded to an Event Processor (co-located or remote), which does two things with it independently: runs it through the CRE's currently-deployed Rules and Building Blocks in real

time (Part 27 §2's architecture, at production scale), and writes it to Ariel's `events` table, on local storage or a connected Data Node, where it becomes searchable via AQL exactly as DEH Part 27 §1.1's query examples assume.

3.2 The flow path

[PLATFORM ENGINEER] Flow data follows a parallel but architecturally distinct route (Part 7 covers the full picture; this is the summary this part needs). Flow records originate from QRadar's own QFlow processing of mirrored/tapped traffic, or from a third-party flow export (NetFlow, sFlow, IPFIX) pointed at QRadar. They arrive at a Flow Processor, which performs flow-specific processing — deduplication of the same flow reported by multiple observation points, superflow aggregation for high-volume, low-information flow bursts — runs flow-based Rule Tests through the CRE, and writes to Ariel's `flows` table. An event and a flow describing the same real network activity (a login attempt and the TCP session that carried it) can arrive through entirely different components, at different processing latencies, and land in different Ariel tables — which is exactly why Part 7 treats "should this analytic use `events`, `flows`, or both" as a real design decision rather than an afterthought.

3.3 Where the CRE and Ariel search actually sit in this picture

[PLATFORM ENGINEER] Both paths converge on the same structural fact Part 27 §2 established at the query-language layer: real-time CRE evaluation and ad hoc Ariel search are two separate consumers of the same underlying data, running at different points and different cadences. The CRE evaluates each event or flow once, as it arrives at its Event/Flow Processor, against whatever Rules are currently deployed — it does not retroactively re-evaluate historical data when a Rule changes. An Ariel search, by contrast, queries whatever has already been written to storage, at the time an analyst or a scheduled search runs it, regardless of what the CRE did or didn't do with that data when it first arrived. A newly deployed Rule catches new matching activity going forward; it does not retroactively create an Offense for a matching event that arrived yesterday, before the Rule existed. Figure 2.1 makes this full path, both branches, concrete as one diagram.

flowchart TB

```

subgraph Sources["Data sources"]
    L["Raw log\n(syslog, WinCollect, JDBC...)"]
    F1["Raw flow data\n(QFlow, NetFlow, sFlow...)"]
end

L --> EC["Event Collector\nDSM parses, assigns QID,\nexttracts custom properties"]
F1 --> FP["Flow Processor\nDedup, superflow creation"]

EC --> EP["Event Processor"]
FP --> FPcre["Flow Processor\nCRE evaluation"]

EP -->|"real-time CRE\n(Rules, Building Blocks)"| CRE1["Offense created/updated\n(Part 3)"]
EP -->|"write"| AE["Ariel events table"]

FPcre -->|"real-time CRE\n(flow-based Rule Tests)"| CRE2["Offense created/updated\n(Part 3)"]
FPcre -->|"write"| AF["Ariel flows table"]

AE -->|"AQL search"| Search["Analyst / hunter\n(DEH Part 27 §1, this book's\nPart 12, Part 14)"]
AF -->|"AQL search"| Search

DN["Data Node(s)\n(storage/search scale-out)"] -.->|"backs"| AE
DN -.->|"backs"| AF

Console["Console\n(Rule/Building Block/Reference Set\nauthoring, Offense + search UI)"] -.
>|"configures"| EP
Console -.>|"configures"| FPcre
  
```

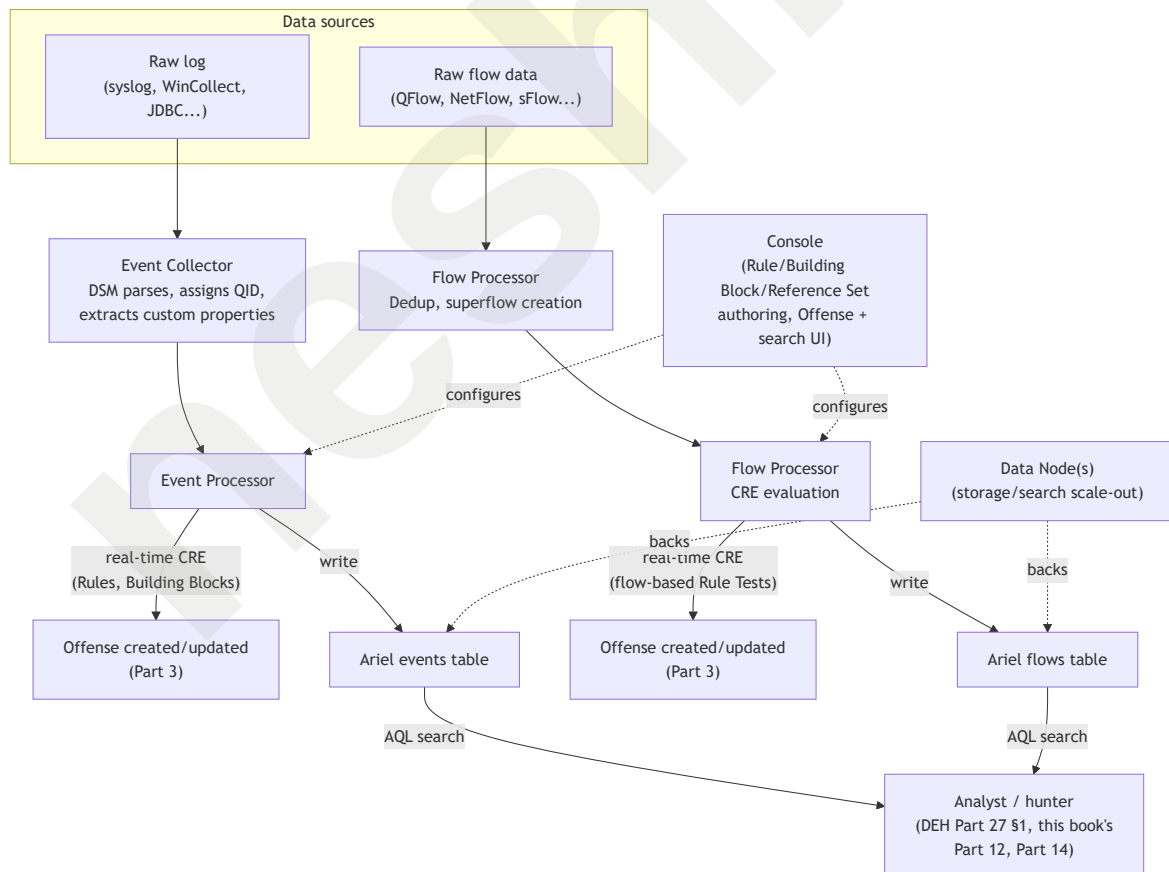


Figure 2.1 — Event and flow data flow from source to Ariel storage and real-time correlation.

CONCEPTUAL. Illustrates the two parallel paths (event, flow) this section describes and the point at which each converges on the same two consumers — real-time CRE evaluation and ad hoc AQL search — that Part 27 §1.1 and §2 describe from the query-language side; not a reproduction of any single IBM architecture diagram, and not a claim about a specific version's exact process names or thread model for these components internally.

Hunter's Note *When a hunt (DEH Part 27 §5's pattern, extended in this book's Part 14) turns up a gap — an event you expected to see and didn't — the first branch point in diagnosing it is exactly this diagram: is the event missing because it never reached an Event Collector at all (a Log Source configuration problem, Part 4), because it reached the collector but the DSM didn't recognize it (a QID-mapping gap, also Part 4), or because it's sitting in **events** but the Rule Test looking for it has a logic gap (a CRE authoring problem, Part 8-11)? Each of those is a different fix, and confusing "the data never arrived" with "the data arrived but the Rule didn't catch it" wastes a hunt's most valuable resource: knowing which team owns the fix.*

4. Packaging and delivery model: the version-sensitive part of this whole topic

[PLATFORM ENGINEER] Everything in §1–§3 describes component roles and data-flow order that have held stable across QRadar's architecture for a long time — an Event Collector's job relative to an Event Processor's is not the kind of thing that changes release to release, which is why this part states it plainly rather than hedging it. What has changed, more than once, in ways this book's author cannot track from outside a live deployment, is how QRadar is actually sold, licensed, and delivered: on-premises physical or virtual appliances you size and patch yourself, versus a cloud-hosted or SaaS-delivered offering where IBM or a partner operates some or all of the component roles in §1 on your behalf, versus hybrid arrangements that split the difference. Which of these delivery models is current, what each one is currently branded, and which QRadar product family (or successor branding) owns which piece of this at any given moment, is squarely the kind of claim DEH's own instinct — and this book's §11 convention — exists to keep out of the "asserted as permanent fact" category.

PRODUCT VERSION NOTE *This book states the component architecture in §1–§3 as stable and does not tie it to a specific delivery model, because the roles themselves (collect, process, store, correlate, manage) hold across on-prem, cloud-hosted, and hybrid arrangements. What this book explicitly does not assert as current fact: which specific packaging tier, cloud-delivery option, or vendor/partner support arrangement is presently available, what each is currently named, or which of the component roles in Table 2.1 a given commercial offering does or doesn't let a customer operate directly versus consume as a managed service. This space has shifted across QRadar's release and business history in ways outside this book's current visibility. Verify current packaging, delivery model, and ownership against IBM's own current commercial documentation before making a purchasing or migration decision on the strength of anything in this part.*

5. What this part deliberately leaves for later parts

[CONCEPT] This part stops at "data has arrived, been normalized or processed, and is now sitting in Ariel storage or has already triggered real-time CRE evaluation." Everything past that point is scoped elsewhere on purpose, so this part stays a topology-and-data-flow foundation rather than a shallow preview of six other parts:

- What actually happens inside the CRE when a Rule evaluates — Building Block composition, Reference Set checks, thresholds — is Part 27 §2's job at the query-language-comparison depth and this book's Section D's job at full operational depth (Parts 8–11).
- What the DSM actually does to a raw log, and what happens when that mapping is missing or wrong, is Part 4 and Part 5's job in full.
- The flow-vs-event design decision this section's §3.2 gestures at is Part 7's job.
- What a Data Node's storage limits mean for retention buckets and search tuning is Part 12's job.
- Capacity planning against the EPS/FPM ceilings this part's components enforce is Part 17's job, building on DEH Part 27 §1.3's single-paragraph flag.

What Would Change My Mind *This part treats the six-component catalog in §1 as the stable core of QRadar's architecture, worth stating plainly rather than hedging generically. That confidence would be misplaced if IBM restructured the platform's own component model itself — not just renamed appliances or changed a delivery tier, but genuinely collapsed or split the collect/process/store/correlate responsibilities differently than they've been split across the releases this book's author could research. A reader on a QRadar release where, say, flow and event processing have been architecturally merged into one component type, or where Data Node-equivalent storage scale-out works on a fundamentally different mechanism, should treat this part's Table 2.1 as out of date and flag it for revision rather than forcing their console into this part's categories.*

Cross-references: DEH Part 27 §1.1 (the `events` / `flows` Ariel tables this part's §3 explains the physical arrival path for) and §2 (the Custom Rules Engine this part's Event/Flow Processor roles run in production); DEH Part 23 (Query Language Strategy, for the general query-language-selection context this book's AQL-adjacent parts build on without re-arguing). Within this book: Part 3 (Offenses, Magnitude, Credibility, and Relevance — what happens the moment §3.3's CRE evaluation fires a Response); Part 4 (DSMs, QID Mapping, and Log Source Onboarding — the Event Collector-side detail this part summarizes); Part 7 (Flow Data Architecture — the flow-vs-event design decision this part's §3.2 defers); Part 12 (Ariel Search Performance and Tuning — Data Node and storage-tier detail); Part 17 (Licensing, EPS/FPM Management, and Capacity Planning — the component-scaling signals in Table 2.2, taken to full capacity-planning depth); Part 19 (Apps, Extensions, and the App Framework — the App Host role in full).

Section B — The Offense Model

neshboy

Part 3 — Offenses, Magnitude, Credibility, and Relevance

Why this part exists

[CONCEPT] DEH Part 27 §6 needed exactly one paragraph to make a single point about QRadar's Offense object — that it "is not quite either an Alert or a Case in TERMINOLOGY.md's strict sense" but a hybrid of both, bundling contributing events under one indexed record over time with "a `magnitude` score QRadar computes from severity, relevance, and credibility rather than a single fixed severity value." That paragraph existed to support a different argument entirely (why AQL alone can't build a QRadar detection); it was never meant to be the full treatment of offense scoring. This part is that full treatment. It inherits DEH Part 27 §6's finding rather than re-deriving it — an Offense stays a hybrid Alert/Case exactly as DEH `TERMINOLOGY.md` defines both terms — and goes deep on what the rest of DEH Part 27 only had room to name: what severity, credibility, and relevance actually measure, how they combine into magnitude, how an Offense is indexed to an entity, and what happens to an Offense across its lifecycle from first contributing event to cleanup.

One constraint governs every specific number in this part: IBM's own QRadar documentation was not reliably fetchable while this book was being outlined (`STYLE-GUIDE.md` §0), and the exact internal weighting arithmetic behind magnitude has never been something this book's author has verified against a live console. Where a claim is architectural — that magnitude has three named inputs, that credibility can rise from cross-source corroboration, that relevance depends on asset context — it is stated plainly, because that structure is stable across the platform's history. Where a claim would require a specific formula, a specific console field name, or a specific default timer, it carries a `PRODUCT VERSION NOTE` instead of an invented number. This part does not re-teach AQL (DEH Part 27 §1 owns that) and does not re-argue Sigma-first-vs-native authoring (DEH Part 23 §5 owns that); every AQL fragment below exists only to illustrate what an Offense's contributing events look like from the search side, not to teach syntax.

1. What an Offense actually is

[CONCEPT] An Offense is the object QRadar's Custom Rules Engine — universally abbreviated CRE — produces when a deployed Rule's Response is configured to create or contribute to one. It is the queue item an analyst opens, which makes it function like DEH `TERMINOLOGY.md`'s Alert. But unlike an Alert, which is one timestamped instance per match, a single Offense keeps absorbing every later event or flow that matches the same Rule (or, as below, a different Rule) against the same indexed entity, which makes it also function like DEH's Case — an investigative container that accumulates evidence rather than closing the moment it opens. Treat "one Offense" as "one entity's accumulated matches over time," not as "one event that fired once."

That accumulation is not limited to repeats of the same Rule. Because QRadar indexes an Offense by an entity value — not by which Rule produced it — a second, entirely different Rule that matches the same indexed value (the same source IP, the same username, whatever the index is) contributes to the already-open Offense instead of opening a new one. A host that first trips a port-scan Rule and later

trips a credential-dumping Rule can end up as one Offense carrying both findings, not two separate queue items an analyst has to manually connect. This is a genuine strength of the model — the Offense already is the correlation DEH Part 27 §2.2's Reference Set pattern builds toward — and it is also why magnitude, credibility, and relevance are computed at the Offense level rather than the event level: they have to summarize a record that can grow more complex over its own lifetime.

Offenses also carry a domain/tenant binding in a multi-tenant deployment, which changes who can even see a given Offense in the first place — full treatment in Part 21; noted here only so the entity-binding discussion in §4 isn't mistaken for the only binding an Offense carries.

2. The three inputs magnitude is built from

[CONCEPT] Every Offense carries three separate scores before it carries one: severity, credibility, and relevance. Each measures a different question, each is visible independently on the Offense record, and magnitude (§3) is what QRadar computes from all three together. Understanding what each one actually asks is the difference between triaging by a single composite number and triaging with judgment.

2.1 Severity — how threatening this activity is rated to be

[CONCEPT] Severity answers: independent of who or what it hit, how dangerous is this pattern? A brute-force authentication failure and a confirmed LSASS memory-read both carry a severity rating, and that rating is a property of the matching Rule and the QID (QRadar's internal event-taxonomy identifier, distinct from any vendor's native event ID — DEH Part 27 §1.2 introduces the term) or category the activity falls under, not a property of which asset it landed on. A brute-force failure against a domain controller and the identical brute-force failure against a decommissioned test VM start with the same severity; the asset's own criticality is relevance's job (§2.3), not severity's.

[RULE ENGINEER] Severity is configured as part of a Rule's or Building Block's Response settings in the Rule Wizard (DEH Part 27 §2.1), and QRadar's own event/category catalog carries default severities for QIDs that haven't had a Rule explicitly override them. The exact numeric scale bounds and the specific default severity QRadar's own catalog assigns to a given category are QRadar's own shipped data, not something this book asserts from memory — verify the current defaults for your own deployment's QID catalog before treating a specific number as fixed platform behavior.

Noisy Offense Trap A **BB:** library that has never had its default severity revisited since initial deployment (left at whatever the Rule Wizard's default happened to be — DEH Part 27 §2.1) inflates the whole queue's magnitude ceiling regardless of what the underlying activity actually warrants: every Offense a high-severity Building Block contributes to starts several points ahead of one built from a more conservatively-rated Rule, whether or not the activity is genuinely more dangerous. The fix is not lowering every threshold uniformly — it's a periodic severity audit against accumulated Disposition data (DEH **TERMINOLOGY.md**'s Disposition entry). A Rule whose Offenses resolve mostly Benign Positive or False Positive over a quarter is a candidate for a severity re-rating, not just a Reference Set exclusion; Part 15's tuning-intake process is the right home for this as a recurring review, not a one-time fix.

2.2 Credibility — how much QRadar trusts that the report is real

[CONCEPT] Credibility answers a different question: how much should this specific report be trusted as an accurate reflection of something that actually happened? A single Log Source's own configured trustworthiness feeds this — QRadar lets an administrator rate a Log Source's baseline credibility, and a source known for noisy or unreliable parsing starts every event it contributes lower than a well-validated one. Credibility can also rise through corroboration: when multiple independent sources report the same underlying activity — a firewall and an IDS both flagging the same connection, or (continuing DEH Part 27's own worked example) a Sysmon-fed Rule and an independently-configured EDR feed both reporting the same process touching `lsass.exe` — QRadar treats that agreement as evidence the activity is real, not an artifact of one source's own parsing quirks, and credibility on the resulting Offense reflects that corroboration rather than a single source's rating alone.

[PLATFORM ENGINEER] A Log Source's baseline credibility rating is an administrator-configurable field, set where Log Sources themselves are configured. The exact menu location and field wording for this setting have shifted across console redesigns the way most navigation claims in this book have — verify the current path in your own deployment's admin area rather than assuming a specific click sequence still matches.

2.3 Relevance — how much the target's context raises the stakes

[CONCEPT] Relevance answers a third question, distinct from both: given where this happened and what it happened to, how much does that context matter? The same event category can carry different relevance depending on the asset's position in the Network Hierarchy, its known vulnerabilities from an integrated vulnerability-scan feed, and any business-criticality weighting an administrator has assigned it — a failed-authentication burst against a domain controller and the identical burst against an isolated lab host are not equally relevant even though they may carry identical severity and identical credibility. Relevance is the input that ties an Offense's score back to *your* environment's own asset model rather than to the activity pattern alone, and it is exactly the reason a generic Rule with generic severity can still produce very differently-prioritized Offenses across two different networks running the same rule base.

Relevance's full mechanics — how the Network Hierarchy, the Asset Database, and vulnerability-feed data actually combine to weight an asset, and the staleness risk of an asset model nobody re-validates after infrastructure changes — belong to Part 20, not here; this part introduces relevance only far enough to make magnitude legible.

3. Magnitude: combining severity, relevance, and credibility into one score

[CONCEPT] Magnitude is the one number QRadar displays prominently on the Offenses list and what actually ranks the queue — a composite QRadar computes from severity, credibility, and relevance so an analyst can sort and prioritize without manually cross-referencing three separate values for every open Offense. The three-input structure itself is stable, documented architecture, not a guess this book is making: it is the same structure DEH Part 27 §6 already named in one sentence. What this book will not do is assert the specific arithmetic — the exact weighting each input contributes, or the exact formula that turns three component scores into one magnitude value.

PRODUCT VERSION NOTE *Magnitude's three-input structure — severity, relevance, and credibility — is architecturally stable and consistent with what DEH Part 27 §6 already established. The exact arithmetic weighting each input contributes to the final magnitude value is not something this book states as a fixed formula: IBM's own documentation was not reliably fetchable during this book's authoring, and the level of arithmetic detail QRadar's own published guidance exposes has itself varied across releases. Treat any specific formula you find in a console tooltip or a given release's documentation as current for that release only, and verify it against your own deployment rather than this book before building a triage rule around a specific numeric threshold.*

Figure 3.1 illustrates the shape of that computation without claiming the arithmetic behind it.

```
flowchart TB
    A["Event / flow matches\na deployed Rule\n(may itself chain BB: + RS-\nper DEH Part 27 sections 2-3)"] --> B["Rule Response:\ncreate or contribute\nto an Offense?"]
    B -->| "no open Offense yet\nfor this index value" | C["New Offense opened,\nindexed on the chosen entity\n(source IP / dest IP /\nusername / custom property – see 4)"]
    B -->| "open Offense already\nexists for this index value" | D["Existing Offense\nabsorbs this event,\nre-scored"]
    C --> E["Offense score inputs"]
    D --> E
    E --> F["Severity\nhow threatening the\nmatching Rule/category is rated"]
    E --> G["Credibility\nsource trust +\ncross-source corroboration"]
    E --> H["Relevance\nasset criticality +\nnetwork/vulnerability context"]
    F --> I["Magnitude\none queue-ranking score"]
    G --> I
    H --> I
    I --> J["Offenses tab,\nranked for analyst triage\n(Part 14)"]
```

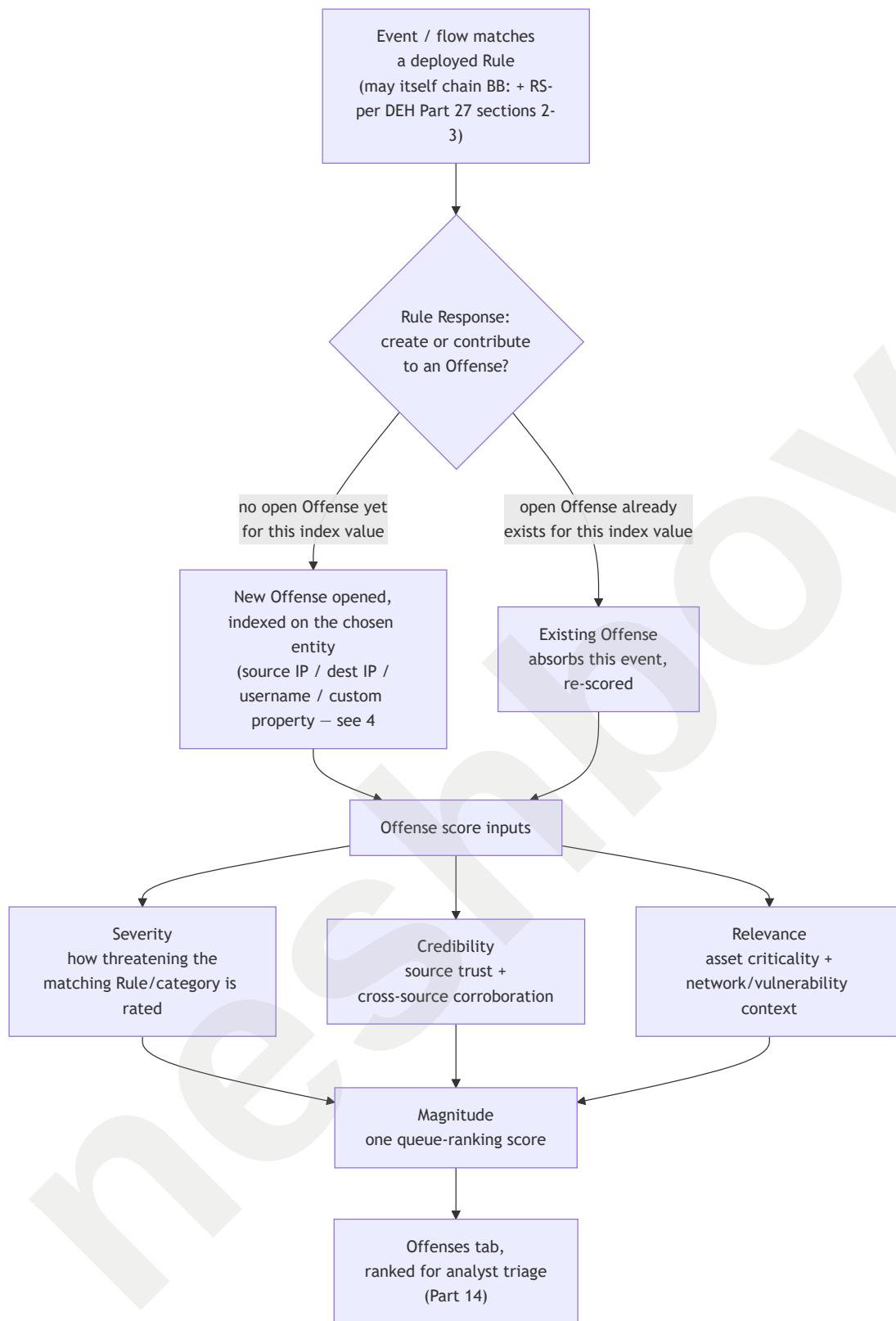


Figure 3.1 — From a matching Rule to a ranked Offense. *CONCEPTUAL.* Illustrates the path from a single Rule match (built, per DEH Part 27 §2–§3, from Building Blocks and Reference Sets rather than one AQL statement) to an Offense carrying a computed magnitude, and the three named inputs that computation draws from. This book has no captured IBM diagram to pair with this figure under **OFFICIAL REFERENCE** — IBM's own offense-management documentation was not reliably fetchable while this book

was authored (see the `qradar_version_scope` front-matter field) — and does not fabricate one; a future revision should add the sourced IBM figure under that tag if that access gap closes, per `STYLE-GUIDE.md` §9.

Table 3.1 collects the three inputs in one place as a working reference.

Table 3.1 — Magnitude's three scoring inputs. Supports deciding which lever actually needs adjusting — a Rule's severity rating, a Log Source's credibility configuration, or an asset's relevance weighting — instead of tuning magnitude as if it were one dial.

Input	What It Measures	Primary Configuration Surface	Covered In Depth
Severity	How threatening this event/category is rated, independent of what it hit	Rule/Building Block Response settings; QID/category default severity	Part 8 (Rule Wizard Catalog), Part 9 (Building Blocks)
Credibility	How much the report should be trusted as real, boosted by independent-source corroboration	Log Source credibility rating; multi-source event aggregation	Part 4 (DSM/Log Source Onboarding), Part 17 (Capacity Planning)
Relevance	How much the target's own context (criticality, vulnerabilities, network position) raises the stakes	Network Hierarchy weighting; Asset Database; vulnerability-feed integration	Part 20 (Asset Model, Network Hierarchy, and Vulnerability Data)

Blind Spot *Magnitude is one number computed from three inputs, and two Offenses can land on the identical magnitude value for very different reasons — one because a single Rule with an aggressively-configured high severity fired once on an uncorroborated event, another because three independently credible sources each reported a moderate-severity event against a highly relevant asset. The CRE has no mechanism to flag "these two 7s got there differently" anywhere in the Offenses list itself; that distinction only surfaces if an analyst opens the Offense and reads severity, credibility, and relevance individually rather than triaging by the composite number alone. A queue sorted purely by magnitude quietly trains analysts to skip exactly the step that would catch this.*

Continuing DEH Part 27 §3.3's worked example makes this concrete. `R: Suspicious LSASS Access – Unapproved Process` (DET-27-01, MITRE T1003.001 (OS Credential Dumping: LSASS Memory)) fires when `BB:LSASS-Access-Candidate` matches and the source process is absent from `RS-Allowlisted-LSASS-Tools`. If only the Sysmon-fed path reports the access, the resulting Offense's credibility reflects that one Log Source's own configured rating. If an independently-configured EDR feed also reports the same process touching `lsass.exe` around the same time, and both feed the same indexed host, credibility rises on the strength of that corroboration — the same underlying analytic, scored differently depending on how many independent sources agree it happened, which is exactly what §2.2 means by credibility not being a synonym for "did the Rule match."

4. Offense indexing and entity binding

[RULE ENGINEER]

When a Rule's Response is configured to create or contribute to an Offense, the Rule Wizard requires a choice of what indexes it — the entity value QRadar uses to decide whether a later match belongs to an already-open Offense or needs to start a new one. Common choices are source IP, destination IP, username, or a custom property (the same kind of DSM-extracted field DEH Part 27 §1.2 covers for AQL). This choice determines what "the same entity" means for bundling purposes, and it is not a cosmetic setting — it changes what magnitude, credibility, and relevance are being computed *about*.

Index on source IP for an attacker-centric Rule (scanning, beaconing) and every contributing event from one apparent source lands in one Offense regardless of which target or account it touched. Index on destination IP for an asset-centric Rule (protecting one high-value server) and every contributing event against that asset bundles together regardless of source. Index on username for an identity-centric Rule (credential misuse) and one account's activity bundles across hosts and source IPs. Choosing wrong doesn't produce an error — it produces an Offense that quietly measures the wrong thing, exactly the kind of failure DEH Part 27 §3.1 already warns fails silently rather than loudly.

Table 3.2 lays out the choice and its failure mode per index type.

Table 3.2 — Offense index choice, what it bundles, and how it fails when chosen wrong. Supports picking the index deliberately at Rule-authoring time rather than discovering the wrong choice only after Offenses have already accumulated against it.

Index Value	What Bundles Into One Offense	Best Fit	Failure Mode If Chosen Wrong
Source IP	Every contributing event from one apparent source, any target, any account	Attacker-centric rules (scanning, C2 beaconing)	An attacker pivoting through several source hosts splits across multiple Offenses instead of one
Destination IP	Every contributing event against one target asset, any source, any account	Asset-centric rules (protecting a specific server or domain controller)	One account's activity across many targeted assets never bundles into a single view of that account's behavior
Username	Every contributing event tied to one account, any source, any target	Identity-centric rules (credential misuse, insider risk)	A shared or service account triggering unrelated Rules from unrelated hosts merges unrelated incidents into one inflated Offense
Custom property	Whatever the DSM-extracted field represents (a session ID, a ticket number)	Narrow, purpose-built correlation	Depends entirely on the property's own extraction reliability — DEH Part 27 §3.1's QID-mapping prerequisite applies directly

Multiple Rules indexed the same way on the same value are what allow the accumulation behavior §1 describes — a port-scan Rule and a credential-dumping Rule both indexed on the same source IP contribute to one Offense, not two. Two Rules meant to correlate the same campaign but indexed on different values (one on source IP, one on username) will never merge, even if a human reviewing both would obviously connect them — a design decision worth checking explicitly when a new Rule is meant to build on an existing one's Offense, not assumed to work automatically.

5. Offense lifecycle: active, inactive, and closed

[ANALYST]

An Offense begins the moment its first contributing event triggers a Rule Response configured to create one, and it stays open — actively able to absorb new contributing events and be re-scored — for as long as new matches keep landing on its indexed entity. This is the behavioral shape that matters for triage: an Offense you closed yesterday as a false positive can, if the same Rule matches the same indexed value again, either reopen or contribute to a fresh Offense depending on how your deployment handles reactivation, which is exactly why closing reasons and notes (Part 14) matter — the next analyst who opens a related Offense needs that history, not a clean slate.

[PLATFORM ENGINEER]

If no new contributing event lands within a configured window, an Offense stops actively accumulating without necessarily being analyst-closed — a state this book describes generically as inactive rather than asserting a specific formal label QRadar's own UI uses for it. Eventually, retention and cleanup policy determines whether an inactive Offense is auto-closed, purged, or simply ages out of default views while remaining queryable. An analyst can also close an Offense directly at any point, assigning a closing reason — the Disposition concept DEH [TERMINOLOGY.md](#) already defines, carried forward unchanged rather than redefined here.

```
stateDiagram-v2
```

```
  [*] --> Active : first contributing event\nmatches a Rule Response
  Active --> Active : new contributing event\nnon same indexed entity\n(magnitude re-scored)
  Active --> Inactive : no new contributing event\nwithin the configured\ninactivity window
  Inactive --> Active : a later event matches\nthe same indexed entity\nbefore cleanup
  Inactive --> Closed : retention/auto-close policy\nelapses, or an analyst\ncloses it first
  Active --> Closed : analyst assigns a\nclosing reason (Disposition)
  Closed --> [*] : eligible for\nretention cleanup
```

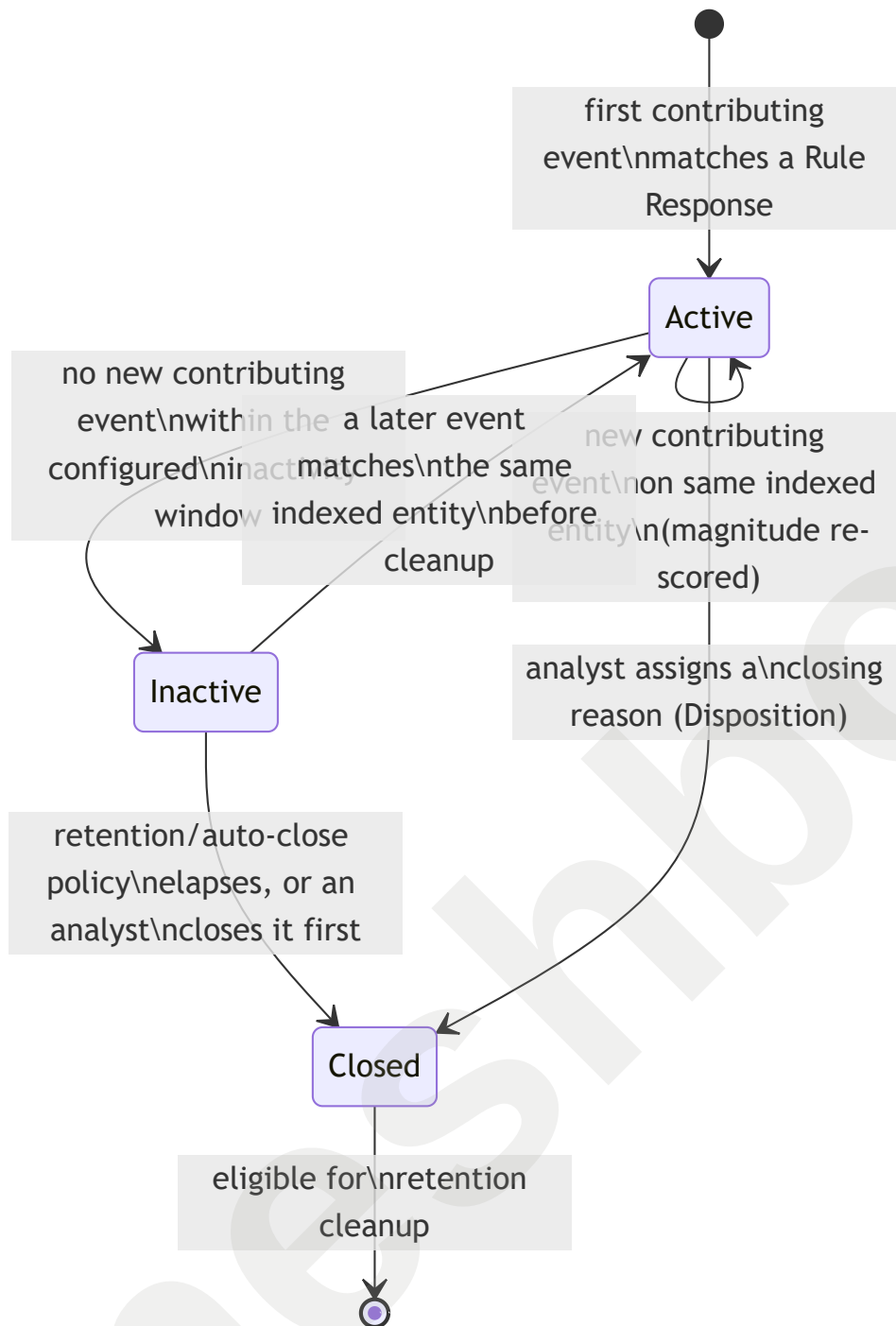


Figure 3.2 — Offense lifecycle: active, inactive, and closed. *CONCEPTUAL*. Illustrates the behavioral shape of an Offense's life — it stays live until either an analyst closes it or it ages out through inactivity and retention policy — without asserting the literal state labels or timer defaults QRadar's own console uses, which the PRODUCT VERSION NOTE below flags explicitly.

PRODUCT VERSION NOTE This section describes Offense state generically as active/open, inactive, and closed because that is the stable behavioral shape: an Offense keeps accumulating contributing events until something stops it, and stops being "live" through either analyst closure or elapsed inactivity. The literal state labels QRadar's own console displays, and the specific inactivity/retention timers — including whether an Offense can auto-close without any analyst action at all, and whether on-prem and cloud-hosted deployments default to the same timers — have differed across releases and packaging models. Verify your own deployment's Offense retention and closing configuration before relying on a specific timer value or state name as current.

6. Retention, cleanup, and offense forwarding

[PLATFORM ENGINEER] Closed and long-inactive Offenses don't disappear immediately — retention policy governs how long they remain queryable in the console before eventual purge, and that retention window is a direct capacity/storage consideration alongside the Ariel event and flow retention buckets Part 12 covers; an Offense's own contributing-event list is a pointer into Ariel data, not a separate full copy, so Offense retention and underlying Ariel retention need to be reasoned about together, not independently. Part 17's capacity-planning discipline is the right home for sizing this rather than treating it as a fixed default.

Offenses also frequently need to leave QRadar entirely — into a SOAR platform, a ticketing system, or an upstream aggregation point in an MSSP architecture (Part 21). Architecturally, that happens through one of a small number of paths: a Rule Response action that fires a notification or script when an Offense is created or updated, a dedicated forwarding/destination configuration for sending Offense (and event) data to an external system, or an external system polling QRadar's REST API for Offense data on its own schedule. Which of these is available, what it's called, and which upstream products ship a native integration, is exactly the kind of claim this book will not assert as fixed.

PRODUCT VERSION NOTE Offense forwarding to an external ticketing or SOAR platform is described here only at the architectural level — a Rule Response action, a dedicated forwarding/destination configuration, or an external system polling the REST API. Which of these paths exists in a given deployment, what it's called in the console, and which upstream products IBM packages a native integration for, has changed across release cycles and business/ownership changes in QRadar's own product line — the packaging/ownership trigger [STYLE-GUIDE.md §11](#) names explicitly applies here. Confirm current forwarding options against your own deployment's admin console and current vendor documentation before designing a workflow around a specific named feature.

7. Reading magnitude, credibility, and relevance at triage time

[ANALYST] The practical payoff of §§2–3 is triage discipline, not trivia: a fired Offense's magnitude tells you where it sits in a ranked queue, but severity, credibility, and relevance individually tell you *why* it's there, and those are different questions an analyst should be able to answer before acting. Is this Offense high-magnitude because three independent sources corroborated the same activity against a genuinely critical asset — strong grounds to escalate fast — or because one aggressively-rated Rule fired once against an asset nobody has weighted for relevance yet? Both can produce the same number. Only opening the breakdown tells you which one you're looking at. Part 14 owns the full day-to-day triage workflow this capsule feeds into; this section only establishes what the three inputs are worth reading for before that workflow starts.

[THREAT HUNTER] If an Offense's magnitude looks lower than the raw activity visible in Log Activity seems to warrant, don't stop at re-reading the Offense — pivot into an ad hoc AQL search scoped to the same indexed entity and time window, extending DEH Part 27 §5's hunting pattern from a cold hypothesis to an Offense you already have in hand, and check what didn't get bundled in. A narrow Building Block condition, a custom property left unpopulated for one of several contributing Log Source types (DEH Part 27 §3.1's silent-zero-rows failure mode), or a second Rule indexed on a different entity value can all leave real activity sitting outside the Offense's own contributing-event list with no error anywhere to flag the gap.

8. What this scoring model costs a SOC program

[SOC MANAGEMENT] "We closed N offenses this week" is not the same claim as "we reviewed N alerts" in a KQL/SPL shop, and it is not a coverage claim at all — DEH [TERMINOLOGY.md](#)'s Coverage entry already warns that bare "coverage" is an incomplete sentence, and an Offense count is one more bare number in the same family. An Offense already bundles however many contributing events one Rule's severity/credibility/relevance mix happened to produce; the count alone says nothing about detection breadth, and reporting it without qualification invites exactly the coverage-count confusion DEH Part 27's own closing paragraph flags for a different metric.

SOC Management View Budget analyst training time specifically for reading the severity/credibility/relevance breakdown behind magnitude, not just the composite number — §3's Blind Spot is a training gap this scoring model creates by design, not a one-time onboarding note. A single-severity-dial platform doesn't need this training line item; a platform whose primary ranking number can mean two structurally different things at the same numeric value does. That recurring cost belongs in the same staffing conversation Part 22 makes explicit for the QRadar practice as a whole, alongside the three-object Rule/Building Block/Reference Set review burden DEH Part 27's own closing paragraph already named.

Cross-references: DEH Part 23 §5 (Query Language Strategy — Sigma-first vs. native authoring, not re-argued here); DEH Part 27 §1.2 (QID and custom-property fundamentals), §2.1–§2.2 (Rule Test/Building Block/Reference Set mechanics behind any Rule that creates an Offense), §3.1/§3.3 (the [R: Suspicious LSASS Access – Unapproved Process](#) worked example continued in §3 above), and §6 (the Offense-as-

Alert/Case-hybrid finding this part inherits rather than re-derives); this book's Part 4 (DSMs, QID Mapping, and Log Source Onboarding), Part 8 (Rule Wizard Catalog), Part 9 (Building Blocks), Part 12 (Ariel Search Performance and Tuning), Part 14 (Analyst Triage Workflow), Part 15 (Noisy-Offense Triage and the Tuning Workflow), Part 17 (Licensing, EPS/FPM, and Capacity Planning), Part 20 (Asset Model, Network Hierarchy, and Vulnerability Data), Part 21 (Multi-Tenancy, RBAC, and MSSP Operations), and Part 22 (Staffing and Running a QRadar Practice).

neshbooy

Section C — Getting Data In

neshboy

Part 4 — DSMs, QID Mapping, and Log Source Onboarding

Why this part exists

[CONCEPT] Detection Engineering Handbook (DEH) Part 27 §3.1 names a failure mode in one paragraph and moves on, because proving one AQL/CRE design point was that part's whole job: "an unmapped or partially-mapped event lands under a generic QID with no custom properties, and the query fails silently." That sentence is doing a lot of unstated work. It assumes a reader already knows what a Device Support Module (DSM) is, how QRadar decides which DSM parses a given Log Source's traffic, what "unmapped" actually looks like operationally, and — most importantly — how a team catches this before it becomes six months of a Rule that has matched zero events and nobody noticed. This part is where that assumption gets paid off in full.

Onboarding a new log source into QRadar is not a single click. It is a protocol decision, a parsing-authority decision (which DSM owns this traffic), a QID-assignment verification step, and a custom-property population check — four distinct failure surfaces, each of which can silently produce "the Log Source is receiving data" while the platform's own event-taxonomy identifier (QID) resolves to nothing useful and every custom property a downstream rule needs comes back empty. This part walks the full lifecycle, then turns DEH Part 27 §3.1's one-paragraph warning into the repeatable checklist a platform engineer runs every time a new source comes online — and hands off, deliberately, to Part 5 (log sources with no native DSM at all) and Part 6 (custom-property extraction and governance in depth) rather than absorbing either topic here. This part also does not re-teach AQL syntax — DEH Part 27 §1 owns that — or re-argue Sigma-first-vs-native authoring, which DEH Part 23 §5 already settles generically; every AQL query below exists only to validate onboarding, not to teach the language.

1. What a DSM actually does, and why nothing downstream works without one

[CONCEPT] A Device Support Module is the parser that turns one vendor or product's raw log format into QRadar's internal event schema. Concretely, a DSM does three things to every raw record it successfully recognizes: it normalizes the record's fields into QRadar's built-in schema (`sourceip`, `destinationip`, `username`, and so on, the same fields DEH Part 27 §1.1's baseline AQL query already selects against), it assigns a QID (QRadar's internal event-taxonomy identifier, distinct from any vendor's own native event ID — DEH Part 27 §1.2's own introduction of the term) drawn from a signature match against the raw payload's structure and content, and — where the DSM has been configured or extended to do so — it extracts custom properties: fields beyond the built-in schema, pulled from the payload by regex or a structured parser and made available in AQL under a quoted display name (`"Granted Access"`, `"Target Process Name"`), exactly as DEH Part 27 §1.2 and §3.2 already show.

None of the platform's correlation machinery cares what the raw log looked like before a DSM touched it. The Custom Rules Engine — the platform's real-time correlation engine, universally abbreviated CRE (DEH Part 27 §2) — evaluates normalized events against Rule Tests keyed on QID, category, and custom-property values. A Building Block condition like `BB:LSASS-Access-Candidate`'s match on `"Target`

`Process Name"` ending in `lsass.exe` (DEH Part 27 §3.3) has nothing to check against if the DSM that parsed the underlying Sysmon Event ID 10 record never extracted that field in the first place. Onboarding is the point in the pipeline where that extraction either happens correctly or doesn't — which makes it upstream of every Rule Engineer decision this book's Section D covers, not a one-time setup task that can be treated as done once a source shows up in the Log Source list.

The distinction that matters for the rest of this part: a **Log Source** is one configured collection point — a specific firewall, a specific domain controller's WinCollect agent, a specific cloud API feed — and a **Log Source Type** is the DSM template QRadar assigns to it, which determines every parsing rule, QID-mapping table, and custom-property extraction pattern applied to that source's traffic from that point forward. Two Log Sources of the same Log Source Type share identical parsing behavior; two Log Sources with the same protocol (both arriving over syslog, say) but different Log Source Types are parsed by entirely different DSMs and can produce completely different QID and custom-property outcomes from visually similar raw text. Getting the Log Source Type assignment right is therefore not a cosmetic labeling choice — it is the decision that determines whether §3 and §4 below succeed or fail.

2. The Log Source object model: protocol, type, and instance

[PLATFORM ENGINEER] Configuring a Log Source means answering two largely independent questions: how does the data physically arrive (the protocol/collection method), and which DSM should parse it once it does (the Log Source Type). QRadar supports several collection paths, and picking the wrong one for a given source is a common early onboarding mistake — a source that could be pulled via a structured API integration but gets configured as a raw syslog feed instead loses whatever structure the API would have preserved, pushing more parsing burden onto a DSM's regex layer than necessary.

Collection Method	Typical Use Case	Push or Pull	Onboarding Note
Syslog (UDP/TCP)	Firewalls, network appliances, most on-prem infrastructure	Push	Highest auto-discovery success rate when the source's syslog format matches a known DSM signature; also the easiest to misassign to the wrong Log Source Type when two vendors' formats look superficially similar.
WinCollect (agent-based)	Windows Event Log, Sysmon channels	Push (agent forwards)	Agent version and the specific Windows Event Log channel configured both affect what reaches the DSM — an agent upgrade can change field layout without changing the Log Source Type assignment (see the Rule Autopsy in §5).
JDBC	Database-resident audit/log tables (many enterprise applications)	Pull (scheduled poll)	Poll interval becomes a real detection-latency factor — a 10-minute JDBC poll means up to 10 minutes before a matching event even reaches the CRE, independent of Rule logic.
LEEF / CEF	Vendor-side structured escape hatch for products emitting IBM's or a common structured log format directly	Push	Skips most DSM regex-parsing burden since the source itself emits a structured, self-describing format; Part 5 covers this as one alternative to a hand-built extension.
REST API / cloud connector	SaaS platforms, cloud provider audit logs	Pull (API poll or webhook)	Field mapping is usually maintained by IBM or the vendor as part of the connector rather than a general-purpose DSM signature — verify which party owns updates to that mapping before assuming it survives a vendor-side API version change unattended.

Table 4.1 — Common QRadar log-source collection methods and their onboarding implications.

Version: general/version-agnostic — the collection-method categories themselves are architecturally stable; the exact protocol list QRadar supports out of the box, and the specific connectors available for a given cloud platform, have grown and changed across release cycles. Verify current protocol/connector availability against your own deployment's Log Source creation wizard before assuming a specific method is present.

Every row in that table answers "how does the data get here." None of them, on their own, answer "does the data get parsed correctly once it arrives" — that is §3 and §4's job, and it is a separate verification step regardless of which collection method a given source uses.

3. Auto-discovery versus manual configuration

[PLATFORM ENGINEER] QRadar can auto-detect certain protocols reaching a known collector and auto-create a Log Source with a guessed Log Source Type, based on matching the incoming traffic's structure against known DSM signatures. This works well for common, high-volume appliance types with a distinctive, stable syslog format — but "auto-detected" is a guess based on pattern matching, not a guarantee that the assigned Log Source Type is the correct one for this specific device's firmware version, configuration, or vendor sub-variant. A platform engineer who accepts an auto-discovered Log Source Type without checking it against the actual device documentation is trusting a heuristic to make a decision that determines every QID and custom property that source will ever produce.

Not every collection path auto-discovers at all. JDBC pulls, most REST/API connectors, and any source requiring credentials or a connection string have to be configured manually from the start — there is no passive traffic pattern for QRadar to match against before a connection is established. Treat auto-discovery as a convenience for a narrow set of common cases, not the default onboarding path to assume for a heterogeneous log-source estate.

PRODUCT VERSION NOTE *The exact auto-discovery workflow — which protocols it covers, how confidently it pre-selects a Log Source Type versus flagging the match as ambiguous for manual review, and the specific wizard steps and screen names involved — has shifted across QRadar releases and is exactly the kind of console-navigation detail this book will not assert as a permanent fact (STYLE-GUIDE.md §11). Verify current auto-discovery behavior and its confidence-reporting UI against your own deployment before building an onboarding runbook that assumes a specific wizard flow.*

4. QID assignment and the generic-QID failure mode

[PLATFORM ENGINEER] A DSM's signature table maps specific raw-payload patterns to specific QIDs — one DSM can and typically does own dozens to hundreds of distinct QIDs, one per event type it recognizes (a firewall DSM might map "connection allowed," "connection denied," and "configuration changed" to three separate QIDs from the same Log Source Type). When an incoming raw record matches none of a Log Source Type's known signatures — because the vendor changed a field order, added a header, shipped a firmware update with new log wording, or because the Log Source Type assigned to this source was simply wrong to begin with — the event still lands in the Ariel `events` table. It has to; QRadar does not discard unparsed data. What it does not get is a specific QID: it lands under a generic, catch-all identifier with no custom properties extracted, because no signature matched well enough to tell the parser which fields exist where in this particular payload.

This is precisely the failure mode DEH Part 27 §3.1 names for one worked example — Sysmon Event ID 10 arriving without a validated DSM path — generalized to every onboarded source: **a Log Source showing green, actively receiving bytes, tells you nothing about whether those bytes are landing under a meaningful QID.** `QIDNAME(qid)` resolving to a generic or unrecognized label, and every custom property a downstream Building Block expects coming back null, is indistinguishable at the ingestion layer from "this event type genuinely never occurs here." Both look identical: a Log Source with a healthy event count and a Rule that has never fired.

PRODUCT VERSION NOTE *The exact display name QRadar uses for an event that matched no specific signature (commonly referred to informally as an "unknown" or generic QID category), the console path for reviewing which events in a given Log Source fell into it, and the exact wording/color coding a Log Source's own status indicator uses to report "actively receiving data" (referred to generically as "green" or "healthy" in this part) are all wording/navigation details that have varied across releases. Confirm the current label, review path, and status-indicator wording in your own Log Activity and Log Source Management interfaces — the architectural point (unmatched events land under a non-specific identifier with no custom properties, and a healthy-receiving-data status is not the same signal as a correct DSM mapping) is stable; the exact strings and colors are not.*

A second, easily overlooked cause of the same symptom is a **parsing-order conflict**: two Log Source Types can plausibly claim the same traffic shape, especially for generic syslog-based devices from different vendors that happen to share a similar wire format. If a source is auto-discovered or manually assigned to a Log Source Type that superficially matches but isn't the actual DSM built for this vendor and product, every event parses "successfully" in the sense of matching some signature — just the wrong DSM's signature, assigning QIDs and (if any) custom properties that describe a different product's event taxonomy entirely. This is worse than the zero-match case in one specific way: a wrong-DSM assignment can produce a plausible-looking, non-generic QID that is nonetheless meaningless for this source, passing a shallow "does it have a specific QID" check while still being wrong.

5. BB:LSASS-Access-Candidate — a DSM mapping that shipped broken

[PLATFORM ENGINEER] The failure mode §4 describes in the abstract has a concrete, dated history in DEH Part 27's own worked example — the onboarding step that has to succeed before BB:LSASS-Access-Candidate can ever match anything.

Rule Autopsy

The rule: A WinCollect-forwarded Sysmon `Microsoft-Windows-Sysmon/Operational` channel, onboarded as the Log Source Type for a community-maintained Sysmon DSM extension, feeding DET-27-01's `BB:LSASS-Access-Candidate` (DEH Part 27 §3.1, §3.3) — the exact scenario DEH Part 27's own Engineering Reality callout warns a reader to validate before writing a single Rule Test against it.

Why it shipped: Onboarding was signed off the day the Log Source showed events flowing and a spot-check AQL search against `events` returned rows with a populated QID for a handful of common event types. That was treated as proof the DSM mapping worked. Nobody ran DEH Part 27 §3.1's specific known-positive test — deliberately generating an Event ID 10 access to `Lsass.exe` and confirming `"Granted Access"` came back populated and in the expected format — before marking the source production-ready.

How it failed: A routine Sysmon agent upgrade six months later changed how the WinCollect-forwarded payload represented the `GrantedAccess` field — from the zero-padded hex string the DSM extension's regex expected to an unpadded variant. The extraction pattern stopped matching for that one field alone; every other field on the same event kept extracting normally, and the Log Source's event count and general health metrics showed nothing unusual. `"Granted Access"` came back empty for every subsequent Event ID 10 record.

`BB:LSASS-Access-Candidate`'s Rule Test checking that field's value against the memory-read access-mask list never matched again — not because the underlying LSASS-access activity stopped happening, but because the field it was built to check had gone silently empty. No error surfaced anywhere in the console; the Rule simply stopped firing, indistinguishable from `"no suspicious access occurred."`

The fix: Onboarding sign-off became conditional on a documented, per-Log-Source-Type known-positive test for every custom property a currently-deployed Building Block or Rule depends on — not just `"the Log Source shows events"` or `"a generic AQL search returns rows."` That test gets re-run, not assumed still valid, after any agent, firmware, or DSM-extension version change touching that source — the same discipline DEH Part 27 §3.1's Engineering Reality callout states for the initial mapping, extended here to cover the mapping's entire operational lifetime rather than treating validation as a one-time gate at onboarding.

This is the sharpest version of the point this part exists to make: the failure DEH Part 27 §3.1 names once, as a caution against skipping validation before writing AQL or a Rule Test, recurs on a longer timescale as a caution against skipping re-validation after any change to the source, the agent, or the DSM extension parsing it. Onboarding is not a milestone reached once; it is a mapping contract that has to survive every subsequent change on either side of it.

6. What onboarding must confirm before a Rule Engineer builds anything

[RULE ENGINEER]

Part 6 owns custom-property extraction, naming, and governance in full depth — the regex-vs-calculated-property choice, Ariel search-performance cost, and the naming discipline that keeps a property referenced in AQL or a Rule Test from silently drifting out of sync with the DSM that populates it. This part's narrower job is to state the handoff condition clearly: a source is not onboarding-complete, in any sense a Rule Engineer should rely on, until every custom property a currently-planned or currently-deployed Building Block needs from it has been confirmed populated, in the expected format, against a real event — not merely present in the DSM's configuration as an extraction rule that has never actually matched live traffic.

Validation Test Setup: *A newly onboarded Log Source, Log Source Type assigned and accepted (auto-discovered or manual), at least one custom property this source is expected to populate for a planned or existing Building Block. **Action:** Generate a known-positive event on the source system that should trigger the specific event type the target custom property depends on — the same discipline DEH Part 27 §3.1's Engineering Reality callout requires for DET-27-01's Sysmon path, generalized to any source: don't wait for organic production traffic to be your first test. **Expected result:** The resulting event in Log Activity resolves to a specific, non-generic QID; the target custom property appears populated with a value in the format the consuming Building Block or AQL query expects (hex vs. decimal, zero-padded vs. not, a full path vs. a bare filename); and an ad hoc AQL search scoped to this Log Source and QID returns the field populated across more than one sample event, not just the first one — a single lucky match doesn't rule out an intermittent extraction pattern.*

Skipping this step doesn't just risk a Building Block that never fires — DEH Part 27 §2.3's Blind Spot already names the sharper consequence for a chained, multi-Rule pattern: if a first-stage Rule depends on a custom property that never actually populates, a Reference Set a second-stage Rule depends on never gets written to, and the second Rule silently never fires either, with the failure's true origin sitting one onboarding step upstream of either Rule's own logic.

7. A repeatable onboarding checklist

[PLATFORM ENGINEER]

The steps below operationalize §4 and §5's failure modes into a sequence a platform engineer can run for every new source, regardless of collection method. None of these steps require AQL syntax beyond what DEH Part 27 §1 already covers — this checklist is a workflow around that syntax, not a reason to relearn it.

Step	Action	Verifies Against
1	Confirm the collection method (Table 4.1) matches what the source can actually deliver, not just what's convenient to configure.	Wrong protocol choice loses structure a native connector would have preserved.
2	If auto-discovered, check the assigned Log Source Type against the device's actual vendor/product/firmware documentation — do not accept a plausible-looking match on trust.	§3's auto-discovery-is-a-heuristic caveat; §4's wrong-DSM parsing-order risk.
3	Generate or identify a known-positive sample event for at least one event type this source is onboarded to support.	§6's Validation Test setup step.
4	Confirm the resulting event resolves to a specific, non-generic QID via <code>QIDNAME(qid)</code> in an ad hoc AQL search (DEH Part 27 §1.2).	§4's generic-QID failure mode.
5	Confirm every custom property a currently planned or deployed Building Block needs from this source is populated, across more than one sample event, in the exact format the consuming query or Rule Test expects.	§5's Rule Autopsy; DEH Part 27 §3.1's Engineering Reality.
6	Estimate this source's sustained event rate and confirm it against the deployment's licensed EPS/FPM budget before going live at full volume (Part 17).	§8's Platform Reality below.
7	Record the Log Source Type, DSM/extension version, validation date, and validating engineer in the onboarding log.	Re-validation trigger for any future agent/DSM/firmware change (§5's fix).
8	Re-run step 3–5's known-positive test after any agent upgrade, DSM content update, or firmware change touching this source — do not assume a passing test stays valid indefinitely.	§5's Rule Autopsy — this is the step that was skipped.

Table 4.2 — Log source onboarding checklist, keyed to the failure mode each step closes. *Version: general/version-agnostic — the checklist steps themselves don't depend on QRadar's exact release; step 4's specific AQL function names and step 2's exact wizard screen are covered by DEH Part 27 §1.2 and this part's own PRODUCT VERSION NOTES respectively.*

flowchart TD

A["New log-generating source identified"] --> B{"Collection method: native protocol\nsupported? (Table 4.1)"}

B -->|"Yes: Syslog / WinCollect / JDBC / LEEF-CEF / API"| C["Configure Log Source:\nprotocol + credentials/collector"]

B -->|"No native DSM path"| D["Route to Part 5:\nUniversal DSM / DSM Editor extension"]

C --> E{"Auto-discovered a\nLog Source Type match?"}

E -->|"Yes"| F["Verify assigned Log Source Type\nagainst vendor/product docs\n(do not trust the guess)"]

E -->|"No / manual required"| G["Manually assign\nincorrect Log Source Type"]

F --> H["Generate known-positive\nsample event"]

G --> H

H --> I{"QIDNAME(qid) resolves to a\nspecific, non-generic QID?"}

I -->|"No"| J["DSM/QID mapping gap –\nDEH Part 27 §3.1 failure mode.\nFix DSM/extension before proceeding."]

I -->|"Yes"| K["Confirm required custom\nproperties populated, correct format,\nacross multiple sample events"]

K --> L{"Properties populated\nand format-correct?"}

L -->|"No"| J

L -->|"Yes"| M["Assess EPS/FPM budget impact\n(Part 17)"]

M --> N["Log onboarding record:\nLog Source Type, DSM version,\nvalidation date, owner"]

N --> O["Source marked production-ready\nfor Building Block / Rule authoring"]

J --> C

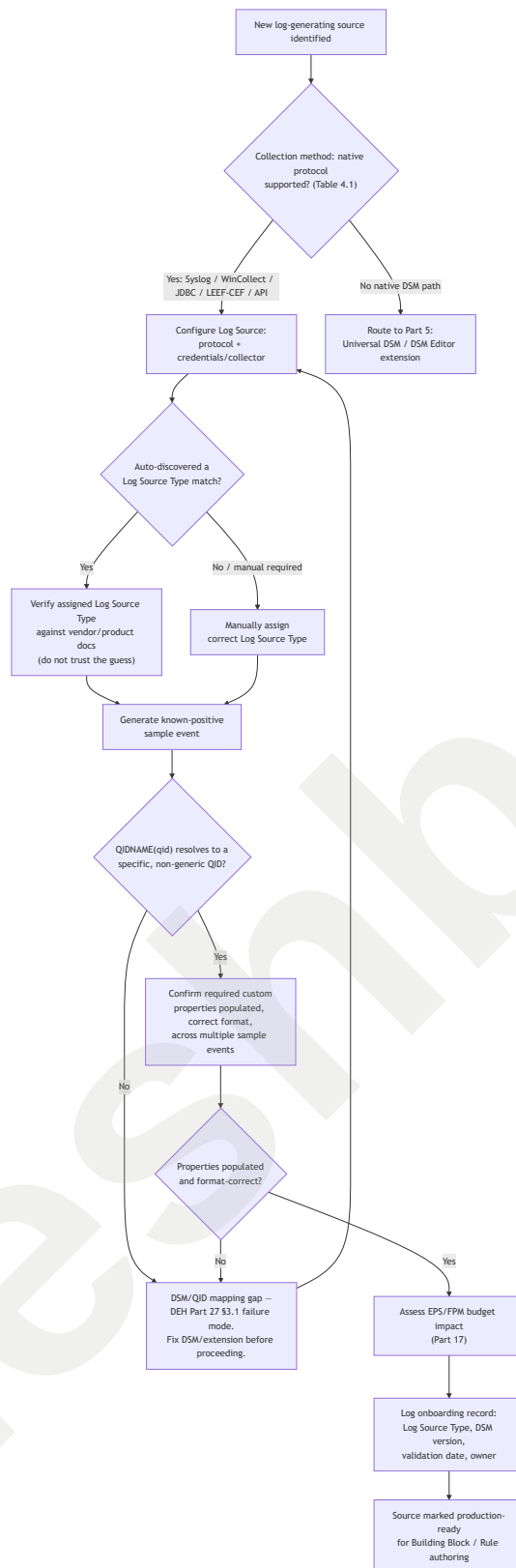


Figure 4.1 — Log source onboarding lifecycle, from source identification to production-ready sign-off. *CONCEPTUAL*. Illustrates this part's own checklist (Table 4.2) as a workflow, including the two loop-back points where a mismatched Log Source Type or an unpopulated custom property routes back to reconfiguration rather than forward to sign-off; not a reproduction of any QRadar console screen or wizard, and not a claim about the exact wizard steps a specific release presents (see §3's

PRODUCT VERSION NOTE). The Mermaid source above is the editable source of truth for this figure per STYLE-GUIDE.md §10.

8. Platform Reality: onboarding doesn't end at sign-off

[PLATFORM ENGINEER] Two forces keep acting on an already-onboarded Log Source after Table 4.2's checklist is complete, and neither is optional to plan for. First, DSM content itself updates — IBM (and, for extensions, whatever team maintains them) periodically ships revised signature tables and field-extraction patterns for existing Log Source Types, the same class of update that Part 19 covers for the App Framework more broadly. A DSM content update can change which QID a given raw pattern resolves to, or add a custom property that didn't exist at onboarding time, without anyone touching this specific Log Source's own configuration. That is generally an improvement, but "generally" is doing real work in that sentence: a signature refinement that narrows an overly broad pattern can, in principle, cause a previously-matching event type to resolve differently after an update most teams apply as routine platform maintenance, not as a change requiring re-validation of every dependent Rule.

Second, and independently of any DSM update, a source's actual sustained event volume rarely matches its estimate from step 6 once real production traffic replaces the sample events used for validation. A newly onboarded source that seemed comfortably within budget during a quiet validation window can push the deployment's total sustained rate over its licensed EPS/FPM tier once it's carrying full production load — Part 17 covers the licensing and capacity-planning mechanics in depth; the operational point here is that onboarding sign-off should include a follow-up check on real sustained volume after the source has been live for a representative period, not just the estimate used to justify going live in the first place.

Platform Reality Documented onboarding treats Table 4.2's sign-off as a completed gate; production treats it as a snapshot that starts decaying immediately. A routine DSM content update can silently change which QID or custom property a Log Source Type produces, and a source's real sustained EPS/FPM under full production load routinely exceeds the estimate from a quiet validation window. Neither is a malfunction — both are ordinary platform behavior a re-validation cadence has to plan for, not a one-time approval.

PRODUCT VERSION NOTE How QRadar surfaces a DSM content update — whether it applies automatically on a schedule, requires a manual review-and-approve step, or is packaged separately from platform/console updates — has varied across release and delivery models (on-prem vs. cloud/SaaS), and this book does not have current visibility into which model your specific deployment uses. Confirm your environment's DSM update cadence and approval workflow directly rather than assuming updates apply silently or require sign-off — either assumption, held wrong, changes how often step 8 of Table 4.2's checklist actually needs to run.

9. Blind Spot: what auto-discovery and a "receiving events" status can't tell you

[PLATFORM ENGINEER] Auto-discovery, by construction, only recognizes traffic shaped like something it already has a signature for. A source using a nonstandard port, an unusual encapsulation (syslog wrapped inside another transport, a proxy that reformats headers before forwarding), or a genuinely proprietary log structure won't be auto-discovered — and won't necessarily fail loudly either. It may register as a Log Source receiving bytes with no confident Log Source Type match, or it may partially match a similar-looking DSM's signature closely enough to get auto-assigned incorrectly, which is the wrong-DSM case §4 already names as worse than an outright non-match. Neither outcome announces itself as an error; both look, from a distance, like "a new source came online and QRadar is handling it."

Blind Spot A "receiving events" status answers one question — is data arriving — and cannot answer a structurally different one: is the assigned Log Source Type the right parser for that data. QRadar tracks these at two separate layers (collection vs. DSM), and no status indicator collapses them into one signal. Only the checklist in §7, run deliberately rather than inferred from a healthy-looking status, closes that gap.

This is the same blind spot as §4's generic-QID failure mode, restated from the discovery side rather than the parsing side: the collection layer and the DSM layer fail independently, and a platform engineer who only checks the collection layer's status has verified nothing about the layer underneath it.

10. What this means downstream, for the analyst and the rule engineer

[ANALYST] When a fired Offense doesn't show up for a pattern an analyst or a Rule Engineer is confident should be firing, the reflex is to suspect the Rule's logic first — a threshold set wrong, a Reference Set membership check inverted, a Building Block condition that's subtly too narrow. This part's whole argument is that the reflex should include one earlier question: has this Log Source's DSM mapping actually been validated for the specific QID and custom properties this Rule depends on, and has it been re-validated since the last agent, firmware, or DSM-content change touching that source? A Rule with flawless logic against a field that's silently empty produces the exact same symptom — nothing — as a Rule with broken logic. Part 18's troubleshooting playbook picks this up as one of its three named failure modes (a DSM/QID mismatch after a vendor agent upgrade) with the full symptom-to-fix decision tree; this part is what makes that playbook's first diagnostic question ("was this source's mapping ever actually validated, and is it still valid") answerable at all, because §7's checklist is what should have produced a documented answer before the question ever needed asking.

Cross-references: DEH Part 27 §1.1–§1.2 (Ariel `events` schema and the `QIDNAME()` / `LOGSOURCETYPENAME()` translation functions this part relies on without re-teaching); DEH Part 27 §3.1 (the DSM/QID mapping prerequisite and generic-QID failure mode this part operationalizes into Table 4.2's checklist); DEH Part 27 §2.3 (the chained-Rule Blind Spot §6 extends to an onboarding-layer root cause); DEH Part 23 §3

(field-mapping completeness as a translation-layer failure mode, the Sigma-pipeline analog of this part's DSM-signature failure mode). Within this book: Part 5 (Universal DSM and DSM Editor extensions for sources with no native DSM); Part 6 (custom-property extraction, naming, and governance in full depth); Part 17 (EPS/FPM licensing and capacity planning); Part 18 (the full troubleshooting playbook for a DSM/QID mismatch and other named failure modes); Part 19 (App Framework and content-update mechanics generally).

neshbooy

Part 5 — The Universal DSM, DSM Editor, and Custom Log Source Extensions

Why this part exists

[CONCEPT] Part 4 walked the onboarding lifecycle a log source follows once QRadar already knows how to parse it: protocol configuration, auto-discovery, and a Device Support Module (DSM) that assigns a QID (QRadar's internal event-taxonomy identifier) and extracts fields from a format IBM (or a community maintainer) has already written a parser for. That part's whole premise — DEH Part 27 §3.1's own framing, that "an unmapped or partially-mapped event lands under a generic QID with no custom properties, and the query fails silently" — assumes a DSM exists to eventually get the mapping right. This part covers what happens before that assumption holds: a log source with no native DSM at all, and no realistic prospect of IBM shipping one on your timeline.

Every SOC running QRadar past its first year accumulates a handful of these: an in-house application with a bespoke log format, a niche security appliance too small a vendor for IBM's DSM catalog to prioritize, a SaaS product whose audit-log export predates any DSM effort, a legacy device nobody has patched in a decade. None of these disappear because QRadar doesn't ship a parser for them — they still need to land in Ariel with a usable QID and usable custom properties, or every rule and every AQL search against that data source in Part 4's sense, and every Building Block DEH Part 27 §2.1 describes, has nothing to match against. This part covers the three escape hatches available when a native DSM isn't coming: build a regex-based extension yourself through the DSM Editor against QRadar's generic Universal DSM path, push the vendor toward emitting a self-describing structured format (LEEF or CEF) QRadar already knows how to parse, or accept the wait for native coverage and operate without full parsing in the meantime. It does not re-teach AQL syntax (DEH Part 27 §1 owns that) or the general onboarding lifecycle Part 4 already covers — it starts exactly where Part 4's assumption runs out.

1. The coverage gap a DSM catalog can't close

[CONCEPT] QRadar's shipped DSM catalog is broad — mainstream operating systems, major firewalls, common identity providers, well-known cloud platforms — but it is not, and structurally cannot be, exhaustive. A DSM is built and maintained against a specific vendor's specific log format, and that format is a moving target the DSM author has to track release-to-release. A log source with a small install base, a format IBM hasn't prioritized, or a home-grown application that was never going to appear in any vendor's catalog in the first place, arrives at your Log Source configuration with nothing to parse it.

Left unaddressed, that source's events still ingest — QRadar doesn't refuse to store data it can't parse — but they land the way DEH Part 27 §3.1 already names for the partially-mapped case, only worse: a generic, low-information QID, no `CATEGORYNAME()` value worth filtering on, and none of the custom properties a Rule Test or an AQL `WHERE` clause needs to do anything useful with the event's actual content. `QIDNAME(qid)` resolves to a generic "unknown log"-style placeholder — the same generic QID DEH Part 27 §3.1 names for the partially-mapped case — rather than a name describing what happened,

and the raw payload sits in the event's payload field, readable by a human scrolling through Log Activity but invisible to every mechanism in this book that expects a parsed field.

Three paths close that gap, each shifting the maintenance burden to a different party:

Table 5.1 — Escape hatches for a log source with no native DSM.

Approach	Where parsing effort lives	Setup cost	Ongoing maintenance risk	Best fit
Universal DSM + DSM Editor regex extension	Your team, in QRadar's console	Moderate — write and test regex against representative sample logs	High — coupled to the exact log line format of the sample used to build it; breaks silently on a source-side format change (\$4)	A source you control, with a stable-enough format and no near-term vendor DSM in sight
Vendor emits LEEF	The log source vendor/application	Low, if the vendor already supports it; otherwise a change request to the vendor	Lower for your team, shifted to trusting the vendor's LEEF implementation stays correct release-to-release	A source whose vendor actively supports LEEF as an IBM-oriented integration target
Vendor emits CEF	The log source vendor/application	Low, if the vendor already supports it (CEF has wide adoption beyond IBM's own ecosystem)	Same shifted-trust profile as LEEF, spread across a wider, less IBM-specific vendor base	A source whose vendor supports CEF for multi-SIEM compatibility rather than QRadar specifically
Wait for / request a native DSM	IBM (or a community DSM maintainer)	None, beyond the RFE/request itself	Timeline outside your control; the source runs under-parsed until it lands	A widely-deployed product likely to get native coverage eventually, where under-parsing in the interim is tolerable

PRODUCT VERSION NOTE Whether IBM's own DSM-request process (an RFE, a support case, or a dedicated request channel) is the fastest path to native coverage, versus a community-maintained DSM extension already existing for your exact source, has shifted across QRadar's release history and IBM's own support-channel structure. Check IBM's current DSM catalog and request process before committing a team to a hand-built extension for a source popular enough that native coverage might already exist or be imminent.

2. The Universal DSM: QRadar's generic parsing path

[PLATFORM ENGINEER] When a Log Source Type has no vendor-specific DSM behind it, QRadar still needs somewhere to route that source's raw events for storage and (optionally) parsing. The generic path for this — commonly referred to as the Universal DSM — accepts events QRadar cannot map to a specific vendor parser and stores them under a generic QID/category rather than rejecting them outright. On its own, a Universal DSM log source is exactly the under-parsed state §1 describes: events

land, but nothing beyond the raw payload and the built-in envelope fields (timestamp, source IP, log source) is queryable as a distinct field.

The Universal DSM becomes useful the moment it's paired with a hand-built parsing definition — a Log Source Extension — that tells QRadar how to read this specific source's format: which QID or category a given line pattern should map to, and which regex capture groups should populate which custom properties. That pairing is built through the DSM Editor.

PRODUCT VERSION NOTE *The exact name QRadar's console uses for the generic, no-native-DSM Log Source Type, and the exact menu path to reach the extension-authoring tool described below, have both been worded and grouped differently across release cycles — this book states the concept (a generic parsing fallback, plus a purpose-built extension-authoring tool) as stable, and defers the literal console labels and click path to your own deployment's Admin console. Confirm both before writing internal documentation that quotes a specific menu sequence.*

DSM Editor — regex-based field and QID extraction

[PLATFORM ENGINEER] The DSM Editor is the authoring surface for a Log Source Extension: an admin tool that takes representative sample events from the source you're onboarding and lets you define, against those samples, the regex patterns that pick out a QID/category and any custom properties QRadar should extract going forward. The core workflow, stable in shape even where the exact wizard steps have shifted release to release:

1. **Capture representative sample events** — not one line, but a set covering every distinct event shape the source actually produces (a success case, a failure case, a warning, whatever variants exist), because a regex written against one sample generalizes only as far as that sample's format actually represents the source's real output.
2. **Define QID/category mapping patterns** — a regex (or a set of them, one per distinct event shape) that QRadar tests against each incoming raw event to decide which QID and category it gets assigned.
3. **Define custom property extraction patterns** — regex capture groups, each named and typed, that pull specific values (a username, an IP, an action code, whatever the source's fields carry) out of the raw payload into queryable custom properties, the same custom-property concept DEH Part 27 §1.2 already introduces for the DSM's own built-in extraction and Part 6 covers in full for governance.
4. **Test against the captured samples before deploying** — confirming every sample event resolves to the intended QID and that every intended custom property populates correctly, not just that the regex compiles without error. A regex that compiles cleanly and matches zero of your samples, or matches but extracts the wrong capture group into the wrong property, fails exactly the way DEH Part 27 §3.1 warns a field-mapping gap fails elsewhere: no error, a query or Rule Test that runs clean and finds nothing or finds the wrong thing.
5. **Deploy and associate the extension** with the Log Source (or Log Source Type, if it should apply to every instance of that source), then re-verify against live traffic, not just the captured samples

— production log volume routinely surfaces a fifth event shape nobody thought to capture in step 1.

The result is a Log Source Extension document — an XML mapping of regex patterns to QID/category assignments and custom-property definitions — that QRadar evaluates against every incoming raw event from the source(s) it's attached to:

CONCEPTUAL SAMPLE – illustrative structure only, not a validated export. Exact element names, attribute schema, and DSM Editor export format depend on your QRadar release; verify against an extension your own console actually exports before treating this as a literal schema reference. Rendered as text, not xml, precisely because it is an annotated illustration rather than a complete, validated artifact – STYLE-GUIDE.md §3.

```
<device-extension xmlns="event-extension-format">
  <pattern id="auth-failure-pattern"
    xmlns="event-extension-format-regex">
    <![CDATA[^(\\S+)\\s+AUTHFAIL\\s+user=(\\S+)\\s+src=(\\S+)]]>
  </pattern>
  <event-match-multiple pattern-id="auth-failure-pattern">
    <values>
      <value index="2" name="identity-source-name">Username</value>
      <value index="3" name="identity-ip-source-address">Source IP</value>
    </values>
    <qid>
      <default-value name="qidname" value="AuthFail-CustomApp"/>
    </qid>
  </event-match-multiple>
</device-extension>
```

This snippet illustrates the underlying idea a DSM Editor session produces — a named regex pattern, applied to a raw event, mapping specific capture groups to specific fields and assigning a QID — without claiming to be a validated export from any specific QRadar version. The disambiguating point worth carrying forward from §1: everything downstream of this extension — every Building Block Rule Test that references "Username" or "Source IP" for this source, every AQL query filtering on them — depends entirely on this regex continuing to match the source's actual output. §4 covers what happens when it stops.

3. LEEF and CEF: pushing the parsing burden to the source

[PLATFORM ENGINEER] A hand-built regex extension is not the only escape hatch, and for a source whose vendor is willing to cooperate, it is often not the best one. QRadar parses two self-describing, structured log formats natively, without requiring a custom regex extension at all, because the format itself carries enough structure for QRadar's built-in parser to extract fields without guessing at delimiters:

- **LEEF (Log Event Extended Format)** — IBM's own key-value log format, purpose-built for QRadar integration. A LEEF-formatted event declares its own event ID/name and a set of `key=value` pairs QRadar's built-in LEEF parser reads directly into QID assignment and custom properties, without a hand-written regex extraction step.

- **CEF (Common Event Format)** — originated at ArcSight, since adopted far more broadly as a de facto multi-SIEM structured-logging format. CEF-formatted events carry a similarly self-describing header-plus-extension-fields structure, and QRadar's built-in CEF parser reads it the same way LEEF is read, without a source-specific regex extension.

The practical effect of either format: if the source device or application can be configured to emit LEEF or CEF instead of its native ad hoc text format, the parsing burden that would otherwise sit in a hand-built DSM Editor extension moves to the vendor's own LEEF/CEF implementation. This is a genuine trade, not a strictly better option — it exchanges one maintenance surface for another, and the party responsible for keeping it correct changes along with the surface.

Table 5.2 — Regex extension vs. LEEF/CEF: who owns the maintenance risk.

Dimension	Hand-built regex extension	LEEF or CEF
Who writes the parsing logic	Your team, via DSM Editor	The source vendor's own LEEF/CEF emitter
What breaks it	Any change to the source's raw log line format that the regex didn't anticipate	A change to the vendor's LEEF/CEF field set or key names between application/firmware versions
Who notices a break first	Your team, if regression-tested (§5); otherwise a Rule that's gone quiet or an analyst pulling an empty search	Same risk, but now also gated behind the vendor's own release notes — a break may not be visible until QRadar-side testing catches it
Who fixes a break	Your team, re-authoring the regex	Your team still has to update the QRadar-side mapping if the vendor added/renamed LEEF/CEF fields, but doesn't have to reverse-engineer a raw format from scratch
Turnaround for a fix	As fast as your team can re-test and redeploy the extension	Bounded by whether the vendor's LEEF/CEF change is backward-compatible; a breaking change on the vendor's side may need a support case, not just a QRadar-side edit

PRODUCT VERSION NOTE Both LEEF and CEF parsing are described here as built into QRadar without a custom extension — a stable architectural fact. The exact protocol configuration options for enabling LEEF/CEF ingestion on a given Log Source Type, and the specific field-name conventions QRadar's built-in parsers expect for either format, have areas of version-specific detail this book does not treat as fixed; verify the current protocol configuration screen and any format-version requirements against your own deployment.

Neither LEEF nor CEF eliminates the coverage gap on its own — a vendor that won't or can't emit either format leaves the regex-extraction path (§2) as the only option — and neither is a substitute for the governance question §5 covers: a LEEF-emitting source can still drift out of sync with QRadar's expectations if the vendor changes its own LEEF field set across a firmware or application release, even though the parsing mechanism itself didn't change on QRadar's side.

4. The maintenance cost of a hand-built extension

[PLATFORM ENGINEER]

A DSM Editor extension is authored against a snapshot: the exact set of sample events captured in step 1 of §2's workflow, produced by one version of one source's software or firmware. That coupling is invisible while the source stays on the version it was captured against, and it is exactly what breaks — usually without any error, warning, or failed deployment — the moment the source changes underneath the extension.

A firmware update that reorders fields, adds a new field between two the regex treats as adjacent, changes a delimiter, or reformats a timestamp does not cause the extension to fail loudly. It causes one of two quieter failures: the pattern stops matching at all, and the event falls back to the generic Universal DSM QID with no custom properties — the exact under-parsed state §1 describes, silently reappearing for a source that used to be fully mapped — or, worse, the pattern still matches syntactically but now captures the wrong token into the wrong named property, so a custom property that used to hold a username now holds a source IP, or an empty string, with no indication anywhere that the value is wrong. Every AQL query and every Rule Test built against that custom property keeps running without error and keeps returning data — just the wrong data, or no matches where there should be some — the identical silent-failure shape DEH Part 27 §3.1 already names for a partially-mapped native DSM, recurring here for a hand-built one with no vendor QA process behind it at all.

Rule Autopsy *The rule:* a DSM Editor extension built two years ago for an in-house billing application's authentication log, extracting "Username" and "Source IP" via regex, feeding `BB:Any-Mapped-AuthEvent` — a Building Block several downstream Rules reference for authentication-failure correlation. **Why it shipped:** the application had no native DSM, the format was stable at the time, and the extension passed its original test pass against the sample events captured that quarter. **How it failed:** a routine application platform upgrade reformatted the log line — the username field moved from the third token to the fifth, and a new session-ID token was inserted where the regex's second capture group expected the source IP. The extension kept matching syntactically. "Username" started silently populating with a session ID instead of an account name, and "Source IP" started capturing whatever token happened to sit where the regex still looked. No deployment failed, no error appeared in any log; the downstream Rules kept firing, just correlating and reporting the wrong identity for every offense this Building Block contributed to for weeks before an analyst noticed the "usernames" on a triage screen looked nothing like real accounts. **The fix:** re-capture a fresh sample set from the upgraded application, rebuild the regex against the new field order, and — the change that should have existed from the start — add this extension to a re-test-on-vendor-upgrade checklist (§5) instead of treating "deployed once, working at the time" as a permanent state.

Noisy Offense Trap *The failure mode above can flood a queue instead of just corrupting one field, depending on where the regex breaks. If the QID/category-mapping pattern stops matching entirely rather than mismatching a capture group, every event from that source falls back to the Universal DSM's generic QID — and a generic QID is exactly broad enough to satisfy LOGSOURCE(DEVICETYPE)-based Rule Tests that several unrelated Building Blocks use to scope themselves loosely ("any event from this Log Source Type," rather than a QID-specific condition). A source that used to produce ten meaningfully distinct, narrowly-matched event types can, the moment its extension stops parsing, start matching every broadly-scoped Rule Test built against its Log Source Type at once — producing a burst of Offenses from Rules that were never designed to fire on unparsed generic events, for a reason that has nothing to do with a real change in attacker behavior. Diagnose a sudden offense-count spike from a hand-extended source by checking whether its events are still resolving to their intended QIDs before assuming the spike reflects anything real; Part 15's tuning-intake workflow applies here directly.*

5. Governance: keeping a hand-built extension honest over time

[SOC MANAGEMENT]

Part 4's onboarding checklist and DEH Part 27 §3.1's own validation instinct — confirm the mapping against a known-positive test event before trusting it — apply to a DSM Editor extension at least as strongly as to a native DSM, and for a reason a native DSM doesn't share: IBM (or a community maintainer) has some incentive and some install base to notice and fix a native DSM's own drift. A one-off, in-house regex extension has exactly the reviewers your team assigns to it, and nobody else.

A workable governance discipline for a hand-built extension, without the git-based pipeline DEH Part 22 describes and this book's Part 16 names as a gap QRadar's console-native workflow doesn't hand you for free, needs at minimum:

- **A retained sample corpus** — the original captured events the extension was built and tested against, kept somewhere durable, not just "whatever was in Log Activity that week." Without it, re-validating after a source-side change means recapturing from scratch with no baseline to diff against.
- **A named maintenance owner** — a specific person or team responsible for this specific extension, the same discipline Part 10 requires for a Reference Set's TTL and membership and Part 9 requires for a BB: -prefixed library entry — not "whoever built it originally," who may no longer be reachable when it breaks.
- **A trigger list for re-validation** — the source-side events that should prompt a re-test, not a calendar-only cadence that misses an out-of-cycle change:

Table 5.3 — Re-validation triggers for a hand-built Log Source Extension.

Trigger	Why it matters	Re-validation action
Source application/firmware/agent upgrade of any kind	The single most common cause of silent regex drift (§4's Rule Autopsy)	Re-capture a fresh sample set post-upgrade; diff against the original corpus before assuming the format is unchanged
Vendor-announced log format change (release notes, changelog)	Sometimes the only advance warning you get before the upgrade actually lands	Pre-stage the regex update against the vendor's documented new format if possible, test on the day of upgrade
A downstream Rule or dashboard using this source's custom properties goes quiet or starts looking abnormal	The first operational symptom of drift is usually here, not in the extension itself	Confirm the custom property is still populating correctly before assuming the Rule logic itself is at fault
Scheduled periodic review (quarterly, or per your team's change-management cadence)	Catches drift with no other trigger — a format change with no vendor announcement and no obviously broken downstream Rule	Re-run the extension against a fresh live sample and confirm QID/property assignment still matches intent

Blind Spot *None of these triggers catch a format change that happens to keep matching the existing regex syntactically while shifting which value lands in which capture group — the mismatched-field failure mode in §4's Rule Autopsy, not the stopped-matching one. A periodic review that only checks "does the extension still assign a QID" without checking "does the value in each custom property still look like what it's supposed to be" (a username field that now contains a session ID, a source-IP field that now contains a port number) will pass a review that should have failed. Validate against expected value shape, not just successful pattern match.*

6. Deciding when to stop maintaining, and what to do instead

[SOC MANAGEMENT] A hand-built extension is a standing engineering liability from the day it deploys, not a one-time cost — every trigger in Table 5.3 is recurring work, scaled by however many niche, unsupported sources your environment has accumulated. That cost is easy to underweight at onboarding time, when the only visible cost is the initial regex-authoring effort, and easy to rediscover expensively later, when a dozen one-off extensions built by people no longer on the team all need simultaneous re-validation after an unrelated infrastructure refresh touched every source at once.

Three questions worth asking before defaulting to "build a DSM Editor extension" for the next unsupported source, rather than after the fact:

1. **Can the vendor emit LEEF or CEF instead?** If yes, §3's trade — moving the parsing-format maintenance burden to the vendor's own release process — is very often the better long-term position, even though it costs a support/feature request up front instead of an afternoon of regex work.

2. **Is this source popular enough that native DSM coverage is plausible on a horizon you can tolerate?** If the answer is genuinely "IBM will likely cover this within a release cycle or two," a temporary under-parsed state (§1) with a documented interim workaround may cost less than building and then decommissioning a full extension.
3. **If a hand-built extension is genuinely the only option, is there a named owner and a re-validation trigger list before it ships, not after the first Rule Autopsy?** Table 5.3's governance discipline is cheap relative to the cost of discovering drift the way §4's Rule Autopsy describes — from a triage screen full of wrong values, weeks after the break happened.

None of these questions have a universally correct answer; they depend on vendor cooperation, your team's regex-maintenance capacity, and how much under-parsing you can tolerate while waiting. What they share is that answering them at onboarding time is governance; discovering the answer only after a silent break is the pattern this part exists to prevent.

flowchart TB

```

A["Log source arrives with\nno native DSM"] --> B["Vendor can emit a\nself-describing format?"]
B -->|"LEEF supported"| C["Configure source to\nemit LEEF"]
B -->|"CEF supported"| D["Configure source to\nemit CEF"]
B -->|"Neither supported"| E["Universal DSM +\nDSM Editor regex extension\n(section 2)"]
C --> F["QRadar's built-in LEEF\nparser assigns QID +\ncustom properties"]
D --> G["QRadar's built-in CEF\nparser assigns QID +\ncustom properties"]
E --> H["Hand-built regex assigns\nQID + custom properties\nfrom raw payload"]
F --> I["Ariel events table,\nfully parsed"]
G --> I
H --> I
H -->|"vendor changes source\nformat post-upgrade,\nno error raised"| J["Regex stops\nmatching,\n\nor mismatches capture\ngroups (section 4)"]
J -->|"unvalidated"| K["Silent drift:\n\nfallback to generic QID,\n\nor wrong field values"]
K --> L["Noisy Offense Trap or\n\nsilently-wrong Rule output"]
I --> M["Custom Rules Engine (CRE)\n\nevaluates Building\n\nBlocks and Rules\n\n(DEH Part 27 section 2)"]

```

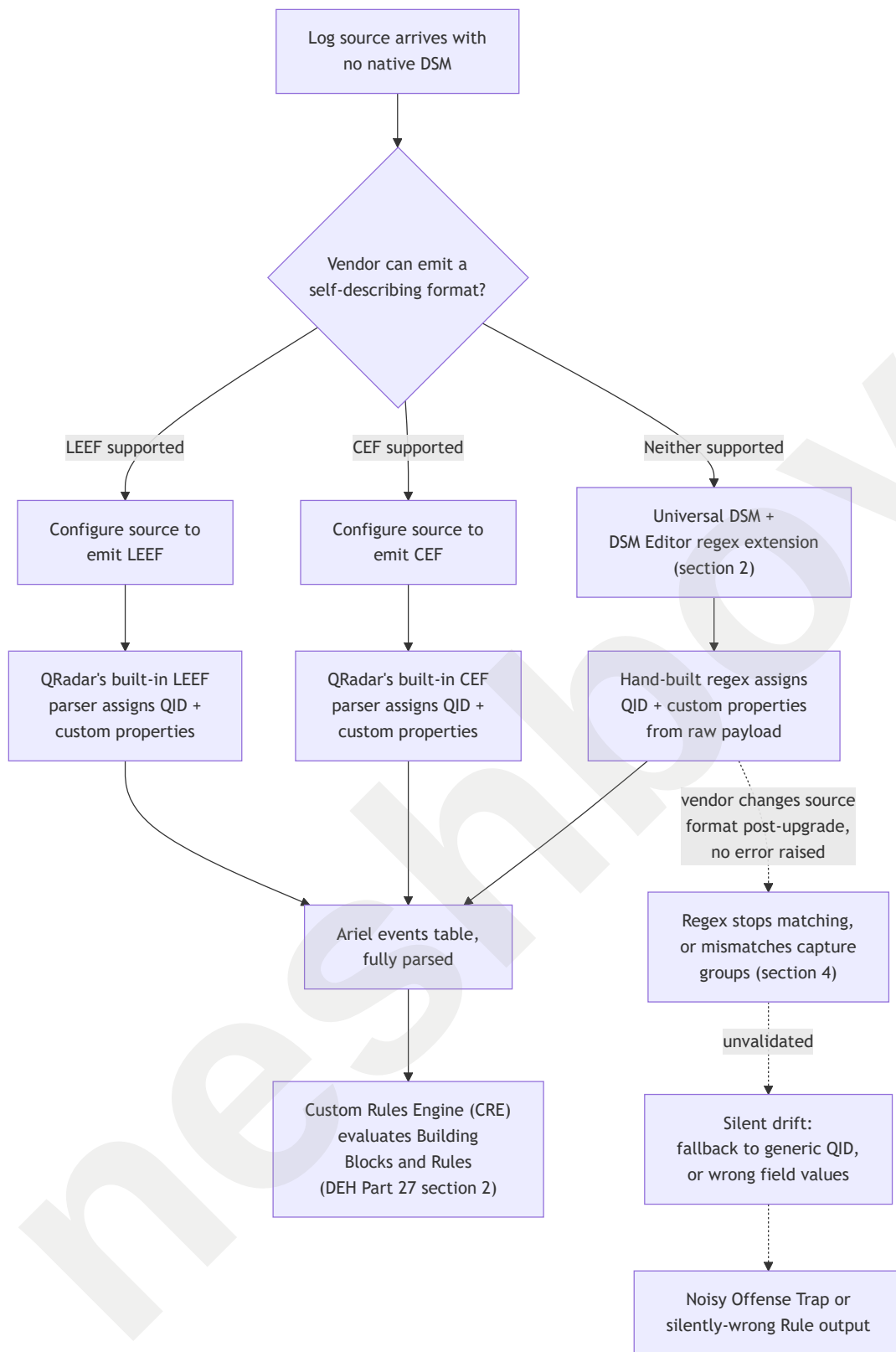


Figure 5.1 — Decision path from an unsupported log source to a parsed Ariel event, and where hand-built extensions drift. *CONCEPTUAL*. Illustrates the three escape hatches this part covers (LEEF, CEF, Universal DSM/DSM Editor regex extension) converging on the same Ariel `events` table destination every other part in this book assumes is already reliably populated, and the specific failure branch — a source-side format change the regex extension doesn't anticipate — that §4 and §5 exist to catch before

it reaches a triage screen or an offense flood, downstream of the Custom Rules Engine (CRE) hand-off DEH Part 27 §2 already covers. Not a reproduction of any IBM architecture diagram and not a claim about internal QRadar processing order beyond what this part states in prose. The Mermaid source above is the editable source of truth for this figure per STYLE-GUIDE.md §10.

Cross-references: DEH Part 27 §3.1 (the unmapped/partially-mapped-event silent-failure pattern this part's regex-drift failure mode inherits); DEH Part 27 §1.2 (QID/DSM function primer — `QIDNAME()`, `CATEGORYNAME()`, `LOGSOURCETYPENAME()` — and the custom-property reference convention this part's extension examples build on); DEH Part 23 §1 (field-mapping and translation-loss framing generalized here to a hand-built extension with no vendor QA process behind it). Within this book: Part 4 (DSM/QID mapping and the onboarding lifecycle this part's scope begins where Part 4's native-DSM assumption ends); Part 6 (custom-property extraction and governance, extended here to properties a hand-built extension defines rather than a native DSM); Part 9 (Building Block naming/governance, referenced for the `BB:` object this part's Noisy Offense Trap and Rule Autopsy examples depend on); Part 10 (reference-data governance discipline, the model for this part's §5 maintenance-owner/TTL analogy); Part 15 (noisy-offense tuning-intake workflow, the diagnostic path for this part's Noisy Offense Trap); Part 16 (rule change management without a detection-as-code pipeline, the same gap this part's §5 names for extension governance specifically); Part 18 (troubleshooting playbook for a log source that stops parsing, the operational counterpart to this part's failure modes).

Part 6 — Custom Properties: Extraction, Calculated Properties, and Governance

Why this part exists

[CONCEPT] Part 4 and Part 5 got a log source parsing cleanly — a Device Support Module (DSM) assigning a QID (QRadar's internal event-taxonomy identifier) and extracting the fields that QID's events carry. DEH Part 27 §1.2 already named the object this part is entirely about, in one sentence: "custom properties — fields a DSM extracts from a log's raw payload via regex or a structured parser, beyond QRadar's built-in schema," referenced in AQL "by their configured display name in double quotes." That sentence is correct and this part does not restate it as new information. What it doesn't cover — because a query-language comparison chapter had no room to — is everything that happens *around* that one fact: that a custom property can come from two structurally different places, not just a DSM; that a second class of custom property, the calculated property, derives a value instead of extracting one; that a badly scoped extraction regex has a real, measurable cost against Ariel search performance (Part 12's subject, flagged here rather than re-taught); and that the display name AQL and every Rule Test depend on is a bare string with no built-in reference-tracking, which makes an unmanaged rename or deletion one of the quietest ways a previously-working detection stops matching anything.

This part also owes a debt to DEH Part 27 §3.1's own worked failure: DET-27-01's "Granted Access" custom property, populated correctly by the DSM but formatted as hex in one deployment and decimal in another, silently defeating a literal-value Rule Test that expected one specific string shape. That is not a DSM-mapping problem — the field is populated, the mapping worked — it is exactly the custom-property design problem this part exists to solve: what a custom property is, where its value comes from, and what happens the moment two different consumers (a query, a Rule Test, a dashboard) expect that value in two different shapes.

1. What a custom property is, and where it comes from

[CONCEPT] QRadar's built-in event schema — `sourceip`, `destinationip`, `username`, `qid`, `category`, and a handful of others — exists identically for every event regardless of source, because it is the platform's own normalized column set. A custom property is anything beyond that set: a field a specific log source's payload happens to carry that QRadar's core schema has no column for, made queryable in AQL and usable inside a Rule Test only because someone configured an extraction (or a derivation) for it. "Source Process Name", "Target Process Name", and "Granted Access" from DEH Part 27's worked example are all custom event properties in exactly this sense — none of them exist for an event whose DSM never defined them, and every one of them is referenced by exactly the quoted-string display name its definition gave it, not by any internal column name a reader could guess.

Two mechanisms populate a custom event property, and they live at different layers of the pipeline:

- **DSM-native extraction** — a mapping or an extension (DSM Editor's regex-based approach, Part 5's subject) that is part of how a specific Log Source Type's payload gets parsed in the first place.

The property exists only for events that DSM parses, and only if that DSM's mapping was written to extract it. This is the mechanism DEH Part 27 §3.1 assumes when it walks through Sysmon Event ID 10's field extraction.

- **Admin-level custom property definition** — a regex applied against an event's raw payload (or an already-parsed field) independently of which DSM produced it, configured centrally rather than inside a specific DSM extension. This mechanism can be scoped to one Log Source Type, one QID, one category, or left unscoped to run against every event QRadar ingests.

Both mechanisms produce the same kind of object from AQL's point of view — a value retrievable by quoted display name — but they differ in where the extraction logic lives, who owns changing it, and what it costs to run at scale (§5). Conflating the two is a common governance failure in its own right: a property that looks DSM-owned in a Rule Wizard's picker may actually be an admin-level definition nobody on the DSM-maintenance side knows exists, and vice versa.

PRODUCT VERSION NOTE *Where exactly the admin-level custom-property editor lives in the console, and the exact wording of the picker that lets you scope a new property to a specific Log Source Type, QID, or category versus leaving it unscoped, has shifted across QRadar releases along with the rest of the Admin tab's own reorganization. Confirm the current path and picker wording in your own deployment before treating any specific click-sequence as durable — the existence of scoped-vs-unscoped custom property definition is the stable claim here, not its exact console location.*

1.1 Table 6.1 — Three mechanisms that put a value behind a custom property's display name

The table below supports one decision: which mechanism to reach for when a new field needs to be queryable in AQL or matchable in a Rule Test. Every row produces a value AQL and the Rule Wizard consume identically once defined; they differ in ownership, scope, and cost.

Mechanism	Where the logic lives	Runs against	Best fit	Version note
DSM / DSM Editor extension property	Inside the Log Source Type's own parsing definition (Part 5)	Only events that specific DSM parses	A field every event of this source type should reliably carry, maintained alongside the rest of that source's mapping	Extension syntax and editor UI have changed across releases — Part 5 owns this detail
Admin-level custom event/flow property (regex)	Central custom-property definition, independent of any one DSM	Every event or flow matching the property's configured scope (one source type, one QID, one category, or unscoped)	A field one team needs from a source whose DSM you don't own or don't want to modify, or a field that spans multiple source types	Editor location and scoping-picker wording — see PRODUCT VERSION NOTE above
Calculated property	A derivation formula referencing one or more existing properties (§2)	Evaluated wherever its input properties are already populated	Normalizing a format inconsistency (hex vs. decimal), unit conversion, string concatenation for a composite key	Available function set for derivations has grown across releases — verify current coverage before assuming a specific transform exists

2. Calculated properties: deriving a value instead of extracting one

[RULE ENGINEER] A calculated property does not read anything new out of a raw payload — it computes a new value from one or more properties (built-in or custom) that are already populated, using QRadar's own function set for string, numeric, or date/time transforms. The distinction matters because the two failure modes are different: an extraction property fails (returns null) when its regex doesn't match the payload it's scoped to; a calculated property fails when one of its *inputs* is null, or when the transform's assumption about the input's format is wrong — which is exactly DEH Part 27 §3.1's "Granted Access" problem restated as something a calculated property can actually fix rather than merely flag.

Consider that problem directly. `BB:LSASS-Access-Candidate` (DEH Part 27 §3.3) matches on "Granted Access" against a literal value set (`0x1010`, `0x1410`). If one Log Source Type's DSM emits that value as hex and another emits it as unpadded decimal, no single literal-match Rule Test condition catches both without listing every format variant by hand — fragile, and a maintenance burden every time a new source joins. A calculated property `"Granted Access (Decimal)"`, defined once as a numeric-base conversion of the raw `"Granted Access"` custom property, gives every downstream consumer — the Building Block's Rule Test, an AQL search, a dashboard widget — one normalized value to match against regardless of which DSM produced the raw field. The normalization work happens once, in one place, instead of being re-solved (or missed) in every consumer separately.

```
-- QRadar AQL, not standard SQL – same disambiguation DEH Part 27 §1.1 states on first
-- appearance: the quoted "Display Name" property references and the LAST n HOURS
-- shorthand below are AQL-specific, not general-purpose SQL syntax, and there is no
-- general-purpose JOIN available. Shown here only to illustrate referencing a calculated
-- property by its configured display name exactly like any other custom property, not
-- to re-teach SELECT/FROM/WHERE syntax, which DEH Part 27 §1 owns. Property names are
-- illustrative, consistent with DEH Part 27's own DET-27-01 worked example (§3 of that
-- part) – this query does not restate that analytic's allowlist boundary, only its
-- custom-property naming.
SELECT sourceip, "Source Process Name", "Granted Access", "Granted Access (Decimal)"
FROM events
WHERE "Target Process Name" ILIKE '%lsass.exe%'
LAST 24 HOURS
```

This query's point is narrow and deliberately so: it shows a raw extracted property and its calculated derivative side by side for validation, before anyone rewrites `BB:LSASS-Access-Candidate`'s Rule Test to match against the normalized column instead of the two-literal hex list DEH Part 27 §3.3 originally shipped. Rewriting that Rule Test is itself a Tuning change in the DEH `TERMINOLOGY.md` sense — a deliberate, logged change to the detection's own logic, not a Suppression — and belongs in this book's Part 16 change-management discipline, not applied silently.

Rule Autopsy The property: A calculated property `"Bytes Transferred (MB)"`, dividing a flow's raw byte count by 1,048,576 for a threshold Rule Test that was tired of comparing against an eight-digit raw byte value. **Why it shipped:** An analyst asked for a Rule Test threshold expressed in megabytes instead of bytes, and a calculated property looked like the clean way to give the Rule Wizard a human-scaled number to compare against. **How it failed:** The underlying raw flow property it divided was itself a custom property scoped to one flow source; a second flow source added months later populated a differently-named raw byte field, and the calculated property's input was null for every flow from the new source. The Rule Test using the calculated property never fired against the new source's traffic — not because the threshold was wrong, but because its input was silently empty, and nothing in the Rule Wizard flags a calculated property whose input has gone null for an entire source. **The fix:** Calculated properties get the same "verify with a known-positive event" discipline DEH Part 27 §3.1's Engineering Reality box already requires for a raw extraction — run a real flow from every source the Rule Test is meant to cover through the calculated property and confirm a non-null result, not just from the source it was originally built against.

3. Event properties vs. flow properties: two separate catalogs

[PLATFORM ENGINEER] Ariel's `events` and `flows` tables (DEH Part 27 §1.1) are separate databases with separate schemas, and custom properties inherit that split completely: a custom *event* property is defined against event payloads and is not visible, referenceable, or extractable when querying `flows`, and a custom *flow* property is the mirror case. There is no shared custom-property namespace that lets a property defined on one side show up automatically on the other, even if the underlying raw data

superficially looks similar (an IP address field extracted from an event payload and a flow's native `sourceip` column are not the same object just because they'd print the same value).

Dimension	Custom event property	Custom flow property
Underlying table	<code>events</code>	<code>flows</code>
Populated from	Log payload parsed by a DSM, or an admin-level regex against that payload	Flow record fields QRadar's own flow collectors or a third-party flow source populate
Typical fields it derives from	Fields a specific log format carries (process names, access-rights values, application-specific identifiers)	Byte/packet counts, application classification, port/protocol combinations
Usable in a Rule Test on an Event rule	Yes	No — a Flow-type Rule Test scope is required
Usable in a Rule Test on a Flow rule	No	Yes

This split is exactly why Part 7's flow-vs-event design decision ("does this analytic belong on `events`, `flows`, or both") is a decision made once, early, rather than something a custom property definition can paper over later — defining the same conceptual field twice, once per table, is the only way to make it available on both sides, and the two definitions then have to be kept in sync by hand.

4. The Ariel search-performance cost of a custom property

[PLATFORM ENGINEER] A custom property is not free enrichment sitting quietly in the background. Every event or flow that falls inside an extraction property's configured scope has that property's regex evaluated against it — at minimum once, at parse/index time, and again at search time for any query using that property unless the property has been marked for indexing (commonly described as "optimizing" the property so it gets its own index rather than requiring a runtime regex scan across every row a search touches). This is the same Ariel indexing-and-quick-filter discipline Part 12 covers in full; this part's job is narrower — naming the specific way a *badly scoped* custom property becomes a search-performance problem rather than a background cost nobody notices:

- **Unscoped regex properties.** A custom property left unscoped ("any Log Source Type," "any QID") runs its regex against every event this deployment ingests, whether or not that event's format could ever match. At sustained EPS that a real production Event Processor is licensed for (DEH Part 27 §1.3's Engineering Reality box on EPS/FPM licensing), that is real, continuous CPU cost competing with the Custom Rules Engine's (CRE's) own per-event Rule Test evaluation and every other admin's unscoped property on the same processor — not a one-time definition cost.
- **Expensive regex patterns.** A greedy, unanchored, or heavily backtracking pattern (`. *something. *` chained repeatedly, alternation-heavy groups) costs measurably more per event than a tightly anchored pattern matching the same field, and that cost is paid once per matching event at parse time and again at search time for every unindexed search that touches it.
- **Un-indexed properties used as a search or `GROUP BY` key.** A custom property never marked for indexing, used routinely as a `WHERE` filter or a `GROUP BY` key over a wide time window, forces a full scan-and-regex-evaluate pass across every row the search's time window and any other filters

leave in scope — the same class of cost Part 12 names generically for any unindexed search field, concentrated here on a property whose whole reason for existing was to make something more queryable, not less.

Platform Reality *"Just add a custom property to make this field searchable" is treated, informally, as a free operation by a lot of admins who've never had to diagnose a slow Ariel search — it isn't. A regex property scoped to one Log Source Type and marked for indexing costs a small, bounded, one-time-per-event overhead that Ariel's own indexing amortizes well. The same property left unscoped, unindexed, and written with a loose pattern, added on top of dozens of other admins' similarly loose unscoped properties over a few years, is a large and cumulative reason a shared Event Processor's search performance degrades in ways no single admin's own query looks slow enough to explain in isolation — because the cost is spread across every property evaluated against every event, not concentrated in any one visible query.*

Blind Spot *QRadar gives you no built-in "find every Rule, Building Block, saved search, and dashboard widget that references this custom property" report. Scoping or re-indexing a property that turns out to be more expensive than expected means finding its consumers by manual review or institutional memory, not by a dependency query the console can answer for you — which is exactly why §6's governance discipline treats an inventory of who-references-what as something the team has to build and maintain itself, outside the console, rather than something to expect QRadar to hand back on demand.*

5. Naming and governance: the display name is the only handle you have

[RULE ENGINEER] Every consumer of a custom property — an AQL query's `SELECT / WHERE` clause, a Rule Test's property picker, a dashboard widget, a saved or scheduled search — references it by exactly one thing: the quoted display name configured when the property was defined. That name is a plain string, case- and whitespace-sensitive, with no structural link back to any of its consumers. Rename `"Source Process Name"` to `"Source Process"` while cleaning up a naming convention, or delete and recreate it with a trailing-space typo nobody notices in the editor, and every one of those consumers keeps running exactly the way DEH Part 27 §3.1 already describes for a missing DSM mapping: no error, no broken-reference warning, a query or Rule Test that executes cleanly and simply never matches anything again. A Rule silently stops firing; a dashboard widget silently starts showing zero; and because nothing in the console surfaces the break, the gap is usually found only when someone notices an Offense that should have fired didn't, days or weeks after the rename.

This is a materially different failure surface than DEH's own `Detection Drift / Schema Drift` concepts describe for a source's own format changing underneath a detection — here the *source data* hasn't drifted at all; the *label* administrators gave to a value has moved, inside a platform with no reference-tracking to catch it. Three practices close most of this gap in a console-native environment with no compiler or CI to catch a broken reference for you:

1. **Treat a live custom property's display name as an interface, not an implementation detail.** Once a `BB:` or `R:`-prefixed Rule references it, or a scheduled report depends on it, renaming it in place is a breaking change to every one of those consumers simultaneously — require the same change-ticket discipline Part 16 asks of a Rule or Building Block change, not a quick admin-panel edit.
2. **Maintain a property-to-consumer inventory outside the console**, because the console won't produce one for you (the Blind Spot above). At minimum: property display name, defining mechanism (DSM extension, admin regex, calculated), owning team, and every known Rule/Building Block/dashboard/report that references it. This is the same review-cost discipline DEH Part 27's own closing SOC Management paragraph names for the three-object Rule/Building Block/Reference Set pattern, extended one layer further to the custom properties those objects actually match against.
3. **Deprecate, don't atomically rename.** Add the new property alongside the old one, migrate consumers deliberately and verifiably (re-running each dependent Rule's Detection Test-equivalent — this book's Validation Test — against a known-positive event after the change), and only retire the old name once the inventory in step 2 shows zero remaining references. An atomic rename-in-place is the fast path to exactly the silent, undiscovered break this section exists to prevent.

SOC Management View *A property-to-consumer inventory is unglamorous, invisible work with no dashboard of its own to show progress on — which is exactly the kind of maintenance task that gets skipped under staffing pressure until a rename breaks something in production. Budgeting a QRadar rule-engineering practice (Part 22's subject) should treat this inventory the same way DEH's own Detection Debt concept treats unreviewed exceptions: a backlog that costs nothing to defer today and a real, undocumented outage risk to defer indefinitely.*

6. Choosing the right mechanism: a decision guide

[RULE ENGINEER] Bringing §1 through §5 together into one practical question — "I need a new field queryable in AQL or matchable in a Rule Test; which mechanism do I reach for?" — the answer depends on where the value already lives and who owns the source it comes from:

- **The value is already present in the raw payload of a source type you (or your DSM-maintenance team) own the mapping for.** Extend the DSM or its extension (Part 5). This keeps the field's definition alongside the rest of that source's mapping, where anyone maintaining the source will actually look for it.
- **The value is present in a payload from a source type whose DSM you don't own, don't want to modify, or need scoped narrower than the whole source type (a single QID or category within it).** Define an admin-level custom property, scoped as narrowly as the need actually requires — not left unscoped by default, per §4's performance argument.
- **The value doesn't exist anywhere in raw form — it's a transform of one or more properties that already do.** Define a calculated property, and validate it against a known-positive event

from every source the transform needs to cover (the Rule Autopsy in §2 is the cautionary case for skipping this).

- **The value is needed on both events and flows.** Define it twice, once per table (§3) — there is no shared definition, and the two copies need to be tracked as two separate inventory entries in the governance discipline §5 asks for, not one.

None of these choices is reversible without cost once a **BB:**, **R:**, dashboard, or scheduled report depends on the result — which is the entire reason this part treats naming and scope as first-class design decisions rather than incidental configuration.

flowchart TB

```
A["Raw log payload\narrives at Log Source"] --> B{"Where does the\nextraction logic live?"}
B -->|"DSM / DSM Editor\nextension (Part 5)"| C["Property extracted at\nparse time, bound to\nthis Log Source Type"]
B -->|"Admin-level custom\nevent/flow property\n(regex, Section 1)"| D["Property\nextracted\nindependent of the DSM,\nscoped or unscoped\n(Section 4 cost)"]
C --> E["Base custom property\ne.g. Granted Access"]
D --> E
E --> F["Calculated property\ne.g. Granted Access\n(Decimal) – Section 2"]
E --> G["Referenced directly\nby display name"]
F --> H["Consumed by AQL search,\nRule Test, dashboard\nwidget, saved report"]
G --> H
H -->|"admin renames,\nretypes, or deletes\nthe property (Section 5)"| I["Every consumer\nabove\nbreaks silently –\nruns clean, zero rows,\nno error anywhere"]
```

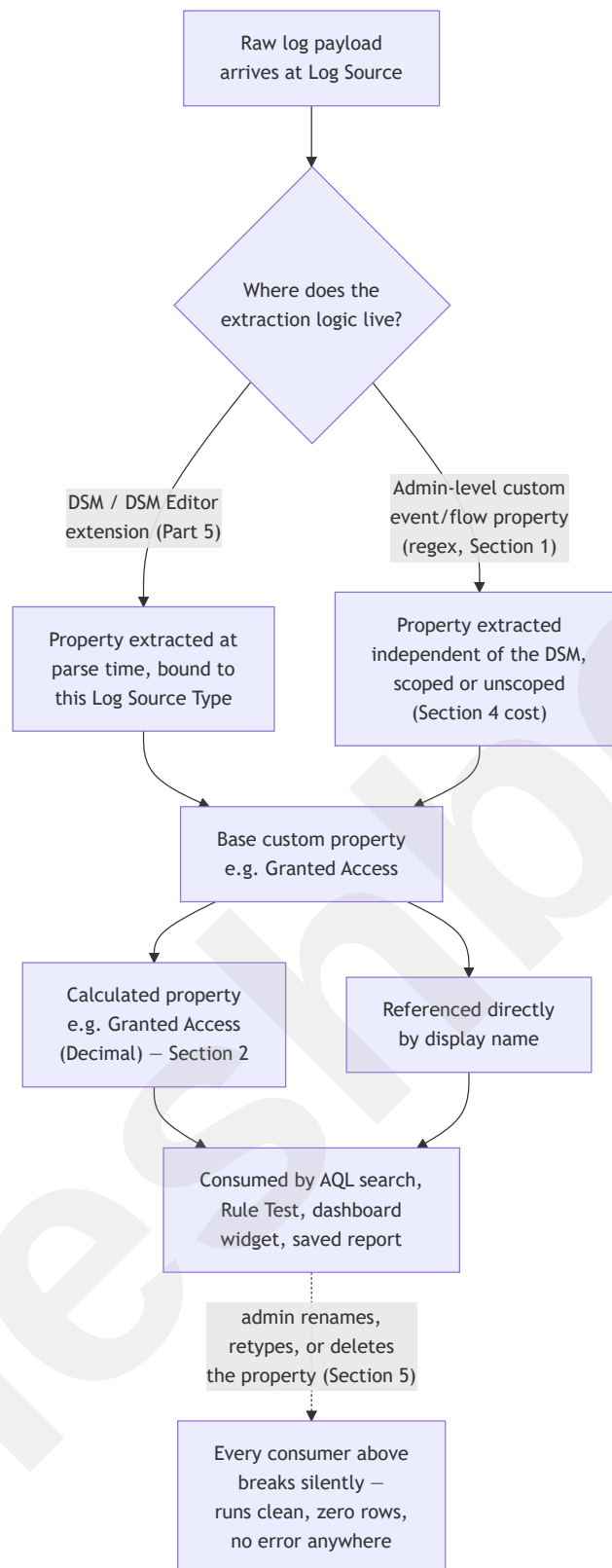


Figure 6.1 — A custom property's path from raw payload to the single point of failure every consumer shares. *CONCEPTUAL*. Illustrates the two structurally different extraction paths (§1) feeding one base custom property, the optional calculated-property derivation layer (§2), and the display name every AQL search, Rule Test, dashboard widget, and saved report references downstream (§5) — the single point every one of those consumers shares once the property is defined, and the reason a

rename or deletion breaks all of them silently, with no error anywhere. Not a reproduction of any QRadar console screen; this book's own structural summary of the naming/governance argument in §5.

Closing

A custom property is a small object with an outsized failure radius: it looks like configuration, but by the time a Building Block, three Rules, a dashboard, and a scheduled report all reference it by name, it functions like a shared interface with no compiler to protect it. The discipline this part asks for — scope extraction properties deliberately (§1, §4), validate a calculated property against every source it needs to cover (§2), keep the event/flow catalogs straight (§3), and track who references what before anyone renames anything (§5) — costs real, unglamorous maintenance time. The alternative, per DEH Part 27 §3.1's own framing carried forward here, is a query or a Rule that runs clean, returns nothing, and tells no one why.

Cross-references: DEH Part 27 §1.2 (custom property syntax in AQL) and §3.1 (DSM/QID mapping prerequisite and the "Granted Access" format-drift example this part's §2 builds on directly); DEH Part 23 §5 (Sigma-first-vs-native tradeoff, not re-argued here); this book's Part 4 (DSM/QID mapping and log source onboarding), Part 5 (Universal DSM, DSM Editor, and custom log source extensions), Part 7 (flow-vs-event design decision), Part 8 (Rule Wizard catalog and Rule Test property references), Part 12 (Ariel search performance and index management), Part 16 (rule change management), and Part 22 (staffing and review-cost budgeting).

Part 7 — Flow Data Architecture: QFlow, Flow Sources, and Flow-vs-Event Design

Why this part exists

[CONCEPT] DEH Part 27 §1.1 introduces the fact that AQL queries two separate Ariel databases — `events` and `flows` — states the mechanics of `events` at length as the basis for that part's own LSASS-access worked example, and disposes of `flows` in a single sentence: network-flow records generated by QRadar's own flow collectors or ingested from a third-party flow source. DEH Part 27 never had to make the events-vs-flows decision, because its worked detection is an events-only analytic from the start — a process-access event, not a network conversation. Parts 4 through 6 of this book closed out the `events` side of the platform: how a DSM (Device Support Module) assigns a QID (QRadar's internal event-taxonomy identifier — DEH Part 27 §1.2's own introduction) and extracts custom properties (Part 4), what happens when no native DSM exists (Part 5), and how a governed custom property survives its own drift (Part 6). This part is `events`' silent counterpart and the last part in Section C — the flow pipeline that runs the whole time next to the event pipeline, watching traffic no Log Source was ever configured to log in the first place.

This part does not re-teach AQL syntax; `SELECT / FROM / WHERE / GROUP BY / LAST n HOURS` work identically against `flows` as they do against `events`, and DEH Part 27 §1 remains the canonical syntax reference for both. It does not re-argue Sigma-first-vs-native authoring — DEH Part 23 §5 owns that generically and DEH Part 27 §2–§4 owns it for QRadar's Rule/Building Block/Reference Set decomposition specifically. What this part does is make the one decision DEH Part 27 §1.1 explicitly left open: whether a given analytic belongs on `events`, on `flows`, or on both — and what QRadar's flow-collection architecture actually does, and fails to do, before that data ever reaches Ariel for a Rule Test or an AQL search to touch it. Section D picks up from here: Part 8's Rule Wizard catalog includes Flow and Common rule types built on exactly the architecture this part describes, and Part 17 picks up the licensing consequence this part only introduces in §7.

1. What a flow is, and why QRadar keeps it separate from an event

[CONCEPT] An event, in QRadar's own vocabulary, is one row per normalized log record — DEH Part 27 §1.1 already fixes this definition and this book inherits it unchanged. A flow is a different unit entirely: one row summarizing a network conversation — source and destination IP and port, protocol, byte and packet counts in each direction, start and end time, and (where the collecting sensor has the visibility to determine it) an application classification such as `HTTP`, `DNS`, or `SMB` — rather than a single log line a device chose to write. Nothing has to log anything for a flow record to exist. A flow is derived directly from the traffic itself, at whatever point in the network a flow source is actually watching.

That last clause is the reason this book gives flow architecture its own part rather than folding it into a paragraph of Part 2's deployment-topology part. An event depends on a Log Source that was deliberately onboarded (Part 4) to a device that was deliberately configured to generate the log in question. A flow depends on nothing configured on the endpoint at all — it depends entirely on where

in the network a sensor sits and what traffic actually crosses that vantage point. Two hosts on the same internal VLAN, transferring a large file over SMB with no security agent installed on either one and no firewall or proxy in the path to log the connection, produce zero rows in `events` for that transfer. They can still produce a `flows` row, if — and only if — some flow source is actually positioned to see that specific segment of the network. Flow data is QRadar's answer to exactly the blind spot a log-source-only deployment has by construction: traffic nobody chose to log.

The two tables' properties differ across nearly every dimension a rule engineer cares about when deciding where an analytic belongs (§5 turns this into a procedure):

Property	events	flows
Row unit	One normalized log record	One summarized network conversation
Source of the data	A configured Log Source + DSM	A flow source — internal QFlow processing or an external flow feed (§2)
Requires endpoint/device configuration	Yes — a Log Source has to be onboarded and generating the log	No — depends only on network vantage point
Typical fields	<code>sourceip</code> , <code>username</code> , <code>QIDNAME(qid)</code> , custom properties (Part 6)	<code>sourceip / sourceport</code> , <code>destinationip / destinationport</code> , protocol, bytes/packets each direction, application ID
License meter	Events Per Second (EPS)	Flows Per Minute (FPM)
Sees traffic with no log source at all	No	Yes, within sensor coverage
Sees application-layer field detail (a username, a process name)	Yes, if the DSM extracts it	No — a flow has no concept of a custom property extracted from a payload field the way a DSM does

This table's last row is the design tradeoff §5 turns into a decision procedure: a flow tells you *that* two hosts talked, in what volume, for how long, and (often) over what application — never *who* was logged on when it happened, unless a separate `events`-side fact gets tied to it through the same Reference Set mechanism DEH Part 27 §2.2 already introduced for a different purpose.

2. Flow collection architecture: QFlow processing and flow sources

[PLATFORM ENGINEER] A flow record does not appear in the Ariel `flows` database by magic any more than an event appears without a DSM assigning it a QID. Something has to watch traffic and produce the summarized conversation record in the first place, and QRadar draws that data from two structurally different kinds of flow source.

2.1 Internal QFlow processing vs. external flow-record ingestion

[PLATFORM ENGINEER] The first kind is internal QFlow processing: a QRadar flow-collection component watches raw packets — via a SPAN/mirror port, a network TAP, or an inline deployment — and builds flow records itself, directly from the packets it sees. Because this path has the actual packet payload available (at least long enough to inspect it before discarding the payload and keeping only the derived

flow record — QRadar's flow model is not a full packet-capture retention product), it can perform deep-packet-inspection-based application classification: identifying that a given conversation on a nonstandard port is actually HTTP traffic, or that a connection using the standard SMB port is doing something a purely port-based classifier would misname. This is also the path that produces the richest flow content, because it's derived from the traffic itself rather than from a third party's own summary of it.

The second kind is external flow-record ingestion: routers, switches, and other network devices that already export their own summarized traffic records — NetFlow (v5/v9), IPFIX, sFlow, or J-Flow, depending on vendor — send those records to QRadar, which parses and stores them without ever seeing the underlying packets itself. Unlike QRadar's own internal processing, these are standardized or vendor-published export formats independent of any one SIEM: NetFlow v9 is Cisco's own template-based export format (Cisco Systems, "Cisco Systems NetFlow Services Export Version 9," RFC 3954, IETF, 2004: <https://www.rfc-editor.org/rfc/rfc3954>), IPFIX is the IETF standards-track protocol NetFlow v9 fed into (IETF, "Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information," RFC 7011, 2013: <https://www.rfc-editor.org/rfc/rfc7011>), and sFlow is InMon's sampling-based alternative (InMon Corporation, "InMon Corporation's sFlow: A Method for Monitoring Traffic in Switched and Routed Networks," RFC 3176, IETF, 2001: <https://www.rfc-editor.org/rfc/rfc3176>). This path is cheaper to deploy (most enterprise-grade network gear already supports one of these export formats; no dedicated collection point needs building), but it inherits whatever classification depth and field detail the exporting device chose to provide — a NetFlow v5 export, for instance, carries less field detail than IPFIX, and QRadar cannot recover detail the exporting device never captured in the first place.

PRODUCT VERSION NOTE *The exact product name, packaging, and appliance-vs-software-vs-virtual-form-factor lineup for QRadar's internal packet-based flow collection (branded, at various points, as the QFlow Collector, with hardware appliance models sitting alongside a related-but-distinct deep-packet-inspection product line) has shifted across release and business-unit cycles, consistent with Part 2's own first PRODUCT VERSION NOTE on delivery-model changes. Verify current product names, supported deployment form factors, and whether a given capability (deep application classification vs. basic flow summarization) ships in the base platform or a separately licensed add-on against your own vendor documentation before committing a design to a specific product name.*

Both kinds land in the same Ariel `flows` table once collected, and — this is the point worth stating plainly rather than leaving implicit — a Rule Test or an AQL `FROM flows` query has no way to tell, from the row alone, which collection path produced a given flow record. A Building Block written against flow fields that assumes deep application classification is available will silently degrade to whatever coarser classification an external NetFlow exporter actually provided, for any flow that came in through that path instead of internal QFlow processing. Confirming which flow sources feed a given network segment, and what classification depth each one actually provides, is the flow-side equivalent of DEH Part 27 §3.1's DSM/QID mapping prerequisite — skipped, it produces the same failure shape: a query or Rule Test that runs cleanly and silently sees less than the engineer designing it assumed.

```

flowchart TB
    subgraph EventPath["Event pipeline"]
        E1["Log Source generates\na log record"] --> E2["DSM parses payload,\nassigns QID,\nexttracts custom properties\n(Part 4)"]
        E2 --> E3["Ariel events table"]
    end
    end
    subgraph FlowPath["Flow pipeline"]
        F1a["Internal QFlow processing\n(SPAN/TAP, packet-derived - 2.1)"] --> F2["Flow record:\n5-tuple + bytes/packets\n+ duration + app ID"]
        F1b["External flow feed\n(NetFlow/IPFIX/sFlow - 2.1)"] --> F2
        F2 --> F2a{"High-fan-out\nhost? (3)"}
        F2a -->|yes| F2b["Superflow aggregation -\nindividual flows collapsed,\nsee Blind Spot"]
        F2a -->|no| F3["Ariel flows table"]
        F2b --> F3
    end
    end
    E3 -->|ad hoc AQL\nFROM events| H["Analyst / hunter"]
    F3 -->|ad hoc AQL\nFROM flows (6)| H
    E3 --> CRE["Custom Rules Engine"]
    F3 --> CRE
    CRE --> BB1["Event, Flow, or Common\nBuilding Blocks (5)"]
    BB1 --> R1["Rule chains BB + RS\nchecks, fires Response"]
    R1 --> OFF["Offense"]

```

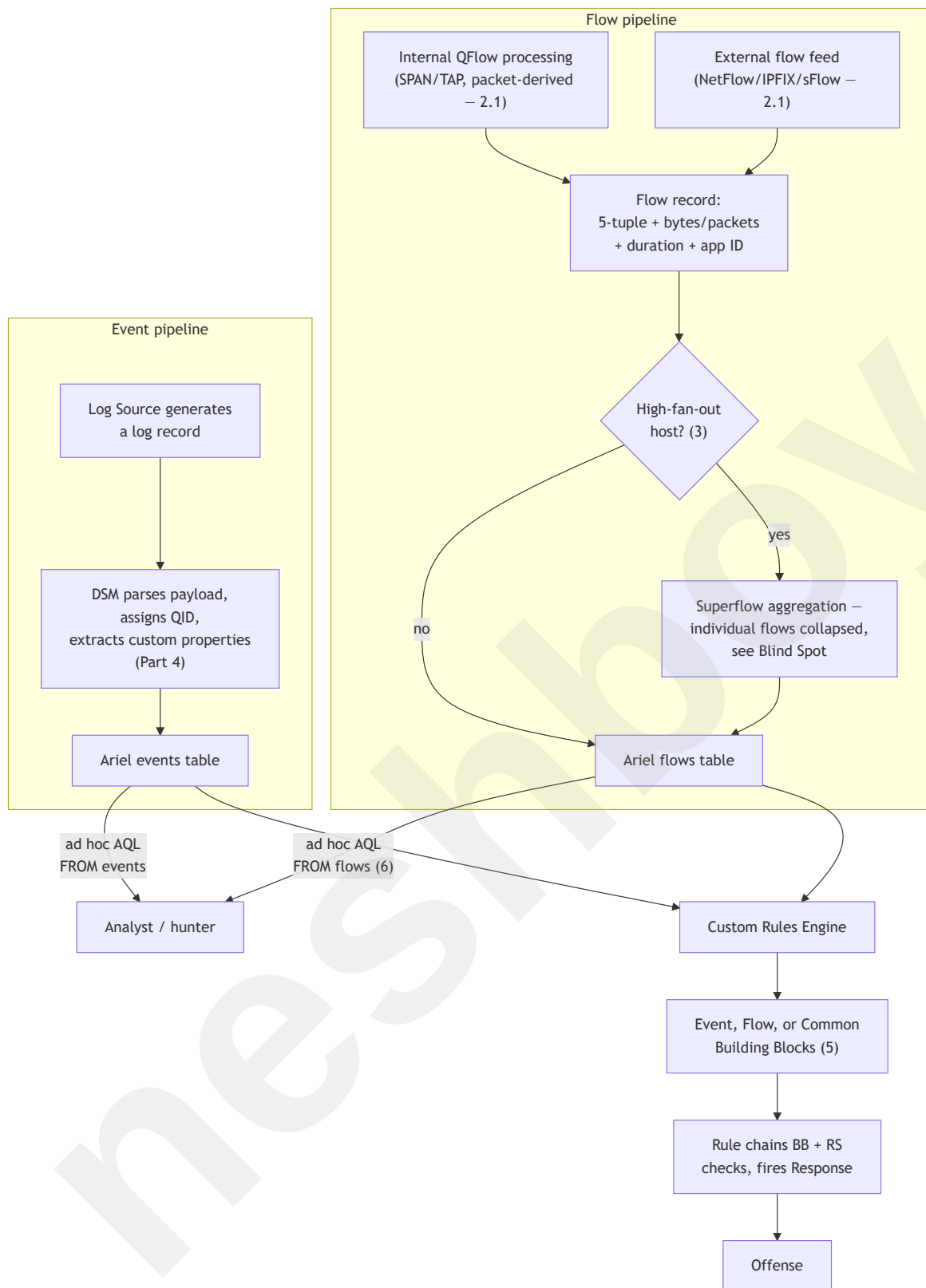


Figure 7.1 — Event and flow pipelines converging on the same Custom Rules Engine.

CONCEPTUAL. Illustrates the two structurally different collection paths this part covers — DSM-parsed events on one side, internally-processed or externally-fed flow records (with superflow aggregation as a possible detour) on the other — both landing in their respective Ariel table and both reachable by an ad hoc AQL search or by a Rule built from Building Blocks and Reference Sets in the Custom Rules Engine

(CRE), extending DEH Part 27's own Figure 27.1 event-only diagram to cover the flow side that part left unillustrated. Not a reproduction of any IBM architecture diagram and not a claim about a specific deployment's actual topology — Part 2 covers real Console/Processor/Collector role placement.

3. Superflows: aggregation, and the blind spot it creates

[PLATFORM ENGINEER] A single busy host — a DNS server answering thousands of clients, an NTP server, a heavily used file server — can generate an enormous number of structurally similar, short-lived flows in a short window. Logging every one of those as a fully discrete flow record would consume FPM budget (\$7) and Ariel storage out of proportion to the actual investigative value of keeping each one separate. QRadar's answer is the superflow: a single aggregate flow record representing many similar underlying flows — same source, same destination pattern, same application — collapsed into one row with summed byte/packet counts and a flow count, rather than one row per individual conversation.

This is a sensible, necessary control for the exact high-fan-out hosts it targets. It is also a real detection blind spot, and it is the blind spot this part exists to name rather than let a rule engineer discover it the hard way after a Rule Test that should have matched something didn't.

Blind Spot *A superflow's aggregate record has no discrete row for any one of the individual flows it consolidated — the specific byte count, destination, or duration of one particular conversation folded into that aggregate is not separately queryable in Ariel once the aggregation has happened, and a Rule Test built to match a specific flow-level condition (an unusually large single transfer, a specific destination among many) may never see the one flow that mattered because it was never stored as its own row. A single malicious DNS-tunneling exchange to a high-fan-out DNS server, or one anomalously large transfer folded into a busy file server's aggregate, can be structurally invisible to flow-based correlation the same way a Reference Set that never got populated makes a second-stage Rule silently never fire (DEH Part 27 §2.3's own Blind Spot names that failure shape for the CRE generally) — except here the cause is upstream aggregation, not a missed Rule Response, and no amount of downstream Rule Test tuning fixes it. The mitigation is scope, not cleverness: identify which hosts are high-fan-out enough to trigger superflow aggregation in your environment, and treat any flow-based analytic aimed at one of those hosts's traffic as needing a different data source (an events-side log from that host itself, or a narrower flow-source configuration scoped to exclude that host from aggregation) rather than trusting flow-based Ariel data to represent every individual conversation that host had.*

PRODUCT VERSION NOTE *The specific threshold (how many similar flows in what window trigger superflow aggregation) and whether it is administrator-configurable in a given deployment has varied across QRadar releases and deployment tiers. Verify the current superflow configuration surface and default thresholds in your own Console before assuming a specific number, and before assuming a specific host is or isn't being aggregated without checking.*

4. Asymmetric routing and capture-location pitfalls

[PLATFORM ENGINEER] Internal QFlow processing builds its richest flow records — full byte counts in both directions, accurate application classification — from seeing both legs of a conversation. A network with asymmetric routing, where outbound and return traffic for the same conversation can take physically different paths (common with multi-homed internet egress, some load-balancer configurations, and routing setups optimized for path cost rather than session symmetry), breaks that assumption at the point where it matters least conveniently: silently. A QFlow sensor watching only the outbound leg of an asymmetric conversation still produces a flow record — it just produces one with an incomplete byte count in the direction it never saw, and potentially a downgraded or incorrect application classification, because DPI-based classification often depends on seeing enough of both directions of the exchange to be confident.

Capture location compounds the same problem at a more basic level: a SPAN or TAP point only sees the traffic that physically crosses it. Two hosts on the same access switch, in the same VLAN, talking to each other without ever crossing a router or firewall uplink, produce traffic a flow source positioned at the network's perimeter or core will never see at all — not degraded, not partial, simply absent, with the resulting Ariel `flows` table showing nothing for that conversation ever having happened.

Platform Reality *The documented expectation behind a flow-based analytic is usually "this network segment's traffic is visible to a flow source." What a real deployment actually has, more often than a design review assumes, is partial visibility assembled opportunistically over years — a SPAN port added to catch one incident, a NetFlow export enabled on one core switch because it was easy, no coverage at all on the east-west traffic between two server racks that were never in scope for any single project that built out flow collection. Before designing a flow-based Rule or Building Block around lateral-movement or internal-scanning detection specifically, verify — segment by segment, not as a blanket assumption — which internal network paths actually have a flow source watching them, the same diligence DEH Part 27 §3.1's own Engineering Reality box asks for DSM/QID mapping before trusting a query's field list. A flow-based Rule that has been silently seeing nothing for a specific internal segment since the day it was deployed fails exactly the same way an unmapped DSM does: no error, a query that returns zero rows, and a false sense of coverage until someone tests it against a known-positive case.*

5. Deciding where an analytic belongs: events, flows, or both

[RULE ENGINEER] DEH Part 27 §1.1 introduces `events` and `flows` as QRadar's two Ariel tables and stops there — proving one narrower point about AQL syntax, not making this decision. Here it is, stated as a procedure rather than left to instinct:

Analytic needs...	Belongs on	Why
A specific application-layer field a device logged (a username, a process name, a Windows Event ID)	<code>events</code>	Flows have no concept of a DSM-extracted custom property (§1's table) — that detail only exists if something logged it.
Visibility into traffic no device was ever configured to log (internal lateral movement, unexpected internal port usage, scan-like fan-out to many destinations)	<code>flows</code>	This is exactly the blind spot §1 names <code>events</code> - only deployments as having by construction.
A volume or duration signal (unusually large transfer, sustained long-lived connection, beaconing-interval regularity)	<code>flows</code>	Byte/packet counts and duration are native flow fields; no DSM extracts an equivalent from most logs.
Correlating "this user logged on" (an events fact) with "this host then moved an unusual data volume" (a flows fact) into one analytic	Both	Requires the same Reference Set decomposition DEH Part 27 §2.2 introduces for a different case — see the worked example below.
A host already known to be high-fan-out (§3)	Neither <code>flows</code> alone, nor without adjustment	Superflow aggregation may already have consolidated the individual conversation away; consider an events-side signal from that host instead.

Two of the Rule Wizard's rule types exist specifically to act on this decision once it's made: a Flow rule evaluates Rule Tests against incoming flow records the same way an Event rule evaluates them against events, and a Common rule can match conditions against either — naming these rule types here as the stable architectural fact that `flows` is a first-class CRE input, not a claim about their exact current labels or availability in every release, which Part 8 flags per its own PRODUCT VERSION NOTE convention. The full Rule Wizard catalog — every rule type, every Rule Test category, and the exact current wording of each — is Part 8's job, not this part's; what matters here is simply that the architecture supports acting on `flows` inside the Custom Rules Engine with the same Building-Block-and-Reference-Set pattern DEH Part 27 §2–§3 already establishes for events, not a separate, lesser mechanism.

R: Suspicious Internal SMB Fan-Out — anatomy of a flow-based Rule

[RULE ENGINEER] A concrete illustration of the "both" row above, deliberately decomposed the same way DEH Part 27 §3.3 decomposes its own worked detection into three objects, and equally illustrative rather than lab-tested — this book has no lab to validate it against (STYLE-GUIDE.md §0):

1. **Building Block** `BB:Flow-SMB-Internal-FanOut-Candidate` — a Flow Rule Test condition matching internal-to-internal SMB traffic (destination port 445, both source and destination inside the internal network hierarchy defined under Network Hierarchy — Part 20 covers how that hierarchy gets built and kept current) where one source host connects to an unusually high count of distinct destination hosts within a short window — QRadar's native threshold/aggregation Rule

Test, the same mechanism DEH Part 27 §4's own threshold example uses for a failed-authentication count, applied here to a distinct-destination-count on flow data instead.

2. **Reference Set** `RS-Approved-SMB-Fileserver-Sources` — the maintained allowlist of hosts expected to make exactly this kind of fan-out connection legitimately: backup servers, patch-management servers, endpoint-management infrastructure that touches many hosts over SMB by design. Populated and re-validated the same way DEH Part 27 §3.3's `RS-Allowlisted-LSASS-Tools` is — by name or IP, re-checked after any infrastructure change that adds or retires one of these legitimate fan-out sources.
3. **Rule** `R: Suspicious Internal SMB Fan-Out` — chains `BB:Flow-SMB-Internal-FanOut-Candidate` AND a Rule Test checking the source host is **not** contained in `RS-Approved-SMB-Fileserver-Sources`, with a Response that creates or contributes to an Offense indexed on the source host asset.

The "both" case from the decision table sharpens this further: if the same source host also has an `events`-side Rule contributing a fact to a shared Reference Set — say, `RS-Recently-Alerted-Hosts`, populated by an unrelated events-based detection that already flagged that host for something else in the last hour — a second Rule Test on `R: Suspicious Internal SMB Fan-Out` checking that Reference Set turns a moderate flow-only signal into a higher-confidence combined one, using exactly the cross-event, cross-table state mechanism DEH Part 27 §2.2 describes, just fed from both `events` and `flows` sources instead of two `events`-side Rules.

PRODUCT VERSION NOTE Whether a specific QRadar release's Rule Wizard lets a single Rule mix a Flow-sourced Rule Test and an events-sourced Reference Set membership check in one object (as opposed to requiring a Common rule type, or requiring the Reference Set check to live in a second, chained Rule) is exactly the kind of Rule Wizard mechanics question Part 8 owns and DEH Part 27 §2.1 already flags as shifting release to release. Verify the current Rule Wizard's supported combinations in your own console before assuming this pattern is buildable as a single object.

6. Threat hunting on flow data

[THREAT HUNTER] Flow data's hunting value is the direct payoff of §1's core claim: it is the one Ariel table that can show a hunter traffic no analyst, no Rule, and no DSM ever had a reason to expect, because nothing had to be configured to log it. AQL against `flows` uses the same syntax family DEH Part 27 §1 already teaches against `events` — `SELECT / FROM flows / WHERE / GROUP BY / LAST n HOURS` — so this section shows the shape of a flow hunt rather than re-explaining any of that syntax.

HUNT-07-01 — Does any internal host show beaconing-shaped flow behavior QRadar's own Rules aren't watching for?

Threat Hypothesis: A compromised internal host communicating with external command-and-control infrastructure often produces a distinctive flow signature — small, regular-interval connections to the same destination over a sustained period — that no Building Block in this environment currently targets, because §5's decision table puts "regularity of interval" outside the native threshold Rule Test's simple count-in-a-window model.

Approach: Pull outbound flows grouped by source/destination pair over a multi-day window and look for pairs with a high connection count but a low total byte volume — the volume-per-connection signature of a small periodic check-in rather than a real data transfer:

```
-- QRadar AQL, not standard SQL — same SELECT/FROM/WHERE/GROUP BY syntax DEH Part 27 §1
-- introduces against events, run here against flows instead; no separate "flow query
-- language" exists, but note the same non-SQL behavior DEH Part 27 §1 flags still applies:
-- LAST n DAYS is AQL's own time-window shorthand, not a standard-SQL construct, and
-- NETWORKNAME() below resolves a raw IP to its network-hierarchy segment rather than
-- naming a queryable column the way a quoted custom property (DEH Part 27 §1.2) would.
-- CONCEPTUAL SAMPLE — a hunting query, not a standing Rule; the count/byte thresholds
-- implied below are illustrative starting points, not validated against any real telemetry.
SELECT sourceip, destinationip, COUNT(*) AS "Connection Count",
       SUM(sourcebytes + destinationbytes) AS "Total Bytes"
FROM flows
WHERE NETWORKNAME(destinationip) = 'Internet'
GROUP BY sourceip, destinationip
LAST 3 DAYS
```

Finding (illustrative — a template, not a captured result): A hunt run this way either turns up nothing worth escalating — itself a negative finding worth recording per DEH [TERMINOLOGY.md](#)'s Hunt definition, as evidence this specific pattern was checked rather than silently assumed absent — or surfaces a source/destination pair with a connection count high enough and a per-connection byte volume small and consistent enough to warrant pulling that destination's reputation and, if it's confirmed suspicious, promoting the pattern into a standing Flow rule the same way DEH Part 27 §5's own hunting finding gets promoted into a Building Block value-list update.

Hunter's Note Pair the connection-count-and-byte-volume view above with an interval-regularity check the raw `COUNT(*)` alone can't give you — export the individual flow start times for any suspect pair and look at the gaps between them by eye, or with a downstream script pulled via the REST API, before treating a merely high connection count as a finding on its own. A host polling a legitimate SaaS API every few minutes and a host beaconing to C2 on the same rough cadence can look identical on count and volume alone; the interval's regularity, and whether that destination has any legitimate business reason to be contacted this often, is what actually separates them.

7. The licensing consequence: FPM, and why superflows exist beyond noise control

[PLATFORM ENGINEER] DEH Part 27 §1.3's own Engineering Reality box already establishes that QRadar licenses events by sustained EPS and flows by sustained FPM — separate meters, separate overage conditions. Superflow aggregation (§3) is not purely a storage-efficiency measure; it is also the platform's own mechanism for keeping a handful of extremely high-fan-out hosts from single-handedly pushing an environment's real FPM consumption over its licensed tier, the same way an unbudgeted Sysmon Event ID 1 rollout can push EPS over budget. That framing matters for how a rule engineer

reads §3's Blind Spot: superflow aggregation is not a bug to route around locally for every affected host, because doing so for enough hosts reintroduces the exact FPM overage problem the aggregation exists to prevent. The real fix, as §3 already states, is scoping — deciding deliberately which specific hosts are worth the FPM cost of granular, non-aggregated flow visibility (a domain controller, a small number of genuinely high-value servers) rather than disabling aggregation broadly. Part 17 covers the full capacity-planning discipline this decision belongs to, including how to forecast FPM growth before a new flow source or a network change pushes an environment into overage the same way Part 4 already flags for a new noisy Log Source's EPS impact.

PRODUCT VERSION NOTE *Support for, and the exact metering/packaging of, flow collection under a cloud- or SaaS-delivered QRadar offering has been a moving target across this platform's business-unit and delivery-model changes, consistent with Part 2's opening PRODUCT VERSION NOTE on packaging generally. Do not assume flow collection is available, licensed identically to EPS, or architected the same way in every current QRadar delivery model — verify against the specific offering and contract you are actually running before a capacity plan or a flow-based detection design depends on it.*

Cross-references

DEH Part 27 §1.1 (the `events/flows` table split), §1.2 (QID), §1.3 (EPS/FPM licensing, Engineering Reality), §2.1–§2.3 (Building Block/Reference Set decomposition pattern, applied here to flow-sourced conditions), §3.1/§3.3 (DSM/QID mapping-prerequisite pattern, mirrored here for flow-source coverage), §5 (hunting-query structure); DEH Part 23 §5 (Sigma-first vs. native authoring, not re-argued here); this book's Part 2 (deployment topology and Flow Processor placement), Part 4 (DSM/QID onboarding, the events-side mirror of this part's flow-source onboarding concerns), Part 8 (full Rule Wizard catalog, including Flow and Common rule types), Part 17 (FPM capacity planning), and Part 20 (Network Hierarchy, referenced by §5's worked example).

Section D — Building the Correlation Layer

neshbooy

Part 8 — The Rule Wizard Catalog: Rule Types Beyond the Canonical Example

Why this part exists

[CONCEPT] DEH Part 27 built exactly one rule: `R: Suspicious LSASS Access – Unapproved Process`, an Event Rule chaining `BB:LSASS-Access-Candidate` against `RS-Allowlisted-LSASS-Tools` (DEH Part 27 §3.3). That was the right amount of Rule Wizard to show in a part whose job was proving one point about QRadar's correlation model against KQL and SPL — not a survey of everything the Rule Wizard can build. This part is that survey. The Rule Wizard's very first screen asks a question DEH Part 27 never had to answer out loud because DET-27-01 only ever needed one answer: **what kind of record is this rule actually evaluating?** An event arriving right now. A flow record arriving right now. Either one, matched on the properties they share. An Offense that already exists, changing state. Or a saved search's aggregate result, checked on a schedule rather than a stream.

Get that first answer wrong and nothing downstream saves you — the wrong rule type doesn't error, it just doesn't expose the Rule Test category you needed, and the rule you built quietly tests something adjacent to what you meant to test. This part works through each rule type in the Wizard's catalog, the Response actions available once a rule matches (which are broader than "create an Offense"), and the Response Limiter mechanism that keeps a matching rule from flooding its own Response. None of this re-teaches AQL (DEH Part 27 §1 owns that) or the Building Block/Reference Set/Rule three-object pattern DEH Part 27 §2 and §3.3 already established — this part assumes both and builds outward into the parts of the Wizard DET-27-01 never had to touch.

1. The Rule Wizard's anatomy, briefly recapped

[CONCEPT] DEH Part 27 §2 established the load-bearing structural fact: the Custom Rules Engine, universally abbreviated CRE, evaluates every incoming record against deployed Rules built from Rule Tests, and Rules can reference Building Blocks — named, reusable Rule Test groupings — by name. A Rule Test is one condition drawn from a fixed, point-and-click catalog; a Rule chains one or more of them with AND/OR logic and attaches a Response.

What DEH Part 27 didn't need to spell out is that the Rule Test catalog itself is not one fixed list. It's scoped by which **rule type** you picked on the Wizard's first screen, because rule type determines which underlying record stream the CRE evaluates the rule against — and a Rule Test that reads a flow-only field simply isn't offered inside a rule type that never sees flow records. This is the decision §2–§4 below work through category by category.

1.1 Rule type as the first decision, not an afterthought

[RULE ENGINEER] In practice, a team that has only ever built Event Rules (because that's what the one worked example in training or in DEH Part 27 showed them) tends to keep building Event Rules for problems that aren't event problems — testing a flow-volume condition by proxy through an event-

side custom property that only approximates it, or building a second, unrelated Event Rule to "watch" an Offense's magnitude instead of using the rule type built for exactly that. None of these substitutes are wrong in the sense of producing an error. They're wrong in the sense of testing something adjacent to the actual question, which is a harder failure to catch in review because the rule deploys cleanly, matches something, and produces Offenses that look plausible until an analyst notices the evidence attached to them doesn't match the name on the tin. §7's Rule Autopsy below is exactly this failure mode, worked through in full.

2. The four data-scoped rule types: Event, Flow, Common, and Offense

[RULE ENGINEER] Four of the Wizard's rule types are distinguished by which record stream they evaluate: two raw-telemetry streams (`events`, `flows` — the same two Ariel tables DEH Part 27 §1.1 introduces for search), one that spans both, and one that steps outside raw telemetry entirely to evaluate QRadar's own correlation output.

2.1 Event Rules — DET-27-01's home

[RULE ENGINEER] An Event Rule evaluates the `events` table as records land, one at a time (or aggregated across a window via a threshold-style Rule Test, the same "at least X events... in Y minutes" category DEH Part 27 §4 covers for DET-27-02). Every Rule Test in an Event Rule's catalog reads event-side fields: QID (QRadar's internal event-taxonomy identifier, per DEH Part 27 §1.2), `LOGSOURCETYPENAME()`, and any custom property a DSM (Device Support Module — the log-source-type-specific parser that assigns a QID and extracts fields, per DEH Part 27 §1.2/§3.1) has populated for that log source. **R: Suspicious LSASS Access – Unapproved Process** is an Event Rule because DET-27-01's entire analytic — a source process, a target process image, a granted-access value — lives on one Sysmon-derived event; there is no flow-side component to the pattern at all.

Event Rules are the default a new QRadar rule author reaches for, and for good reason: most detection content maps naturally onto discrete log records rather than network-flow aggregates. The failure mode isn't choosing an Event Rule for an event pattern — it's reaching for one when the pattern actually needs flow evidence, covered next.

2.2 Flow Rules — evaluating network behavior QRadar itself observed

[RULE ENGINEER] A Flow Rule evaluates the `flows` table (DEH Part 27 §1.1) as flow records arrive from QFlow or a third-party flow source (Part 7 of this book covers flow collection architecture in full). Its Rule Test catalog exposes flow-side properties an Event Rule's catalog does not: bytes/packets transferred in each direction, flow duration, detected application, and the source/destination network-hierarchy segment a flow traversed. A Flow Rule cannot see event-side custom properties at all, for the same structural reason an Event Rule can't see flow volume — it evaluates a different table.

Example, illustrative: **R: Large Outbound Flow to Unowned Network – Exfil Candidate** — a Flow Rule chaining a Rule Test on bytes-transferred-outbound exceeding a configured value against a Rule Test checking the destination network segment is **not** contained in a Reference Set of known-owned or known-partner networks. This is a flow-native pattern: nothing about it involves a parsed log record, and

building it as an Event Rule would leave the author hunting for a bytes-transferred Rule Test that the Event Rule catalog simply does not carry.

2.3 Common Rules — one rule, two streams

[RULE ENGINEER] A Common Rule evaluates both **events** and **flows**, but only against the subset of properties both tables actually share — source/destination IP and port, protocol, and a handful of others QRadar's schema maps onto both record types identically. A Common Rule's Rule Test catalog is narrower than the union of Event and Flow catalogs for exactly this reason: it can't offer a flow-only bytes-transferred test any more than it can offer an event-only QID test, because either one would only ever be populated on half the records a Common Rule evaluates.

Common Rules earn their place when the correlation genuinely needs "the same source IP whether it shows up as an event or a flow," not as a general substitute for choosing between Event and Flow. §7's Rule Autopsy is a team learning that distinction the hard way.

2.4 Offense Rules — correlating on QRadar's own output

[RULE ENGINEER] An Offense Rule evaluates Offenses themselves — the record QRadar's CRE already produced — as they're created or updated, not the events or flows that fed them. Its Rule Test catalog reads Offense-level state: magnitude crossing a value, an Offense going a configured period without analyst assignment, an Offense's event/flow count growing past a value, or an Offense matching a specific source Rule or Building Block by name.

Example, illustrative: **R: Escalate Unassigned Critical Offense** — an Offense Rule testing for magnitude above a configured value AND no analyst assignment after a configured dwell time, with a Response that sends an email notification and adds an annotation, rather than creating a second Offense for the same underlying activity.

Blind Spot An Offense Rule tests conditions on the Offense record itself — magnitude, assignment state, contributing-record count, source Rule name — not on the event or flow fields that produced it. A rule built to catch "an Offense whose magnitude jumped because of a specific technique" can only see that the magnitude jumped, never which contributing event drove the jump, unless the Event or Common Rule that fed the Offense already wrote that context somewhere an Offense Rule can read it back (an annotation, a Reference Set entry keyed to the Offense ID). Meta-correlation on Offenses is a coarser signal layered on top of already-fired detections — an escalation and notification mechanism, not a substitute for getting the underlying Event, Flow, or Common Rule right in the first place.

3. The saved-search-driven rule family: Threshold, Anomaly, and Behavioral

[RULE ENGINEER] The Wizard groups three rule types together under a distinct authoring model from the data-scoped four above: rather than evaluating individual records as they stream in, each of these three wraps a saved AQL search's *aggregate result* and re-evaluates that result on a schedule. This is

architecturally different from the threshold-style Rule Test DEH Part 27 §4 already covers for DET-27-02 — worth disambiguating explicitly, since both use the word "threshold" and a reader moving between the two parts should not conflate them:

- DEH Part 27 §4's threshold Rule Test (*when at least X events are seen with the same property in Y minutes*) is a condition available *inside* an Event, Flow, or Common Rule — it evaluates a rolling window of individual records against a count, in near-real time, as part of a rule that also carries other Rule Tests.
- This section's Threshold rule *type* is a standalone rule built directly on top of a saved search's own aggregate output, re-run on a configured interval, with no requirement that the underlying records look anything like a single Building Block's match condition.

The distinction matters because the second model is the one that answers a genuinely different class of question: not "did this specific pattern happen at least X times against one entity," but "did some aggregate metric across the whole environment move in a way worth flagging."

```
-- QRadar AQL – illustrative saved search only, disambiguated from standard SQL exactly
-- as DEH Part 27 §1.1 flags on first appearance: no general-purpose JOIN, and the
-- LAST/GROUP BY shorthand here follows AQL's own time-window and aggregation syntax,
-- not a portable SQL dialect. Property names and QID text are illustrative – see
-- DEH Part 27 §3.1 on confirming both against your own DSM mapping before reuse.
SELECT UNIQUECOUNT(sourceip) AS "Distinct Sources"
FROM events
WHERE QIDNAME(qid) = 'Process accessed'
GROUP BY QIDNAME(qid)
LAST 1 HOURS
```

A saved search shaped like this one — how many distinct source IPs triggered a process-access QID in the last hour, across the whole environment, not any one host — is the kind of aggregate a Threshold, Anomaly, or Behavioral rule type wraps. DEH Part 27 §2.3 already flags that whether a given QRadar version lets a Rule Test anchor directly to a saved AQL search, and on what evaluation cadence, is a version-dependent capability; the same uncertainty applies with at least as much force to which saved-search shapes the Anomaly Detection Rule family accepts as a valid basis in your specific release.

3.1 Threshold rules

[RULE ENGINEER] A Threshold rule fires when a saved search's aggregate result crosses a fixed number you configure — "more than 50," not a baseline the platform computed for you. This is the simplest of the three: no history, no seasonality, just a number and a comparison operator, checked each time the saved search re-runs.

3.2 Anomaly rules

[RULE ENGINEER] An Anomaly rule compares a saved search's current aggregate result against a baseline QRadar builds from that same search's own recent history, and fires on a meaningful percentage change from that baseline rather than a fixed number — the mechanism aimed at catching "today looks different from this metric's usual pattern" without an author having to guess the right

static number for an environment whose baseline itself drifts over time (seasonal traffic, headcount growth, a new business unit's log volume).

3.3 Behavioral rules

[RULE ENGINEER] A Behavioral rule targets the same class of question as an Anomaly rule — has this metric moved outside what's normal — but through a different statistical treatment of "normal," oriented toward catching a sudden deviation from a metric's typical behavior rather than a gradual seasonal shift. The practical distinction an author cares about is which of the two better matches the kind of change worth flagging for a given metric: a slow drift away from baseline versus a sharp, out-of-character spike or drop.

PRODUCT VERSION NOTE *The exact statistical model each of Threshold, Anomaly, and Behavioral rule types uses — the specific percentage-deviation formula, the baseline window length, and even which of the three names groups under "Anomaly Detection Rules" in the Wizard's own menu — has not been independently verified against a live console for this book (STYLE-GUIDE.md §0's no-lab constraint) and is exactly the kind of literal-wording claim §11 requires flagging rather than asserting. Treat this section's descriptions as the conceptual shape of each mechanism, and verify the current math, defaults, and menu grouping against your own deployment's Rule Wizard and release notes before designing a specific rule around any one of the three.*

4. User Rules and the identity-record exception

[RULE ENGINEER] A User rule type evaluates QRadar's own User records — identity data the platform builds and updates from authentication and identity-correlation events, distinct from any single raw event or flow — as those records are created or changed. Where the four data-scoped types in §2 test what a log record or flow record says, and the three saved-search types in §3 test what an aggregate metric is doing, a User rule tests what QRadar currently believes about an identity: a new administrative-group membership appearing on a User record, or an identity attribute changing in a way worth flagging independent of any single authentication event that triggered the change.

PRODUCT VERSION NOTE *User rules sit closest to QRadar's identity-correlation and user-behavior-analytics capabilities, which have been packaged, renamed, and re-scoped across releases and business decisions more than most of this book's other subject matter (per STYLE-GUIDE.md §11's packaging/ownership trigger) — including periods where deeper user-behavior analytics lived in a separate app rather than the core Rule Wizard (Part 19 of this book covers the App Framework and UBA-class apps at a survey level). Whether "User" appears as a first-class rule type, what Rule Test categories it exposes, and how it relates to any UBA app installed alongside it are all things to verify against your own deployed release before building on this section's description as a literal current-state claim.*

5. Choosing a rule type: a decision table

[RULE ENGINEER] The table below supports the one decision this part exists to get right before any Rule Test gets picked — which of the Wizard's rule types actually matches what you're correlating. Treat the Version column as a floor, not a ceiling: it flags where this book has the least confidence the exact menu grouping or availability matches your release, not a claim that everything else in the row is guaranteed unchanged either.

Rule Type	Evaluates	Typical Use	Version
Event	events table, per record or windowed	Log-record patterns — DET-27-01's home	Stable core CRE concept
Flow	flows table, per record or windowed	Network-volume/behavior patterns with no log-record component	Stable core CRE concept
Common	Shared properties across events and flows	One entity (IP, port) correlated regardless of which stream reported it	Stable core CRE concept
Offense	Existing Offense records, on state change	Escalation, notification, meta-correlation on already-fired detections	Stable core CRE concept
Threshold	A saved search's aggregate result vs. a fixed number	Simple environment-wide count crossing a known bad number	See §3's PRODUCT VERSION NOTE
Anomaly	A saved search's aggregate result vs. its own rolling baseline	Gradual/seasonal drift worth flagging	See §3's PRODUCT VERSION NOTE
Behavioral	A saved search's aggregate result vs. typical-behavior deviation	Sudden, sharp deviation worth flagging	See §3's PRODUCT VERSION NOTE
User	QRadar's own User (identity) records, on change	Identity-attribute or group-membership change	See §4's PRODUCT VERSION NOTE

flowchart TD

```
A["What are you correlating?"] --> B["Raw telemetry arriving now,\nor QRadar's own derived state?"]
B -->| "Raw events/flows" | C["Which stream(s)\ndoes the pattern need?"]
C -->| "Event-side fields only" | E["Event Rule\n(DET-27-01's home)"]
C -->| "Flow-side fields only" | F["Flow Rule"]
C -->| "Same entity, either stream" | G["Common Rule\n(shared-property tests only)"]
B -->| "An existing Offense\nchanging state" | H["Offense Rule\n(meta-correlation, see Blind Spot 2.4)"]
B -->| "A saved search's aggregate\nresult, on a schedule" | I["What shape of change\nmatters?"]
I -->| "Fixed number crossed" | J["Threshold Rule"]
I -->| "Gradual/seasonal drift\nfrom baseline" | K["Anomaly Rule"]
I -->| "Sudden deviation from\ntypical behavior" | L["Behavioral Rule"]
B -->| "QRadar's own User\n(identity) record changing" | M["User Rule"]
```

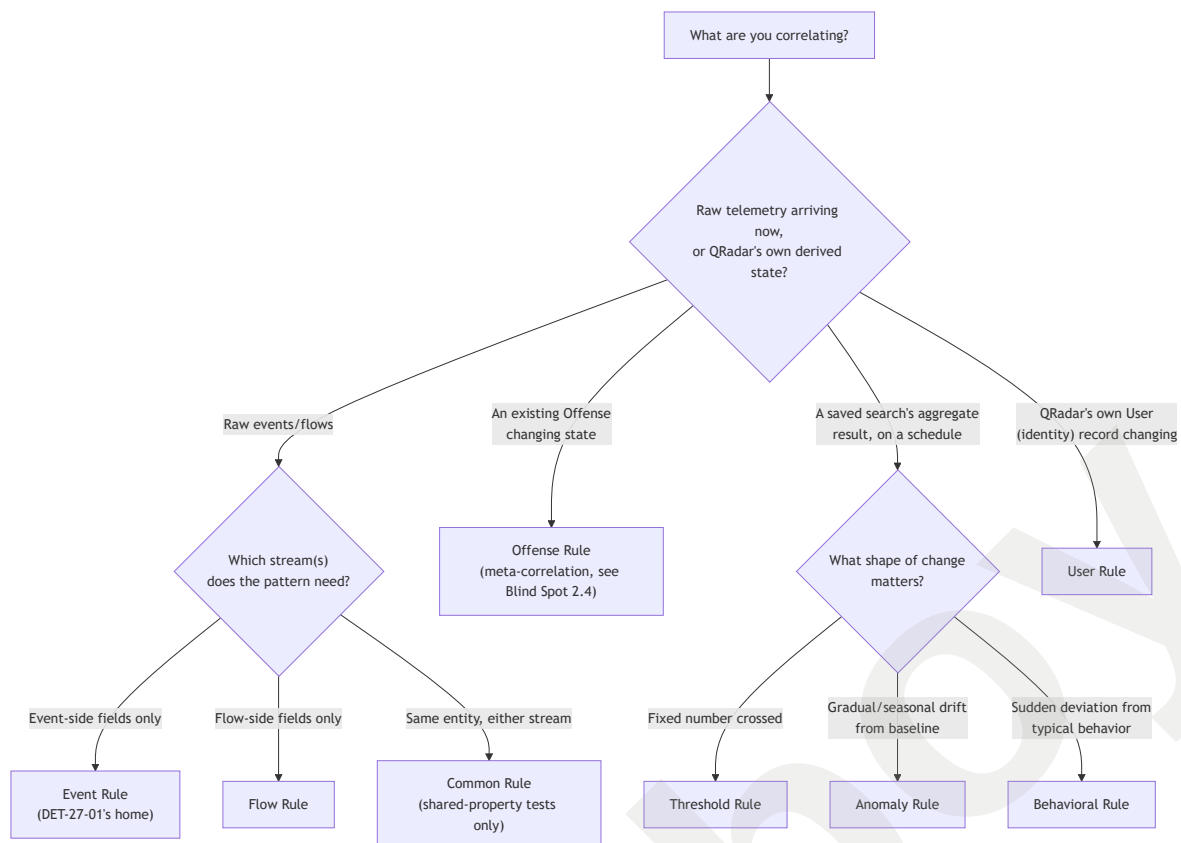


Figure 8.1 — Choosing a Rule Wizard rule type from what's actually being correlated.

CONCEPTUAL. Illustrates the decision this part argues from prose — rule type is a first-class authoring choice made before any Rule Test is selected, not an incidental setting — and maps each of the eight types in §2–§4 to the question it answers. Not a reproduction of the Rule Wizard's own screen flow, which DEH Part 27 §2.1 and this part's PRODUCT VERSION NOTES already flag as wording/grouping that shifts across releases; this diagram is this book's own structural summary of that catalog, not a captured UI walkthrough.

6. Response actions beyond "create an Offense"

[RULE ENGINEER] Every rule type above shares the same Response tab once a Rule Test chain matches. DEH Part 27 §2.1 names "create or contribute to an Offense" as one Response option among several without enumerating the rest; this section is that enumeration.

6.1 Offense-shaping responses

[RULE ENGINEER] Beyond simply creating an Offense, a Response can contribute a match to an existing Offense (indexed on a chosen field, exactly as `R: Suspicious LSASS Access – Unapproved Process` indexes on source host or username per DEH Part 27 §3.3), set or adjust Offense attributes such as name, description, or an annotation, and — the mechanism §2.4's Offense Rules depend on to see anything beyond raw magnitude — write a note or attribute an Offense Rule can later read back.

6.2 Reference-data responses

[RULE ENGINEER] A Response can add to, or remove from, a Reference Set, Reference Map, or another reference-data collection (Part 10 of this book covers the full typed taxonomy). This is the mechanism behind DEH Part 27 §2.2's entire Reference Set correlation pattern — the first-stage Rule's Response is what actually populates `RS-Stage1-Seen` for the second-stage Rule Test to check.

6.3 Notification and integration responses

[RULE ENGINEER] A Response can send an email notification, send an SNMP trap, forward a syslog message to an external system, dispatch a new correlated event (chaining into another rule's evaluation), or execute a script or app-based custom action. Each of these is how QRadar's correlation output leaves the platform — into a ticketing system via email-to-case, into another monitoring tool via SNMP or syslog, or into a bespoke remediation action via a script.

PRODUCT VERSION NOTE *The exact name, availability, and execution surface of the script/custom-action Response — historically a raw script executed on the Console, increasingly framed as an App Framework "Custom Action" in more recent packaging — has shifted across releases and delivery models (on-prem vs. cloud/SaaS, per STYLE-GUIDE.md §11's packaging trigger). Confirm what your specific deployment actually offers, and where it actually executes, before designing a Response around this action; Part 19 covers the App Framework's own resource footprint for the case where a custom action runs as an app.*

Platform Reality *A Response that sends an email, an SNMP trap, or runs a script executes once per match, not once per Offense — a Rule matching in bulk (a genuine flood, or a newly enabled Rule evaluated retroactively against a large backfill) can generate that same multiplied volume of outbound notifications or script executions, independent of how many Offenses the CRE consolidates the matches into. A Response that looks cheap at expected match volume can back up mail queues, SNMP receivers, or the Console's own resources at flood volume — the same resource-sharing concern DEH Part 27 §1.3's Engineering Reality callout names for EPS/FPM licensing applies to the Response subsystem, not just to ingestion.*

6.4 Response Limiters — the throttle nobody notices until it's missing

[RULE ENGINEER] A Response Limiter caps how often a Rule's Response actually fires per matched value within a configured interval — for example, sending at most one email per source IP per hour even if the underlying Rule Test chain matches that source IP forty times in the same hour. The Rule still evaluates and still contributes to the Offense every time it matches; the Limiter governs only the Response side, which is exactly the distinction that makes it easy to configure once, forget, and then be surprised twice in opposite directions — once when a Limiter that's too permissive floods a downstream mailbox during a genuine incident, and once when a Limiter that's too aggressive silently drops the second, third, and further legitimate notification an escalation actually needed.

7. Rule Autopsy: the Common Rule that tested only half its own condition

[RULE ENGINEER] §2.3's Common Rule/Event Rule boundary is the exact distinction this failure worked through in full.

Rule Autopsy The rule: *R: Same Source IP Seen in Event and Flow Category – Candidate C2 Beacon*, built as an Event Rule. **Why it shipped:** The author wanted one rule instead of two — a single source IP showing both a suspicious event-side indicator and a high-volume outbound flow — and picked Event Rule in the Wizard's first screen because that was the same rule type the team's one existing worked example (*R: Suspicious LSASS Access – Unapproved Process*, an Event Rule per DEH Part 27 §3.3) already used, without checking whether a flow-volume Rule Test was even offered inside that catalog. **How it failed:** It wasn't. Event Rules evaluate the *events* table only; the flow-side bytes-transferred Rule Test category the author needed simply never appeared in the list, so the "flow" half of the intended condition was quietly dropped rather than flagged as missing. The rule deployed cleanly, matched on the event-side indicator alone, and produced Offenses named after a C2-beacon pattern with zero flow evidence attached to any of them — eight closures in before an analyst asked why a "candidate beacon" Offense never showed the flow tab populated. **The fix:** Rebuilt as a Common Rule, whose Rule Test catalog exposes the source-IP property shared by both streams, chained against the existing event-side Building Block and a new flow-volume condition backed by a Reference Set of known-owned destination networks (§2.2's pattern). Re-validated against a simulated beacon (a test host generating both the event-side indicator and a matching high-volume outbound flow) before redeployment, confirming both halves of the condition actually contributed before trusting the Offense name again.

8. Governance consequence: one more object type to review

[SOC MANAGEMENT] DEH Part 27's closing SOC MANAGEMENT paragraph named the three-object review burden a QRadar rule set imposes relative to a single-file Sigma or native query — a Building Block, a Reference Set, and a Rule, each separately versioned and separately permissioned. This part's catalog adds a dimension to that review burden that isn't about counting objects but about verifying a category choice: a reviewer checking a new Rule needs to confirm not just that its Rule Tests and Response are correct, but that its rule *type* actually matches the data shape the author intended — the exact gap §7's Rule Autopsy shows slipping past a review that only checked the Rule Test logic and never asked whether an Event Rule could see the flow evidence the name promised. Budgeting review time for "one new rule" should account for that category check as a distinct line item, not assume it's implied by reading the Rule Test list — a rule that tests exactly what its Wizard-limited catalog let it test can still look complete to a reviewer who doesn't independently ask what the author actually meant to correlate.

Cross-references: DEH Part 27 §1.1, §1.2, §2.1, §2.2, §2.3, §3.1, §3.3, §4 (AQL fundamentals, CRE/Building Block/Reference Set mechanics, DET-27-01/DET-27-02 worked examples); DEH Part 23 §1, §5 (DET-23-01 origin; Sigma-first-vs-native authoring tradeoff, not re-argued here); this book's Part 7 (flow collection architecture), Part 9 (Building Block governance at scale), Part 10 (the full reference-data type taxonomy), Part 11 (thresholds, Anomaly, and Behavioral rules in operational depth), Part 15 (Noisy-Offense tuning workflow), Part 16 (rule change management), Part 19 (App Framework and UBA-class apps), Appendix A2 (Rule Wizard and Rule Test quick reference).

neshbooy

Part 9 — Building Blocks: Design Patterns and Governance at Scale

Why this part exists

[CONCEPT] DEH Part 27 §2.1 introduces the Building Block for exactly one purpose: to prove that a QRadar Rule can share a named, reusable Rule Test grouping instead of re-specifying the same condition in every Rule that needs it, using a single worked example (`BB:LSASS-Access-Candidate`, carried forward in this book's own worked example below). That is the right amount of Building Block theory for a part whose job is comparing six query languages. It is not enough Building Block practice for a team running a QRadar deployment that has been live for three years, has forty rule authors who have rotated through the team, and has a `BB:` library that has grown — quietly, one reasonable one-off decision at a time — past four hundred entries, most of them unreviewed since the day they were created.

This part is about that library, not about how to build one Building Block. It covers naming and taxonomy patterns that keep a `BB:` library navigable past the point where any one person can hold its contents in their head; where sprawl and drift actually come from at scale, and how to detect both before a Noisy Offense Trap or a silent detection gap forces the discovery; and the change-control gap this book has to name honestly rather than paper over — QRadar's console-native authoring surface does not hand a team DEH Part 22's git-based, pull-request-gated, three-role-separation-of-duties pipeline for free, and a disciplined team has to build a compensating discipline instead. It does not re-teach Rule Wizard mechanics (Part 8 owns the full Rule Test catalog) or the reference-data type taxonomy a Building Block often works alongside (Part 10 owns Reference Set/Map/Table selection). It does not own the day-to-day tuning-intake workflow a flooding Building Block eventually feeds into (Part 15) or the ticket-level change-management process a single rule change should follow (Part 16) — this part is about the library those processes operate against, and the governance discipline that keeps the library itself from becoming the source of the next incident.

1. A Building Block library is a dependency graph, not a list

[CONCEPT] A Building Block is a named Rule Test grouping — the same authoring mechanism as a Rule's own conditions, saved under a `BB:`-prefixed name so it can be referenced by name instead of rebuilt (DEH Part 27 §2.1). What that section's single example does not have room to show is that a Building Block's Rule Test can itself include the condition "when the event matches any/all of the following Building Blocks" — a Building Block can reference another Building Block. At the scale of a handful of custom Building Blocks sitting alongside QRadar's own shipped content, this nesting is trivial: everyone remembers that `BB:LSASS-Access-Candidate` feeds `R: Suspicious LSASS Access – Unapproved Process` and nothing else does. At the scale of hundreds, it is a real dependency graph with real failure modes a flat list hides: a single foundational Building Block quietly referenced by forty Rules, where an edit intended to narrow one Rule's false-positive rate silently reshapes the other thirty-nine; a Building Block that references a second Building Block that has since been disabled; a Building Block with zero

remaining Rule references at all, sitting in the library as dead weight nobody has confirmed is safe to remove.

Figure 9.1 makes this structure explicit. The point is not the specific shape drawn — it is that a **BB:** library big enough to matter has a topology, and "how many Rules does this Building Block feed, directly or through another Building Block" is a question a team needs to be able to answer before editing anything, not after an edit has already gone live.

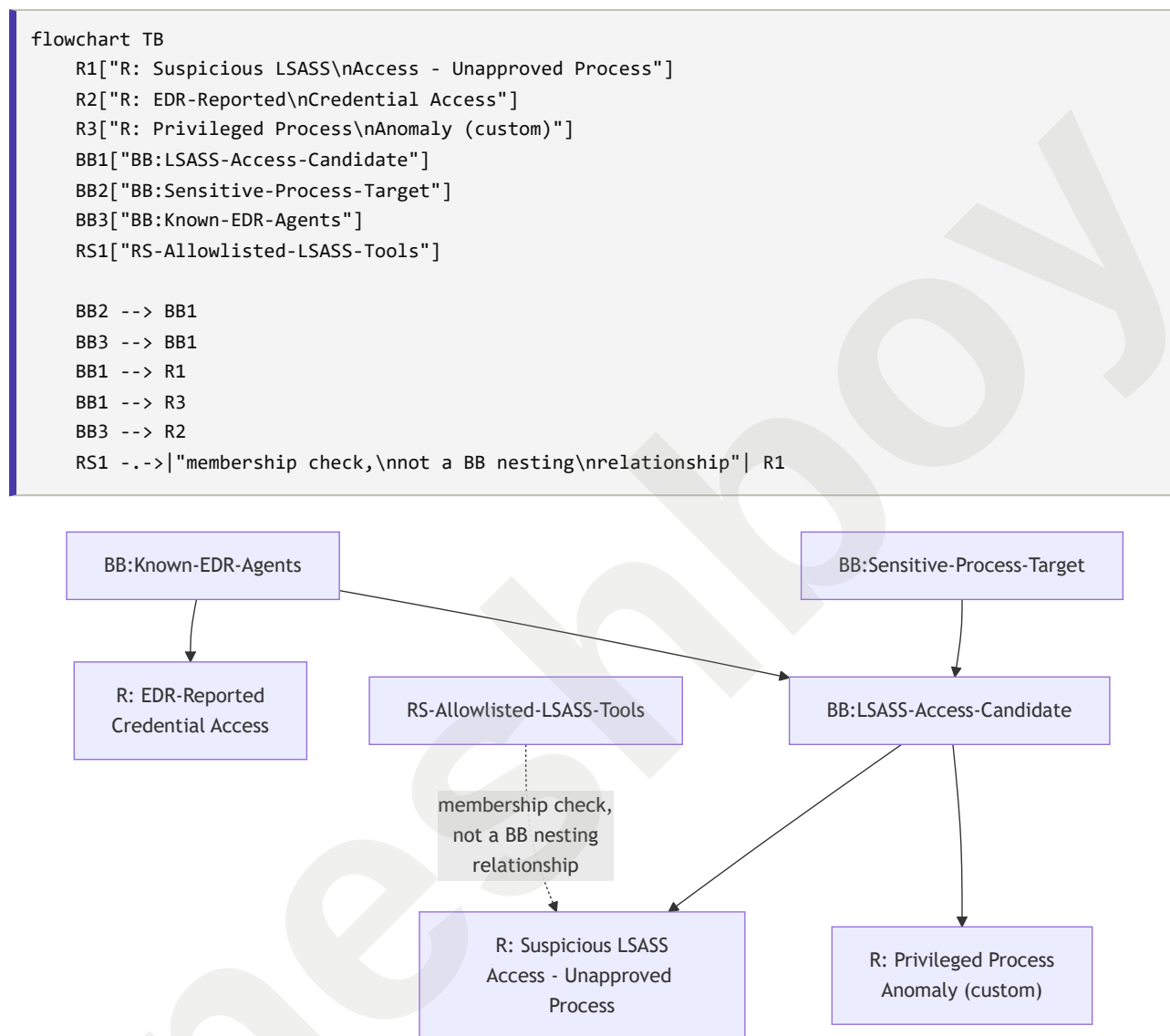


Figure 9.1 (FIG-09-01) — A Building Block library as a dependency graph. CONCEPTUAL. Illustrates the two ways sprawl compounds at scale: a foundational Building Block (**BB:LSASS-Access-Candidate**) that several Rules and a second-order Building Block all depend on, and a Reference Set consulted by a Rule directly rather than through Building Block nesting — the same distinction DEH Part 27 §2.2 draws between Building Block reuse and Reference Set state. Not a capture of any real rule repository or dependency-visualization tool; this book has no lab access to produce that capture (STYLE-GUIDE.md §9).

[RULE ENGINEER] The practical consequence: before editing any Building Block that has existed longer than the most recent onboarding cycle, the first question is not "what does this condition currently say"

but "who depends on it, directly and through another Building Block" — §3 covers how to answer that question without a native dependency-visualization tool guaranteed to exist in your specific deployment.

2. Naming and taxonomy patterns that survive scale

[RULE ENGINEER] QRadar ships with its own out-of-box Building Block library organized, in broad strokes, around a small set of functional categories — Building Blocks that define a category of events, a category of hosts, a category of network segments, a category of ports, a category of users, or a known-false-positive condition, each intended to be referenced by many downstream Rules rather than rebuilt per Rule. A custom **BB:** library that grows past the first few dozen entries benefits from adopting the same functional-category discipline for its own additions, rather than letting every new Building Block get whatever name felt clearest to the analyst who wrote it that week.

PRODUCT VERSION NOTE *The exact prefix strings, punctuation, and grouping QRadar's out-of-box content uses for its own categorical Building Blocks (definitional patterns along the lines of a category, host, network, port, user, or device grouping) are believed stable in spirit across recent releases but have not been verified against a live console for this book (STYLE-GUIDE.md §0/§9 — no lab access). Treat the functional taxonomy in Table 9.1 below as this book's own recommended convention for a custom library, informed by that shape, not as a literal transcript of IBM's current out-of-box naming — verify your own deployment's shipped Building Block names in Admin before assuming an exact string match.*

Table 9.1 states that convention as a decision aid: which functional bucket a new custom Building Block belongs in, and what naming pattern keeps that bucket searchable once the library holds hundreds of entries across multiple bucket types.

Table 9.1 — A functional taxonomy for a custom **BB: library.** Supports the decision of which bucket a new Building Block belongs in and what its name should communicate before a second author has to guess.

Category	Purpose	Example name	Typically reused by
Candidate-match	Defines the raw pattern a downstream Rule tests against, before any allowlist exclusion is applied	BB:LSASS-Access-Candidate	One or a small number of closely related Rules
Definitional / scope	Names a reusable set of hosts, networks, ports, users, or device types other logic references by name instead of by literal value	BB:Domain-Controllers	Many Rules and other Building Blocks across unrelated analytics
Exclusion / allowlist-adjacent	Narrows a candidate-match condition against a known-benign source — distinct from a Reference Set (Part 10) when the exclusion is a fixed logical condition rather than a maintained membership list	BB:Known-Backup-Agent-Behavior	The specific candidate-match Building Block it's paired with
Category rollup	Groups several QID (QRadar's internal event-taxonomy identifier) or event-category values under one reusable label, standing in for a repeated CATEGORYNAME() -style condition	BB:CategoryDefinition-Auth-Failures	Any Rule needing a broad authentication-failure signal
Deprecated / retiring	Marks a Building Block scheduled for removal, with the removal date and replacement named in the description field, not just implied by the name	BB:LSASS-Access-Candidate-DEPRECATED-2026Q4	Nothing, by design — see §5's deprecation policy

[RULE ENGINEER] Two naming disciplines matter more than the specific taxonomy chosen: first, a Building Block's name should say what it *matches*, not what *Rule* it was built for — BB:LSASS-Access-Candidate survives being reused by a second Rule cleanly; a name like BB:Rule14-Condition2 does not, and is itself a sprawl symptom (§3). Second, every custom Building Block's description field — not just its name — should state its functional category, its intended reuse scope, and an owner, because the name alone runs out of room fast once a library holds several hundred entries across five categories.

3. Where sprawl actually comes from, and how to catch it before it costs you

[PLATFORM ENGINEER] A BB: library rarely sprawls from one bad decision. It sprawls from many individually reasonable ones, each made under time pressure by an author who did not have visibility into the rest of the library:

- **Building-Block-per-Rule.** An analyst under deadline pressure builds a new Building Block for a condition that's 90% identical to an existing one, because searching the existing several hundred entries for a near-match takes longer than writing a new one from scratch. The library gains a near-duplicate; neither copy gets the benefit of the other's future tuning.
- **Orphaned Building Blocks.** A Rule gets retired or rewritten, and the Building Block it depended on — built specifically for that Rule and referenced nowhere else — is left in place because removing it wasn't part of the Rule-retirement checklist. It sits in the library indefinitely, consuming review attention every time someone audits "what does this library actually contain" without contributing to any live detection.
- **Silent breakage on rename or duplication.** §1's dependency graph is exactly what makes this failure mode possible: an author "cleans up" an old Building Block by duplicating it under a

corrected name and deleting the original, intending the swap to be transparent to every dependent Rule. Whether that swap is actually transparent depends on how QRadar's Rule Wizard internally resolves a Building Block reference — and that is exactly the kind of implementation detail this book will not assert as fact without a source (see the PRODUCT VERSION NOTE below, and the worked failure in §3.1's Rule Autopsy).

- **Unclear ownership.** A Building Block written by an analyst who has since left the team, with no owner field and no description beyond its name, becomes something nobody wants to touch — not because it's known to be load-bearing, but because nobody can rule that out quickly. The safe-feeling response, leaving it alone indefinitely, is how a library accumulates permanent uncertainty instead of resolving it.

PRODUCT VERSION NOTE *Whether your specific QRadar release exposes a built-in "where is this Building Block referenced" or dependency view from inside the Rule Wizard or the Building Block editor is not a claim this book makes either way — it has shifted across releases and this book has no lab access to confirm current behavior. If your deployment does not expose one, the compensating control is a periodic content export (§4's REST API pattern) parsed for cross-references by name, run before any Building Block is edited or deleted, not after.*

[RULE ENGINEER]

Absent a guaranteed native dependency view, the audit pattern that catches all four sprawl causes above without one is the same in each case: export the full Rule and Building Block catalog on a fixed schedule (§4), parse it for which Building Block names appear inside which Rule's and other Building Block's Rule Test definitions, and flag three conditions — a Building Block with zero referencing Rules or Building Blocks (orphan candidate), a Building Block whose logic is a near-exact duplicate of another's (consolidation candidate), and a Building Block referenced by an unusually high count of Rules relative to the library's median (a de facto foundational dependency that any edit needs extra review before touching, regardless of how small the edit looks).

BB:LSASS-Access-Candidate — anatomy of a well-governed Building Block

[RULE ENGINEER]

This book does not re-derive **BB:LSASS-Access-Candidate**'s matching logic — DEH Part 27 §3.3 already defines it as the Rule Test equivalent of DET-27-01's candidate-match condition (QID resolves to the Sysmon process-access category, target process name ends **lsass.exe**, granted access matches the memory-read-capable value set), feeding **R: Suspicious LSASS Access – Unapproved Process** alongside **RS-Allowlisted-LSASS-Tools**. What this part adds is the governance metadata a well-run library attaches to that same object, none of which QRadar's own description field is guaranteed to structure for you — so a disciplined team adopts a fixed convention and writes it into every Building Block's free-text description consistently:

- **Category:** Candidate-match (Table 9.1).
- **Owner:** the individual or team accountable for reviewing this Building Block, not the person who happened to author it three years ago and may no longer be on the team.
- **Known dependents:** **R: Suspicious LSASS Access – Unapproved Process** — kept current by the audit pattern in §3, not by memory.

- **Last reviewed:** a date, bumped every time the audit in §3 confirms this Building Block's logic and dependents are still accurate — the same discipline DEH Part 22 §4 requires of a `Required Fields/ Known FP` metadata block for a Detection Rule, adapted to an object QRadar's own console does not give a dedicated structured field for.

Rule Autopsy The Building Block: `BB:LSASS-Access-Candidate`, feeding `R: Suspicious LSASS Access – Unapproved Process` exactly as DEH Part 27 §3.3 wires it. **Why it shipped:** A quarterly naming-cleanup pass (§2) renamed the Building Block by duplicating it under a corrected, standardized name and disabling the original — a "safe-looking" two-step migration, not a reckless one — on the assumption that the dependent Rule's Building Block condition would carry over automatically. **How it failed:** The dependent Rule's Rule Test had been wired to the specific Building Block object the original author selected in the Rule Wizard at authoring time. Whether QRadar resolves that reference by a stable internal object identifier or by display name at evaluation time is exactly the kind of implementation detail this book flags rather than asserts (see the *PRODUCT VERSION NOTE* above) — and it is precisely the detail that determines whether a duplicate-then-disable migration like this one is transparent or silently breaks the link. In this scenario, it broke the link: the Rule stayed enabled, logged no error, and simply stopped matching anything, indistinguishable from a quiet week for `Lsass.exe` activity across the whole fleet. **The fix:** Treat "duplicate the object, then disable or delete the original" as a two-step migration that is not complete until every dependent Rule and Building Block identified by the §3 audit has been individually re-verified against the new object — not assumed to follow automatically — and prefer an in-place rename of the existing object over a duplicate-and-delete migration whenever the change is naming-only and no logic actually needs to change.

4. Change control without a git-based pipeline

[PLATFORM ENGINEER] DEH Part 22 describes a rule repository where the console is a deployment target, not an editing surface: a protected branch, a pull request per change, automated lint and regression testing that blocks the merge button, and a structural three-role separation — author, reviewer, merge-approver — enforced by branch protection settings rather than trusted to memory. None of that exists by default for a Building Block edited directly in QRadar's own console. A user with Rule/Building Block edit permission opens the editor, changes a condition, and saves — live, immediately, with no required second reviewer, no CI gate, and no built-in mechanism forcing a record of what the object looked like five minutes earlier. This is not a QRadar defect distinct from every other console-native platform's default behavior; DEH Part 22 §1's own Engineering Reality names the identical gap for detection consoles generally. It is, however, a gap this book states plainly rather than implying a Building Block library gets DEH Part 22's guarantees for free just because the underlying authoring mechanism (a named, reusable, versionable logical object) looks similar on the page.

PRODUCT VERSION NOTE *This part states plainly that no built-in second-reviewer approval gate and no automatic revision history exist for a Building Block edit made directly in the console. That absence is stated as this book's working assumption, not a claim verified against every current release — this book has no lab access (STYLE-GUIDE.md §0/§9), and IBM has added platform-management capabilities (content/extensions export-import tooling, and administrative auditing features) over successive releases that could narrow this gap in ways not reflected here. Verify what your own deployment's specific release actually exposes before assuming the fully manual compensating workflow below is the only option available to you.*

[PLATFORM ENGINEER] What a disciplined team substitutes, mapped against the three roles DEH Part 22 §3 makes structural:

- **In place of a pull request:** a change ticket in whatever system of record the team already uses for infrastructure changes, opened before the console edit is made, naming the specific Building Block by its `BB:` name, the current logic (captured from the export in the pattern below, not from memory), the proposed change, and the reason — the same fields DEH Part 22 §4's metadata standard requires of a Detection Rule, filled in as ticket text instead of a YAML front-matter block, because the console gives no equivalent structured field.
- **In place of CI lint/test gates:** a manual pre-change step that runs the §3 dependency audit specifically against the Building Block being changed, and a manual post-change validation against a known-positive test event or saved search — the same intent as DEH Part 22 §2's automated regression check, performed by a human because the platform will not gate the save button on it.
- **In place of branch-protection-enforced separation of duties:** a two-person rule at the process level — the proposer opens the ticket and stages the exact before/after text, a second, named individual reviews the ticket and only then makes (or explicitly authorizes) the console edit — accepting, as DEH Part 22 §3's own SOC Management View names for a two-person detection team, that a console with no native reviewer-cannot-also-be-merger toggle makes this a staffing discipline to enforce, not a setting to turn on.
- **In place of `git log` as the audit trail:** a scheduled export of the full Rule and Building Block catalog into an external version-controlled store — not a live gate, since nothing blocks a console edit from happening outside this export cycle, but a genuine after-the-fact diff and rollback aid once it exists, and the same raw data the §3 dependency audit runs against.

```
# CONCEPTUAL SAMPLE – illustrative shadow-version-control export, not a validated
# call against a live deployment (STYLE-GUIDE.md §0/§9 – no lab access). Confirm
# the exact endpoint path, the required API "Version" header value, and the
# authentication scheme against your own deployment's REST API reference before
# reusing this unmodified – all three have changed across QRadar releases.
curl -s -H "SEC: $QRADAR_API_TOKEN" \
  -H "Version: 20.0" \
  "https://qradar.example.internal/api/analytics/rules?fields=id,name,type,owner,base_host_id" \
  -o rules_and_building_blocks_export.json
```

```
{
  "id": 4021,
  "name": "BB:LSASS-Access-Candidate",
  "type": "BUILDING_BLOCK",
  "owner": "detection-engineering-team",
  "base_host_id": 1
}
```

PRODUCT VERSION NOTE *The REST API request above is illustrative of the pattern — export the Rule/Building Block catalog on a schedule, diff it against the last export, feed the diff into an external, version-controlled store — not a literal, tested call. The exact endpoint path, the required `Version` header value, field names in the response body, and whether Building Blocks are exposed through the same endpoint as ordinary Rules (distinguished only by a type field) or a separate one entirely, are all specifics that have shifted across QRadar releases and API versions. Verify against your own deployment's in-console REST API reference before building automation on top of this pattern.*

```
flowchart LR
    subgraph GIT["DEH Part 22: git-based pipeline"]
        A1["Author opens PR"] --> A2["CI lint + regression\ntests gate the merge"]
        A2 --> A3["Reviewer:\ntechical-adversarial review"]
        A3 --> A4["Merge-approver merges\nto protected main"]
        A4 --> A5["main branch IS\nthe deployed state"]
    end
    subgraph QRADAR["This part: console-native compensating workflow"]
        B1["Proposer opens change\nticket, stages before/after text"] --> B2["Manual dependency audit\n(3) + manual test validation\n(4) against the specific object"]
        B2 --> B3["Named second reviewer\napproves the ticket"]
        B3 --> B4["Console edit made\n(or authorized) by reviewer\nor a distinct third party"]
        B4 --> B5["Scheduled export captures\nnew state into external VCS\n(after the fact, not a gate)"]
    end
```

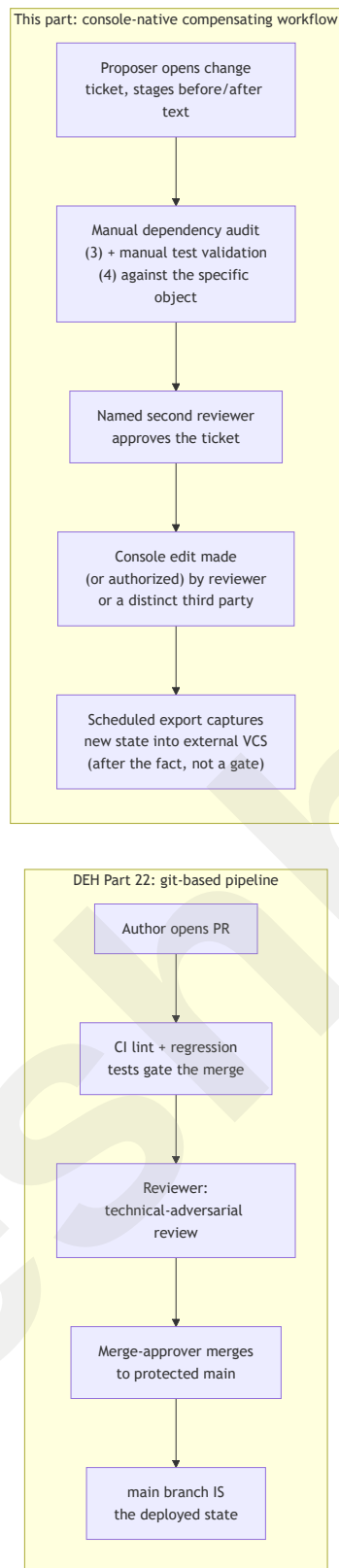


Figure 9.2 (FIG-09-02) — A git-based pipeline's structural gate versus a console-native compensating workflow's process gate. *CONCEPTUAL*. Illustrates the same intent — no single person authors, reviews, and ships a change unilaterally — achieved by two different mechanisms: DEH Part 22's branch-protection-enforced structural gate on the left, where the merge button is literally unavailable until conditions are met, versus this part's process-level compensating workflow on the

right, where every step depends on staffing discipline rather than a setting the platform enforces. Not a capture of any real ticketing system or QRadar console.

[PLATFORM ENGINEER] The honest limitation Figure 9.2 is drawn to surface, not hide: the right-hand workflow is a discipline, not a control. Nothing in QRadar's own console stops the proposer from skipping the ticket and editing the Building Block directly — the compensating workflow's only enforcement mechanism is restricting console edit permissions to people who have agreed to follow it, and auditing console-side changes against the last export to catch the ones that didn't (the same gap DEH Part 22 §1's own Engineering Reality names for a git-managed rule repository generally, applied here to a Building Block library specifically). Part 16 covers this ticket-to-rollback process in full for a single rule change; this section states the version of it a **BB:** library's dependency structure (§1) specifically requires — a change to a foundational Building Block needs the same two-person review a change to a single-Rule Building Block does, but the audit step beforehand has to check every dependent, not just the one Rule the proposer had in mind.

5. Governance roles and review cadence at scale

[SOC MANAGEMENT] DEH Part 27's own closing paragraph names a review-cost consequence specific to QRadar's multi-object model: reviewing "one detection" here means reviewing a Building Block, a Reference Set's maintained membership, and a Rule's chained logic as three separately-changeable artifacts, not one file. A **BB:** library of a few dozen custom entries makes that cost tolerable through informal team memory. A library of several hundred does not — and the review-cost multiplication compounds specifically because of §1's dependency graph: reviewing a change to a foundational Building Block referenced by forty Rules is not one detection's review cost, it is potentially forty.

SOC Management View *A team sized for "review each new detection as it ships" runs out of capacity the moment a **BB:** library crosses from a flat handful of entries into the hundreds, because the review unit stops being one Rule and starts being a dependency graph with unknown blast radius per change — exactly the cost this part's §1 and §4 name mechanically. Budget for a named Building Block library owner (a role, not necessarily a dedicated headcount line) whose job includes running the §3 audit on a fixed cadence, maintaining the ownership/last-reviewed metadata §3.1 recommends, and holding authority to block a change to a high-fan-in Building Block until its full dependent list has been re-verified — not just the one Rule the requester had in mind. A team that skips this role does not avoid the cost; it defers it to the day a foundational Building Block edit breaks silently across every Rule that depended on it, discovered only when offense volume drops and someone finally asks why.*

[SOC MANAGEMENT] A minimum viable cadence for a library past a few hundred entries: a quarterly pass running the §3 audit in full (orphan detection, near-duplicate detection, fan-in outlier detection), a rolling deprecation policy — a Building Block marked **-DEPRECATED-<date>** per Table 9.1's convention is removed after a stated grace period with zero new dependents added during that window, not left in the library indefinitely out of caution — and an ownership review confirming every Building Block's

assigned owner is still on the team, reassigning the orphaned ones rather than letting "unclear ownership" (§3) silently regenerate itself one departure at a time.

6. A minimal governance checklist

[RULE ENGINEER] Table 9.2 collects this part's recommendations into the form a Building Block library owner can actually run against a single object during a change review, rather than holding the whole part in mind at once.

Table 9.2 — Building Block governance checklist, applied per change. Supports a reviewer's go/no-go decision on a proposed Building Block edit, rename, or deletion.

Check	Confirms	Owning section
Name states what it matches, not which Rule it was built for	Reusability signal is legible to the next author	§2
Category and owner recorded in the description field	Findable during an audit without relying on memory	§2, §3.1
Dependency export run against this specific object before editing	Full list of dependent Rules and Building Blocks is current, not assumed	§1, §3
Near-duplicate check run against the rest of the library	Change isn't solving a problem an existing Building Block already solves	§3
Change ticket opened with before/after text, before the console edit	Compensates for the missing PR gate	§4
Named second reviewer approved before the edit was made	Compensates for the missing branch-protection separation of duties	§4
Post-change validation run against a known-positive test event	Compensates for the missing automated regression gate	§4
Export captured after the change, diffed into external VCS	Audit trail exists for this change going forward	§4
If renamed or duplicated: every dependent individually re-verified	Closes the exact failure mode named in §3.1's Rule Autopsy	§3, §3.1

Cross-references: DEH Part 27 §2.1 (Building Blocks and Rule Tests) and §3.3 ([BB:LSASS-Access-Candidate](#) worked example, carried forward unchanged in §3.1 above); DEH Part 22 (Detection-as-Code: the git-based pipeline and three-role separation of duties this part's §4 builds a compensating workflow against); DEH Part 23 (Query Language Strategy — not re-argued here); this book's Part 8 (Rule Wizard Catalog), Part 10 (Reference Data Collections), Part 15 (Noisy-Offense Triage and the Tuning Workflow), Part 16 (Rule Change Management Without a Detection-as-Code Pipeline), and Part 22 (Staffing and Running a QRadar Practice).

Part 10 — Reference Data Collections: Sets, Maps, Map of Sets, Map of Maps, and Tables

Why this part exists

[CONCEPT] DEH Part 27 introduced exactly one reference-data object — the Reference Set — for two purposes, in two different sections: §2.2's generic illustrative example, giving a two-stage correlation pattern (`RS-Stage1-Seen`) a place to persist state between an event's first-stage match and a second, later event's evaluation; and §3.3's worked DET-27-01 case study, giving `R: Suspicious LSASS Access – Unapproved Process` a maintained allowlist (`RS-Allowlisted-LSASS-Tools`) to check membership against. That part was explicit about scope: it needed one worked example to prove that QRadar's Reference Set is the platform's structural answer to what a KQL `join` or an SPL `transaction` expresses natively, not a full survey of the reference-data type catalog.

This part is that survey. A Reference Set is the simplest member of a five-shape taxonomy — Set, Map, Map of Sets, Map of Maps, and Table — and picking the wrong shape for a given correlation question either forces awkward workarounds (cramming a key-value relationship into a bare Set by concatenating the key and value into one string) or silently fails to answer the question at all (checking "is this IP anywhere in the set" when the real question was "is this IP associated with *this specific user's* recent history"). It also covers the maintenance dimension DEH Part 27 §2.2 flagged once and moved past: time-to-live (TTL) management, and the three distinct paths — Rule Response, the REST API, and a bulk console import — a piece of reference data can be populated through, each with its own drift risk when more than one of them touches the same object without coordination.

This part does not re-teach AQL syntax or the Custom Rules Engine's basic Rule Test/Building Block mechanics; DEH Part 27 §1 and §2.1 own that, and this book's Part 8 (Rule Wizard catalog) and Part 9 (Building Block governance) own the adjacent authoring-surface ground. What follows assumes a reader already comfortable with "a Rule Test can check whether a property is contained in a Reference Set" and wants the rest of the typed reference-data picture DEH Part 27 gestures at — "Reference Set (or its typed relatives — Reference Map, Reference Table)" — but never expands.

1. From one Reference Set to a typed taxonomy

[CONCEPT] Every reference-data object in QRadar solves the same architectural problem: the Custom Rules Engine (CRE) evaluates each incoming event or flow against a Rule's conditions as it streams in, with no native mechanism for holding an arbitrary multi-event join open across an unbounded time window the way a KQL `summarize` or an SPL `transaction` can inside one query. DEH Part 27 §2.2's Blind Spot already names the consequence: state that needs to survive from one event to a later, differently-shaped event has to live somewhere the CRE can write to and read from outside any single query's time bound. Reference data is that somewhere — a named, persistent collection a Rule Response can add to or remove from, and a Rule Test can check against, independent of Ariel's own retention buckets (Part 12) or any AQL search's `LAST n HOURS` clause.

A bare Reference Set answers exactly one kind of question well: "is this value present, yes or no." The moment a correlation question needs a second dimension — "is this value present *for this specific key*," "what set of values has accumulated under this key," "what value is stored two keys deep" — a Set alone forces an awkward encoding (string-concatenating a username and a source IP into one Set entry, then parsing it back apart in a Rule Test) that a properly typed collection avoids by construction. QRadar's reference-data catalog covers that range directly:

Type	Structure	Correlation question it answers	Illustrative object
Reference Set	Unordered collection of single values	Is this value present in the set?	RS-Allowlisted-LSASS-Tools
Reference Map	One key maps to one value	What single value is associated with this key?	RM-Asset-To-Owner
Reference Map of Sets	One key maps to a set of values	What set of values has accumulated under this key? Is this specific value already in that key's set?	RMOS-User-To-Source-IPs
Reference Map of Maps	One key maps to a second key, which maps to a value	For this first key, what value is stored under this second key?	RMOM-Host-ProcessBaseline
Reference Table	Keyed row with one or more named, typed inner keys ("columns") holding a value	What multi-column record is associated with this key?	RT-Asset-Inventory

PRODUCT VERSION NOTE *The five rows above name the shapes this book's correlation examples actually need, not a guaranteed one-to-one match to a specific QRadar release's Admin console labels or REST API wording. The reference-data type catalog itself (Set, Map, Map of Sets, Map of Maps, Table) is a stable architectural feature of the platform rather than something that has gained or lost a member across recent releases, but the Admin console's exact menu labels for each type have not stayed constant across versions. Check Admin > Reference Set Management (or your version's equivalent menu) for the exact wording your console uses before assuming any specific label from this table exists verbatim in your deployment.*

flowchart TB

```

subgraph Population["Three population paths – Section 6"]
    RR["Rule Response\n(a firing Rule writes\nthe entry itself)"]
    API["REST API\n(threat intel, CMDB,\nSOAR playbook)"]
    BULK["Bulk import\n(Admin console,\nhuman-curated)"]
end

RR --> RD
API --> RD
BULK --> RD

RD["Reference data object\nSet / Map / Map of Sets /\nMap of Maps / Table\n– Section 1"]
RD -->|"Rule Test:\nmembership or lookup"| CRE["Custom Rules Engine\nreal-time evaluation"]
RD -->|"ad hoc lookup\nSection 8"| AQL["Analyst / hunter\nAQL search"]
RD -->|"TTL – Section 5"| EXP["Entry ages out,\nor never does\nif no TTL is set"]
CRE --> OFF["Offense created\nor updated"]

```

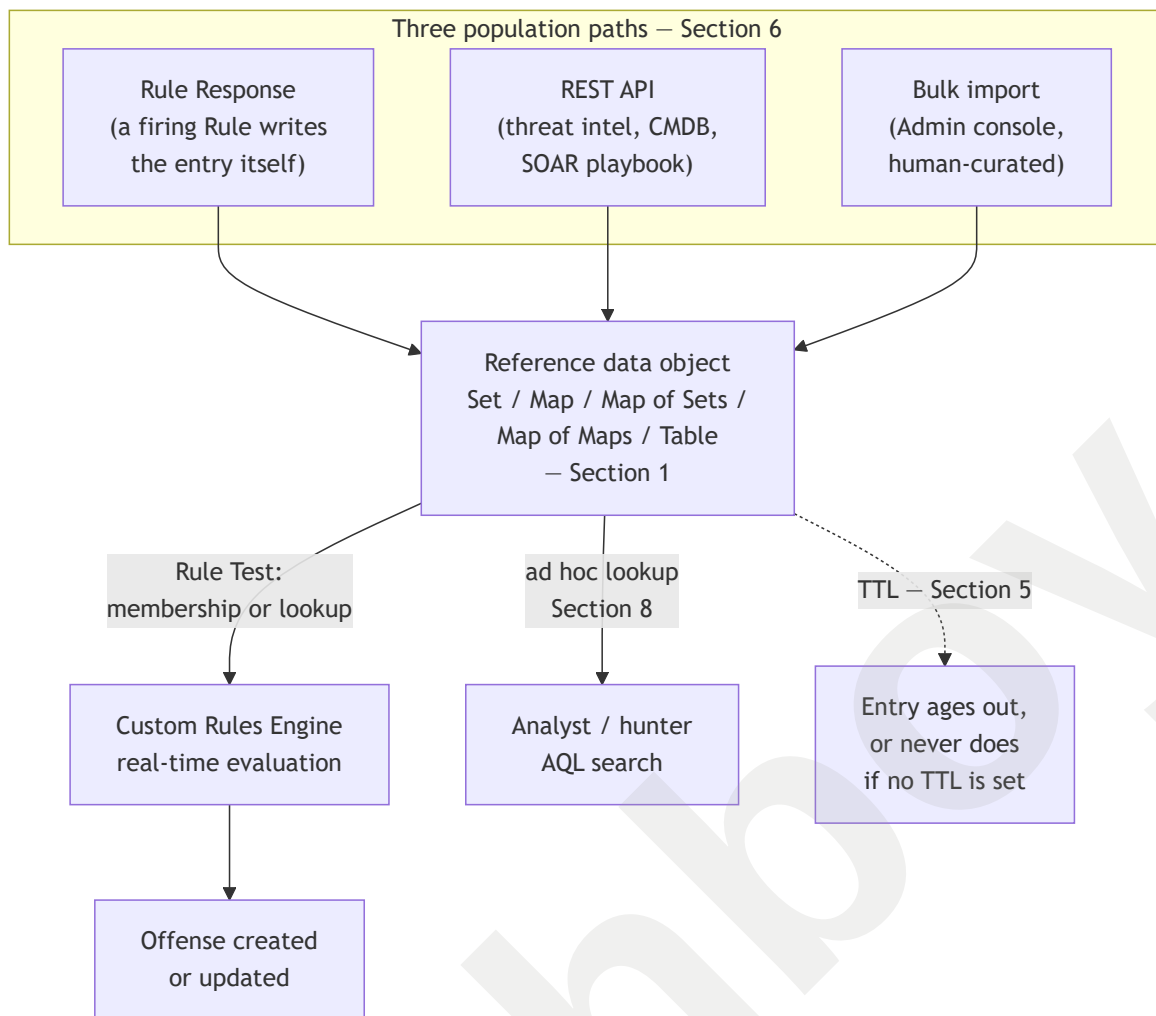


Figure 10.1 — Reference data as the CRE's shared, mutable state layer. *CONCEPTUAL.* Illustrates how any of the five collection types in the table above sits between three independent population paths and two independent consumers — real-time Rule Test evaluation and ad hoc AQL lookup — with TTL as the one expiry mechanism any of them has, or lacks if never configured. Not a reproduction of any IBM architecture diagram; a structural summary of this part's own §1, §5, §6, and §8 read together. Compare Figure 27.1 in DEH Part 27 for the narrower single-Reference-Set version of the same left-hand population/right-hand consumption pattern.

The rest of this part walks the taxonomy from simplest to most structured, then covers the two dimensions every type shares regardless of shape: TTL/expiry (§5) and population path (§6).

2. Reference Set: the single-value collection, generalized beyond the allowlist

[RULE ENGINEER] A Reference Set is an unordered collection of single, independent values — no key, no structure beyond "is this value a member." `RS-Allowlisted-LSASS-Tools` is one instance of a broader pattern that recurs constantly once you stop thinking of a Reference Set as "the allowlist object" and start thinking of it as "the cheapest possible persistent membership test the CRE has." Common shapes beyond an allowlist:

- **A denylist fed by an external feed** — a Reference Set of known-malicious source IPs, populated by a threat-intel integration, checked by a Rule Test on every inbound connection event. This is not a hypothetical integration shape: IBM's own public `qradar-misp-ioc-importer` app does exactly this, polling a MISP threat-intelligence instance on a configurable interval and writing matching indicator types (IPs, domains, URLs) into a named QRadar Reference Set (IBM, "qradar-misp-ioc-importer," GitHub, 2023: <https://github.com/IBM/qradar-misp-ioc-importer>).
- **A short-lived "seen recently" marker** — `RS-Stage1-Seen` from DEH Part 27 §2.2, holding whatever a first-stage Rule's Response wrote to it until a second-stage Rule Test checks membership or the entry's TTL expires.
- **A scope-narrowing set for a broad Rule** — Part 8's Rule Wizard catalog covers this book's threshold-rule generalization of DET-27-02 (repeated authentication failure); a Reference Set of `RS-VIP-Usernames` lets a single Rule's Response branch its severity or routing without duplicating the whole Rule for one privileged-account carve-out.

RS-Recent-Failed-MFA-Sources — a Reference Set with a maintenance lesson

[RULE ENGINEER] Consider a Reference Set built to catch MFA fatigue/push-bombing: a Rule's Response adds the source IP to `RS-Recent-Failed-MFA-Sources` on every failed MFA challenge, and a second Rule fires an Offense when that same source IP accumulates five failed challenges in the set within a short window, distinguishing genuine push-bombing from an isolated fat-fingered approval. The object works exactly as designed for the first several weeks. Six months later, nobody remembers this Reference Set exists, its TTL was never configured, and it has quietly accumulated every source IP that has ever failed an MFA challenge — legitimate typo-driven failures included — going back to the day it was created. §5 below covers why that specific failure mode (a Reference Set with no TTL, still doing exactly what it was built to do, just against a stale and unbounded population) is this part's Noisy Offense Trap, not a hypothetical.

3. Reference Map and Reference Map of Sets: when membership alone isn't enough

[RULE ENGINEER] A Reference Map associates one key with exactly one value — a lookup, not a membership test. Where a Reference Set answers "is X present," a Reference Map answers "what is the value associated with X."

RM-Asset-To-Owner — a Reference Map anatomy

[RULE ENGINEER] `RM-Asset-To-Owner` maps an asset identifier (hostname or IP, depending on which key your asset model — Part 20 — uses consistently) to a single value: the email address or ticketing-queue identifier of the team responsible for that asset. A Rule Response on a fired Offense can look up the contributing asset in this Map and route a notification directly, without the Rule itself needing to encode organizational ownership as CRE logic. The map's value is exactly as current as whoever maintains it — the same staleness risk Part 20 names for the asset model generally, inherited here rather than solved by the Map's own mechanics.

A Reference Map of Sets extends this one step further: instead of one value per key, each key accumulates a *set* of values, and a Rule Test can check either "does this key have any accumulated values at all" or "is this specific value already in this key's set."

RMOS-User-To-Source-IPs — impossible-travel via Map of Sets

[RULE ENGINEER] `RMOS-User-To-Source-IPs` maps each username to the set of source IPs (or, more usefully, the set of resolved network-hierarchy segments or geolocated regions) that username has authenticated from recently. A Rule Response adds each new authenticating source to the set on every successful login. A second Rule Test checks: is the source on *this* login event new to this username's accumulated set — and, if the population feeding the Map also carries a rough geolocation, is the new source geographically implausible given the most recent prior entry's timestamp. This is the shape of an impossible-travel heuristic for **T1078 (Valid Accounts)** built entirely from native reference-data mechanics, with no external UEBA integration required, though Part 19 covers where a dedicated UBA app does this more precisely once one is licensed.

A Reference Set alone cannot express this: a flat Set of "all source IPs anyone has ever logged in from" tells you nothing about *which user* a given IP belongs to. Cramming a `username|sourceip` concatenated string into a plain Set would technically work for the membership check but breaks the moment you need to enumerate everything associated with one user — exactly the operation a Map of Sets is built to do natively, and exactly the workaround §1 warned against.

Blind Spot A Map of Sets used for impossible-travel or new-source detection is only as good as its own bootstrap period. On the day `RMOS-User-To-Source-IPs` is first deployed, every user's set is empty, so the first login event for every account in the environment reads as "new source" — either a flood of low-value Offenses on day one if the Rule fires unconditionally on any new-source match, or a silent false negative for weeks if the team responds by suppressing new-source matches broadly rather than giving the Map a defined backfill/warm-up period. Populate the Map from a bulk import of each account's genuinely established recent history (§6) before enabling the Rule that reads it, the same discipline DEH Part 27's Detection Test requires before trusting any new correlation object's first-week output.

4. Reference Map of Maps and Reference Table: two-key and multi-column state

[RULE ENGINEER] A Reference Map of Maps adds a second level of keying: an outer key maps to an inner key, which maps to a value — two lookups deep, still resolved in one Rule Test evaluation rather than a chained pair of Rules. This is the right shape whenever the correlation question is naturally "for *this* entity, what do I know about *this specific sub-thing*," and a single flat key would either collide across entities or need to be reconstructed by string-concatenating two identifiers, the same workaround §3 flagged for Map of Sets.

RMOM-Host-ProcessBaseline — a Map of Maps anatomy

[RULE ENGINEER] `RMOM-Host-ProcessBaseline` maps a hostname (outer key) to a nested structure keyed by process image path (inner key), whose value is the timestamp that process was first observed executing on that host. A Rule Response on every process-creation event checks whether the inner key already exists for this host; if not, it adds the entry and, on that specific transition, fires a low-severity "new process observed on this host" Offense — a first-seen-per-host baseline, maintained natively, without exporting process telemetry to an external baselining system. The same object structure, read rather than written, lets a separate Rule Test suppress noise for genuinely routine software rollouts: a process that already has an entry under every host in a given asset group is evidently not new to that group, regardless of whether this specific host has seen it yet.

This is a heavier object than a Map of Sets in exactly the way its structure suggests — two keys deep means two lookups' worth of write-and-read cost on every evaluating event, and a host fleet numbering in the thousands with a process-diversity count in the hundreds per host is an outer-key-times-inner-key cardinality problem worth sizing before deployment, not after the Console's reference-data memory footprint becomes a Part 17 capacity conversation.

MITRE: a first-seen-process baseline of this kind is one practical, native-CRE approach to surfacing **T1059 (Command and Scripting Interpreter)** and living-off-the-land tooling generally — it does not name a single technique on its own, since "a process ran on this host for the first time" is a weak signal in isolation and needs the same allowlist/context discipline DEH Part 23's False Positive Trap already requires for any first-seen heuristic.

RT-Asset-Inventory — a Reference Table anatomy

[RULE ENGINEER] `RT-Asset-Inventory` uses a hostname or IP (outer key, matching whichever asset identifier Part 20's asset model uses consistently) plus a fixed set of named, typed inner keys — `owner`, `criticality`, and `last_patched` — each holding its own value for that asset: the owning team's identifier, a criticality tier, and the timestamp of the asset's most recent patch cycle. A Rule Test can pull any one column for a given asset (the `criticality` column feeding an Offense's severity weighting) without needing three separate Reference Maps kept in sync by hand, and a Rule Response updating one column (a patch-management integration writing `last_patched` on every completed cycle) never touches the others. This is the structural difference from a Map of Maps: a Map of Maps' inner key space is open-ended and per-entry (process image paths differ host to host, as `RMOM-Host-ProcessBaseline` above shows), where a Reference Table's inner keys are a fixed, named column set defined once when the table is created and shared by every outer key stored in it.

Ordered and sequence-tracking as a pattern, not a type

[RULE ENGINEER] Some correlation questions add a third dimension on top of "which key, which sub-key": *order*. DEH Part 27 §2.2's two-stage pattern (`RS-Stage1-Seen` gating a second Rule) already tracks a two-step sequence with nothing more than a plain Reference Set — stage 1 writes, stage 2 reads, and the ordering is implicit in "stage 2's Rule Test only fires after stage 1's Response has run." Extending that past two stages, or adding an explicit *within-order* constraint (stage 2 must follow stage 1, not merely coexist with it), does not call for a sixth object type — QRadar's reference-data catalog has no dedicated ordered- or sequence-tracking collection in any release. It is a pattern built on top of the Reference Map or Reference Table shapes already in this section: a keyed entry (`RM-KillChain-Stage-`

Tracker, or RT-KillChain-Stage-Tracker if more than one attribute needs tracking per stage) whose value records which stage of a defined sequence has been reached and when, so a later Rule Test can check both "has this key reached stage N" and "was the interval between stages plausible" against logic you write yourself rather than a mechanism the platform provides natively.

PRODUCT VERSION NOTE *Whether your Admin console packages any convenience wizard for this kind of multi-stage tracking on top of a Reference Map or Reference Table, versus leaving the stage-number-and-timestamp bookkeeping entirely to the Rule Response and Rule Test logic you write yourself, is the kind of console-convenience detail that has varied across releases — the underlying object shape (a Map or a Table, per §1) does not. Confirm what your own Rule Wizard offers before assuming either path.*

Whichever shape you build it on, the general lesson generalizes past three stages the same way DEH Part 27 §4's threshold-rule discussion generalizes past one: a native aggregation test handles "count of one event type, one key, one window" cheaply; anything with a state dimension — membership, lookup, or now order — needs some shape of reference data, and the shape should match the actual question, not the first object type that happens to be already deployed for something else.

5. TTL and expiry: the maintenance dimension every type shares

[PLATFORM ENGINEER] Every reference-data type above shares one property DEH Part 27 §2.2 states once and does not revisit: none of them expire an entry automatically unless a time-to-live is explicitly configured on the collection. A Reference Set, Map, Map of Sets, Map of Maps, or Table with no TTL configured behaves exactly as designed for as long as it exists — which is precisely the trap in RS-Recent-Failed-MFA-Sources from §2: the object never malfunctions, it simply keeps answering a question ("has this source ever failed an MFA challenge") that stopped being the question anyone meant to ask the moment "recent" quietly became "ever."

TTL posture is not one-size-fits-all across the taxonomy — it depends on what the object represents, not what shape it is:

Object example	Typical TTL posture	Reasoning
RS-Allowlisted-LSASS-Tools	No TTL, or very long, human-reviewed	Curated by a person; auto-expiring a legitimate entry silently reopens the exact false-positive flood the allowlist exists to prevent.
RS-Stage1-Seen	Short, sliding — minutes to a few hours	State should bridge only the attacker's actual plausible multi-stage window; anything longer accumulates entries the second-stage Rule was never meant to match against, exactly the DEH Part 27 §2.2 Blind Spot.
RMOS-User-To-Source-IPs	Short, sliding, ideally per-value	An impossible-travel or new-source check is only meaningful against genuinely recent history; a set that never forgets an old source IP eventually contains so much history that nothing reads as new.
RMOM-Host-ProcessBaseline	No TTL, or very long with a scheduled re-validation	A baseline is meant to represent "has this ever been seen," not "was this seen recently" — expiring entries defeats the object's entire purpose and reintroduces the day-one bootstrap flood from §3's Blind Spot on every expiry cycle.

PRODUCT VERSION NOTE *Whether TTL is configured per collection (a single expiry policy applying to every entry) or per entry (each value or key carrying its own expiry clock, resettable on each write), and whether a Rule Response's write to an existing entry resets that entry's TTL clock or leaves the original expiry untouched, are both configuration-surface details that have varied across QRadar releases and are exactly the "exact wording, default, or availability" category §11 requires a note for. Confirm both behaviors against your own console's Reference Set Management configuration for the specific collection before relying on either assumption — a Rule Response that you expect to keep refreshing an entry's recency, but which only extends the TTL clock on true insert rather than on every touch, produces the same "silently stale after all" failure mode as having no TTL at all, just delayed.*

Noisy Offense Trap *RS-Recent-Failed-MFA-Sources from §2, six months in with no TTL ever configured: every source IP that has ever failed an MFA challenge — a legitimate typo, a cached-credential retry, a genuine push-bombing attempt from eighteen months ago — is still a member. The Rule reading it still fires exactly as designed on "five accumulated failures within the set," except the set's population has drifted from "recent failure sources" to "every failure source in this object's entire lifetime," which inflates the false-positive rate on any account with a long history of occasional typos without anyone changing a single Rule Test. The fix is not to raise the failure-count threshold — that just makes a real push-bombing attempt harder to catch too, the identical mistake DEH Part 27 §4's own Noisy Offense equivalent warns against for a threshold rule. The fix is to add a TTL to the Reference Set that matches the actual correlation window the Rule was designed around, backfill-test it against a known-positive push-bombing simulation (Part 14's Validation Test discipline), and audit every reference-data object without a TTL the same way Part 15's tuning-intake process audits a flooding Rule — because an unmanaged reference-data object is a tuning problem that has nothing to do with the Rule that reads it.*

6. Populating reference data: three paths, and the drift risk of running more than one

[PLATFORM ENGINEER]

A reference-data object gets values into it through exactly three mechanisms, and a team that uses more than one against the same object without coordinating them is the single most common way reference data silently drifts from what the Rules reading it assume it contains.

Path	Typical use	Coupling to a Rule	Drift risk
Rule Response	Self-populating state — the Stage1/Stage2 pattern, a running baseline, an auto-added denylist entry	Tight — the object's contents only change because a Rule fired	If the populating Rule stops matching (a DSM mapping breaks per Part 4, a Building Block gets edited and quietly stops matching), the object silently stops updating with no error anywhere — indistinguishable from "nothing to add" the same way a missing custom property is indistinguishable from "no matches" in DEH Part 27 §3.1.
REST API	External feed — threat intel, a CMDB export, a SOAR playbook, a scheduled script	Loose — no Rule is aware the API call happened	A scripted job that does a full replace of an object's contents (delete-and-reload) rather than a merge silently erases any entry a Rule Response or a manual console edit added since the last scheduled run, with no audit trail pointing at the API call as the cause.
Bulk import (Admin console)	Human-curated allowlist seed, a one-time backfill, an analyst's tuning-ticket addition	None	The moment it's imported, it's already as stale as the source list it was copied from — nobody re-validates a bulk-imported allowlist against current reality unless a process explicitly requires it (Part 16's rule-change-management discipline is the compensating control here).

```
# Illustrative only – a REST API call adding one value to an existing Reference Set.
# CONCEPTUAL EXAMPLE: the exact endpoint path, authentication header name, and request
# shape below are illustrative of the kind of call this population path uses, not a
# verified transcript of a specific QRadar REST API version. Confirm the current
# Reference Data endpoint path and required headers against your own deployment's
# REST API documentation (usually browsable from the Console itself) before reusing
# this verbatim.
curl -X POST "https://qradar.example.internal/api/reference_data/sets/RS-Allowlisted-LSASS-Tools" \
  -H "SEC: <api-token>" \
  -H "Content-Type: application/json" \
  -d '{"value": "C:\\Program Files\\Vendor\\scanner.exe"}'
```

PRODUCT VERSION NOTE The Reference Data REST API's exact endpoint paths, whether a POST appends a value versus a PUT replacing the entire collection, and the specific authentication header/token mechanism, have all been the kind of implementation detail that shifts across QRadar releases without changing the underlying concept. Treat the `curl` example above as illustrating the shape of this population path — an authenticated call that mutates a named collection — and verify the literal request against your own version's REST API reference before building a scheduled job against it.

The drift risk table's middle row is worth naming as a concrete failure, not an abstraction: a threat-intel team pushes a nightly REST API job that reloads `RS-Allowlisted-LSASS-Tools` from a CMDB export of approved EDR agent paths, using a full-replace call because that was the simpler script to write. An analyst, acting on a same-day tuning ticket, manually adds a newly-approved diagnostic tool's path through the Admin console that afternoon. That night's scheduled reload silently deletes the addition — the script only knows the CMDB export it was handed, not what changed since the last run. The Rule fires again on the diagnostic tool the next morning, the analyst re-adds it, and the cycle repeats until someone fixes the reload script to merge rather than replace, or moves ownership of that allowlist to one path exclusively. Neither population path was misconfigured on its own; the drift is entirely a consequence of two uncoordinated paths writing to the same object.

7. Choosing the right type: a decision guide

[RULE ENGINEER] The taxonomy in §1 is easiest to apply correctly by starting from the question a Rule Test actually needs answered, not from whichever object type is already familiar:

- **"Is this value present at all?"** — Reference Set. No key, no lookup, cheapest option; do not reach for a Map when a Set answers the question.
- **"What single fact is associated with this key?"** — Reference Map. One key, one value, a lookup rather than a membership check.
- **"What has accumulated under this key, and is this new value already part of it?"** — Reference Map of Sets. Any "per-entity recent history" question — source IPs per user, destination ports per host, escalation targets per team — belongs here rather than encoded into a flat Set's concatenated strings.
- **"For this entity, what do I know about this specific sub-thing?"** — Reference Map of Maps. Two genuinely independent keys (host and process, asset and vulnerability ID, tenant and rule) that would otherwise need string concatenation to fit a flatter type.
- **"What multi-column record is associated with this key?"** — Reference Table. A per-entity record with more than one fixed, named attribute (an asset's owner, criticality, and last-patched date) belongs here rather than several parallel Reference Maps that have to be kept in sync by key.
- **"Have these stages happened, in this order, within a plausible interval?"** — not a distinct object type at all; build it as a pattern on top of whichever of the two prior answers fits (Reference Map for one tracked attribute per stage, Reference Table for more than one) per §4.

A Rule Engineer who defaults to the Reference Set for every new correlation need — because it was the first type learned, and DEH Part 27 §2.2 only demonstrated that one — eventually produces exactly the string-concatenation workarounds §3 and §4 both warn against: technically functional, materially harder for the next reviewer to read, and a worse fit for Part 9's Building Block governance discipline than an object whose type already documents what question it answers.

8. Validating and hunting against reference data state

[THREAT HUNTER]

A Rule that silently stops matching because the reference data it depends on has drifted — an allowlist missing a recently-approved tool, a Map of Sets whose bootstrap period was skipped, a Rule Response that stopped firing after a DSM mapping broke — produces no error anywhere in the CRE. The only way to catch that drift before an Offense flood (or an Offense silence) forces the issue is to periodically check a reference-data object's actual current contents against what the Rules reading it assume is there, the same discipline DEH Part 27 §3.1's Engineering Reality box requires for a DSM/QID mapping.

```
-- QRadar AQL – this checks whether a specific value is currently a member of a named
-- Reference Set from an ad hoc search, which is a different operation from a Rule
-- Test's own membership check inside the CRE: this runs against Ariel search
-- infrastructure on demand, not the real-time evaluation path. The exact function
-- name and argument order for reference-data lookups from AQL have not been
-- confirmed against a specific QRadar release for this book – verify the current
-- function signature in your own AQL function reference before reusing this
-- pattern, per the PRODUCT VERSION NOTE immediately below.
SELECT sourceip, username, "Source Process Name"
FROM events
WHERE QIDNAME(qid) = 'Process accessed'
  AND "Target Process Name" ILIKE '%lsass.exe%'
LAST 7 DAYS
```

PRODUCT VERSION NOTE AQL exposes some mechanism for checking reference-data membership or looking up a value from an ad hoc search — the general capability is a reasonable, stable assumption given that a Rule Test can already do the equivalent check inside the CRE. The exact function name, its argument order, and whether it can address a Map's inner key or only a Set's flat membership, are details this book has not independently verified against a specific QRadar release and will not assert as fact. Check your own console's AQL function reference (typically browsable from the Log Activity > Advanced Search interface) before writing a hunt or a saved search that depends on a specific reference-data function signature.

The more reliable validation path, and the one that needs no unverified AQL function at all, is direct: open the Reference Set Management (or equivalent) view for the object in question and review its actual current membership against what you expect — exactly the DEH Part 27 §3.1 discipline of confirming a mapping is populated *and* usable, not just assumed correct because the Rule has been deployed a long time without complaint.

Hunter's Note Before concluding that a quiet Rule means "nothing to catch," pull the reference-data object it depends on and check its last-modified timestamp and current size, not just whether the Rule itself looks enabled. A `BB:LSASS-Access-Candidate`-style Building Block paired with a Rule Test against `RS-ALLOWlisted-LSASS-Tools` that has silently grown from twelve entries to four hundred over two years of unreviewed additions — every tuning ticket's fix, never once revisited — is functioning exactly as configured and is also functionally closer to "the Rule is disabled" than anyone treating it as a live control realizes. A Rule's silence and a Reference Set's drift are two separate things to check, and checking only the Rule misses the second one entirely.

9. Governance: the review burden of five object types instead of one

[SOC MANAGEMENT] DEH Part 27's own closing paragraph named the three-object review burden a QRadar detection carries relative to a single Sigma file or native query — a Building Block, a Reference Set, and a Rule, each separately versioned and permissioned. This part's taxonomy adds a dimension to that burden rather than replacing it: "reviewing the reference data" is no longer one predictable object shape. A reviewer now has to check a Reference Set's membership list, a Reference Map's key coverage, a Map of Sets' per-key cardinality growth, a Map of Maps' two-level consistency, and a Reference Table's per-column consistency across its outer keys — each with its own TTL posture and population path per §5 and §6 — and none of that collapses into the single "check the query" pass a native SPL or KQL detection's usually-stateless logic allows.

SOC Management View A team with forty Rules but two hundred reference-data objects behind them — some Sets, some Maps, several Maps of Sets from an impossible-travel program, a handful of baseline Maps of Maps nobody has re-validated since the host fleet's last refresh — has a materially larger review surface than the Rule count alone suggests. "We reviewed forty rules this quarter," reported without naming how many reference-data objects back them, is the same coverage-count confusion DEH Part 1 and Part 23 §5 warn against, restated here for the object layer Part 9 and Part 15 both depend on staying honest.

Cross-references: DEH Part 27 §2.2 (Reference Set introduction and the `RS-Stage1-Seen` worked example this part generalizes from) and §3.3 (the `RS-Allowlisted-LSASS-Tools` worked example this part also generalizes from), together with §2.3's Blind Spot (unmanaged Reference Set drift, extended here into the TTL discussion in §5); DEH Part 23 §5 (Sigma-vs-native authoring tradeoff, inherited rather than re-argued) and DEH Part 27's closing SOC MANAGEMENT paragraph (the three-object review burden this part's §9 extends to the reference-data layer specifically). Within this book: Part 4 (DSM/QID mapping — the silent-failure pattern §6 and §8 both borrow); Part 8 (Rule Wizard catalog and Response actions, the mechanism that populates reference data from inside a Rule); Part 9 (Building Block governance — the sibling object-sprawl problem this part's §9 runs alongside); Part 12 (Ariel search performance, distinct

from reference-data lookup cost); Part 14 (analyst triage) and Part 15 (Noisy-Offense tuning workflow, the compensating process for §5's TTL failures); Part 16 (rule change management, the compensating control for §6's bulk-import staleness); Part 17 (licensing/capacity, for reference-data memory footprint at scale); Part 19 (App Framework/UBA apps, the licensed alternative to the hand-built impossible-travel pattern in §3); Part 20 (asset model, the other major consumer of Map-of-Maps-style lookups).

neshboy

Part 11 — Thresholds, Anomaly and Behavioral Rules, and the Limits of Native Aggregation

Why this part exists

[CONCEPT] DEH Part 27 left two threads dangling on purpose. Its §2.3 Blind Spot named, in the abstract, exactly what the Custom Rules Engine (CRE) cannot do: "an arbitrary multi-event join running continuously the way a KQL/SPL scheduled query does." Its §4 then built exactly one worked example of the CRE's own answer to *some* multi-event problems — DET-27-02, a single Rule using QRadar's built-in aggregation Rule Test to catch three failed authentications against one account from one source within five minutes — and closed by naming two gaps that one Rule's grouping key leaves wide open: password spraying and distributed credential stuffing. Part 27 stopped there, by design; a query-language comparison chapter had one worked example's worth of room, not a survey of every native correlation shape QRadar's Rule Wizard actually supports.

This part picks up both threads. First, it generalizes DET-27-02's single grouping-key threshold into the fuller parameter space the Rule Wizard's aggregation Rule Test actually offers — including the distinct-value-count variant that closes the spraying and stuffing gaps DEH Part 27 §4 named without building. Second, it introduces the saved-search-baseline mechanisms — Anomaly, Threshold, and Behavioral rule types — that operate on an entirely different evaluation model from the per-event streaming Rule Test, and that exist specifically to catch drift-from-normal patterns no fixed count-in-a-window test can express. Third, and this is the part's actual job per DEH Part 27 §2.3, it draws the boundary precisely: exactly which correlation shapes these three native mechanisms cover between them, and which shapes still force the Building Block/Reference Set/Rule decomposition pattern this book's Part 9 and Part 10 cover in depth. This part does not re-teach AQL syntax (DEH Part 27 §1) or re-argue Sigma-first-vs-native authoring (DEH Part 23 §5); it assumes both and builds the native-aggregation half of the CRE's correlation toolkit that DEH Part 27 had no room to finish.

1. The aggregation Rule Test, generalized

[RULE ENGINEER] DEH Part 27 §4's DET-27-02 used one shape of QRadar's built-in aggregation Rule Test: `when at least X events are seen with the same property in Y minutes`, grouped on the pair (`sourceip`, `username`), threshold 3, window 5 minutes. That single worked example undersells the test's actual parameter space, which is what this section fills in before §2 uses it to close DET-27-02's own named gaps.

Three independent knobs make up this Rule Test, whatever its exact on-screen wording in your deployment:

- **Grouping key.** One property (`sourceip` alone) or a composite of several (`sourceip` and `username` together, as DET-27-02 used). More properties in the key means a narrower, more specific match — and a correspondingly easier evasion by an attacker who varies just one of them.

- **Threshold and window.** A count (at least X) and a time span (in Y minutes) evaluated on a rolling basis as events stream in — this is real-time, per-event CRE evaluation, the same evaluation model DEH Part 27 §2 describes for any Rule, not a periodic batch job.
- **Same-value count vs. distinct-value count.** The variant DET-27-02 used counts *occurrences* of the same grouping-key combination. A second variant — matching against **distinct** values of a named property within the window, rather than raw occurrence count — asks a structurally different question: not "did this pair repeat enough times" but "did this one key see enough *different* values of some other property." That second variant is the piece DEH Part 27 §4 needed and didn't have room to build.

PRODUCT VERSION NOTE *The Rule Wizard exposes both variants — same-value repetition count and distinct-value count — as separate aggregation Rule Test entries, and both have existed across recent QRadar release lines. The exact on-screen wording of the distinct-count variant (something to the effect of when at Least X events are seen with at Least Y different values of property in Z minutes) has shifted in phrasing and Rule Wizard grouping before, the same drift risk DEH Part 27 §2.1 already flags for the Rule Test catalog generally. Confirm the current wording and exactly which properties are eligible as the "different values" target in your own deployment's Rule Wizard before treating this section's phrasing as a literal transcript.*

Table 11.1 lays out the shape space this generalization actually covers, stated against the specific gap each shape closes rather than as an abstract feature list — including, in its first row, a bare QID (QRadar's internal event-taxonomy identifier; DEH Part 27 §1.2) matched with no grouping composite at all.

Table 11.1 — Aggregation Rule Test shapes and what each one catches.

Correlation shape	Rule Test variant	Grouping key	Example catch	Version sensitivity
Repeated hit, same actor	Same-value count	Single property (sourceip)	One host repeatedly triggering a scan-detection QID	Stable concept; wording flagged above
Repeated hit, same actor pair	Same-value count	Composite (sourceip + username)	DET-27-02: 3+ failed auths, one account, one source, 5 minutes	Stable concept; wording flagged above
One actor touching many targets	Distinct-value count	sourceip, distinct username	Password spraying (§2)	Wording flagged above
One target touched by many actors	Distinct-value count	username, distinct sourceip	Credential stuffing (§2)	Wording flagged above
Sustained deviation from a historical norm	Not covered by this Rule Test at all	—	"This user logs in from twice as many countries this week as their 90-day average"	See §4 — a different mechanism entirely

The bottom row is the pivot point for the rest of this part: every shape above it is a bounded, fixed-window, single-pass count evaluated as events arrive — cheap for the CRE to maintain in memory and genuinely real-time. The bottom row needs a rolling historical baseline, and no fixed-window streaming count expresses "compared to normal for this entity" no matter how the grouping key is composed. §3 draws that line formally; §4 covers the mechanism QRadar actually offers for it.

2. Closing DET-27-02's two named gaps

[RULE ENGINEER] DEH Part 27 §4's own Blind Spot named the failure mode precisely: DET-27-02's single Rule, keyed on (`sourceip`, `username`) same-value count, "never accumulates enough failures against any single (source, account) pair to fire" against either password spraying (many accounts, one source, one or two attempts each) or distributed credential stuffing (many sources, one account, one or two attempts each) — and warned that lowering the threshold or widening the window on that *same* Rule makes it noisier without closing the gap, because the grouping key itself is the wrong shape for either pattern.

The distinct-count variant from §1 closes both gaps as two separate Rules, not as a modification to DET-27-02's existing one — each reusing the same `BB:Auth-Failure-Candidate` Building Block DET-27-02's own failed-authentication matching logic already established, per this book's Part 9 governance pattern for sharing match conditions across Rules:

1. **R: Password Spray Candidate – Distinct Accounts** — Rule Test: at least 10 events matching `BB:Auth-Failure-Candidate`, same `sourceip`, with at least 5 distinct values of `username`, in 10 minutes. Catches one source hitting many accounts with low per-account attempt counts — **MITRE:** T1110.003 (Brute Force: Password Spraying).
2. **R: Credential Stuffing Candidate – Distinct Sources** — Rule Test: at least 10 events matching `BB:Auth-Failure-Candidate`, same `username`, with at least 5 distinct values of `sourceip`, in 10 minutes. Catches many sources hitting one account — **MITRE:** T1110.004 (Brute Force: Credential Stuffing).

Both Rules are still native aggregation — no Reference Set, no second Rule reading state a first Rule wrote. That distinction matters enough to state directly: distinct-value counting looks like it needs to "remember" every username or source IP seen for a given key, and in a sense the CRE does exactly that internally for the duration of the window — but that bookkeeping lives entirely inside one Rule Test's own evaluation, scoped to one window, discarded once the window rolls off. It is not the same mechanism as DEH Part 27 §2.2's Reference Set pattern, where one Rule's Response deliberately writes state a *different* Rule reads back with no shared window at all. Confusing the two is a real risk: an engineer who reads "this needs to count distinct values across events" and reaches for a Reference Set to do it has built a slower, more failure-prone version of a test the Rule Wizard already offers natively.

Validation Test Setup: *BB:Auth-Failure-Candidate* deployed and confirmed matching real failed-authentication events (same DSM/QID verification discipline DEH Part 27 §3.1 requires before trusting any Building Block); *R: Password Spray Candidate – Distinct Accounts* deployed with the distinct-count Rule Test from above. **Action:** From one test source IP, generate failed authentication attempts against 6 distinct test accounts, one or two attempts each, within a 10-minute window. **Expected result:** No single (source, account) pair crosses DET-27-02's own same-value threshold — confirming DET-27-02 correctly stays silent on this pattern, as DEH Part 27 §4 predicted it would — while *R: Password Spray Candidate – Distinct Accounts* fires once the sixth distinct account crosses the 5-distinct-value threshold, visible as a new Offense indexed on the source IP.

This closes DEH Part 27 §4's specific Blind Spot as stated, but it does not close every gap a real spraying or stuffing campaign can present — the next section names why on purpose, rather than letting two working Rules read as "solved."

3. Where native aggregation still hits a wall

[CONCEPT] Every shape in Table 11.1's top four rows shares three properties: one event type (or one Building Block's worth of matching conditions), one fixed evaluation window chosen at authoring time, and a count — same-value or distinct-value — as the only statistic computed. That combination is deliberately narrow, and naming the boundary precisely is this section's job, because DEH Part 27 §2.3's Blind Spot deserves a real answer, not just a restatement.

Three correlation shapes fall outside every variant of the aggregation Rule Test, regardless of how the grouping key is composed:

- **Multi-event-type sequential correlation.** Stage-one event (a distinct QID, a distinct schema) followed by a *different* stage-two event, where the link between them is identity or asset, not a shared count within one window. This is exactly DEH Part 27 §2.2's Reference Set pattern — a first Rule's Response writes to a Reference Set, a second Rule's Rule Test checks membership — covered in full in this book's Part 9 (Building Block governance) and Part 10 (the reference-data type taxonomy). No aggregation Rule Test, however parameterized, expresses "this event, followed later by a structurally different event."
- **Long-horizon historical baselines.** "More than twice this user's 90-day daily average" is not a fixed count in a fixed window — it needs a stored statistical baseline computed over a horizon far longer than any aggregation Rule Test's window is practically evaluated at, and a live comparison against that baseline rather than a literal threshold. This is Anomaly/Behavioral territory, covered in §4.
- **Peer-group or population-relative deviation.** "This asset's flow volume relative to other assets in its same asset group" needs a population statistic, not a per-entity count — also Anomaly/Behavioral territory, and the sharper of the two once §4 gets into it.

Blind Spot Table 11.1 and this section together are the answer DEH Part 27 §2.3 deferred: the CRE's native aggregation Rule Test covers exactly "count or distinct-count of one matched-condition's occurrences, grouped by up to a few properties, within one bounded window" — full stop. It does not matter how creative the grouping key gets; a shape that needs a second, structurally different event type, or a comparison against a rolling historical baseline rather than a fixed number, is outside this mechanism's reach by construction, not by an implementation gap IBM might close with a wider Rule Wizard menu. An engineer who spends a week trying to force a multi-stage, cross-event-type pattern into one aggregation Rule Test by widening the grouping key is fighting the architecture, not tuning it — the Reference Set decomposition (Part 9/10) or the saved-search baseline (§4) is the actual tool for that job, and recognizing which of the two applies before authoring anything is the design decision this whole part exists to support.

flowchart TB

```
E["Incoming event stream\n(Ariel events / flows)"] --> R1["Real-time CRE evaluation\nper event, as it arrives"]
```

```
R1 --> AGG{"Aggregation Rule Test\n(same-value or\ndistinct-value count,\none fixed window)"}
AGG -->|"shape fits:\none event type,\ncount within a window"| OFF1["Offense\n(e.g. DET-27-
```

```
02,\nspray/stuffing Rules, Sec.1-2)"]
```

```
AGG -->|"shape doesn't fit:\nneeds a second,\ndifferent event type"| RSPATH["Reference Set\npattern\n(Part 9 / Part 10;\nDEH Part 27 Sec.2.2)"]
```

```
RSPATH --> OFF2["Offense\n(two-stage correlation)"]
```

```
E --> SAV["Saved AQL search,\nre-run on a schedule\n(not per-event)"]
```

```
SAV --> BASE{"Anomaly / Threshold /\nBehavioral rule type\n(compares current result\nto stored\nbaseline, Sec.4)"}
BASE -->|"deviation exceeds\nconfigured bound"| OFF3["Offense\n(baseline-relative,\nSec.4-5)"]
BASE -->|"within normal range"| NOOFF["No Offense"]
```

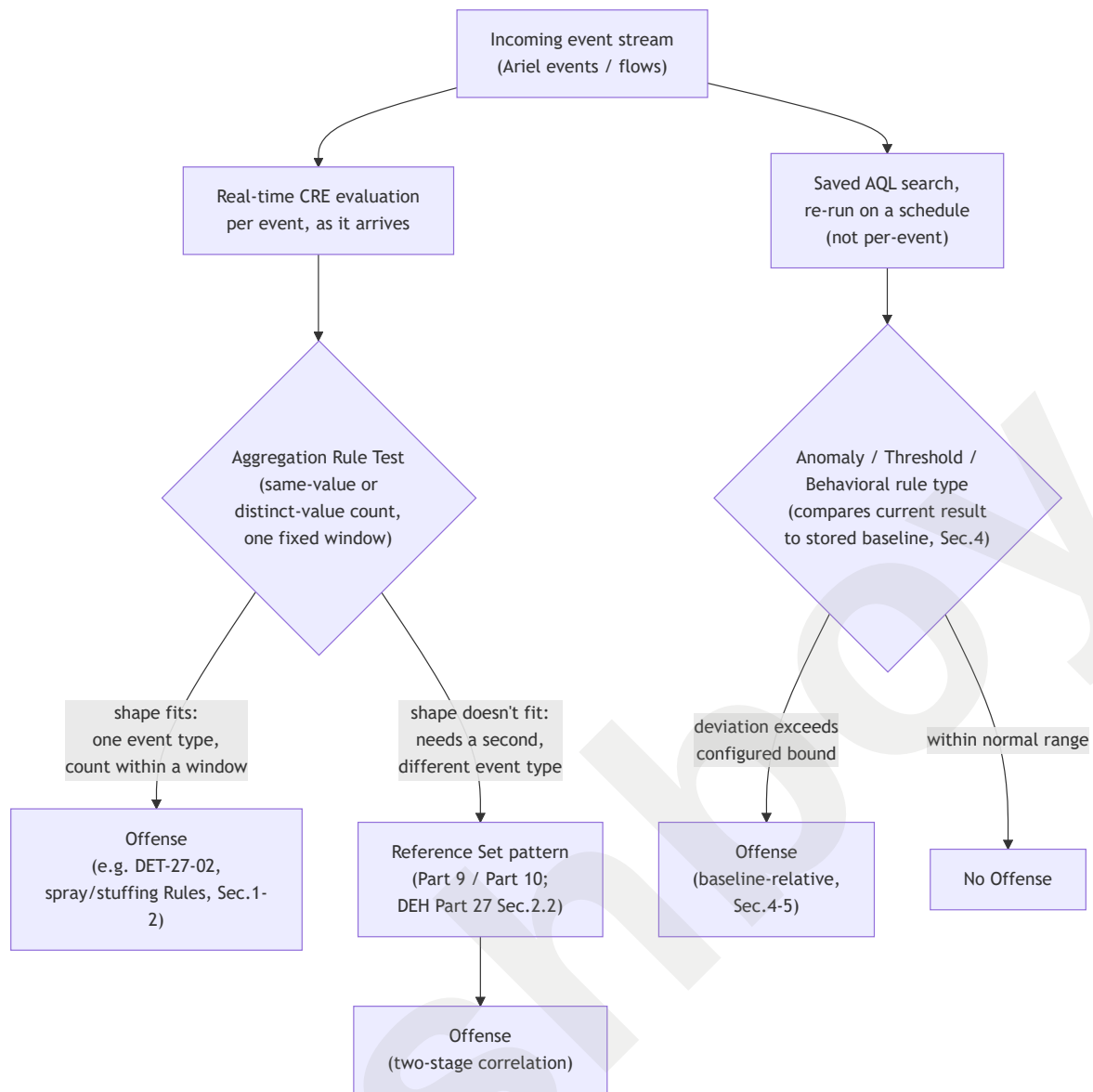


Figure 11.1 — Three native correlation paths from one event stream, and where each one's reach ends. CONCEPTUAL. Illustrates the two structurally different evaluation models this part covers — real-time per-event aggregation on the left branch, periodic saved-search-baseline comparison on the right branch — converging on the Reference Set decomposition path only when neither native mechanism's shape fits. Not a reproduction of any IBM architecture diagram; this book's own structural sketch of the boundary §3 and §4 argue from prose. Compare DEH Part 27's Figure 27.2 for the KQL/SPL-vs-QRadar deployment-unit contrast this diagram's middle branch extends.

4. Anomaly, Threshold, and Behavioral rule types: saved-search baselines, not streaming events

[PLATFORM ENGINEER] Everything in §1–§3 evaluates inline, per event, as the CRE processes the live stream — the same real-time model DEH Part 27 §2 describes for any ordinary Event, Flow, or Common Rule. QRadar's Rule Wizard also exposes a second family — commonly presented as **Anomaly**, **Threshold**, and **Behavioral** rule types, distinct from the Event/Flow/Common rule types Part 8 covers in

full — built on an entirely different evaluation model: each references a **saved search** (an AQL search against `events` or `flows`, saved rather than run ad hoc) and re-runs that search on a recurring interval, then evaluates the search's *result set* — not the raw incoming events — against a Rule Test comparing the current result to a stored baseline or a configured bound.

The distinction that matters operationally: an ordinary aggregation Rule Test (§1) sees every event the instant the CRE processes it, and its "window" is a rolling span computed continuously. A saved-search-based rule sees only what its underlying search returns *when it happens to run* — detection latency for this family is bounded by the search's re-run interval, not by CRE stream processing, and a search scheduled to re-run every 15 minutes cannot detect a pattern faster than 15 minutes after it starts, no matter how tight the Rule Test's own comparison logic is.

Within that family, the three named rule types differ in what they compare the saved search's result to, in spirit:

- **Threshold** rule types compare the saved search's result against a fixed or configured bound — conceptually adjacent to §1's aggregation Rule Test, but evaluated on the search's own schedule against a broader or more complex result set than a single Rule Test condition can express inline (a multi-column `GROUP BY` result, for instance).
- **Anomaly** rule types compare the current result against a stored historical baseline computed from the same saved search's own past runs — the "twice this user's 90-day average" shape §3 named as outside the aggregation Rule Test's reach.
- **Behavioral** rule types compare an entity's current pattern against either its own recent history or a peer-group pattern — the population-relative shape §3 also named, and the one most sensitive to what "normal" was computed from in the first place (§5).

PRODUCT VERSION NOTE *The existence of Anomaly, Threshold, and Behavioral as named rule types built on saved-search comparison is treated in this part as a stable architectural fact, consistent with this book's Part 8 scope. The exact baseline-computation mechanics available in a given release — how many days of history feed a baseline, which statistical measure (average, standard deviation, a configured percentage change) the comparison uses, and how much of that is user-configurable versus fixed — is not something this book states as certain fact, because IBM's own current documentation was not reliably available at the time this book was outlined. Verify the current baseline-configuration options in your own deployment's Rule Wizard before designing a specific Anomaly or Behavioral rule around an assumed statistical method.*

Platform Reality *A saved search feeding an Anomaly or Behavioral rule type is still an Ariel search, competing for the same search-resource pool DEH Part 27 §1.3 already flags for any AQL query and this book's Part 12 covers in depth — it is not a lightweight background process just because it's wrapped in a rule type rather than run from Log Activity. A saved search scoped too broadly (an unbounded `GROUP BY` over a busy environment's full event volume) and scheduled to re-run every few minutes competes directly with every analyst's own ad hoc search for the same shared resources, and a deployment with several such rules can turn "detect drift from normal" into "starve the search tier" without a single Offense ever explaining why. Scope the underlying saved search as narrowly as the detection goal allows, and treat its re-run interval as a genuine capacity-planning input, not a free dial to tighten for lower detection latency.*

5. Baseline drift and the false-positive cost of an unmaintained "normal"

[ANALYST] Every mechanism in §1–§2 has no concept of "normal" at all — a count either crosses a fixed threshold or it doesn't, and the threshold is exactly what an engineer typed into the Rule Wizard. Anomaly and Behavioral rule types are different in a way that changes what tuning even means for them: the comparison point is a *computed* baseline, derived from historical data the rule itself (or its underlying saved search) accumulated, not a number anyone chose directly. That has a direct and under-discussed consequence for false-positive management.

A baseline computed from an organization's own recent history encodes whatever was already true during the window it learned from — including activity nobody would call "normal" in any meaningful sense if asked directly. A department that ran an unusual, temporary data-migration project for three weeks bakes that volume into a 90-day rolling baseline; once the project ends, the department's genuinely normal traffic can read as an *anomalous drop*, and the reverse is equally real — a baseline learned during an unusually quiet stretch flags the first ordinary busy week as a deviation. Neither failure is a bug in the Rule Test's comparison logic; both are the baseline itself having drifted out of sync with what "normal" should mean by the time the rule evaluates against it, the same staleness risk this book's Part 10 already names for an unmaintained Reference Set and Part 20 names for an unvalidated asset model.

Noisy Offense Trap A Behavioral rule flooding the queue during a specific recurring period — month-end reporting, a quarterly close, a seasonal traffic pattern — is rarely a sign the underlying detection logic is wrong; it is usually a sign the baseline window is too short to have ever seen that recurring pattern as "normal," or too long to have adapted after the organization's actual behavior changed. The fix is not to raise the deviation threshold until the noise stops, which — the same discipline DEH Part 27 §4's own False Positive Trap already states for a different mechanism — also raises the bar for a real deviation the rule exists to catch. The fix is to name a baseline maintenance owner (the same role this book's Part 10 requires for Reference Set TTL policy) who reviews the baseline window length and re-validates it against a known-normal recent period on a fixed schedule, not just when an analyst complains.

6. Validating a candidate baseline before deploying it

[THREAT HUNTER] DEH Part 27 §5's hunting pattern — loosen a deployed detection's fixed condition in an ad hoc AQL search and see what falls out, without touching production correlation logic — applies just as directly to a *candidate* baseline as it does to a deployed Building Block's value list. Before wiring a saved search into an Anomaly or Behavioral rule type at all, run the same search's underlying logic as a plain AQL query over the exact historical horizon the baseline would learn from, and look at the actual distribution rather than trusting that "90 days of history" produces a sane average by default.

```
-- QRadar AQL, not standard SQL – same disambiguation DEH Part 27 Sec.1.1 states on first
-- appearance: SELECT/FROM/WHERE reads like standard SQL, but LAST n DAYS is AQL-specific
-- shorthand with no ANSI SQL equivalent, and there is no general-purpose JOIN available.
-- CONCEPTUAL SAMPLE – hunting query validating a candidate Behavioral-rule baseline before
-- deployment, not a standing detection. As in DEH Part 27's own AQL examples, the exact
-- property names and QID display strings are illustrative – confirm both against your own
-- Log Activity > Advanced Search field list first. GROUP BY DATEFORMAT(...) here approximates
-- a per-day bucket for eyeballing the distribution; AQL's own date-bucketing syntax and
-- function names are covered in full by DEH Part 27 Sec.1, not re-taught here.
SELECT DATEFORMAT(starttime, 'yyyy-MM-dd') AS "Day", username, COUNT(*) AS "Events"
FROM events
WHERE username = 'test-svc-account'
GROUP BY "Day", username
LAST 90 DAYS
```

Approach: Run this per candidate entity (a specific service account, a specific asset group) before the baseline goes live, and look explicitly for the failure modes §5 named — a sustained anomalous stretch that would skew a 90-day average, a period of near-zero activity that would make ordinary activity read as a spike once the baseline resets, or a level shift partway through the window (a role change, a migration, a decommission date) that makes "the last 90 days" internally inconsistent as a single baseline.

Finding (illustrative — treat as a template, not a captured result): A hunt run this way either confirms the candidate window is reasonably stable — worth documenting as a deliberate validation

step, not silence about having checked — or surfaces a specific anomalous stretch that becomes an explicit exclusion window when the baseline is configured, the same targeted fix DEH Part 27 §5's own worked hunt produces for a Building Block's value list rather than a blanket threshold change.

Hunter's Note *Plot the distribution, don't just compute the average before you look at it. A 90-day average of 40 events/day can hide a distribution that was actually 10/day for 60 days and 130/day for a 30-day migration project — the mean lands on a number that never actually occurred on any single day, and a Behavioral rule comparing live data against that mean will misfire in both directions once the migration project's traffic pattern is no longer current. A day-by-day **GROUP BY** like the one above costs one extra query and catches this before it becomes a production noise incident instead of after.*

7. The review-cost consequence, and what would change this boundary

[SOC MANAGEMENT] DEH Part 27's own closing paragraph named a staffing consequence specific to QRadar's multi-object model: reviewing "one detection" here means reviewing a Building Block, a Reference Set's maintenance, and a Rule's chained logic as three separately-changeable artifacts. This part's mechanisms add a fourth review surface with its own distinct maintenance discipline: a saved-search-based Anomaly or Behavioral rule's *baseline*, which §5 already established drifts on its own schedule independent of anyone editing the rule itself. Budgeting review time for "we run N Anomaly/Behavioral rules" without separately budgeting a recurring baseline-revalidation task is the same undercount risk DEH Part 27 §5's SOC Management paragraph warns against for coverage-count reporting generally — a rule that hasn't been touched in a year reads as stable, and a baseline that hasn't been revalidated in a year reads exactly the same way right up until it starts either flooding the queue or going silent.

Table 11.2 — Native correlation mechanisms, their review burden, and where the boundary sits.

Mechanism	Evaluation model	Objects to review	What goes stale	Version sensitivity
Aggregation Rule Test (§1–§2)	Real-time, per event	One Rule (plus any shared BB:)	Threshold/window relative to actual traffic growth	Wording of distinct-count variant
Reference Set decomposition (Part 9/10)	Real-time, two-stage	BB: , RS- , R: — three objects	Reference Set membership (Part 10)	Low — architecture is stable
Anomaly / Threshold / Behavioral (§4–§5)	Periodic, saved-search re-run	Saved search, rule type config, and the baseline itself	The baseline's learned "normal" (§5)	Baseline-mechanics detail, rule-type wording

What Would Change My Mind This part draws its central boundary — aggregation Rule Tests cover single-event-type counting within a fixed window; anything needing a second event type or a rolling historical comparison needs either the Reference Set pattern or a saved-search-based rule type — from the CRE's architecture as DEH Part 27 §2.3 described it and as this part's own worked examples confirm. If a future QRadar release lets an ordinary aggregation Rule Test reference a saved search directly as its own grouping input, or extends the distinct-count variant to span more than one event type in one condition — either of which DEH Part 27 §2.3 already flagged as a version-dependent capability worth checking rather than assuming — the sharp three-way split this part draws would soften, and the guidance to reach for Part 9/10's decomposition "by construction, not by an implementation gap" (§3's Blind Spot) would need to be re-evaluated against whatever the new capability actually covers. Recheck this boundary against your own deployment's current Rule Wizard before treating it as permanent architecture rather than this book's best current read of it.

Cross-references: DEH Part 23 §5 (Sigma-first-vs-native authoring tradeoff, not re-argued here); DEH Part 27 §1.3 (Ariel search-resource cost, extended in §4's Platform Reality), §2.1–§2.3 (Rule Test/Building Block catalog and the CRE Blind Spot this part resolves), §4 (DET-27-02 and its named spray/stuffing gap, closed in §2), §5 (the AQL hunting pattern extended in §6). This book's Part 8 (full Rule Wizard rule-type catalog), Part 9 (Building Block governance and the shared `BB:` pattern used in §2), Part 10 (Reference Set/Map/Table taxonomy and TTL discipline referenced in §3 and §5), Part 12 (Ariel search performance, referenced in §4's Platform Reality), Part 15 (noisy-offense tuning workflow, referenced in §5's Noisy Offense Trap), Part 20 (asset-model staleness, the same drift pattern named for baselines in §5), Part 22 (staffing/review-cost model extended in §7).

Section E — Operating the Search and Reporting Layer

neshbooy

Part 12 — Ariel Search Performance and Tuning

Why this part exists

[CONCEPT] DEH Part 27 §1.3 raises Ariel search cost as a single paragraph inside a section about AQL time windows: an unbounded search against a busy Ariel database "will run long enough to time out or exhaust search resources shared with every other analyst's concurrent search," backed by an Engineering Reality callout about QRadar's EPS/FPM licensing model. That paragraph was doing one job for DEH — flagging that AQL's authoring convenience and its runtime cost are different axes — and it explicitly deferred the rest. This part is the rest: what actually makes one Ariel search fast or slow, which properties are worth indexing and why a `SELECT *` against a wide time window is a different kind of expensive than a narrow indexed lookup, the difference between a saved search an analyst reruns by hand and a scheduled search quietly feeding a dashboard tile every few minutes, how QRadar's retention-bucket model changes the cost of searching data that's aged out of hot storage, and what actually happens when several analysts and a batch of scheduled reports compete for the same finite search resources at once.

One naming note before this part uses the word "tuning" in its own title: this is search-**performance** tuning — index choices, search scope, resource contention — not the Tuning DEH `TERMINOLOGY.md` defines ("a deliberate, logged change to a detection rule's logic, thresholds, or exclusions... in response to observed false-positive or false-negative patterns") or this book's own Part 15, which covers exactly that sense for a flooding Rule. Both senses of "tuning" are real work on a live QRadar deployment, and they occasionally touch — a Reference Set growing large enough to slow the Rule Test that checks it is a genuine intersection point, flagged in §5 below — but this part stays on the performance side of that line and points to Part 15 rather than absorbing its scope.

This part assumes DEH Part 27 §1.1's `events/flows` table split, §1.2's translation functions (`QIDNAME()`, `LOGSOURCETYPENAME()`, custom properties referenced by quoted display name), and §1.3's mandatory time-window practice, and does not re-teach any of that AQL syntax again — see DEH Part 27 §1 directly for the language itself. It also assumes DEH Part 23's general framing that a query language's authoring convenience and its runtime cost on a given backend are separable concerns; this part is that second axis, worked through for one platform's real search-execution model rather than argued abstractly.

1. How an Ariel search actually executes

1.1 Ariel is a distributed, purpose-built search store, not a relational database

[PLATFORM ENGINEER] Ariel is QRadar's own event and flow data store, not a general-purpose relational database sitting behind AQL — there is no query planner an admin inspects with an `EXPLAIN`-equivalent statement, and no arbitrary index the way a DBA would add one to a SQL table. Each Event Processor and Flow Processor in a deployment (Part 2 covers the full topology) writes and retains its own local slice of the Ariel data it processes, as compressed, time-partitioned flat files, not as rows in a

shared database a single node owns. When an analyst or a scheduled job submits a search, the Console decomposes it into sub-searches distributed to every processor holding data in the requested time range, each processor searches its own local files, and the Console (or, in a distributed deployment, a coordinating processor) merges the partial results back into the answer the analyst sees.

That architecture has one direct consequence this whole part builds on: **a search's real cost is not just "how many rows came back" — it is "how many processors had to be asked, how much of each one's local data had to be read to answer, and how much of that reading could be skipped because the condition matched against an index instead of a full record scan."** A query that returns ten rows from a year-wide search across every processor in a twelve-node deployment can be far more expensive than a query that returns ten thousand rows from one processor's last hour of data.

PRODUCT VERSION NOTE *The distribution-and-merge model above — Console decomposing a search across Event/Flow Processors and merging partial results — is architecturally stable and consistent with how QRadar's own documentation describes a distributed deployment, but the exact terminology QRadar uses for the coordinating role in a given topology (Console vs. a designated processor acting as a search head for a subset of nodes), and any specific concurrent-search-slot limits tied to appliance tier or license, have varied across releases and hardware generations. Verify current terminology and any documented concurrency limits against your own deployment's System and License Management pages before treating a specific number as current.*

1.2 Indexed properties: the one lever that changes a search's cost class, not just its speed

[PLATFORM ENGINEER] QRadar lets an admin flag specific normalized fields and custom properties as indexed, through a dedicated index-management screen. An indexed property lets a search filter on that field without QRadar having to decompress and scan every event's full payload to evaluate the condition — the difference is not a modest speedup, it is the difference between a search that can answer in seconds against weeks of data and one that has to touch every byte of every event in the requested window to find out whether the condition matched at all. `sourceip`, `destinationip`, `username`, and `qid` are indexed by default in a standard deployment because they are exactly the fields most searches, most Rule Tests, and most dashboard widgets filter on. A custom property a DSM extracts from raw payload — the kind DEH Part 27 §1.2 shows referenced in AQL as `"Source Process Name"` or `"Granted Access"` — is **not** indexed by default, and stays that way until someone deliberately flags it, because indexing every custom property unconditionally would cost storage and processing overhead on properties nobody actually filters on.

Property Type	Indexed by Default	Typical Search Role	Version
sourceip, destinationip	Yes	Primary filter/pivot field for nearly every investigation	Stable across releases
username	Yes	Identity-centric pivot, Offense triage starting point	Stable across releases
qid / QIDNAME(qid)	Yes (on the underlying qid integer)	Event-type filtering, the anchor condition in most Building Blocks	Stable across releases
Custom property (e.g., a DSM-extracted process name or access-rights field)	No, until manually enabled	High-value filter for a specific analytic (e.g., DET-27-01's "Target Process Name") once that analytic runs often enough to justify the cost	Screen name/navigation — see PRODUCT VERSION NOTE below
Free-text payload search (no property reference at all)	Never indexed	Ad hoc keyword search when the relevant field isn't known yet	Always a full scan

This table supports one decision: **before promoting a hunt query into a saved search someone will re-run daily, or into a Building Block's Rule Test that the Custom Rules Engine (CRE) will evaluate against every incoming event, check whether the property it filters on is indexed** — and if the analytic is important enough to run routinely, that is the point at which indexing it stops being optional overhead and starts being the fix. Part 6 covers the extraction-time cost of a poorly written custom property (an expensive regex evaluated on every incoming event regardless of whether it's ever searched); this section's cost is a separate, search-time concern layered on top of whatever that property already costs to populate.

PRODUCT VERSION NOTE The index-management screen's exact name and navigation path (commonly referenced as "Index Management" under the Admin tab in recent releases) and the specific mechanics of flagging a custom property as indexed have carried the same underlying concept across QRadar's history but are exactly the kind of literal UI detail this book's §11 convention exists to flag rather than assert as a fixed click sequence. Confirm the current path in your own Admin tab before treating any specific navigation as current.

1.3 Quick Filter versus Advanced Search: two different cost profiles for the same question

[THREAT HUNTER] QRadar's Log Activity tab offers a Quick Filter search bar — a fast, keyword-oriented search that leans on indexed properties and a lighter-weight matching path — alongside Advanced Search, where an analyst writes the AQL DEH Part 27 §1 teaches directly. The two are not interchangeable for every question: a Quick Filter search across a known indexed field (an IP address, a username) is usually the cheapest way to answer "show me everything touching this entity in the last day," while a question that needs a custom property comparison, a `GROUP BY` aggregate, or a join-style Reference Set lookup (§2.2's territory from DEH Part 27) has no Quick Filter equivalent and has to go through Advanced Search / AQL.

For an analyst pivoting off a fired Offense — say, `R: Suspicious LSASS Access – Unapproved Process` from DEH Part 27 §3.3's DET-27-01 — the practical sequence is: start from the Offense's own contributing-event list (already scoped, already indexed on the entity QRadar bound the Offense to), and only drop into an unscoped Advanced Search across a wide time window if the Offense's own contributing events don't answer the question. Reaching for a fresh, wide AQL search first when the Offense already narrowed the field for you is the single most common way an otherwise reasonable investigation turns into an expensive search competing with everyone else's.

PRODUCT VERSION NOTE "Quick Filter" and "Advanced Search" as the two named entry points under the Log Activity tab are stable in concept — a lightweight indexed-field search versus full AQL authoring — but the literal tab/button labels and where each lives in the console have shifted before and are not this book's claim to freeze. Verify the current labels and layout in your own deployment before treating this section's naming as a literal transcript.

2. Search-scope narrowing: the three levers that matter more than query cleverness

[PLATFORM ENGINEER] Given §1's cost model, three scoping decisions change a search's cost class far more than any amount of clever AQL syntax inside an already-wide search:

1. **Time window.** `LAST 1 HOUR` versus `LAST 90 DAYS` is not a tenfold difference in cost, it can be a difference measured in orders of magnitude, because it directly controls how many processors' worth of historical data — and, per §4 below, which retention tier — the search has to touch at all.
2. **Log source / log source type filter.** Narrowing with `LOGSOURCETYPENAME(devicetype) = 'Microsoft Windows Security Event Log'` (DEH Part 27 §1.1's own example) restricts the search to data from a known subset of sources instead of every DSM-mapped source in the deployment, which matters most when the question is genuinely source-specific rather than "search everything and see what turns up."
3. **Indexed-property filter first, unindexed condition second.** AQL does not guarantee a specific evaluation order the way a query planner might, so writing the indexed condition first is a habit, not a documented optimization — but structuring a query so its cheapest, most-selective condition is the one doing the heavy narrowing (an indexed `sourceip` match) before an unindexed `ILIKE` payload scan keeps the expensive part of the search working against a much smaller candidate set.

```
-- QRadar AQL, not standard SQL – same SELECT/FROM/WHERE shape DEH Part 27 §1.1 introduces,  
-- including its non-SQL LAST n HOURS/DAYS window shorthand and no general-purpose JOIN.  
-- The point of this pair is search COST, not new syntax: neither query introduces a  
-- construct DEH Part 27 §1 doesn't already cover.
```

```
-- Wide form: scans every log source type, 90 days, unindexed payload match first.  
-- This is the version a hunter should run ONCE to confirm a pattern exists, per  
-- DEH Part 27 §5's hunting discipline -- not the version to save and re-run daily.
```

```
SELECT sourceip, username, "Target Process Name"  
FROM events  
WHERE "Target Process Name" ILIKE '%lsass.exe%'  
LAST 90 DAYS
```

```
-- Narrowed form: indexed QID filter and a log source type filter do the heavy lifting  
-- before the unindexed payload condition ever runs, and the window matches the actual  
-- investigation (the last 24 hours around a fired Offense), not a hunting-scale range.
```

```
SELECT sourceip, username, "Target Process Name", "Granted Access"  
FROM events  
WHERE QIDNAME(qid) = 'Process accessed'  
  AND LOGSOURCETYPENAME(devicetype) = 'Microsoft Windows Security Event Log'  
  AND "Target Process Name" ILIKE '%lsass.exe%'  
LAST 24 HOURS
```

The narrowed form's main limitation: it assumes the QID display name and log source type string are already confirmed for this environment, exactly the DSM/QID mapping prerequisite DEH Part 27 §3.1 requires before trusting any of DET-27-01's field names — a search scoped against the wrong QID string doesn't error, it just returns nothing, the same silent-failure mode DEH Part 27 already names.

Hunter's Note Run the wide, unscoped form of a query exactly once, to confirm the pattern exists and to see the real distribution of values (DEH Part 27 §5's HUNT-27-01 does exactly this against "Granted Access"), then narrow every subsequent run of that same investigation. A hunter who reruns the wide form out of habit every time they revisit a hypothesis is paying the full-scan cost repeatedly for a question the first run already answered.

2.1 What this feels like from the analyst's chair

[ANALYST] None of §2's scoping discipline is abstract to the person triaging a live queue. A Log Activity search that takes ninety seconds to return during a fast-moving incident is not a minor inconvenience — it's ninety seconds a Tier-1 analyst spends staring at a spinner instead of deciding whether an Offense is a true positive, multiplied across every analyst on shift doing the same thing against the same shared search resources. The habits that keep an investigation fast are the same ones §2 already names: pivot from the Offense's own contributing-event list before reaching for a fresh search, scope the time window to the actual incident timeframe rather than defaulting to the widest option in the picker, and prefer a Quick Filter search against a known indexed field over an Advanced Search AQL query when the question doesn't actually need one.

3. Saved searches, scheduled searches, and where dashboards fit

[PLATFORM ENGINEER]

A **saved search** is a stored AQL query or Quick Filter definition an analyst reruns on demand — it costs Ariel search resources exactly once, at the moment someone actually runs it, and its stored form (§1.2's scoping discipline baked in) is what makes it cheap or expensive each time. A **scheduled search** runs on a defined interval whether or not anyone is watching — commonly to populate a report (cross-reference Part 13's dashboard and reporting scope) or to feed a Rule Test anchored to a saved search, where a given QRadar version supports that anchoring at all (DEH Part 27 §2.3 already flags whether a Rule Test can anchor directly to a saved AQL search, and on what evaluation cadence, as a version-dependent capability — this part inherits that exact hedge rather than resolving it).

The distinction matters for capacity planning, not just semantics: a saved search that scans ninety days of unindexed payload data costs nothing until someone clicks run, but the same query saved as a scheduled search that fires every fifteen minutes costs that full scan every fifteen minutes, indefinitely, whether or not its results ever change meaningfully between runs. A handful of such scheduled searches, each individually reasonable, can add up to a standing load on Ariel search resources that a one-time audit of "what's scheduled and how often" is the only reliable way to catch — nothing in the platform proactively warns an admin that a scheduled search's own AQL was never scoped as tightly as the ad hoc version it started life as.

Platform Reality *A dashboard tile backed by an AQL query (Part 13's scope) behaves like a scheduled search from a resource standpoint even though it presents as a passive chart — it re-runs on its own refresh interval, competing for the same finite search resources as every analyst's live Advanced Search and every scheduled report, and a dashboard with a dozen AQL-backed widgets each refreshing every few minutes is a standing search load that is easy to build one widget at a time without anyone totaling the sum. Audit dashboard widget refresh intervals with the same scrutiny §3 gives scheduled searches — a widget nobody has looked at in months but that still refreshes every two minutes is pure overhead with zero analyst value on the other side of it.*

4. Retention buckets and the rising cost of searching old data

[PLATFORM ENGINEER]

QRadar routes incoming event and flow data into named retention policies — commonly called retention buckets — that match data against filter criteria (a log source type, a specific QID, a network segment) and assign it a retention period, and in some configurations a distinct storage/compression treatment, independent of the deployment's overall default retention. Two consequences follow for search performance specifically, beyond the compliance and storage-planning purpose retention buckets primarily exist for:

1. **A search spanning a wide time window may cross multiple retention tiers**, and data nearing the edge of its retention period, or already compressed more aggressively than recently ingested

data, does not necessarily search at the same speed as this morning's events — a ninety-day hunt is not a uniform ninety days of cost.

2. **Data a retention bucket has already aged out and deleted is not searchable at all**, regardless of how the AQL time window is written — a `LAST 180 DAYS` search against a bucket configured for 90-day retention returns nothing for the uncovered range, silently, the same failure shape DEH Part 27 §3.1 already names for a missing DSM mapping: the query runs cleanly and simply doesn't have the data behind it anymore.

Retention Consideration	Search-Performance Impact	Compliance/Capacity Impact
Shorter retention bucket for high-volume, low-value log source type	Keeps hot-tier data smaller, faster to scan for recent windows	Reduces storage footprint fastest for the noisiest sources
Longer retention bucket for sources tied to a compliance mandate	Wide-window hunts against this bucket stay possible further back, at proportionally higher scan cost	Meets retention obligations independent of search convenience
No explicit bucket (default retention applies)	Cost and availability follow the deployment-wide default, not a tuned exception	Easiest to configure, least tailored to actual data value
Bucket boundary crossed mid-search (<code>LAST N</code> spans two tiers)	Search cost is not uniform across the window; the older portion may be slower or, past the retention edge, simply absent	Bucket policy, not the search, determines what's still there to find

Blind Spot A well-scoped, well-indexed search is still bounded by whatever retention buckets have kept around. A retrospective hunt built on the assumption that "QRadar keeps everything for a year" fails the moment it runs against a high-volume log source type that was quietly given a 30-day bucket to control storage cost — and because the failure is a clean, empty result set rather than an error, a hunter who doesn't check bucket policy first can walk away from a genuine negative finding without realizing the finding is "the data was already gone," not "the pattern didn't occur." Confirm retention-bucket coverage for every log source type a retrospective hunt depends on before treating an empty result as evidence of absence.

PRODUCT VERSION NOTE "Retention bucket" is this part's working term for QRadar's data-retention-policy mechanism; the exact admin screen name, the specific criteria types available for matching data into a bucket, and whether a given release exposes per-bucket storage-tier or compression settings distinctly from retention duration have all varied across releases. Confirm the current terminology and configuration surface (commonly under a Data Retention area of the Admin tab) against your own deployment before assuming this section's vocabulary matches your console exactly.

5. Concurrency: what happens when your search competes with everyone else's

[PLATFORM ENGINEER]

Ariel search execution shares finite resources — CPU, disk I/O, and a bounded number of concurrent search slots tied to the deployment's hardware/appliance tier — across every analyst running an ad hoc Advanced Search, every scheduled search and report, and every dashboard widget on its own refresh timer, all at once. None of that load is prioritized by urgency: a Tier-1 analyst's time-sensitive pivot off a live Offense competes on equal footing with an unattended scheduled report someone set up eighteen months ago and forgot about, unless the deployment has been deliberately configured otherwise.

Two practical consequences follow. First, a single unscoped search — the wide form from §2's worked example, run against a deployment's full retention window instead of the narrow form — can measurably slow every other concurrent search on the same processors, not just the one that submitted it; this is why §2's scoping discipline is a platform-health practice, not just a personal-productivity tip. Second, capacity planning for "how many analysts can search comfortably at once" is a real sizing question with the same shape as the EPS/FPM licensing question DEH Part 27 §1.3's Engineering Reality callout raises for ingest — this book's Part 17 covers that sizing question in depth; this part's job is naming that search concurrency is a shared, boundable resource in the first place, not a bottomless one.

Platform Reality *The degree to which Ariel search execution shares underlying compute/I/O resources with the same processor's real-time CRE evaluation and event/flow indexing work is a genuine architectural interaction, but the precise resource-isolation guarantees (whether search load can measurably delay correlation latency, versus being fully partitioned from it) are not something this book asserts as settled fact without a real deployment to measure it against — see this book's own no-lab constraint ([STYLE-GUIDE.md §0](#)). Treat "a search-heavy period might slow other search-heavy work" as the safe claim, and "a search-heavy period delays Offense creation" as a claim to test against your own deployment's monitoring, not to assume from this book alone.*

5.1 Where performance tuning and detection tuning genuinely intersect

[RULE ENGINEER]

One specific interaction is worth naming rather than leaving implicit: a Reference Set checked by a Rule Test (DEH Part 27 §2.2's [RS-Stage1-Seen](#) pattern, or this book's own [RS-Allowlisted-LSASS-Tools](#) from Part 9/10's scope) is itself a lookup the CRE performs on every relevant incoming event, and an unmanaged Reference Set that grows into the tens or hundreds of thousands of entries because nobody configured a TTL is not just Part 10's data-hygiene problem — it is also a standing per-event cost on whatever Rule checks it. When a noisy or slow Rule shows up for tuning under Part 15's workflow, "how large is the Reference Set this Rule Test checks, and does it have a TTL" is a legitimate first question, sitting exactly on the boundary between this part's performance scope and Part 15's detection-tuning scope.

6. Validating a performance change before trusting it

[PLATFORM ENGINEER]

A performance-tuning change — indexing a custom property, narrowing a scheduled search's scope, adjusting a retention bucket — is only real once it's measured, not assumed

from the change description alone.

Validation Test Setup: A candidate custom property (e.g., "Target Process Name") not currently flagged as indexed, referenced by an existing saved search that analysts or a scheduled report run routinely; a recorded baseline run time for that saved search against a fixed, repeatable time window. **Action:** Flag the property as indexed through the deployment's index-management screen (§1.2), allow any documented reindexing/propagation period the platform requires, then rerun the identical saved search against the identical time window. **Expected result:** A measurably shorter run time for the post-change search than the recorded baseline, with identical result rows — a performance change that alters what a search returns, not just how fast it returns, is a correctness bug in the change, not a tuning win. If run time doesn't improve, confirm the property was actually referenced by an indexed condition in the query (not just present in the `SELECT` list) before concluding indexing doesn't help this case.

7. A practical Ariel search-hygiene checklist

[PLATFORM ENGINEER]

The individual sections above resolve into a short list worth keeping visible for anyone writing, saving, or scheduling a search against a shared Ariel deployment:

Practice	Why It Matters	Who Owns It
Scope every search to the narrowest time window the question actually needs	Direct multiplier on data scanned across every processor (§1.1, §2)	Analyst / hunter running the search
Filter on an indexed property before an unindexed payload condition	Changes the search's cost class, not just its speed (§1.2)	Whoever authors a saved or scheduled search
Prefer the Offense's own contributing-event list over a fresh wide search when pivoting from a triage	Avoids re-scanning data QRadar already scoped for you (§1.3)	Triaging analyst
Audit scheduled searches and dashboard widgets for refresh interval versus actual value	A forgotten scheduled search costs the same every run, forever, until someone looks (§3)	Platform admin / dashboard owner
Confirm retention-bucket coverage before trusting a wide-window hunt's empty result	An empty result past a bucket's retention edge means "gone," not "absent" (§4)	Hunter running a retrospective query
Check whether a Rule Test's Reference Set has a TTL before assuming a slow Rule is purely a logic problem	Reference Set size is a standing per-event CRE cost, not just a data-hygiene issue (§5.1)	Rule engineer doing Part 15-style tuning intake
Measure a performance change against a recorded baseline, same query, same window	Confirms the change actually helped and didn't alter results (§6)	Whoever makes the tuning change

8. Mermaid diagram: how one Ariel search competes for shared resources

[PLATFORM ENGINEER]

The flowchart below collects §1's distribution-and-merge model and §5's concurrency argument into one picture: every submission path funnels into the same per-processor search execution, bounded by whatever the retention-bucket policy (§4) has kept physically available.

flowchart TB

```
A["Analyst submits Advanced Search\n(AQL, Log Activity tab)"] --> D["Console decomposes\nsearch\nacross Event/Flow Processors\nholding data in the time window"]
S["Scheduled search / report\n(fires on its own interval)"] --> D
W["Dashboard AQL-backed widget\n(fires on refresh interval, Part 13)"] --> D
D --> P1["Event Processor 1:\nsearch local Ariel files\nindexed condition first (1.2)"]
D --> P2["Event Processor 2:\nsearch local Ariel files"]
D --> PN["Event Processor N:\nsearch local Ariel files"]
P1 --> R["Console merges\npartial results"]
P2 --> R
PN --> R
R --> OUT["Result set returned\nto analyst, report, or widget"]
RB["Retention bucket policy\n(4) governs what's still\nphysically present to search"] -.->|"bounds\nwhat P1..PN\ncan find at all"| P1
RB -.-> P2
RB -.-> PN
```

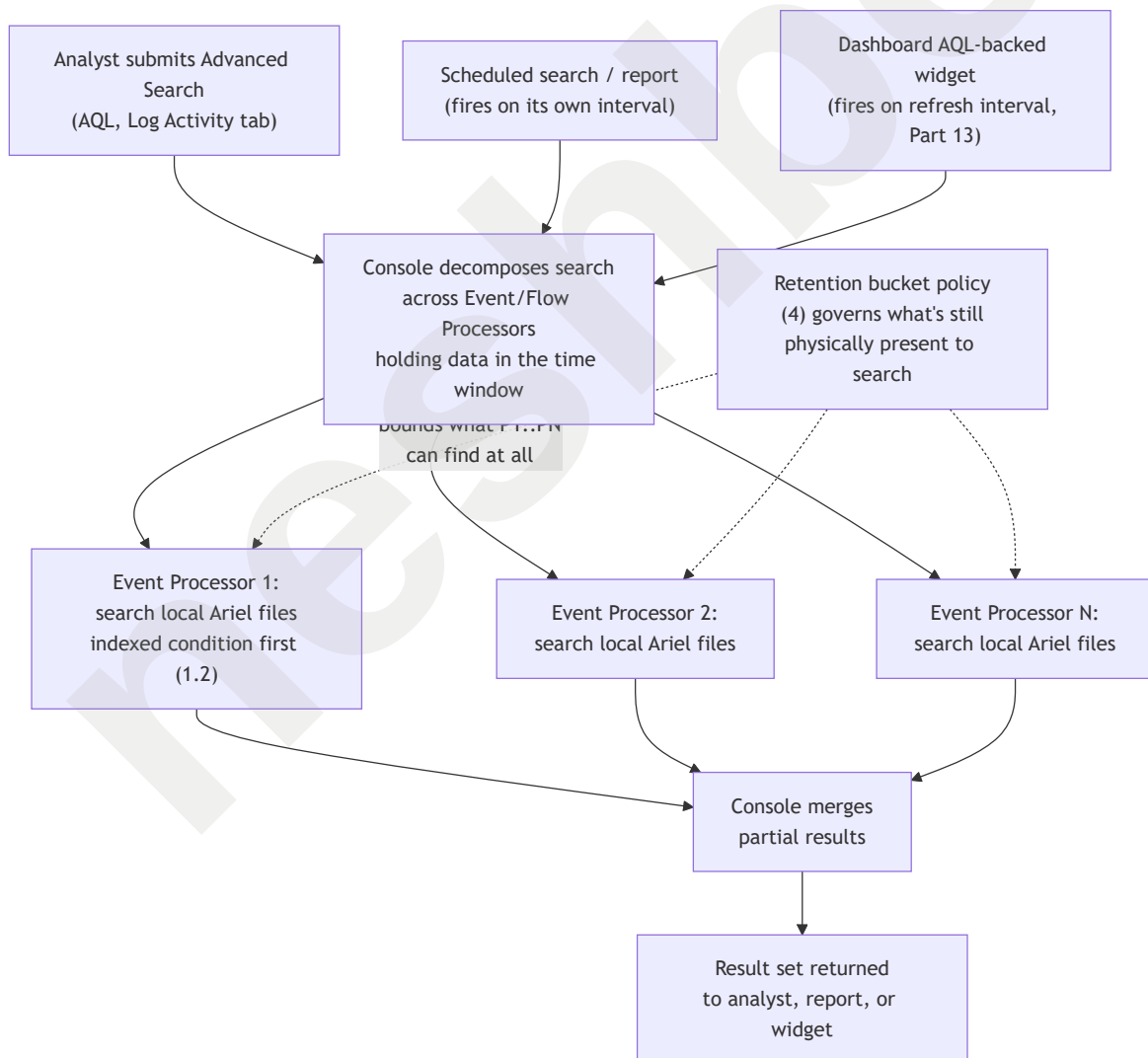


Figure 12.1 — One Ariel search's real path, and what it shares with everyone else's. *CONCEPTUAL.*

Illustrates §1's distribution-and-merge model and §5's concurrency argument together: an ad hoc analyst search, a scheduled search, and a dashboard widget all funnel into the same per-processor search execution and the same merge step, bounded throughout by whatever the deployment's retention-bucket policy has kept physically available. This is a structural illustration of this part's own argument, not a capture of any real QRadar console or a claim about a specific vendor-published architecture diagram.

Cross-references: DEH Part 27 (QRadar AQL) §1.1–§1.3 (Ariel table structure, translation functions, time-window/search-cost discipline this part expands), §2.2 (Reference Set lookup cost intersecting §5.1), §3.1 and §5 (DSM/QID mapping and hunting discipline this part's scoped-query examples build on); DEH Part 23 (Query Language Strategy) §5 (authoring convenience vs. runtime cost as separable axes); this book's Part 6 (Custom Properties — extraction-time cost distinct from this part's search-time indexing cost), Part 9/10 (Building Blocks and Reference Data — TTL and sizing referenced in §5.1), Part 13 (Dashboards, Pulse, and Executive Reporting — AQL-backed widget refresh cost referenced in §3), Part 15 (Noisy-Offense Triage and the Tuning Workflow — the detection-tuning sense of "tuning" this part deliberately does not cover), and Part 17 (Licensing, EPS/FPM Management, and Capacity Planning — search concurrency sizing referenced in §5).

Part 13 — Dashboards, Pulse, and Executive Reporting

Why this part exists

[CONCEPT] Every part before this one in Section D built the objects that produce an Offense — a Rule, a **BB:**-prefixed Building Block, an **RS:**-prefixed Reference Set — and Part 12 covered the Ariel search performance discipline that keeps ad hoc investigation usable at scale. None of that work is visible to anyone who doesn't open the console and run a search. This part covers the layer that actually gets looked at by people who aren't running searches: the Dashboard tab, the Pulse app's purpose-built widgets, and QRadar's scheduled-report engine — the surfaces a shift lead, a SOC manager, or an executive sponsor actually reads.

This part does not re-teach AQL (Ariel Query Language) syntax — DEH Part 27 §1 owns that in full, and every saved search backing a dashboard item or report chart in this part is written the way that section already teaches. It does not re-argue Sigma-first-vs-native query authoring — DEH Part 23 §5 owns that tradeoff, and it is orthogonal to this part's subject anyway: a dashboard widget consumes whatever AQL search sits behind it regardless of how that search's logic was originally designed. What this part adds is QRadar-specific and genuinely new: how a dashboard item or Pulse widget is actually backed (§1–§2), how scheduled reports compose and distribute that same data (§3), the specific honesty problem a bare offense-count number creates for anyone reporting metrics upward (§4, building directly on DEH **TERMINOLOGY.md**'s Coverage entries), a worked structure for a report that avoids that trap (§5), the silent-failure mode a dashboard shares with a broken DSM mapping (§6), and the access-control boundary a shared dashboard needs (§7).

This part assumes DEH Part 27 §1's AQL fundamentals and §6's Offense-triage framing, and this book's own Part 3 (magnitude, credibility, relevance), Part 6 (custom-property governance), Part 9 (Building Block governance), and Part 12 (Ariel search performance) — cross-referenced by section below, not restated.

1. Dashboard architecture: default items vs. custom AQL-backed widgets

[CONCEPT] The Dashboard tab is a tiled surface of independently refreshing panels, each called a dashboard item. Some ship with the product as defaults — an Offense summary, a most-recent-Offenses list, a system-notification panel, an event/flow throughput chart. Others are added by an analyst or admin, most commonly by saving an Advanced Search (an AQL query, per DEH Part 27 §1) and adding its result to the dashboard as a new item. The tile itself is dumb: it re-runs whatever is behind it on its own refresh interval and renders the result as a chart, table, or single-value counter. Nothing about the tile's own logic differs from what §2's Pulse widgets do — the distinction between "Dashboard tab item" and "Pulse widget" is which app renders the tile and how much layout/typing flexibility it offers, not a different data model underneath.

PRODUCT VERSION NOTE *The exact names QRadar's console gives its default dashboard tabs ("Offense," "Network Activity," "Application Overview," or similar wording) and the specific set of dashboard items that ship pre-built have shifted in wording and grouping across releases — the same drift risk DEH Part 27 §2.1 already flags for the Rule Wizard's own Rule Test catalog. Treat this section's category names as illustrative and verify the current tab and default-item list against your own console before citing one as a literal on-screen label.*

1.1 What a dashboard item actually is

[PLATFORM ENGINEER] Structurally, a dashboard item is one of three things, and knowing which one you're looking at matters for everything in §4 and §6:

1. **An Offense-index tile.** Reads directly from QRadar's own Offense index — the same magnitude/credibility/relevance-scored records Part 3 covers — not from an AQL search against `events` or `flows` at all. A "Top 10 Offenses by Magnitude" tile is this category.
2. **A saved-search tile.** Backed by an Advanced Search saved against `events` or `flows`, exactly the query surface DEH Part 27 §1.1–§1.3 teaches. A "Failed Authentication Events, Last 24 Hours" chart grouped by source IP is this category, and it carries the same search-cost profile any AQL search does — see §3's callout on report-time contention.
3. **A system-metric tile.** Reads QRadar's own operational telemetry — sustained EPS/FPM against your licensed tier (Part 17), disk/storage utilization, system-notification feed — not user data at all. This category is the one place a dashboard genuinely reflects platform health rather than security signal, and conflating the two is a real failure mode: a green "system healthy" tile says nothing about whether any Rule is still matching real telemetry correctly.

1.2 Dashboard item categories, at a glance

[PLATFORM ENGINEER] The table below supports deciding what to check first when a dashboard item looks wrong — an empty Offense-index tile and an empty saved-search tile fail for entirely different reasons.

Category	Backing mechanism	Refresh behavior	Search-resource cost	Version note
Offense-index tile	QRadar's own Offense index (Part 3)	Near-continuous; reflects live Offense state	Low — not an Ariel search	Field names on Offense-index tiles are stable across releases; layout options are not (see above)
Saved-search tile	AQL search against Ariel <code>events / flows</code> (DEH Part 27 §1)	On a configured interval, re-running the search	Shares Ariel search capacity with every concurrent analyst search (Part 12)	Default refresh-interval bounds and options have varied by release
System-metric tile	Platform telemetry (EPS/FPM, storage, notifications)	Near-continuous	Low — not an Ariel search	Exact metrics surfaced have expanded across releases; do not assume an older deployment exposes the same set
Reference-data tile	Reference Set/Map contents or membership count (Part 10)	On the item's refresh interval	Low to moderate depending on set size	Whether a given release exposes reference-data contents directly as a dashboard item, versus only via a saved search that checks membership, is version-dependent — verify before designing a widget around it

2. Pulse: purpose-built dashboards beyond the default tab

[PLATFORM ENGINEER] Pulse is an app built on QRadar's App Framework (Part 19 covers App Host resource footprint and governance in full) that provides a more flexible dashboarding surface than the default Dashboard tab — more widget types, more layout control, and a design intent closer to a single-pane executive or shift-handoff view than a working analyst's tiled search results. Underneath, a Pulse widget is backed by the same three mechanisms §1.1 names: an Offense-index read, a saved AQL search, or a platform metric. Nothing about Pulse changes what data is available; it changes how that data can be laid out and combined into one view meant for someone who isn't going to open a search bar.

PRODUCT VERSION NOTE *Whether Pulse ships pre-installed with a given QRadar deployment or has to be installed separately from IBM's App Exchange, and which packaging tier or delivery model (on-prem vs. cloud/SaaS) bundles it by default, has varied across release cycles and business-unit ownership changes to QRadar's own commercial packaging — exactly the category of claim §11 of this book's style guide flags as aging fastest. This part assumes Pulse is available as an installable app rather than asserting it ships by default in your deployment; verify its install status and any separate licensing before promising an executive-dashboard capability that depends on it.*

The practical reason to reach for Pulse rather than stacking more tiles on the default Dashboard tab is audience separation: an analyst's working dashboard is optimized for "what needs my attention right now" (a live Offense list, a saved search tracking a rule under active tuning per Part 15), while a Pulse view built for a weekly SOC-management sync is optimized for trend and volume over a longer window.

Building both audiences into one tiled surface tends to produce a dashboard that serves neither well — too dense for a quick triage glance, too granular for a management summary.

3. Scheduled reports: composition, distribution, and the search-cost tax

[PLATFORM ENGINEER] QRadar's Reports function composes chart and table sections — each backed by a saved search or an Offense-index query, structurally identical to §1.1's tile categories — into a formatted document generated on a schedule (daily, weekly, monthly, or ad hoc) and distributed by email attachment or to a shared location, typically as PDF or CSV depending on the section type. A report is a dashboard's contents, reformatted for people who read a document instead of a live console, generated once per schedule interval rather than refreshed continuously.

The operational detail worth naming explicitly: **a scheduled report's generation time is a real Ariel search event, not a free export of already-computed dashboard data.** A report section covering a 30-day window, grouped across every Log Source, runs that search against Ariel at generation time — the exact search-resource contention Part 12 covers for ad hoc analyst searches, except now scheduled and recurring. A report scheduled to generate at 9:00 AM on a Monday, timed to land in a manager's inbox before a standup, runs its underlying searches at the exact moment the overnight shift is also running its own end-of-shift searches.

Platform Reality *A wide historical report section — 30 days, all Log Sources, grouped by several fields — competes for the same Ariel search resources as every analyst's live search, the same shared-resource constraint Part 12 names for concurrent ad hoc use. Scheduling a heavy report to generate during business hours does not just risk a slow report; it can measurably slow down every analyst's concurrent search while it runs. Schedule wide historical reports for off-peak hours, and treat report-generation time as a line item in the same search-capacity planning Part 12 asks you to do for analyst headcount, not as a separate, free reporting layer sitting outside Ariel's actual load.*

Distribution and branding options (a logo, a cover page, per-recipient access scoping) are cosmetic on top of this same generation mechanism — worth configuring for a report leaving the SOC for an executive audience, but they do not change the underlying search-cost fact above.

4. What a dashboard number actually tells you: the coverage trap

[SOC MANAGEMENT] DEH [TERMINOLOGY.md](#) §5 states a rule this book inherits rather than re-derives: bare "coverage" is an incomplete sentence, and a claim has to be qualified as Telemetry Coverage, Detection Coverage, or a specific tier on DEH's six-tier scale (NO VISIBILITY through TESTED/RECENTLY VALIDATED) before it means anything. A QRadar dashboard is exactly where that discipline gets skipped by default, because the platform hands you a tile that looks like a coverage statement without being one.

SOC Management View An "Offenses Closed This Week" tile, or a scheduled report's raw offense-count trend line, is a volume metric — how many Offenses the Custom Rules Engine (the CRE; see this book's Part 8 for the full Rule Wizard catalog behind it) produced and how many an analyst dispositioned — not a Detection Coverage statement in DEH [TERMINOLOGY.md](#)'s sense. It says nothing about which adversary behaviors this deployment can reliably detect, only how much the deployed Rule set fired and how the queue was worked. Reporting a rising or falling offense count to leadership without naming which of DEH's six Detection Coverage tiers backs it repeats the exact "bare coverage" ambiguity DEH's [TERMINOLOGY.md](#) warns against generically — applied here to a specific tile that looks like progress on its own merits.

Blind Spot Every stock dashboard item and every scheduled report section in this part draws from Offenses the CRE already produced or events/flows already indexed in Ariel. None of it can show a technique sitting at NO VISIBILITY (no telemetry exists — a collection gap) or a Log Source sitting at TELEMETRY ONLY with no analytic ever built against it — the Visibility Debt DEH [TERMINOLOGY.md](#) names as a gap that "looks like progress on a source-count dashboard while contributing nothing to actual detection capability." A perfectly clean, low-offense-count QRadar dashboard is consistent with excellent tuning (Part 15) and separately consistent with a deployment that has never onboarded telemetry for half its ATT&CK-relevant surface — the two look identical from inside QRadar's own reporting layer. Closing that gap needs the source-and-analytic inventory DEH's six-tier matrix asks for, tracked as a program artifact outside QRadar's dashboards entirely, not as a widget QRadar itself can render for you.

4.1 Reframing four common tiles honestly

[SOC MANAGEMENT] The table below is not a claim that these tiles are wrong to build — they answer real operational questions. It is a claim that each one answers a narrower question than its label suggests, and the honest reframing is the sentence that belongs in a report's caption or a management readout, not the raw label alone.

Common dashboard metric	What it actually measures	What it doesn't measure	Honest reframing
Offenses Closed This Week	Analyst queue throughput	Whether closed Offenses were correctly dispositioned, or whether anything went undetected	"Queue throughput: N Offenses closed; see Part 14's disposition-accuracy sampling for triage quality"
Offense Count by Severity	Volume of CRE matches, bucketed by the Rule's configured severity	Detection Coverage tier for any specific technique; a high-severity Rule that never fires contributes zero to this chart either way	"Volume by configured severity, not a coverage statement — cross-reference the technique-level Detection Coverage tier for anything this chart is being used to argue"
Top 10 Source IPs by Event Count	Which sources are noisiest in raw log volume	Which sources are attacking anything; a chatty, benign application server routinely outranks a slow, deliberate intrusion	"Noisiest sources by volume — a triage starting point, not a threat ranking"
Log Sources Ingesting (count)	Telemetry Coverage — sources present in the pipeline (DEH TERMINOLOGY.md)	Detection Coverage — whether any Rule uses that data	"N sources ingesting; see the Visibility Debt tracker for which have an analytic built against them (Part 15, Part 22)"

5. Building an executive-safe report: a worked structure

[SOC MANAGEMENT] A report structure that survives the trap in §4 leads with tiered coverage language instead of raw counts, and treats offense volume as one input among several rather than the headline number. A structure that holds up under a second read:

1. **Detection Coverage summary, by DEH's six-tier scale** — a distribution across NO VISIBILITY through TESTED/RECENTLY VALIDATED for the technique set your program tracks (Part 22's capstone staffing model assumes this exists as a maintained artifact), not a single aggregate percentage DEH [TERMINOLOGY.md](#) explicitly warns against reporting.
2. **Offense volume and disposition, contextualized** — the raw counts §4.1's table reframes, presented with their disposition breakdown (true positive, benign positive, false positive per Part 14's closing-reason taxonomy) rather than as a bare trend line.
3. **Top firing Rules and Building Blocks, with tuning status** — which named **R: -** and **BB: -** prefixed objects (Part 8, Part 9) generated the most volume, and whether each is an intentional high-volume detector or a Part 15 tuning candidate already in progress. Naming the object, not "a large number of alerts from one rule," is the same concrete-over-vague discipline this book's style guide requires throughout.
4. **Mean time to triage and close** — Part 14's queue-handling metric, reported alongside coverage rather than instead of it, since a fast queue and a wide detection gap are both true at once and neither metric substitutes for the other.
5. **Tuning and onboarding actions taken this period** — new Log Sources onboarded (Part 4), Reference Sets updated (Part 10), Rules tuned or retired (Part 15) — the concrete evidence that Visibility Debt from §4's Blind Spot is being worked down, not just tracked.

None of this requires new QRadar functionality beyond what §1–§3 already describe — every section above is a saved search, an Offense-index query, or a manually-maintained coverage-tracking artifact assembled into one report template. The change is entirely in what gets reported first and how each number is captioned, not in the tooling.

6. Dashboard and report drift: the silent failure mode

[PLATFORM ENGINEER] A saved-search tile or report section fails exactly the way DEH Part 27 §3.1 describes for a broken QID (QRadar's internal event-taxonomy identifier) mapping: silently, with no error surfaced anywhere an analyst or manager would see it.

Platform Reality *If a custom property a widget's saved search references gets renamed during a DSM update (Part 6's custom-property governance), or a `BB:/R:`-prefixed object referenced by name in the search's filter gets renamed without the search being updated to match, the widget does not break visibly — it keeps rendering, and it renders a flat line or a zero. A shift lead skimming the Dashboard tab reads that as "nothing happened this period," not "this widget stopped measuring anything." The same failure shape §3.1 of DEH Part 27 documents for a Rule silently matching nothing since deployment applies here one layer up: the query runs cleanly, returns zero rows, and nobody is alerted that the number means "broken" rather than "quiet." Re-validate a dashboard item's or report section's backing search after any DSM change, custom-property rename, or Rule/Building Block rename that touches a field the search filters or groups on — the same trigger list Part 6 already asks you to track for AQL searches generally, extended here to cover dashboards and reports as additional consumers of the same fields.*

Validation Test Setup: A dashboard item or Pulse widget built on a saved AQL search grouping Offense volume by `"Source Process Name"` for `R: Suspicious LSASS Access – Unapproved Process` (DEH Part 27 §3.3's worked example; T1003.001 (OS Credential Dumping: LSASS Memory)). **Action:** Generate one known-positive event against the underlying Rule using the same test procedure DEH Part 27 §3.3's own Detection Test describes, then refresh the widget and re-run the equivalent report section. **Expected result:** The widget's count increments by one and the newly-charted value matches the test event's actual `"Source Process Name"`, confirming the widget's backing search still resolves the same custom property and Rule name it was built against — not merely that the widget renders without a console error, which a stale or mis-scoped search can do indefinitely while measuring nothing real.

7. Access and sharing across roles

[SOC MANAGEMENT]

A dashboard or report built for one audience becomes a liability shared with the wrong one: a working analyst dashboard exposing raw event detail to an audience that will misread a single noisy tile as an incident, or a management-facing report exposing per-analyst disposition counts in a way that reads as a performance scoreboard rather than a queue-health metric. QRadar's own sharing controls for dashboards and reports scope who can view or edit a given item or template; in a multi-tenant or MSSP deployment this interacts directly with the Domain/tenant boundaries Part 21 covers for Rules and Offenses generally.

PRODUCT VERSION NOTE *The specific sharing/permission model for dashboards and reports — whether sharing is scoped per-item, per-tab, or per-user-group, and what the exact console workflow for granting that access looks like — is a UI/navigation detail of the kind this book's style guide §11 requires flagging rather than asserting as stable fact. Verify the current sharing model in your own console, and cross-reference Part 21 for how it interacts with Domain-based tenant separation, before designing a shared-dashboard rollout around an assumed permission structure.*

flowchart TB

```
A["Custom Rules Engine\n(CRE) evaluates events/flows\nin real time"] --> B["Offense\nindex\n(magnitude / credibility /\nrelevance – Part 3)"]
C["Ariel events / flows tables\n(DEH Part 27 §1.1)"] -->|"saved AQL search"| D["Dashboard item\n/\nPulse widget"]
B -->|"Offense-index read,\nno AQL search"| D
C -->|"saved AQL search,\nscheduled"| E["Report chart / table\nsection"]
B -->|"Offense-summary\nreport section"| E
D --> F["Analyst-facing\nDashboard tab / Pulse view"]
E --> G["Scheduled report\nngeneration (§3)"]
G --> H["Distribution:\nemail / PDF / CSV"]
H --> I["SOC Management\nreadout (§4-5)"]
```

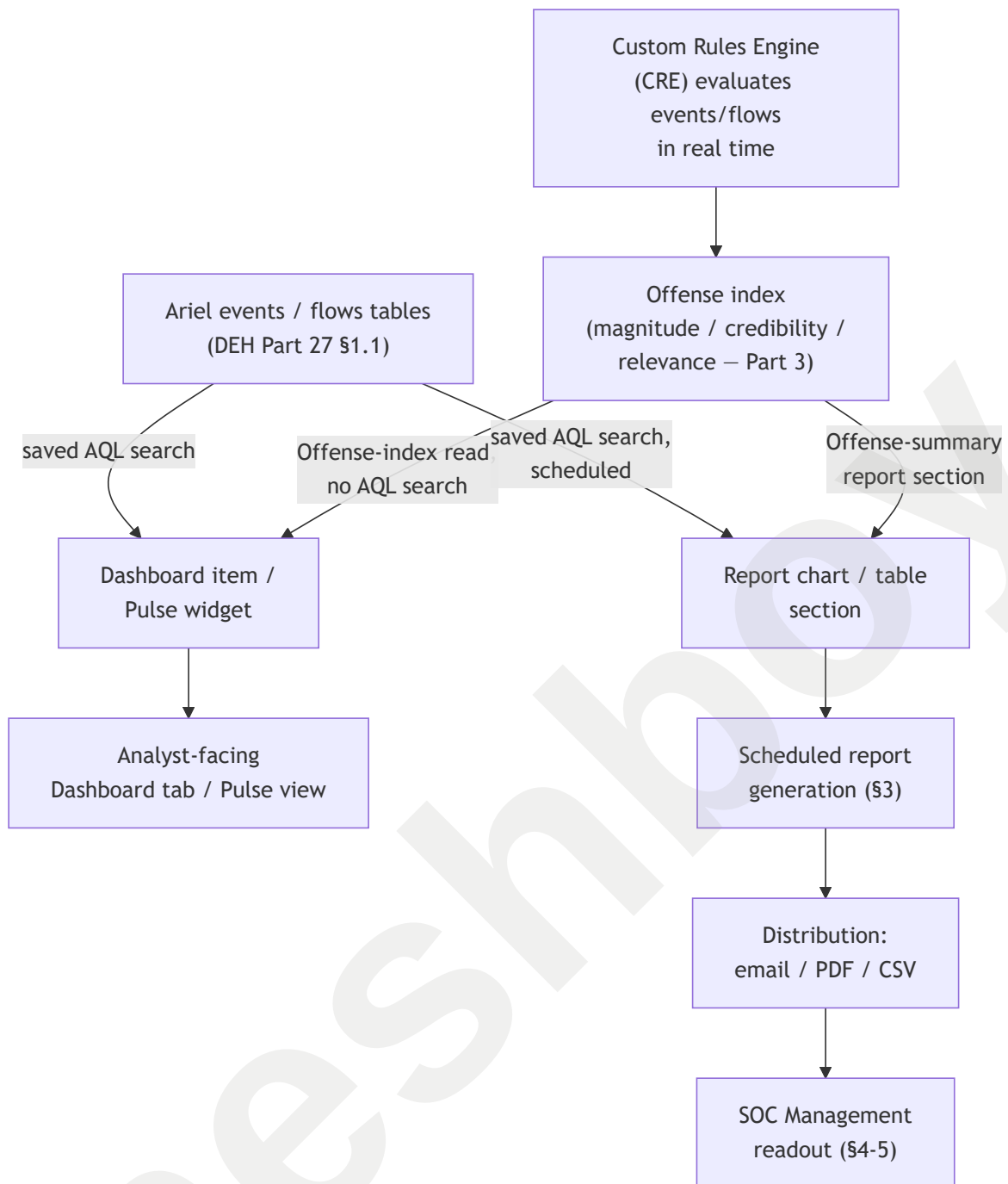


Figure 13.1 — Two backing paths into every dashboard tile and report section. *CONCEPTUAL.*

Illustrates that a QRadar dashboard item, Pulse widget, or report section draws from one of two structurally different sources — the Offense index QRadar itself maintains (Part 3's magnitude/credibility/relevance model) or a saved AQL search against the Ariel `events / flows` tables (DEH Part 27 §1.1) — and that both paths converge on the same scheduled-report and distribution mechanism before reaching a management-facing readout. Not a reproduction of any single IBM architecture diagram, and not a claim about any specific dashboard item's internal query implementation, which IBM does not publish.

Cross-references: DEH Part 27 §1 (AQL fundamentals, [events / flows](#)), §3.1 (silent DSM/QID mapping failure — the same failure shape §6 applies to dashboards), §3.3 ([R: Suspicious LSASS Access – Unapproved Process](#) worked example reused in §6's Validation Test), §6 (Offense as Alert/Case hybrid); DEH Part 23 §5 (Sigma-first vs. native authoring, orthogonal to this part's subject); DEH [TERMINOLOGY.md](#) §5 (Coverage, Telemetry Coverage, Detection Coverage, Visibility Debt — the basis for §4); this book's Part 3 (magnitude/credibility/relevance), Part 4 (Log Source onboarding), Part 6 (custom-property governance), Part 8 (Rule Wizard catalog), Part 9 (Building Block governance), Part 10 (Reference Data types), Part 12 (Ariel search performance and resource contention), Part 14 (triage disposition and MTTR), Part 15 (tuning workflow and Visibility Debt paydown), Part 17 (EPS/FPM capacity), Part 19 (App Framework and Pulse's App Host footprint), Part 21 (multi-tenancy and dashboard/report sharing), Part 22 (staffing and coverage-tracking capstone).

Section F — Running the Queue

neshboy

Part 14 — Analyst Triage Workflow for Offenses

Why this part exists

[CONCEPT] DEH Part 27 §6 spent one section proving a structural point: a QRadar Offense is not cleanly either an Alert or a Case in DEH `TERMINOLOGY.md`'s strict sense — it behaves like a hybrid of both, a queue item an analyst opens (the Alert instance) that also bundles every contributing event matched against the same Rule and the same indexed entity into one accumulating record over time (the Case). That section closed with a two-line worked example: pull the contributing events, decide whether the source process should have been on the allowlist (a Benign Positive that becomes a Tuning change) or is genuinely unrecognized (a True Positive candidate). This part is the rest of that workflow, generalized past one Building Block and one Rule to the whole queue an analyst actually works a shift against.

Part 3 (`part03-offenses-magnitude-credibility-relevance.md`) owns the scoring model itself — what severity, relevance, and credibility each measure, how they combine into magnitude, and the Offense lifecycle states (active, dormant, closed). This part assumes that model is already understood and asks the operational question Part 3 doesn't: given a queue of Offenses each carrying a magnitude number, what does an analyst actually do, in what order, and how do they record the outcome so the next shift — and every downstream metric in this book — can trust it? Part 15 picks up where this part's escalation-vs-tuning fork leads when the same Rule keeps flooding the queue with the same disposition; this part is scoped to one analyst working one Offense at a time, correctly, before that systemic question is in scope.

This part does not re-teach AQL — every drill-down query below is a pointer to DEH Part 27 §1 and §3.2's syntax, not a new tutorial — and it does not re-derive the Offense-as-hybrid finding Part 27 §6 already established; it cites that finding and builds the day-to-day discipline on top of it.

1. The queue an analyst actually sees

[ANALYST] An Offense list is not a flat, chronological feed the way a raw alert stream from a simpler platform can be. Because QRadar indexes an Offense on an entity (an IP, an asset, a username — the exact indexing key depends on how the firing Rule's Response was configured) and keeps updating that same Offense record as new contributing events match, a queue an analyst opens mid-shift already contains a mix of:

- Offenses that fired once, cleanly, and are waiting on first review.
- Offenses that have been open for hours or days, accumulating additional contributing events every time the same Rule matches the same entity again — magnitude and event count both climbing since the last analyst touched it.
- Offenses that another analyst already reviewed and left an internal note on, without formally closing, because the investigation needed information that hadn't arrived yet (an asset owner's response, a EDR console lookup).

[ANALYST] The practical consequence: before touching the top-magnitude Offense, an analyst scans for whether it's new or accumulating. An Offense sitting at magnitude 7 that fired once an hour ago reads very differently from one sitting at magnitude 7 that has been open for three days and just picked up its ninth contributing event in the last ten minutes — the second is actively escalating, and the queue-sort-by-magnitude view alone won't surface that unless the analyst also checks last-event-time and event count per Offense, not magnitude in isolation.

PRODUCT VERSION NOTE *The specific Offense-list columns available by default (magnitude, last-event time, event count, source/destination summary, assigned-user field) and whether a given deployment exposes a one-click "sort by last updated rather than magnitude" toggle are console details that have shifted in exact placement across QRadar releases. The underlying data — an Offense's magnitude, its event count, and its last-event timestamp — is architecturally stable; the specific column layout and default sort a reader's own console presents is not something this part asserts as a fixed screenshot. Verify your own Offense list's available sort/filter fields before building a shift-handoff procedure around a specific column that may not be in the position this part describes.*

1.1 Reading magnitude, credibility, and relevance as triage inputs — not as a verdict

[ANALYST] Part 3 defines what each of the three inputs measures; this section is about what an analyst does with the number, which is a different question. Magnitude is a queue-ranking score, not a confirmed-severity label — a high-magnitude Offense is one QRadar's own weighting considers worth looking at first, not one that has been validated as a true positive by anything other than the Rule's own match logic. Treat magnitude the way DEH [TERMINOLOGY.md](#)'s Alert entry treats any alert score: a triage-ordering signal, disposed only after the analyst actually looks.

A magnitude number by itself also collapses three different questions into one figure, and a triage workflow that only ever reads the collapsed number loses information a shift handoff needs:

- **Is this a severe pattern in the abstract?** (severity — the Rule/offense-type's own inherent weight, largely fixed by how the firing Rule was authored.)
- **Is this pattern relevant to *this* asset specifically?** (relevance — pulls from the asset model and Network Hierarchy weighting Part 20 covers; an identical Rule match against a domain controller and against an unmanaged guest-network laptop should not triage identically, and relevance is the mechanism that's supposed to reflect that difference.)
- **How much does QRadar trust the event data behind this match?** (credibility — lower for a Log Source type QRadar's own scoring treats as less reliable, or for a match built from a single low-confidence event rather than corroborated by multiple independent sources.)

[ANALYST] A queue-triage discipline worth building: when two Offenses show similar magnitude, look at which of the three components is driving it before assuming they're equally urgent. An Offense scoring high mostly on severity against a low-relevance asset is a different next action than one scoring the same total mostly on relevance against a high-value asset with only moderate severity underneath

— the first might genuinely wait behind the second even though the queue's raw sort shows them adjacent.

2. First-look triage: what to pull before deciding anything

[ANALYST] Continuation of DEH Part 27 §6's own worked example, generalized past one Rule. For any fired Offense, before a disposition decision is made, pull:

1. **The contributing event(s).** Open the Offense's event list and read the actual matched fields — not just the Rule name. A Rule name like `R: Suspicious LSASS Access – Unapproved Process` (DEH Part 27 §3.3's own example) tells you what pattern matched; it does not tell you which specific process, host, and user triggered it this time.
2. **The entity's recent history.** Has this same asset or user shown up on other Offenses recently, from the same Rule or a different one? A single Offense reviewed in isolation misses a pattern a second Offense on the same entity, twenty minutes later, would make obvious.
3. **The asset context.** Pull whatever Part 20's asset model and Network Hierarchy placement say about this host or user — is this a domain controller, a workstation in a segment with known vulnerability findings, a service account with an expected automated behavior pattern this match might just be describing normally?
4. **Whatever Reference Set membership the firing Rule actually checked.** If the Rule chains a Building Block match against a Reference Set exclusion — DEH Part 27 §3.3's `RS-Allowlisted-LSASS-Tools` pattern — check directly whether the matched value was *close to* being on that list rather than cleanly off it. A source path that's one character off from an allowlisted entry (a version-bump path change, a typo introduced in a prior maintenance edit) is a strong signal toward Benign Positive with a Tuning follow-up, not toward escalation.

[ANALYST] For deeper investigation — pivoting from the Offense's summary view into the full underlying event set with a broader time window, or checking whether the same pattern shows up against other entities the Offense list doesn't currently group together — drop into an ad hoc AQL search anchored on the Offense's own indexed entity and time range. DEH Part 27 §1 and §3.2 already cover AQL syntax and the search-cost caveats (`LAST n HOURS`, no unbounded search) that apply here unchanged; this part adds no new syntax, only the triage question the search is answering.

Hunter's Note A drill-down search run from inside an Offense's own context should almost always narrow, not widen, relative to the Rule Test that fired it — for example, dropping the specific `GrantedAccess` value filter to see the full range of access-rights values this entity has produced against `Lsass.exe` recently, the same loosening DEH Part 27 §5's `HUNT-27-01` demonstrates. If a full-context drill-down search on a live Offense turns into an open-ended hunt with no time bound and no specific question it's trying to answer, it has stopped being triage and become a hunt — which is a legitimate activity, but belongs under `[THREAT HUNTER]` discipline with its own hypothesis and time budget (DEH Part 27 §5's pattern), not an unbounded detour inside a triage shift with a queue still waiting behind it.

3. Disposition and closing-reason codes

[ANALYST] DEH `TERMINOLOGY.md`'s Disposition entry is unchanged here by definition, per this book's STYLE-GUIDE §12 — an Offense's recorded outcome is still one of True Positive, Benign Positive, False Positive, or Unable to Determine, and every downstream quality metric this book or DEH discusses is only as trustworthy as that recorded disposition. What QRadar adds operationally is a closing-reason mechanism attached to the close action itself, letting an analyst record *why* a given disposition applies without writing free-text every time.

Disposition	Typical QRadar closing-reason pattern	Next action
True Positive	A specific reason naming the confirmed technique or activity (e.g., "confirmed credential-dumping tool execution")	Escalate per §4; do not close the Offense as the terminal record of a confirmed incident — hand off to the Case/Incident process DEH <code>TERMINOLOGY.md</code> defines, keeping the Offense as supporting evidence.
Benign Positive	A reason naming the known-legitimate cause (e.g., "authorized EDR scan," "known maintenance window")	Tuning follow-up if the cause is likely to recur — add the source to the relevant <code>RS-</code> Reference Set or file a Part 15 tuning-intake request; do not just close and move on if the same cause will fire again tomorrow.
False Positive	A reason naming the logic gap (e.g., "Rule matched on stale asset classification")	Tuning follow-up owned by the Rule/Building Block author, not a Reference Set entry — this is a logic defect, not an expected-but-unwanted match; route through Part 15's diagnosis workflow.
Unable to Determine	A reason naming what's missing (e.g., "source host offline, could not confirm process")	Re-open on new information if it arrives; do not let "unable to determine" become a silent default for closing a queue faster — DEH <code>TERMINOLOGY.md</code> 's Disposition entry treats this as a real, trackable outcome, not a non-answer.

PRODUCT VERSION NOTE Whether QRadar's console presents closing reasons as a fixed, admin-managed picklist, a free-text field, or both, and the exact default reason list shipped out of the box, are UI specifics that have varied across releases and deployment configurations. The claim this part relies on — that a closing action can carry a structured reason distinct from the four-value disposition itself — is treated as architecturally available; the exact picklist contents and whether custom reasons can be added without vendor support involvement should be verified against your own deployment before building a reporting pipeline that assumes a specific reason taxonomy exists by default.

3.1 The Tuning-vs-Suppression fork, at the point of disposition

[ANALYST] DEH `TERMINOLOGY.md` draws a hard line between Tuning (a logged change to the detection's own logic) and Suppression (a mechanism that stops an otherwise-matching condition from reaching an analyst, without touching the underlying logic) — carried forward unchanged per this book's STYLE-GUIDE §12, and given full console-native treatment in Part 15. The disposition step is where that fork actually gets decided, even though the mechanical change happens later:

- A Benign Positive caused by a legitimate tool that should have been excluded from the start (DEH Part 27 §3.3's False Positive Trap — an AV engine, `WerFault.exe`, a sanctioned diagnostic tool) points toward **Tuning**: add the tool to the relevant `RS-` allowlist, a change to what the Rule's logic treats as excluded.
- A Benign Positive caused by a known, bounded, temporary condition — a scheduled penetration test, a one-off maintenance window — points toward **Suppression** if it recurs during a known window, without permanently changing what the Rule matches once that window closes.

[ANALYST] Recording the disposition and reason correctly at triage time is what lets Part 15's tuning-intake process later distinguish "this Rule needs a permanent logic change" from "this was a one-off, no systemic action needed" — a queue full of Benign Positive dispositions with no reason code attached gives the tuning workflow nothing to act on beyond "this Rule is noisy," which is a much weaker starting point than a reason-coded pattern showing exactly which Reference Set entry is missing.

4. Escalation criteria

[ANALYST] Escalation is the point where an Offense's disposition moves from "an analyst's individual judgment call" to "a decision other people and processes now depend on." A workable escalation trigger set, generalized from DEH Part 27 §6's own single-Rule example:

- **Confirmed True Positive on any asset**, regardless of the firing Rule's severity weighting — a confirmed malicious action on a low-relevance asset still needs the same incident-response handoff a high-relevance asset would get; relevance affects triage *order*, not whether escalation happens at all once confirmed.
- **Multiple Offenses on the same entity within a short window**, even if no single one individually reads as severe — the pattern across Offenses is often the signal a single Offense's magnitude cannot carry, since magnitude is computed per-Offense, not across an entity's full recent history.
- **An Unable to Determine disposition where the missing information itself is concerning** — a source host that's gone offline specifically because it's mid-compromise reads differently from one that's offline for a scheduled reboot; escalate on the ambiguity itself when the missing-information cause can't be independently confirmed as benign.
- **Any Offense that would, if true, cross a threshold your organization has pre-defined as requiring formal incident declaration** — credential-dumping activity against a domain controller, confirmed lateral movement, anything matching your own incident-response plan's declaration criteria. Escalation criteria here should be pre-agreed with the incident-response process, not improvised per-analyst per-shift.

[ANALYST] Escalating hands the Offense's supporting evidence — the contributing events, the drill-down search results, the reason the analyst believes this is a True Positive rather than the next-most-likely alternative — into DEH `TERMINOLOGY.md`'s Case or Incident process. The Offense itself stays open or closes-with-escalation-noted depending on your organization's own workflow convention; what matters for this book's purposes is that the Offense's own disposition record and the downstream Case/Incident record both point at each other, so a later reviewer reconstructing what happened doesn't have to guess which Offense triggered which Case.

Validation Test Setup: A test Offense fired against a non-production or clearly test-labeled asset, from a Rule with a known, reproducible Rule Test condition (for example, DEH Part 27 §3.3's R: Suspicious LSASS Access – Unapproved Process, triggered per its own Detection Test procedure). **Action:** An analyst works the Offense through this part's triage sequence — pulls contributing events, checks entity history and asset context, checks Reference Set proximity, assigns a disposition with a reason code, and (for this test) escalates per §4's criteria as if the finding were confirmed. **Expected result:** The disposition and reason code are recorded on the Offense; the escalation handoff produces a Case/Incident record that cross-references the originating Offense; a second analyst reviewing only the recorded disposition and reason (without re-reading the raw events) can correctly state why the Offense was escalated. If the second analyst cannot reconstruct the reasoning from the recorded disposition alone, the reason-code discipline in §3 failed, independent of whether the underlying triage judgment was correct.

5. A shift-handoff workflow, end to end

[ANALYST] The following diagram ties §1 through §4 into one repeatable per-Offense path, including the two forks (tune vs. suppress, escalate vs. close) this section has built up separately.

flowchart TD

```
A["Offense appears in queue"] --> B["Check: new or accumulating?\n(event count, last-event time)"]
B --> C["Read magnitude components:\nseverity / relevance / credibility"]
C --> D["Pull contributing events\n+ entity history + asset context"]
D --> E["Matched value close to\nRS- allowlist entry?"]
E -->|yes, looks like allowlist gap| F["Disposition: Benign Positive\nreason: allowlist gap"]
E -->|no clean match either way| G["Ad hoc AQL drill-down\n(DEH Part 27 sec1, sec3.2)"]
G --> H["Confirmed malicious\nor confirmed benign?"]
H -->|benign, one-off| I["Disposition: Benign Positive\nreason: known cause"]
H -->|benign, recurring pattern| J["Disposition: Benign Positive\n-> Part 15 tuning intake"]
H -->|cannot confirm either way| K["Disposition: Unable to Determine\nreason: what's missing"]
H -->|malicious| L["Disposition: True Positive"]
F --> M["Part 15 tuning intake\n(add to RS- set)"]
L --> N["Meets Section 4\nescalation criteria?"]
N -->|yes| O["Escalate: hand off to\nCase / Incident process"]
N -->|no, contained/minor| P["Document and close,\nno escalation needed"]
K --> Q["Missing info arrives\nlater?"]
Q -->|yes| D
Q -->|no, ages out| R["Close as Unable to Determine,\nreason recorded"]
```

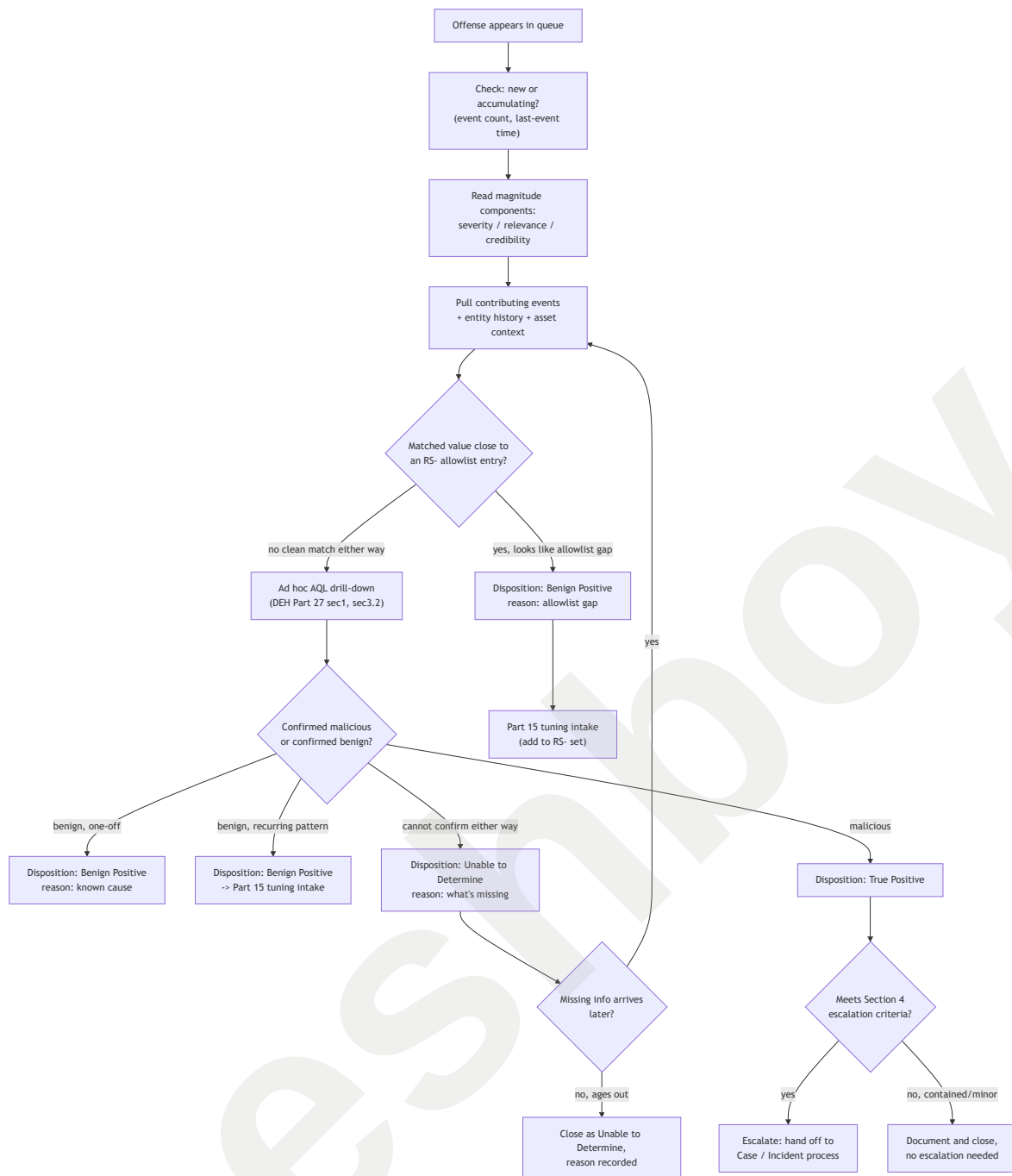


Figure 14.1 — Per-Offense triage decision path. CONCEPTUAL. Illustrates this part's own decision sequence — queue-state check, magnitude-component read, first-look pull, the allowlist-proximity shortcut, the drill-down fork into the four disposition outcomes, and the tuning/escalation handoffs into Parts 15 and the Case/Incident process. Not a reproduction of any QRadar-provided workflow diagram or a claim about a specific console feature automating this path; it is this book's own representation of a repeatable analyst procedure built on top of the architecture DEH Part 27 §6 and this book's Part 3 already establish.

[ANALYST] The two shortcut branches worth calling out explicitly: a matched value that's obviously and cleanly on the allowlist already (not shown as a separate branch above because a Rule correctly chaining a Reference Set check should not have fired at all in that case — see DEH Part 27 §2.2) and a matched value that's obviously, unambiguously malicious with no plausible benign explanation, which

should skip straight to escalation without spending the shift's limited drill-down time re-confirming what's already clear. The decision path exists to structure the *ambiguous* middle case, which is most of what a real queue actually contains — the false economy in a triage workflow is treating every Offense as equally uncertain when most queues have a recognizable long tail of "this again" alongside a much smaller set of genuinely novel matches.

5.1 What a shift handoff needs to record

[ANALYST] An Offense still open at shift change needs more than "I looked at it" passed to the next analyst. At minimum: which step in Figure 14.1's path the review had reached, what evidence has already been pulled (so the next analyst doesn't re-pull the same event list from scratch), and what specific missing piece — if any — is blocking a disposition. A queue-management practice worth adopting from DEH [TERMINOLOGY.md](#)'s own Triage discipline: an open Offense with no note at all reads, to the next shift, identically to an Offense nobody has touched yet — which wastes the first analyst's work and risks a duplicate escalation or a duplicate dismissal from two analysts reasoning independently about the same evidence.

Rule Autopsy *The rule:* `R: Suspicious LSASS Access – Unapproved Process` (DEH Part 27 §3.3), deployed correctly with `BB:LSASS-Access-Candidate` and `RS-ALLOWlisted-LSASS-Tools` all wired together as designed. **Why it shipped:** The Rule Autopsy pattern here is not the Rule's logic — that's DEH Part 27's own worked example, and it's sound — it's a triage-workflow gap around it: an analyst disposed three consecutive Offenses from this Rule as Benign Positive with no reason code, no note, and no Part 15 tuning-intake ticket, because the shift was busy and "same as last time" felt self-evident. **How it failed:** A fourth Offense from the same Rule, actually a True Positive using a renamed copy of a legitimate tool's binary, got the same fast "same as last time" disposition from a different analyst who had no recorded reasoning from the first three to compare against — only a pattern of three quick Benign Positive closes with nothing explaining why, which read as routine rather than as a queue that should have already had a Part 15 tuning ticket open and, more importantly, a documented reason the prior three were safely benign. **The fix:** No disposition closes without a reason code, full stop, regardless of how routine it feels in the moment — §3's reason-code discipline is the actual fix, not a smarter Rule. The Rule's logic was never the defect; the missing paper trail was.

6. Cross-role handoffs this workflow depends on

[ANALYST] Two boundaries worth naming so this part doesn't quietly claim ownership of decisions that belong elsewhere in this book's tag taxonomy:

- **Tuning a Rule, Building Block, or Reference Set is** **[RULE ENGINEER] work**, not an extension of triage. An analyst's disposition and reason code are the *input* to a tuning decision (Part 15's intake process consumes exactly this data); the analyst does not themselves edit `RS-Allowlisted-LSASS-Tools` or the Building Block's Rule Test as part of closing an Offense, even when the fix looks obvious from the triage seat. This boundary exists for the same change-control reason DEH Part

27 §2.2 flags a Reference Set's unmanaged growth as a drift risk — an allowlist entry added ad hoc, without the review Part 9 and Part 15 both require for `BB:/RS-` changes, is exactly how an allowlist becomes the next `Taskmgr.exe`-shaped evasion gap DEH Part 27 §3's own Blind Spot describes.

- **Relevance weighting drift is [PLATFORM ENGINEER] territory (Part 20)**, not something an analyst corrects by mentally discounting a specific asset's Offenses shift after shift. If an analyst repeatedly finds that magnitude's relevance component doesn't match the asset's actual current importance — a decommissioned host still weighted as high-value, a newly critical host still weighted as generic — that observation is worth escalating as an asset-model staleness finding, not silently worked around at the triage layer indefinitely.

[SOC MANAGEMENT] The reason-code discipline in §3 is not free — it costs an analyst a few extra seconds per closed Offense, multiplied across an entire queue's volume every shift. It is worth budgeting that time explicitly rather than treating disposition speed alone as the throughput metric a SOC manager optimizes for: a queue closed fast with no reason codes produces exactly the ambiguous, unusable Disposition data DEH `TERMINOLOGY.md`'s own Disposition entry warns against, and every downstream metric this book or DEH discusses — Part 15's tuning backlog, Part 22's staffing model — is built on Disposition data that has to actually mean something to be worth aggregating.

What Would Change My Mind *This part treats reason-coded disposition at the point of triage as worth the added seconds per Offense. If a SOC running this discipline for a full quarter found no measurable difference in Part 15's tuning-intake accuracy or escalation-quality outcomes between reason-coded closures and bare four-value dispositions — genuinely no signal, not just an unmeasured assumption — that would be a real argument for dropping the reason-code requirement on high-volume, low-severity Rule types specifically, while keeping it for anything that can escalate. This part has not seen that data; it states the discipline as the default, not as a claim already proven against a production queue this book has access to.*

Cross-references: DEH Part 27 §1 and §3.2 (AQL syntax and the drill-down query form this part's §2 builds on, not re-teaches), DEH Part 27 §6 (the Offense-as-Alert/Case-hybrid finding this part's whole workflow inherits), DEH Part 23 (query-language strategy generally, and the field-mapping-gap discipline behind confirming a drill-down search actually returns the fields it claims to), DEH `TERMINOLOGY.md` (Alert, Triage, Disposition, Case, Incident, Tuning, Suppression — all inherited unchanged, none redefined here); this book's Part 3 (magnitude/credibility/relevance scoring model), Part 15 (the systemic tuning-intake workflow this part's disposition data feeds), Part 20 (asset model and relevance weighting), and Part 22 (staffing and review-cost consequences of the disposition discipline in §6).

Part 15 — Noisy-Offense Triage and the Tuning Workflow

Why this part exists

[CONCEPT] Part 14 ended at a fork: an analyst disposes an Offense as Benign Positive, attaches a reason code, and — if the cause looks likely to recur — files a tuning-intake request rather than just closing the queue item and moving on. That fork is where Part 14's scope stops on purpose. One analyst reasoning about one Offense cannot see whether "same as last time" is actually the fortieth Offense this week from the same `R: / BB: / RS-` combination, whether the underlying cause is a Building Block condition that was always too broad, or whether a Reference Set that should have blocked the match silently missed one entry. This part picks up exactly there: not "what does an analyst do with one noisy Offense," but "what does a Rule Engineer do when a Rule, Building Block, or Reference Set is flooding the queue systemically, and how does a team run that diagnosis-and-fix cycle repeatably on a platform that hands you no CI gate for free."

DEH `TERMINOLOGY.md` §4 draws the line this whole part stands on: **Tuning** is a deliberate, logged change to a detection's own logic, thresholds, or exclusions; **Suppression** is a mechanism that stops an otherwise-matching condition from reaching an analyst without touching that logic. That distinction is inherited here unchanged, per this book's `STYLE-GUIDE.md` §12 — this part does not redefine it, it operationalizes it against QRadar's specific objects: a `BB:` Rule Test condition, an `RS-` Reference Set's membership, and a Rule's chaining logic are the concrete things a Tuning change edits; a Response Limiter or a time-bounded allowlist entry are the concrete things a Suppression mechanism edits instead. DEH Part 27 §2.1–§2.2 and §3.3 already built the vocabulary this part diagnoses against — Rule Tests, Building Blocks, and the Reference Set pattern that stands in for a join QRadar's correlation model cannot express natively — and this part assumes that mechanics discussion is settled rather than re-explaining it. Where a diagnosis step needs an ad hoc query against historical events, this part points to DEH Part 27 §1 and §3.2 for AQL syntax and DEH Part 23 for the broader question of when a native, backend-specific query is the right tool rather than a portable one — it does not re-teach either.

This part is scoped to the recurring, steady-state discipline of keeping a Rule base tuned. Part 16 owns the harder problem this part deliberately does not solve: what a disciplined team builds to replace the git/PR/CI pipeline DEH Part 22 assumes is available, for the *change-management* half of a tuning edit (tickets, approvals, a staging tenant, rollback). Part 18 owns the acute version of the same failure — an offense-flooding incident in progress, walked as symptom → cause → fix under incident-response time pressure — where this part is the discipline that should have caught the same Rule before it flooded anything.

1. Diagnosing a flooding Rule: symptom versus cause

[RULE ENGINEER] A Rule that is producing far more Offenses than it should always looks the same from the queue: a spike in Offense count against one `R:` name. It does not always have the same cause, and treating every flood as "the Rule Test is wrong" leads to fixes that don't hold. Three distinct failure

surfaces produce the identical symptom, and a diagnosis has to tell them apart before touching anything:

- **The Building Block's Rule Test condition is genuinely too broad** for what the analytic is trying to catch — it matches a wider set of real-world activity than the analytic intends, independent of any exclusion list. DEH Part 27 §4's generalized failed-authentication example (`DET-27-02`) is exactly this shape at the design stage: a Building Block written to span every Log Source type QRadar has mapped to an authentication-failure category, rather than one known-consistent source, inherits a cross-source conflation risk before a single Reference Set is even in the picture.
- **The Reference Set that should be excluding known-legitimate activity is missing an entry, or the entry it has no longer matches.** DEH Part 27 §3.3's own worked example — `BB:LSASS-Access-Candidate` chained against `RS-Allowlisted-LSASS-Tools` — floods for this reason specifically when an EDR or AV agent's own binary path changes after a vendor upgrade and the allowlist entry, written against the old path, stops matching. The Building Block's logic did not change; the environment moved out from under a static exclusion list.
- **The threshold or aggregation window is miscalibrated for the population it's actually evaluating,** not for the population it was tuned against originally. A Rule built and validated against one business unit's traffic pattern, then deployed fleet-wide without re-checking whether a legitimately bursty but benign population (a backup job's authentication retries, a scheduled scan's connection volume) crosses the same threshold the original analytic assumed only an attacker would cross.

[RULE ENGINEER] These three surfaces point to different fixes, and confusing them is the single most common way a tuning change makes things worse instead of better. The table below is the first diagnostic pass — before opening the Rule Wizard, work out which row the flood actually belongs to.

Symptom	Likely cause	Where to look first	Tuning-shaped fix
Same R: fires across many unrelated entities, no obvious common exclusion	Building Block condition too broad for the analytic's real intent	BB: Rule Test logic itself — is it matching a category, not a specific pattern?	Narrow the Rule Test (a specific QID — QRadar's internal event-taxonomy identifier — or QID range, a specific property value set) — DEH Part 27 §3.1's QID-mapping discipline applies before narrowing further.
Same R: fires repeatedly against a small, recognizable set of known-legitimate sources	Reference Set missing an entry, or an entry that no longer matches (path/hash drift after an agent upgrade)	RS- membership, checked against the actual matched value on a recent Offense, not just the exclusion list's intent	Add or correct the specific RS- entry; re-verify the match format (path string, hash, case sensitivity) against what the DSM actually extracts.
A threshold-based R: fires on a population that wasn't part of the original validation	Threshold or grouping key miscalibrated for a wider or different population than tuned against	The Rule Test's grouping key (DEH Part 27 §4's sourceip / username example) and which Log Source types actually feed the Building Block behind it	Narrow the Building Block to a specific Log Source type, or split into population-specific Rules with separately tuned thresholds, rather than raising one shared threshold for everyone.
Offense count climbs gradually over weeks with no single triggering change	Reference Set drift — entries never expired, never re-reviewed, or a legitimate change (org restructuring, tool retirement) never reflected	RS- TTL configuration (Part 10) and last-reviewed date, if your team tracks one	Scheduled Reference Set review, not a one-off fix — this symptom is a governance gap (Part 9), not a single bad entry.

PRODUCT VERSION NOTE Whether QRadar's console exposes a built-in "top firing Rules" or "Offense count by Rule" report out of the box, and where that report lives in the Admin or Offenses tab, are console specifics that have moved across releases. The diagnostic question this section asks — which Rule, Building Block, and Reference Set combination is actually producing the volume — is answerable in every recent release through some combination of the Offense list's own grouping/filtering and an ad hoc AQL search grouped by rule name and time; verify the specific built-in reporting surface your deployment offers before building a runbook around a menu path this part does not claim as fixed.

1.1 Confirming the cause before changing anything

[RULE ENGINEER] Every row in §1's table has the same next step: confirm the cause against real matched values before editing the Building Block, the Reference Set, or the Rule. Pull a representative sample of the flooding Offenses' contributing events — the same first-look pull Part 14 §2 already teaches at the single-Offense level, just run across a sample large enough to see the pattern rather than one instance of it. A Building Block that looks too broad in the abstract sometimes turns out to be correctly scoped, with the real cause sitting in a Reference Set that just happens to be stale for one specific tool — narrowing the Building Block in that case would have fixed today's symptom while

quietly reopening exactly the evasion gap DEH Part 27 §2.3's Blind Spot warns about for an under-scoped two-stage chain.

Noisy Offense Trap *A Rule Engineer under queue-volume pressure reaches for the fastest available lever — widening a threshold, adding a broad wildcard exclusion, or simply disabling the Rule — because any of those stops the flood immediately and the pressure to clear the backlog is real. Each of those three moves also removes detection coverage the analytic was built to provide, and none of them is a diagnosis: a threshold widened without checking whether the flood was a Reference Set gap just made a genuinely malicious burst below the new threshold invisible too. The fix that actually holds is almost always narrower and slower than the fix that stops the pain fastest — confirm the cause per §1.1 before choosing which lever to pull, even when the queue is on fire.*

2. The console-native constraint: tuning with no CI gate

[PLATFORM ENGINEER] A team used to DEH Part 22's detection-as-code model reaches for a branch, a pull request, and a CI pipeline that runs a regression suite before a rule change merges. QRadar's Rule Wizard, Building Block editor, and Reference Set management screens do not hand a team that pipeline — a saved edit to a **BB:** Rule Test or an added **RS-** entry takes effect immediately, evaluated by the Custom Rules Engine against live traffic from the moment it's saved, with no staging environment most licenses include and no automated test gate between the edit and production.

This part is scoped to the diagnosis-and-fix cycle that happens *within* that constraint, not to replacing it — Part 16 covers what a disciplined team builds instead (change tickets, a rule revision-history-based rollback plan, a staging tenant where licensing allows one, manual regression testing against known-positive events before and after a change). What this part assumes from Part 16, without re-deriving it: every tuning change gets logged somewhere a later reviewer can find — what changed, why, and what was tested — because the console's own revision history, where available, records *that* an object changed and by whom, not necessarily *why* in terms a future Rule Engineer can act on without asking the person who made the change.

PRODUCT VERSION NOTE *Whether a given license tier or deployment includes a separate staging/test tenant to evaluate a Rule or Building Block edit before it touches production is a packaging and licensing question, not an architectural constant — it has varied across QRadar offerings and sales agreements. Part 16 covers what to build "where licensing allows" one; do not assume the absence of a staging tenant applies uniformly to every deployment without checking your own license and deployment model first.*

PRODUCT VERSION NOTE *Rule and Building Block revision-history depth — whether a deployment can show a prior version of a Rule Test's exact condition and revert to it directly from the console, versus only showing a last-modified timestamp and user — has varied across QRadar releases and is not a capability this part assumes exists uniformly. Confirm what your specific deployment's revision history actually retains before building a rollback plan around a console feature that may show less detail than a git-based diff would.*

3. Tuning versus Suppression, at the systemic level

[RULE ENGINEER] Part 14 §3.1 drew this fork at the point of one Offense's disposition. At the systemic level the same fork decides which of two different objects gets edited, and getting it backward is how a temporary fix quietly becomes permanent scope creep or a permanent fix gets treated as disposable:

- **Tuning** edits what the Rule, Building Block, or Reference Set actually matches — adding a corrected **RS-** entry, narrowing a Rule Test's condition, splitting an overbroad Building Block into population-specific ones. This is the right tool when the cause is a **permanent** mismatch between the analytic's logic and the environment it runs against: a tool that will always be there, a population the analytic was never validated against, a field format the DSM will always emit this way.
- **Suppression** stops a match from reaching the queue without touching the logic underneath — a Response Limiter capping how many times a Rule contributes to an Offense or fires a notification within a window, or a time-bounded **RS-** entry with a TTL scoped to a known, temporary condition. This is the right tool when the cause is **bounded**: a scheduled penetration test, a one-off maintenance window, a vendor's temporary scanning campaign that will end on a known date.

[RULE ENGINEER] The failure mode DEH **TERMINOLOGY.md**'s Tuning Debt concept names — a known false-positive source identified repeatedly but never fed back into the rule — is exactly what happens when a Suppression mechanism gets used past its bounded window because it was easier to leave in place than to circle back and ask whether the cause turned out to be permanent. A Response Limiter silences the symptom every time regardless of cause; it produces a quieter queue whether the underlying condition was ever actually tuned or not, which makes it a dangerous default to reach for first rather than a considered choice for a genuinely temporary condition.

Situation	Right mechanism	Why
AV/EDR agent upgrade changed a known tool's binary path	Tuning — correct the RS- entry	Permanent environmental fact; the old path will never come back.
A scheduled two-week penetration test will generate expected LSASS-access activity from an authorized tool	Suppression — time-bounded RS- entry with a TTL matching the engagement window, or a Response Limiter for the engagement's duration	Bounded, known-end condition; the exclusion should expire on its own rather than requiring a second person to remember to remove it.
A Building Block spans multiple Log Source types and floods on one specific source's normal behavior	Tuning — narrow the Building Block or split by Log Source type	The mismatch is structural, not temporary — it will recur every day until the logic changes.
A single noisy source is under active incident-response investigation and the team wants matches to keep accumulating evidence without paging on-call every few minutes	Suppression — Response Limiter on the notification path only, Offense still accumulates contributing events	The team wants the detection logic fully intact and still recording; only the paging volume is the problem.

PRODUCT VERSION NOTE *Response Limiter configuration — exactly which Rule types expose it, what granularity it rate-limits at (per Rule, per Response action, per matched entity), and whether it is named "Response Limiter" consistently across the Rule Wizard in your specific release — has shifted in wording and placement across QRadar versions. The underlying capability (cap how often a Rule's Response fires within a window, independent of how often the underlying condition matches) is architecturally stable; confirm the exact configuration surface against your own deployment before writing a runbook step that names a specific wizard screen.*

4. A repeatable tuning-intake workflow

[RULE ENGINEER] The workflow below is this part's core deliverable: a four-stage pipeline a team can run every time a Rule, Building Block, or Reference Set floods, so the outcome doesn't depend on which Rule Engineer happened to be on shift that day.

4.1 Intake

[SOC MANAGEMENT] A tuning request needs a minimum data set to be actionable, and the most common reason a tuning backlog stalls is that requests arrive without it. At minimum, an intake record needs: which **R: / BB: / RS-** combination is involved, how many Offenses over what window, a sample of the reason codes Part 14 §3 already requires analysts to attach at disposition time, and — if the requesting analyst has one — a guess at which of §1's three failure surfaces looks most likely. A queue full of reason-coded Benign Positive dispositions naming the same allowlist gap is a far stronger intake signal than an analyst's Slack message saying "this Rule is noisy again," precisely because Part 14's disposition discipline exists to make that signal available in the first place.

4.2 Diagnosis

[RULE ENGINEER]

Apply §1's diagnostic table against the intake sample, then confirm the hypothesis against real historical data before writing a single change. This is the point where an ad hoc AQL search earns its place in the workflow — DEH Part 27 §1 and §3.2 already cover the syntax and the search-cost discipline (bounded time windows, no unbounded search against a shared Ariel deployment) this step inherits without restating. Query the historical event population the flooding Rule's Building Block would have matched, with the proposed fix's condition removed or loosened, and check what the result set actually contains before assuming the fix is safe:

```
-- QRadar AQL, not standard SQL – this is a diagnostic query, not a standing rule.
-- Non-SQL behavior in play here: LAST n DAYS is AQL's time-window shorthand, not a
-- standard-SQL construct, and there is no general-purpose JOIN to correlate against a
-- second table. Syntax per DEH Part 27 sec1/sec3.2; not re-taught here. Illustrative
-- property names ("Source Process Name") per this book's convention of following DEH
-- Part 27's own worked LSASS example – confirm your own custom property names against
-- Log Activity > Advanced Search before reuse.
SELECT "Source Process Name", COUNT(*) AS "Occurrences"
FROM events
WHERE QIDNAME(qid) = 'Process accessed'
  AND "Target Process Name" ILIKE '%lsass.exe%'
LAST 30 DAYS
GROUP BY "Source Process Name"
```

Grouping the flooding population by the field a proposed RS- fix would key on shows, directly, whether one dominant source explains the volume (a strong candidate for a clean Tuning fix) or whether the volume is spread across many distinct, unfamiliar values (a signal that the real cause is closer to §1's "Building Block too broad" row than "one missing allowlist entry"). This diagnostic query is itself a QRadar-native artifact, not a portable one — DEH Part 23 §5's finding about when Sigma-first authoring pays off and when it doesn't applies here in the negative: a diagnosis built against this platform's own RS- / BB: constructs and this platform's own Offense population has no meaningful portable form, and isn't trying to have one. It exists to answer one platform-specific question, once, and gets discarded or promoted into documentation afterward, the same way DEH Part 27 §5's hunting queries are used and retired.

4.3 Change

[RULE ENGINEER]

Make the confirmed fix — the RS- entry, the narrowed Rule Test, the split Building Block — through whatever change-authorization your team requires (Part 9's Building Block governance discipline and Part 16's fuller change-management model both apply; this part does not repeat either). Record what changed and why in whatever log Part 16's model designates as the record of truth, even if that log is a change ticket rather than a git commit message — the "why" matters more here than in a git-based workflow precisely because there is no diff a future reader can inspect to reconstruct intent.

4.4 Re-validation

[RULE ENGINEER]

A tuning change is not done when it stops the flood — it's done when two things are both confirmed: the flood has actually stopped, and the analytic still catches what it was built to catch. Skipping the second half is how a tuning fix becomes a coverage regression nobody notices until a red-team exercise or a real incident finds the gap.

Validation Test Setup: A tuning change has just been made to `BB:LSASS-Access-Candidate` or `RS-AllowListed-LSASS-Tools` in response to a confirmed diagnosis (§4.2) — for example, correcting an allowlist entry's binary path after an EDR agent upgrade. **Action:** Re-run §4.2's historical AQL query against the same window to confirm the previously-flooding source no longer appears in the unmatched population, then separately run DEH Part 27 §3.3's own Detection Test — a credential-dumping tool against a test host's live LSASS process — to confirm the Rule still fires on genuinely unapproved access. **Expected result:** The corrected source's activity no longer produces new Offenses from `R: Suspicious LSASS Access – Unapproved Process`, and the Detection Test's known-malicious pattern still produces one. If the flood stopped but the Detection Test also stopped firing, the fix over-corrected — the Building Block itself was narrowed rather than the Reference Set, or the Reference Set entry was written broader than the specific tool it needed to cover — and the change should be rolled back to the state Part 16's revision history retains, not patched further under continued time pressure.

flowchart TD

```
A["Intake: R:/BB:/RS- combo,\nOffense count + window,\nreason-coded dispositions\n(Part 14 sec3)"]
--> B["Diagnosis: apply Section 1\nfailure-surface table"]
B --> C{"Which surface?"}
C -->|"BB too broad"| D["Narrow Rule Test\n(specific QID / property set)"]
C -->|"RS- gap or drift"| E["Correct RS- entry\n(path, hash, format)"]
C -->|"Threshold miscalibrated"| F["Split by population /\nnarrow Building Block scope"]
D --> G["Confirm hypothesis:\nad hoc AQL against historical\nevents (Section 4.2)"]
E --> G
F --> G
G --> H{"Bounded or\npermanent cause?\n(Section 3)"}
H -->|"permanent"| I["Tuning change,\nauthorized per Part 9 / 16"]
H -->|"bounded, known end"| J["Suppression: TTL entry\nor Response Limiter"]
I --> K["Re-validation (Section 4.4):\nflood stopped +\nDetection Test still fires"]
J --> K
K -->|"both confirmed"| L["Close tuning request,\nlog change per Part 16"]
K -->|"Detection Test fails"| M["Rollback via revision\nhistory, re-diagnose"]
M --> B
```

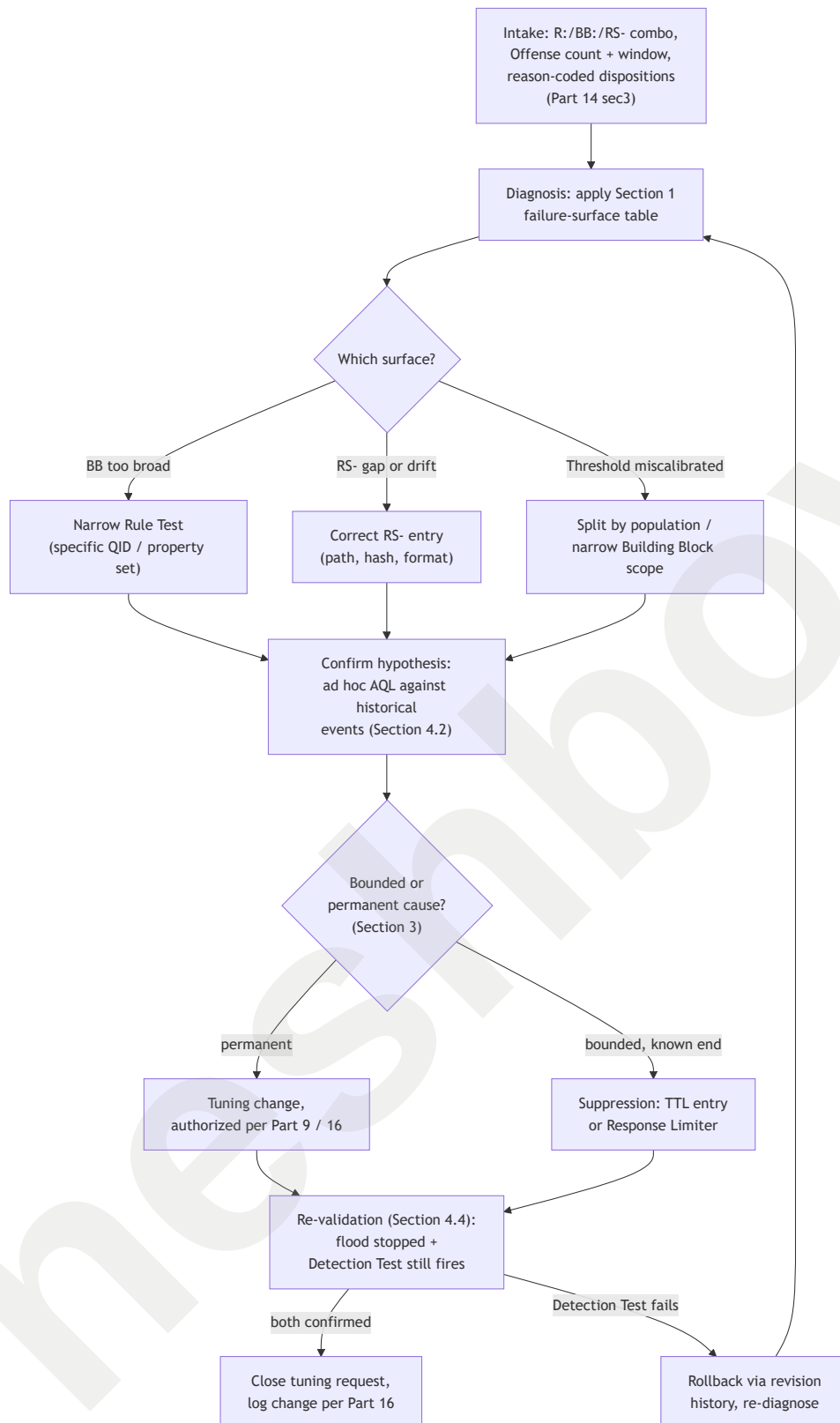


Figure 15.1 — The tuning-intake pipeline: request, diagnosis, change, re-validation. *CONCEPTUAL.* Illustrates this part's own four-stage workflow, including the Tuning-vs-Suppression fork from Section 3 and the re-validation loop that catches an over-corrected fix before it ships as a silent coverage regression. Not a reproduction of any QRadar-provided workflow feature or a claim that the platform

automates this sequence — it is this book's own representation of a repeatable practice built on top of the object model DEH Part 27 §2.1–§2.2 and §3.3 already establish.

5. Case walkthrough: the LSASS Rule, flooded and tuned

[RULE ENGINEER] DEH Part 27 §3.3's False Positive Trap named the mechanism in one sentence: an AV/EDR agent upgrade can change a scanning process's binary path and either reopen a false-positive flood or silently stop matching an exclusion nobody re-tested. This walkthrough is that sentence run through §4's pipeline end to end.

Rule Autopsy *The rule:* `R: Suspicious LSASS Access – Unapproved Process`, chaining `BB:LSASS-Access-Candidate` against `RS-ALLOWlisted-LSASS-Tools` (DEH Part 27 §3.3). MITRE ATT&CK T1003.001 (OS Credential Dumping: LSASS Memory), mapping inherited from DEH Part 23/27 and not re-derived here. **Why it shipped:** `RS-ALLOWlisted-LSASS-Tools` was populated by full binary path for the environment's EDR agent, correctly, at the time it was written — the allowlist did its job for months. **How it failed:** A fleet-wide EDR agent upgrade moved the scanning binary to a new installation path as part of the vendor's own directory restructuring. `RS-ALLOWlisted-LSASS-Tools` still held the old path. Every endpoint running the upgraded agent started generating a `BB:LSASS-Access-Candidate` match that no longer found its source process on the allowlist, and `R: Suspicious LSASS Access – Unapproved Process` began firing across the fleet within hours of the upgrade's rollout — hundreds of Offenses, all from one root cause, all reading identically to a genuine credential-dumping wave at first glance. **The fix:** Intake (§4.1) arrived within the hour as a spike of reason-coded Benign Positive dispositions all naming the same EDR process. Diagnosis (§4.2) ran the historical AQL query grouped by `"Source Process Name"` and found one dominant, single value driving the entire spike — the new EDR path, not a spread of unfamiliar tools, which pointed cleanly at §1's Reference Set row rather than a Building Block or threshold problem. The change (§4.3) corrected `RS-ALLOWlisted-LSASS-Tools`'s entry to the new path, logged with the EDR vendor's change bulletin as justification. Re-validation (§4.4) confirmed the flood stopped and separately confirmed the Detection Test's credential-dumping tool still fired.

[RULE ENGINEER] The version of this same flood that goes wrong looks almost identical up to the diagnosis step, with one difference: an under-pressure Rule Engineer, seeing hundreds of Offenses and a queue backing up, disables `R: Suspicious LSASS Access – Unapproved Process` entirely rather than running §4.2's confirmation query first. The flood stops immediately — and so does every legitimate detection of the technique the Rule exists to catch, for however long it takes someone to notice the Rule is disabled and ask why. This is the exact shape of §1.1's Noisy Offense Trap, playing out with a real deadline attached rather than a hypothetical one.

[RULE ENGINEER] A second, less tidy example belongs in this walkthrough because it doesn't resolve as cleanly as the LSASS case: DEH Part 27 §4's generalized failed-authentication Rule (`DET-27-02`), built on QRadar's native threshold Rule Test across every Log Source type mapped to an authentication-failure

category. That Rule floods differently — not from one identifiable source, but from cross-source conflation, where a `username` shared coincidentally across two unrelated identity namespaces (an SSH account and an unrelated application login sharing a common name like `admin` or `svc-backup`) merges into one grouping and crosses the threshold on combined, unrelated activity. Diagnosis here does not find one dominant value to add to an allowlist; it finds that the Building Block itself needs to be split by Log Source type, because the grouping key was never wrong, but the population it was applied across was too broad from the start. This is why §1's diagnostic table treats "Building Block too broad" and "Reference Set gap" as genuinely different rows rather than two names for the same fix — the LSASS case needed a data correction, this case needs a logic change, and running the wrong one first (adding usernames to an exclusion list instead of splitting the Building Block) would have chased an ever-growing allowlist without ever closing the actual gap.

6. Measuring tuning debt and reporting it honestly

[SOC MANAGEMENT] DEH `TERMINOLOGY.md`'s Tuning Debt concept — a known false-positive source identified repeatedly but never fed back into the rule — is directly measurable from the intake records §4.1 asks a team to keep, and a SOC manager budgeting a QRadar practice should be tracking it as its own metric rather than folding it into a generic "queue volume" number that hides where the actual cost sits.

Metric	What it tells you	Reads as a problem when
Open tuning requests, by age	Backlog size and staleness	Requests routinely age past a week with no diagnosis started — intake is outpacing Rule Engineer capacity.
Offenses suppressed vs. Offenses tuned, per period	Whether the team is fixing causes or muting symptoms	Suppression count grows relative to Tuning count over successive periods — Response Limiters and time-bounded exclusions accumulating past their intended bounded use.
Mean time from intake to re-validated fix	Process health, not individual speed	Consistently long even for single-dominant-cause diagnoses (the LSASS case above) — a sign the pipeline itself, not the diagnosis difficulty, is the bottleneck.
Repeat intake requests against the same <code>R: / BB: / RS-</code> combination	Whether a prior "fix" actually held	A Rule that generates a new tuning request every few months against the same root cause — the earlier fix treated a symptom, not the cause, or the Reference Set drifted again with no scheduled review in place.

SOC Management View *A tuning-intake pipeline that reports "flood resolved" without recording which of Tuning or Suppression resolved it, and without a re-validation step confirming the Detection Test still passes, produces exactly the same false confidence a bare offense-count dashboard produces for detection coverage generally (Part 13's own caution against that metric). Budget the pipeline's overhead explicitly — the diagnosis query, the Detection Test re-run, the change log entry — as the cost of a tuning fix that actually holds, rather than measuring the team only on how fast the queue quiets down. A fast fix that was really a Suppression wearing a Tuning label shows up as solved this quarter and reopens, with interest, the quarter an auditor or a red team actually checks whether the underlying gap ever closed.*

[SOC MANAGEMENT] The same review-cost argument DEH Part 27's own closing SOC MANAGEMENT paragraph makes about a QRadar detection's three separately versioned objects applies again here, sharpened: reviewing whether a tuning fix was correct means reviewing which of the `BB:`, `RS-`, or `R:` chaining logic actually changed, not just confirming the Offense count went down. Staff and schedule that review the same deliberate way Part 9 asks a team to review a new Building Block before it ships — a tuning change is a production change to live correlation logic, not a queue-management housekeeping task, even when it looks like one from the analyst's side of the fork.

7. Where this part's scope ends

[PLATFORM ENGINEER] This part covers the diagnosis-and-fix discipline; it does not cover the paperwork and rollback infrastructure around it, and it does not cover what to do when the flood is bad enough to be an active incident rather than a queued tuning request. Part 16 owns the change-management ceremony — tickets, staging where licensing allows it, revision-history-based rollback — this part's §4.3 gestures at without designing. Part 18 owns the acute failure-mode version of this same problem, walked as a live troubleshooting scenario under time pressure rather than a steady-state intake queue; its offense-flooding decision tree assumes this part's diagnostic table as a starting point and adds the specific triage speed a live incident demands. Part 10's Reference Set TTL and governance discipline is the upstream fix that prevents §1's "gradual drift" row from recurring at all — a Reference Set reviewed on a schedule floods from drift far less often than one nobody revisits until it's already causing a problem.

Cross-references: DEH Part 27 §2.1 and §2.2 (Rule Test, Building Block, and Reference Set mechanics this part diagnoses against, not re-derives), DEH Part 27 §3.3 (the `BB:LSASS-Access-Candidate/RS-Allowlisted-LSASS-Tools/R: Suspicious LSASS Access` worked example this part's §5 walkthrough extends), DEH Part 27 §4 (the generalized threshold-rule example and its cross-source-conflation Blind Spot, extended in §5), DEH Part 27 §1 and §3.2 (AQL syntax used in §4.2's diagnostic query, not re-taught), DEH Part 23 §5 (when native, non-portable query authoring is the right tool — applied to this part's diagnostic queries), DEH `TERMINOLOGY.md` §4 (Tuning, Suppression, Exception, Tuning Debt, Alert Fatigue — inherited unchanged, none redefined); this book's Part 3 (magnitude/credibility/relevance scoring

underlying an Offense's queue position), Part 9 (Building Block governance this part's change step relies on), Part 10 (Reference Set TTL and drift discipline), Part 14 (the disposition and reason-code discipline that feeds this part's intake stage), Part 16 (the change-management model this part's §4.3 defers to), Part 18 (the acute offense-flooding troubleshooting scenario this part's steady-state discipline exists to prevent), and Part 22 (staffing and review-cost consequences of the tuning-intake pipeline in §6).

neshboy

Part 16 — Rule Change Management Without a Detection-as-Code Pipeline

Why this part exists

[CONCEPT] DEH Part 22 built a whole discipline on one mechanical premise: a Detection Rule is a piece of software with a lifecycle, and every failure mode software engineering already solved — an untested change shipping broken, one person's typo reaching production with no second set of eyes, nobody knowing which version is actually running, a bad change with no way back — happens to detection rules too, usually with less visibility, because a broken rule doesn't crash or throw an error. DEH Part 22's fix is a git-based rule repository: a protected branch, a pull request per change, automated lint and regression tests that gate the merge button, and a three-role separation of duties (author, reviewer, merge-approver) enforced by branch protection settings rather than trusted to memory.

QRadar's Rule Wizard does not hand a team that pipeline for free. A **Rule**, a **BB:**-prefixed Building Block, or an **RS-**-prefixed Reference Set is edited directly in the Console by anyone holding the right administrative role, saved, and — subject to the propagation behavior this part flags with a **PRODUCT VERSION NOTE** below — live. There is no protected branch a change has to merge into, no required status check that blocks a save the way a red CI job blocks a merge button, and no platform-native concept of "this Rule is under review" that stops a second administrator from editing it mid-review. This is the honest gap Part 9 already named for Building Block governance at scale and Part 15 already named for the tuning-intake workflow; this part is where the gap gets a full, dedicated treatment rather than a paragraph each.

None of what follows is a claim that QRadar secretly has a hidden CI system this book failed to find. It's the opposite: a console-native platform genuinely does not give a team DEH Part 22's structural guarantees, and pretending otherwise — assuming "we have a change-ticket process" delivers the same protection as "the merge button is red until tests pass" — is exactly the kind of process-versus-mechanism confusion DEH Part 22 §3 warns against for its own git-based model. What a disciplined team builds instead is a set of compensating practices: a change-ticket workflow standing in for a pull request, a manual export-before-you-edit habit standing in for **git log**, a staging environment where licensing allows one, and a before/after regression-testing discipline built on the AQL search syntax DEH Part 27 §1 and §3.2 already teach. Each one is weaker than its DEH Part 22 counterpart in a specific, named way; this part states each weakness plainly rather than dressing up a paper process as equivalent to a CI gate.

This part does not re-teach AQL syntax — every baseline-capture query below points back to DEH Part 27 §1 and §3.2 rather than re-explaining **SELECT / WHERE / LAST n HOURS** from scratch — and it does not re-argue Sigma-first-vs-native authoring, which DEH Part 23 §5 already resolves generically: Sigma-first authoring pays for itself when the same logic runs on more than one backend, and pays for itself less clearly in a single-platform shop that has invested in that platform's own native constructs. A QRadar-only shop's Building Blocks, Reference Sets, and Rules are exactly that second case — natively authored,

natively edited, with no Sigma intermediary and no git repository underneath them by default. This part is about the compensating discipline that gap requires.

1. The gap, named precisely

[PLATFORM ENGINEER] Table 16.1 maps DEH Part 22's pipeline stages directly onto what QRadar's Console gives a team at each stage, without softening the comparison — the point of naming the gap this specifically is that a vague "QRadar is less mature here" claim doesn't tell an engineer which specific control they need to build by hand.

Table 16.1 — DEH Part 22 pipeline stage vs. QRadar's console-native equivalent. Supports deciding exactly which control a team has to build itself, rather than assuming any given DEH Part 22 guarantee transfers for free.

DEH Part 22 pipeline stage	What it guarantees	QRadar Console equivalent	What's missing
Git repository, one file per rule	Full change history, diff view, permanent audit trail	No native git-backed repository for <code>R: / BB: / RS-</code> objects	No diff view; revision depth is a PRODUCT VERSION NOTE item (§4)
Feature branch per change	Isolated draft, no risk to production until merged	No branching concept — an edited Rule is the live Rule	A "draft" edit and a "live" edit are the same object; there is no isolated workspace
Required CI lint/schema check	Structurally blocks a malformed change from merging	Rule Wizard field validation only (syntax-level, not logic-level)	No automated check that a Building Block's Rule Test still matches its documented intent
Automated regression test against fixtures	Objective, repeatable pass/fail before merge	None native	Manual AQL baseline-and-diff is the substitute (§6)
Human technical review, recorded approval	A named second person attested the change, structurally required to merge	No required-approval concept on a Rule save	A change-ticket approval field is a norm, not a gate (§3, §8)
Merge-approver, distinct third role	A named third person confirmed release-readiness	No merge step at all — saving <i>is</i> releasing	Same weakness as above, compounded: there is no moment between "reviewed" and "live"
CD deploy to production	Deliberate, logged promotion event	Saving the Rule Wizard form	No separate deploy action to gate or log independently of the edit itself (see §1's PRODUCT VERSION NOTE)
Rollback to a prior committed version	<code>git revert</code> , redeploy, done	None native beyond whatever revision history the Console retains	Export-before-edit is the substitute (§4)

[PLATFORM ENGINEER] The pattern across every row is the same: DEH Part 22's guarantees are *structural* — enforced by a tool that will not let a bypass happen quietly — and QRadar's console-native equivalents are, at best, *procedural*: a team can build a process that looks the same on paper, but nothing stops an administrator with the right role from skipping every step and saving the change

directly. §8 returns to this distinction as this part's closing Blind Spot: none of what follows makes a bypass impossible, only harder to do by accident and easier to catch after the fact.

PRODUCT VERSION NOTE *Whether saving a change in the Rule Wizard (or a Building Block or Reference Set edit) takes effect against live, incoming events immediately, or requires a separate explicit deploy/activate action first, and how long that propagation takes to reach every Event Processor in a distributed deployment (Part 2's component catalog), are specifics that have varied across QRadar releases and deployment topologies. Some QRadar administrative changes elsewhere in the platform (network hierarchy edits, for instance) are known to require an explicit "deploy changes" step rather than taking effect on save alone; whether Rule/Building Block edits behave the same way in your specific release is not a claim this part asserts as fixed. Verify the exact save-to-live behavior, and the propagation latency across your own distributed deployment, before treating a change as "not yet live" or "already live" without checking.*

2. What breaks first when there's no pipeline: three named failure modes

[RULE ENGINEER] Before building the compensating workflow, it's worth naming the specific ways a console-native, ungated edit actually goes wrong in practice — not as an abstract risk, but as three concrete failure shapes a change-ticket discipline is built to catch.

The silent ripple edit. A Building Block referenced by a dozen Rules gets its Rule Test tightened to fix one Rule's noise problem, without anyone checking which other Rules depend on that same **BB:** name. Part 9's governance discipline already names this risk for Building Block sprawl generally; this part's concern is narrower — the *moment* of the edit. A **BB:** edit has no analog to a compiler's "this symbol is referenced in 11 other files" warning; the only way to know is to check manually, before editing, which Rules currently reference the Building Block being changed.

The overlapping-edit collision. Two administrators, working from two different change tickets, edit the same Reference Set's TTL setting within the same afternoon, each unaware of the other's change. Without a lock or a branch, the second save silently overwrites the first — not merged, not flagged as a conflict, just gone, with no record the first administrator's intended change ever existed except whatever the change-ticket system captured on its own (§3).

The untested "obvious" fix. An analyst's Part 14 triage disposition correctly identifies that **RS-Allowlisted-LSASS-Tools** is missing an entry, and — under queue pressure — someone with Reference Set write access adds it directly, skips the change-ticket process because "it's just one line," and never checks whether the added value is specific enough (full path and hash, not a bare filename) to avoid reopening the exact evasion gap DEH Part 27 §3.3's own False Positive Trap already names for this Reference Set.

Platform Reality None of these three failure modes require malice or even carelessness in the ordinary sense — each one is a reasonable individual decision made without visibility into the platform's own dependency graph, because QRadar's Console does not surface that graph proactively. Before editing any **BB:**-prefixed Building Block, the practical habit that substitutes for a missing "used by" warning is running an AQL search (per DEH Part 27 §1's syntax, not re-taught here) against your own Rule/Building Block export or REST API listing for the literal string of the Building Block's name, to enumerate every Rule that currently references it. This is a five-minute manual check standing in for something a real dependency-aware IDE would surface instantly — worth budgeting the five minutes rather than skipping it under queue pressure.

3. The change ticket: a paper pull request

[RULE ENGINEER] A change ticket is this workflow's substitute for DEH Part 22's pull request — the single artifact that records what changed, why, who proposed it, who reviewed it, and what evidence justified calling it done. Table 16.2 defines the fields a change ticket needs to carry the same weight a PR's description and review thread carry in DEH Part 22's model, adapted from that part's metadata standard (§4 there) to the object types this book's Rule Wizard actually edits.

Table 16.2 — Required fields on a QRadar rule-change ticket. Supports giving a reviewer everything DEH Part 22's PR template gives a reviewer, in a system with no PR template.

Field	Populated by	Purpose
Object(s) changed	Requester	The exact <code>R:</code> , <code>BB:</code> , or <code>RS-</code> name(s) — never a paraphrase, per this book's STYLE-GUIDE §4 object-naming rule
Reason for change	Requester	The Tuning-vs-Suppression fork stated explicitly (DEH <code>TERMINOLOGY.md</code> 's definitions, carried forward unchanged) — is this a logic change or a filter change, and why
Dependency check	Requester, before submission	The result of §2's "what else references this Building Block" check — a named list, not "nothing that I know of"
Before-state export	Requester	The exported current definition, per §4's export-before-you-edit discipline — attached to the ticket, not just described
Proposed after-state	Requester	The literal proposed Rule Test wording, threshold value, or Reference Set entry — quoted, not summarized
Baseline AQL evidence	Requester or reviewer	The §6 before-change match count/sample, so the reviewer isn't reviewing logic in the abstract
Reviewer	A different named individual	Technical-adversarial review — does the dependency check look complete, does the baseline evidence support the claimed effect
Approver	A third named individual, where staffing allows (§9)	Confirms release-readiness; on a two-person team, may collapse with reviewer per DEH Part 22 §3's own staffing-constrained allowance, but never with the requester
Change window	Approver	When the edit will actually be made — deliberately scheduled, not "whenever," so a post-change regression check (§6) has a known point to measure from
Post-change validation evidence	Requester, after the change window	The §6 after-state AQL comparison, confirming the change produced the intended delta and nothing else
Rollback plan	Requester, before submission	The specific action to restore the before-state export if post-change validation fails

[SOC MANAGEMENT] A ticket carrying every field above is not a bureaucratic tax for its own sake — it is the only mechanism this workflow has for reconstructing, six months later, why a given Building Block's Rule Test reads the way it does. DEH Part 22's `git log` answers that question automatically; a QRadar deployment with no equivalent audit trail answers it only if the ticket discipline was actually followed at the time — which is exactly why §8's Blind Spot matters: a template with every field defined is worthless the moment someone with Admin access decides a change is too small to bother filing one for.

4. Revision history and the export-before-you-edit habit

[RULE ENGINEER] DEH Part 22's rollback mechanism is mechanical and cheap: `git revert` the merge commit, redeploy, done, with the full prior state recoverable because it was never overwritten in the first place — only ever committed alongside the new version. QRadar's Console does not give a rule author that same guarantee by default, which is exactly why the export-before-you-edit habit exists: before touching any `R:`, `BB:`, or `RS-` object under a change ticket, export its full current definition and attach that export to the ticket (Table 16.2's before-state field) *before* making the edit, not after.

This is deliberately the cheapest possible substitute for version control, not an elegant one: a manually-triggered snapshot, taken once per change, stored wherever the change-ticket system stores attachments — not a continuously tracked history. If the change-ticket discipline lapses for one edit, that edit's prior state is not recoverable from this mechanism at all, which is the direct cost of a manual habit standing in for an automatic one.

PRODUCT VERSION NOTE *Two specifics in this section are not fixed, universal capabilities and both vary across QRadar releases. First, whether the Console retains its own native revision history for a Rule or Building Block — a viewable, restorable "prior versions" list versus only a "last modified by/at" audit-log entry with no restorable content — and how far back it extends. Second, the exact mechanism for exporting a Rule's, Building Block's, or Reference Set's full definition (the QRadar REST API, a content-extension export bundle through Admin > Extensions Management, or another current path) and the format that export takes. Do not build a rollback plan that assumes native history will save a team that skipped the export-before-you-edit habit; verify both specifics against your own deployment before relying on either as this workflow's rollback attachment. Whichever export mechanism is current, the requirement that matters is that it captures the object's full logic — every Rule Test, every Building Block reference, the Reference Set's membership or TTL/type configuration — not just its name and a timestamp.*

[RULE ENGINEER] A rollback under this model mirrors the export step: re-apply the attached before-state export through the same Rule Wizard (or reimport it, if the mechanism round-trips directly), verify against a second baseline AQL check (§6) that the restored object behaves as the pre-change export implies, and close the ticket noting the rollback and its trigger. This is slower and more error-prone than `git revert` — a manual re-entry of Rule Test conditions is itself an opportunity to introduce a second mistake while undoing the first — the honest cost of not having a mechanical rollback, not a gap this workflow can fully close.

5. A staging tenant, where licensing allows one

[PLATFORM ENGINEER] DEH Part 22's CI stage runs every proposed change against test fixtures in an environment that cannot affect production, before a human even looks at it. The closest QRadar equivalent is a genuinely separate staging deployment — its own Console, its own licensed EPS/FPM budget, ideally fed a realistic sample of the same log sources — where a proposed Rule, Building Block, or Reference Set change can be deployed, evaluated against live-shaped traffic, and observed for a validation window before the same change is made in production.

[SOC MANAGEMENT] The honest constraint this section exists to state plainly: a second QRadar deployment is not a free sandbox the way a CI runner spinning up a disposable container is. It is commercial infrastructure with its own licensing cost (Part 17 covers EPS/FPM licensing and capacity planning in full), and "stand up a staging tenant" is not a recommendation this part can make without naming that tradeoff. Where a staging deployment genuinely isn't in the budget, three partial substitutes carry real, if lesser, value:

- **A disabled or clearly-labeled test copy of the Rule inside the same production deployment**, its Response deliberately configured to log or update a test-only Reference Set rather than create a live Offense, evaluated against a narrow window of real traffic before the production Rule is edited — tests against real data with no second license, at the cost of running overlapping logic side by side temporarily, which needs its own change-ticket-tracked cleanup step so it doesn't become permanent clutter.
- **A phased rollout scoped by Network Hierarchy or log source**, deploying the changed logic against one segment or one Log Source type first and widening scope only after a defined observation window — a partial substitute for staging isolation, at the cost of tracking which segment is running which version for the duration of the phase.
- **Domains and tenants (Part 21) as partial isolation**, where a deployment already runs multi-tenant — but only where the object under test is genuinely scoped to one Domain and shares no state with Domains outside the test; a Reference Set referenced across Domain boundaries defeats this isolation immediately, so confirm scope before relying on it.

PRODUCT VERSION NOTE *Whether a given QRadar licensing tier or delivery model (on-prem, cloud-hosted, hybrid — Part 2 §4's own PRODUCT VERSION NOTE covers this space in full) permits or discounts a genuinely separate non-production/staging deployment, and what that costs relative to a single production license, is commercial-packaging information this part does not have current visibility into and does not assert as fact. Confirm current licensing options for a staging environment with IBM or your reseller before budgeting one, rather than assuming either that it's readily available or that it isn't.*

6. Manual regression testing: baseline, change, re-baseline

[RULE ENGINEER]

This is the section that substitutes for DEH Part 22's automated fixture-based regression test, and it is where this part leans hardest on AQL syntax DEH Part 27 already teaches rather than reinventing it. The method is a before/after comparison run as an ad hoc AQL search — exactly the investigative use DEH Part 27 §1 and §3.2 already establish for AQL, applied here to a change-validation purpose rather than a hunt or an investigation.

Before the change: express the *current* Building Block or Rule Test condition as an AQL search over a representative historical window, and record the match count and a sample of matched entities. For a proposed tightening of `BB:LSASS-Access-Candidate` (DEH Part 27 §3.3's own worked example, carried forward here as the running case), that baseline search is structurally the same shape as DEH Part 27 §3.2's own investigative query:

```
-- QRadar AQL, per DEH Part 27 sec1/sec3.2 syntax and disambiguating comment convention --
-- not standard SQL, no general-purpose JOIN, LAST n HOURS is QRadar's own time-window
-- shorthand. Baseline capture for a proposed BB:LSASS-Access-Candidate tightening: run
-- BEFORE the change, over a representative window, and keep the result attached to the
-- change ticket (Table 16.2's "Baseline AQL evidence" field) as the pre-change reference
-- point. Property names and the QID display string are illustrative per DEH Part 27 sec3.2's
-- own caveat – verify against your own Log Activity field list before reuse.
SELECT "Source Process Name", "Target Process Name", "Granted Access", COUNT(*) AS "Matches"
FROM events
WHERE QIDNAME(qid) = 'Process accessed'
  AND "Target Process Name" ILIKE '%lsass.exe%'
  AND ("Granted Access" = '0x1010' OR "Granted Access" = '0x1410')
GROUP BY "Source Process Name", "Target Process Name", "Granted Access"
LAST 30 DAYS
```

Apply the change inside the scheduled window Table 16.2's ticket records, per §1's PRODUCT VERSION NOTE on save/deploy propagation.

After the change: re-run the same search, adjusted only for the new condition, over an equivalent-length window starting after the change took effect, and compare the resulting match set against the baseline by hand: did the entries the change intended to exclude actually disappear, did anything unexpected also disappear, and did any previously-unmatched activity newly appear. This is a manual diff, not an automated one — DEH Part 22's fixture-based CI test asserts pass/fail automatically; this workflow's equivalent is a human reading two AQL result sets side by side and reasoning about whether the delta matches the change's stated intent.

Validation Test Setup: A change ticket proposing to add a specific process path to **RS-ALLOWListed-LSASS-Tools** (a Tuning change per DEH **TERMINOLOGY.md**'s definition, carried forward unchanged per this book's **STYLE-GUIDE §12**), with the before-state export and a baseline AQL search (matching the query above) attached, showing the process path's current match count and sample entities over the prior 30 days. **Action:** Apply the Reference Set change inside the ticket's scheduled window; after the window, re-run the identical baseline AQL search over the next comparable period. **Expected result:** The specific process path no longer appears in the post-change result set (confirming the exclusion took effect), every other previously-matching process path still appears at a comparable rate (confirming the change didn't accidentally broaden the exclusion or affect unrelated matches), and the ticket is closed with both AQL result sets attached as the recorded before/after evidence. If the specific path still appears post-change, the Reference Set edit didn't propagate as expected — check §1's PRODUCT VERSION NOTE on save/deploy timing before assuming the logic itself is wrong.

[RULE ENGINEER] This method has a real limitation worth stating rather than glossing over: it validates against *historical* traffic, not against the specific live event a genuine attacker will eventually generate. A baseline-and-diff check confirms the change behaves as intended against what has already happened; it says nothing about a novel technique variant that hasn't appeared in the 30-day window either before or after the change. This is the same distinction DEH Part 27 §3's own Detection Test callout draws between confirming a Rule Test's field logic (§3.1 there) and confirming the Rule actually fires against a

live, deliberately generated test event — where staffing and environment allow it, pair this section's historical baseline-diff with an actual Detection Test-style live trigger (DEH Part 27 §3.3's own Setup/Action/Expected result) rather than treating the historical comparison alone as sufficient proof the change works.

7. Putting it together: an end-to-end change workflow

[RULE ENGINEER] Figure 16.1 ties §3 through §6 into one repeatable path, explicit about where this workflow's compensating controls sit relative to DEH Part 22's pipeline stages in Table 16.1.

flowchart TD

```
A["Change request\n(analyst disposition, tuning\nintake, or engineer-initiated)"] --> B["Change ticket opened\n(Table 16.2 fields)"]
B --> C["Dependency check:\nwhat else references\nthis BB:/RS-/R: object?"]
C --> D["Before-state export\nattached to ticket"]
D --> E["Baseline AQL search\n(sec 6) attached to ticket"]
E --> F["Reviewer: technical-adversarial\nreview of proposed change"]
F --> G["Staging tenant or\nphased-rollout substitute\navailable? (sec 5)"]
G -->|yes| H["Validate change in\nstaging/phased scope first"]
G -->|no| I["Approver schedules\nproduction change window"]
H --> I
I --> J["Change applied in\nproduction (Rule Wizard\nsave/deploy, sec 1)"]
J --> K["Post-change AQL search,\ncompared against baseline\n(Validation Test, sec 6)"]
K --> L["Delta matches\nchange intent?"]
L -->|yes| M["Ticket closed,\nevidence attached"]
L -->|no| N["Rollback: re-apply\nbefore-state export (sec 4)"]
N --> O["Ticket updated with\nrollback trigger and cause"]
```

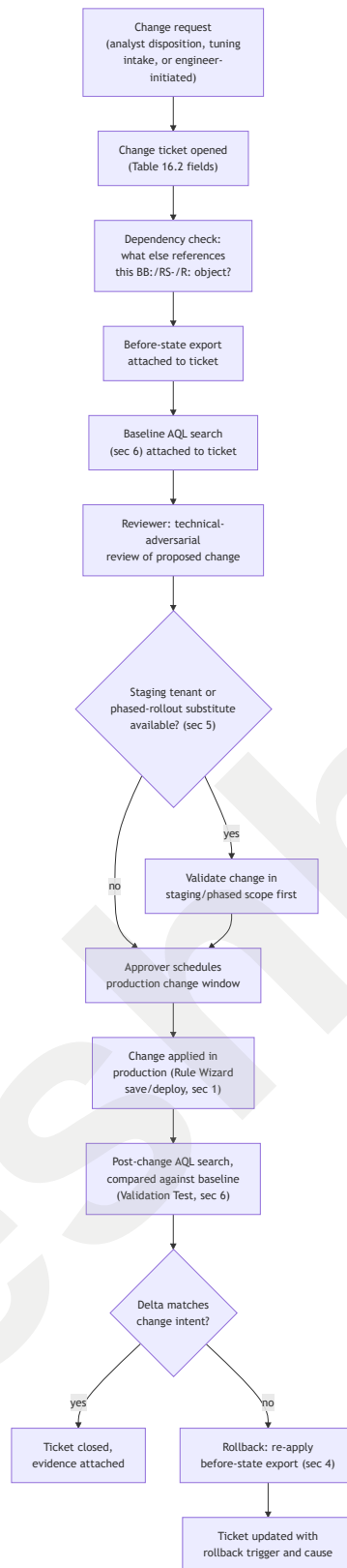


Figure 16.1 — A console-native rule-change workflow, end to end. *CONCEPTUAL*. Illustrates this part's own compensating-control sequence — change ticket, dependency check, export-before-edit, staging or phased validation where available, baseline-and-diff regression testing, and a rollback path keyed to the before-state export — mapped against DEH Part 22's pipeline stages from Table 16.1. Not a reproduction of any QRadar-provided workflow feature or a claim that the platform enforces any of

these steps natively; every arrow in this diagram is a process discipline a team has to actually follow, not a gate the Console itself enforces.

8. Where the substitute still falls short

[RULE ENGINEER] Every recommendation above is a procedural control layered on top of a platform that enforces none of it. That's worth stating once, directly, rather than leaving it implied by the diagram's own honesty.

Blind Spot Nothing in Figure 16.1's workflow is structurally enforced by QRadar the way DEH Part 22 §3's branch-protection settings structurally block a self-approved merge. Any administrator holding Rule Wizard, Building Block, or Reference Set write permission can open the Console, edit `BB:LSASS-Access-Candidate` directly, and save — no ticket, no dependency check, no baseline capture, no reviewer, no approver — and the platform will not stop them or even distinguish that edit from one made correctly through the full workflow. The change-ticket process in §3 is a norm the team has agreed to follow, not a mechanism the Console enforces the way a required CI status check enforces DEH Part 22's gate. The only real mitigations available inside the platform itself are permission scoping (restricting write access to a small, named group, the principle DEH Part 22 §1's Engineering Reality box applies to console-managed detections generally) and after-the-fact review of whatever administrative audit trail QRadar retains (subject to the same revision-depth *PRODUCT VERSION NOTE* as §4) — neither prevents a bypass; both only make one detectable after it's already happened.

[SOC MANAGEMENT] This is the honest cost line a budget conversation about QRadar rule governance needs stated plainly: the discipline in this part substitutes people and process for tooling DEH Part 22's git-based model gets for the cost of a repository and a CI runner. That substitution is not free — it costs the change-ticket overhead §3 names, the dependency-check time §2's Platform Reality box names, and the ongoing management attention required to keep a norm from eroding the way §2's "obvious" fix scenario describes. A team that budgets zero time for this discipline is budgeting for a governance outcome the platform cannot provide without the people-hours this part describes.

SOC Management View *Sizing this discipline against headcount is a direct, answerable question: every `BB:`/`RS-`/`R:` change needs a requester's time to file the ticket and run the dependency check and baseline query, a reviewer's time to read the ticket and evidence critically, and — separately, staffing allowing — an approver's time to schedule and confirm the change window and post-change validation. On a two-person function, reviewer and approver commonly collapse into the same senior person, mirroring DEH Part 22 §3's own staffing-constrained allowance; what never collapses is the requester also serving as their own reviewer — a self-approved edit is exactly the "it's just one line" bypass §2 names as this workflow's most common real failure mode. Budget the review time explicitly in whatever staffing model Part 22 sizes a QRadar practice against, rather than treating rule-change review as free because no licensing invoice makes the cost visible the way a CI runner's bill would.*

9. What would make this gap close

[PLATFORM ENGINEER] This part's whole premise — that QRadar's console-native workflow forces a procedural substitute rather than handing a team DEH Part 22's structural guarantees — describes the platform's architecture as understood at the time of writing, not a permanent verdict. A native, version-controlled rule-authoring surface (built into the Console itself, or delivered as an App Framework app per Part 19's survey) that gave Rules, Building Blocks, and Reference Sets a real diff view, a required-approval gate before a save takes effect, and a one-click revert would close most of Table 16.1's gap rows directly, and this part's compensating workflow would shrink to whatever residual gap remained.

PRODUCT VERSION NOTE *Whether any current QRadar release or App Framework app (Part 19's survey names Use Case Manager, Pulse, and UBA as commonly deployed apps, at a survey level) provides a rule-versioning, approval-gating, or diff capability closer to DEH Part 22's git-based model than the plain Rule Wizard does, is not something this part asserts either way. Such a capability would not eliminate the dependency-check and baseline-regression discipline in §2 and §6 — those substitute for CI-style testing, which a versioning app alone doesn't provide — but it would strengthen the review-gate and rollback rows of Table 16.1. Check your own deployment's current App Exchange catalog and release notes before assuming the plain Rule Wizard is the only authoring surface available.*

Cross-references: DEH Part 22 (Detection as Code — the pipeline this part's workflow substitutes for, stage by stage in Table 16.1); DEH Part 23 §5 (Sigma-first vs. native authoring — why a QRadar-only shop's Rules have no Sigma intermediary to version); DEH Part 27 §1, §3.2, and §3.3 (AQL syntax and the `BB:LSASS-Access-Candidate` / `RS-Allowlisted-LSASS-Tools` / `R: Suspicious LSASS Access – Unapproved Process` worked example this part's §6 reuses without re-teaching); DEH `TERMINOLOGY.md` (Tuning, Suppression — inherited unchanged). Within this book: Part 2 §4 (the packaging/delivery-model **PRODUCT VERSION**

NOTE §5 extends); Part 9 (Building Block governance — the dependency-sprawl risk §2 names); Part 15 (the tuning-intake workflow generating most of this part's change requests); Part 17 (licensing/capacity planning behind §5's staging-tenant tradeoff); Part 19 (App Framework — where §9's version-control-app possibility would live); Part 21 (Domains/tenants — the partial isolation substitute §5 names); Part 22 (the staffing model §8's SOC Management View feeds).

neshbooy

Section G — Keeping the Platform Healthy

neshbooy

Part 17 — Licensing, EPS/FPM Management, and Capacity Planning

Why this part exists

[CONCEPT] DEH Part 27 §1.3 flags QRadar's licensing model in exactly one paragraph — a QRadar deployment is licensed by sustained Events Per Second (EPS) and Flows Per Minute (FPM), not by indexed data volume or analyst seat count, and a Sysmon rollout that widens its collection scope or a new Log Source that pushes sustained throughput past its licensed tier "does not silently get ingested for free." That paragraph exists in DEH Part 27 to make one narrow point: a detection engineer designing an analytic against QRadar cannot borrow the mental model a storage-priced, volume-billed cloud-native SIEM might teach — where ingesting more data mostly just costs more money at the end of the month — because QRadar's own correlation and search capacity is bounded by a rate limit that behaves differently under sustained overage. This part is the deep dive that single paragraph promises but has no room to deliver: what the EPS/FPM licensing model actually measures, what a burst allowance is and how overage actually plays out operationally, how to size a candidate Log Source before it goes live rather than after the license-overage notification fires, what growth forecasting looks like as a recurring discipline instead of a one-time appliance-sizing exercise at initial deployment, and — the concrete question every SOC eventually asks — what onboarding one new, genuinely noisy Log Source really costs.

This part also picks up a thread DEH Part 23 §5 states in general terms for cross-platform analytic design: whether a Sigma-first authoring investment pays off depends on backend-specific constraints that a single canonical example doesn't need to enumerate. Licensed EPS/FPM capacity is exactly that kind of constraint for QRadar specifically — a detection design ported from a volume-priced backend, or authored with no backend in mind at all, can be logically correct and still be unaffordable to run the way it was designed, once its actual event rate is measured against a real license tier. This book's own Part 2 (deployment architecture) already named the Event Processor as the component actually running the Custom Rules Engine (CRE) and writing to Ariel storage — this part is about the rate limit that component's licensed capacity imposes on everything upstream of it, and how a platform engineer plans for that limit rather than discovering it during an incident.

1. What EPS and FPM actually meter

[CONCEPT] A QRadar license entitles a deployment to a specific sustained Events Per Second rate and a specific sustained Flows Per Minute rate — two independent metered dimensions, because event data (parsed log records) and flow data (network-traffic records, Part 7) arrive through different collection paths and are licensed separately. "Sustained" is the operative word: EPS/FPM licensing is not a hard per-second ceiling that instantly rejects the 501st event in a second licensed for 500, any more than a highway's posted speed limit rejects a single moment of acceleration — it is a rate the deployment is licensed to sustain over time, with tolerance for short-term variance above that rate (§2) and consequences that accumulate specifically under *sustained* overage rather than a single burst.

This is a genuinely different resource from the one Part 12 covers for Ariel search performance. EPS/FPM capacity governs how much data the deployment is licensed and provisioned to *ingest and correlate* in real time; Ariel search capacity governs how much of that already-ingested data can be *queried* concurrently without analysts competing for the same search resources. A deployment can be well within its EPS tier and still have slow, contended Ariel searches during a heavy report-generation window (Part 13 §3), and a deployment can have fast, uncontended search performance and still be in sustained license overage — the two constraints are adjacent but not the same conversation, and conflating them is a common capacity-planning mistake: buying more search-side hardware does not raise a licensed EPS ceiling, and raising an EPS tier does not by itself fix a slow, contended search.

PRODUCT VERSION NOTE *Whether a given QRadar deployment's EPS/FPM entitlement is delivered as a fixed on-premises appliance tier, a flexible cloud/SaaS consumption model, or some hybrid of the two — and which IBM business unit or partner currently owns sale, packaging, and support for that entitlement — has shifted across release cycles and business decisions this book does not have current visibility into at time of writing, exactly the packaging/ownership risk [STYLE-GUIDE.md §11](#) flags as aging fastest. State a specific tier number, contract structure, or delivery model to your own organization's procurement or IBM account team as something to verify directly rather than something this part can hand you as settled fact.*

1.1 Tiers, not a single number

[PLATFORM ENGINEER] A license entitlement is packaged as a tier — a bundled EPS ceiling and FPM ceiling sized together, because most deployments' event and flow load scale together with the same underlying infrastructure growth (more hosts, more network segments, more traffic). A distributed deployment (Part 2) may also license individual Event Processors or Flow Processors against a portion of the total entitlement, so that "are we within license" is not always a single deployment-wide number to check — it can be a per-processor question, particularly once a deployment has grown past its original All-in-One or single-processor footprint. Confirm which shape your own entitlement takes before treating any single EPS figure quoted in a vendor conversation as the complete answer for a multi-processor deployment.

2. Burst allowance and what sustained overage actually does

[PLATFORM ENGINEER] QRadar's licensing model tolerates short-term bursts above the sustained licensed rate — a brief spike during a batch job, a log-rotation event, or a burst of authentication failures during a real incident should not immediately degrade correlation the moment it crosses the licensed ceiling for a few seconds. What counts as "short-term" and how far above the sustained rate a burst is tolerated before it registers as an overage condition is itself part of the licensing model, not an emergent side effect — but the exact burst window and percentage are the kind of specific, version- and packaging-sensitive figures this book will not assert as a fixed number (see the PRODUCT VERSION NOTE below). What is stable across the platform's history, and consistent with DEH Part 27 §1.3's own

framing, is the shape of the consequence: a *sustained* rate above the licensed tier — not a momentary spike — is what triggers a logged license-overage condition, and sustained overage is what risks degraded event or flow processing rather than a brief burst being absorbed and forgotten.

Platform Reality *The documented behavior most teams can find without much digging is that sustained overage gets logged as a notification-level event and is visible somewhere in the deployment's own system-notification feed or Offense/system dashboard (Part 13 §1.1's system-metric tile category). What is less consistently understood, and matters more operationally, is what happens to the data itself once a deployment sits in sustained overage rather than a brief burst: throughput above the licensed ceiling is not guaranteed to be processed with the same priority or timeliness as licensed-rate traffic, and a deployment that has been quietly over its tier for weeks — not because anyone made a deliberate decision, but because a Log Source's own logging verbosity crept upward after a vendor agent upgrade — can have events delayed into Ariel later than their actual timestamp, or dropped outright, well before anyone reads the notification that first flagged the condition. Treat a license-overage notification as an incident-severity finding, not a background housekeeping item: the gap between "overage was first logged" and "someone looked at it" is exactly the window in which correlation against real telemetry can be silently degraded.*

PRODUCT VERSION NOTE *The specific burst-tolerance window and percentage above the sustained licensed rate, the exact overage grace period before enforcement behavior changes, and whether enforcement means throttling, delayed processing, selective dropping, or a purely advisory notification with no data-plane effect at all, have been described differently across QRadar releases and licensing packaging changes. Verify the current overage-handling behavior for your specific license tier and deployment version directly against IBM's current documentation and your own support contract before designing a capacity margin around an assumed grace period.*

3. Sizing a candidate Log Source before it goes live

[PLATFORM ENGINEER] The failure mode this section exists to prevent is the one DEH Part 27 §1.3 names in miniature: a team widens a Sysmon deployment to log Event ID 1 (Process Create) fleet-wide, or onboards a new EDR agent's full telemetry stream, without first estimating what that change actually costs in sustained EPS — and finds out from a license-overage notification instead of a sizing exercise. The estimation itself does not require exotic tooling: it requires a baseline sample of the source's actual logging behavior, measured the way it will actually be configured in production, not a vendor's marketing figure for "typical" volume.

A workable sizing sequence: stand the source up against a small, representative subset of its eventual footprint (a handful of hosts, one collection channel, the actual verbosity level planned for production — not a minimal default that understates the real rollout); let it run long enough to capture a realistic

mix of quiet and busy periods, not just a quiet baseline; measure the sustained rate that subset produced (§5 covers how); and extrapolate linearly to the full planned footprint, treating the extrapolation as a floor rather than a ceiling, since per-host log verbosity rarely decreases as an environment scales and a rollout that starts at ten hosts routinely ends up logging more per host once real production activity — not a quiet test environment — is what's generating the events.

Table 17.1 supports the first, coarser pass of that estimate: which category of Log Source tends to drive load for structural reasons, before any specific vendor's per-event size or per-host rate is measured.

Table 17.1 — Log Source categories and what drives their EPS/FPM footprint. Supports deciding which candidate sources need a measured baseline before onboarding versus which are safe to size by rough headcount alone.

Source category	Primary load driver	Typical scaling factor	Verification note
Windows Security auditing (baseline: logon/logoff, object access)	Host count, audit-policy scope	Roughly linear with host count at a fixed audit-policy setting	A single Group Policy change widening audit scope (e.g., enabling object-access auditing broadly) can multiply this source's rate without any host-count change — re-baseline after any audit-policy change, not just after fleet growth
Sysmon (process creation, network connection, image load)	Host count, which Sysmon event IDs are enabled, config verbosity	Materially higher per-host rate than baseline Windows auditing; scales with both host count and config breadth	DEH Part 27 §1.3's own named example — verify per-host rate against the <i>specific</i> Sysmon config being deployed, not a generic figure, since two Sysmon configs of differing verbosity can differ by an order of magnitude on the same fleet
Firewall/perimeter (allow+deny logging)	Traffic volume, whether allowed traffic is logged in addition to denied	Highly deployment-specific; a policy that logs all allowed connections, not just denies, can dominate total EPS	Confirm whether "log all" or "log denies only" is the actual configured policy before sizing — this single setting is often the largest lever on this source category's contribution
EDR/endpoint telemetry (full behavioral stream)	Host count, telemetry tier selected (alert-only vs. full behavioral)	Full behavioral telemetry (process, network, file, registry events) is typically the single highest per-host rate among common source categories	The exact rate difference between an "alerts only" and a "full telemetry" integration tier is vendor- and version-specific on both the EDR product and QRadar's own DSM (Device Support Module)/app integration — measure both tiers against the same host subset if choosing between them
Cloud platform audit logs (IaaS/SaaS control-plane activity)	Tenant activity level, API call volume, breadth of services in scope	Can spike sharply and unpredictably with automated tooling (CI/CD pipelines, infrastructure-as-code runs) rather than scaling smoothly with headcount	Size against a period that includes at least one automated deployment cycle, not only interactive human activity, since the spike shape differs from every other row in this table
Flow data (NetFlow/sFlow/QFlow-derived)	Network segment breadth, sampling rate configured on the flow source	FPM, not EPS — scales with distinct flow count, which is sensitive to flow-timeout and sampling configuration independent of raw traffic bytes	A flow source's sampling rate is often configured outside QRadar's own console (on the router/switch/exporter) — confirm that setting directly at the source, not by assumption, before sizing FPM contribution

4. The real cost of one new noisy Log Source — a worked walkthrough

[PLATFORM ENGINEER] Put §3's sizing discipline into a concrete sequence, because "measure before onboarding" is easy to state and easy to skip under deadline pressure. A SOC decides to extend EDR coverage from alert-only integration to full behavioral telemetry across 4,000 endpoints, because a recent incident review found the alert-only feed didn't retain enough raw process/network detail to answer a question the team needed answered during triage. The naive path is to flip the integration tier

in the EDR console and the corresponding Log Source setting in QRadar the same afternoon. The sized path looks like this:

1. **Baseline a subset.** Enable full behavioral telemetry on 100 representative endpoints (a mix of workstation and server roles, not just one) for a week that includes normal business activity, not a holiday week.
2. **Measure the subset's sustained contribution** (§5) and compare it against the alert-only tier's already-known contribution from the same 100 hosts, to isolate the *delta* the tier change actually introduces — not the full new rate, which conflates the change with load the source was already contributing.
3. **Extrapolate to 4,000 hosts**, treating the 100-host measurement as a floor per §3's guidance, and add that delta to the deployment's current total sustained EPS.
4. **Compare the projected total against the current license tier's headroom** — not against the tier's raw ceiling, since some headroom needs to remain for organic growth and burst tolerance (§7).
5. **If the projected total exceeds available headroom**, the decision is no longer a configuration change — it is a licensing and budget decision (§8), with the tradeoffs a filtering approach can and cannot buy back (§6) as the alternative to a straight tier upgrade.

SOC Management View *The number that actually needs to reach whoever approves budget is not "we want better EDR telemetry" — it's the projected sustained EPS delta from step 3, translated into whatever unit your license tier is actually priced in, compared against current headroom from step 4. A request framed as a security-value argument alone ("this closes a real visibility gap the incident review found") is true and still incomplete without the capacity number attached, because the approver's actual decision is between a licensing cost and one of the mitigations in §6 — a decision that cannot be made well from the security argument alone. Bring both numbers to the same conversation: what the gap costs to close in security terms, and what it costs in licensed capacity terms, since a program that always argues the first without the second trains its own budget holders to discount the ask the next time, whether or not this specific request was justified.*

5. Measuring actual load: AQL approximation and system-metric utilization

[PLATFORM ENGINEER] Two different views of "how much are we actually using" exist side by side in a QRadar deployment, and they answer slightly different questions. The system-metric dashboard tile category Part 13 §1.1 already names — a tile reading QRadar's own operational telemetry rather than an Ariel search — is the canonical source for total sustained EPS/FPM utilization against the licensed tier, because it reflects QRadar's own internal accounting of what it considers metered load, not an approximation. What it typically does not hand you in one glance is a per-Log-Source breakdown fine-grained enough to answer "which specific source is driving this number up" — for that, an ad hoc AQL search against the `events` table (DEH Part 27 §1.1–§1.2 owns the full syntax; not re-taught here) is the practical tool:

```
-- QRadar AQL – DEH Part 27 §1 owns full AQL syntax; shown here only to illustrate a
-- load-diagnostic search pattern, not to re-teach SELECT/FROM/WHERE/GROUP BY. LOGSOURCENAME()
-- resolves the internal log source ID to its configured display name (DEH Part 27 §1.2).
-- This query approximates per-source event volume for capacity diagnosis – it is not a
-- substitute for QRadar's own licensed-EPS accounting; see the Platform Reality note below.
SELECT LOGSOURCENAME(logsourceid) AS "Log Source", COUNT(*) AS "Event Count"
FROM events
LAST 1 HOURS
GROUP BY LOGSOURCENAME(logsourceid)
ORDER BY "Event Count" DESC
```

Divide the top result's `Event Count` by 3,600 to get an approximate average EPS contribution for that source over the hour searched — useful for identifying *which* source to investigate further, and a reasonable first estimate, but it carries two real limitations worth stating plainly rather than treating the number as authoritative. First, an hour-wide average smooths over exactly the short-term bursts §2 discusses — a source that spikes hard for five minutes and sits quiet the rest of the hour shows the same hourly average as one that sustains a flat, lower rate the whole time, and only the first of those two patterns is likely to matter for burst-tolerance planning. Second, this query counts events as stored in Ariel, which may not be identical to QRadar's own point of license metering in the ingestion pipeline (§6 covers why that distinction matters for filtering decisions specifically) — treat this as a strong diagnostic signal for *which* source to investigate, not as a replacement for the system-metric tile's own accounting when the two numbers need to be reconciled for a licensing conversation.

Platform Reality *A discrepancy between this section's AQL approximation and the system-metric tile's own reported utilization is not automatically a bug in either number — it is a reminder that "events counted in Ariel over an hour" and "QRadar's own licensed-EPS accounting" are not guaranteed to be defined identically, and a capacity-planning conversation that hinges on a precise number should use the system-metric tile (or QRadar's own licensing-status reporting, where your deployment exposes one) as the number of record, with the AQL breakdown used only to identify which source to act on.*

6. Reducing load without losing detection value

[PLATFORM ENGINEER] When §4's projected total doesn't fit inside available headroom, a straight license-tier upgrade is one option, but it is not the only one, and it is not always the right one. Three categories of load-reduction are worth evaluating before defaulting to a bigger tier:

- **Scope the source's own verbosity.** Table 17.1's Windows-auditing and Sysmon rows both name this directly: an audit policy or Sysmon config that logs more event categories than any deployed Rule or Building Block actually consumes is pure licensed-capacity cost with no detection value behind it. Cross-reference which QIDs (QRadar's internal event-taxonomy identifiers) a given source's telemetry actually feeds into a deployed `BB: -`prefixed condition (Part 9's Building Block governance) before assuming every enabled event category is earning its EPS cost.

- **Filter at or near the source, not after ingestion.** Dropping a known-noisy, low-value event category at the collection point — a source-side logging policy change, or a filter configured on the Event Collector itself (Part 2 §1.2) before the event is parsed and counted — reduces load earlier in the pipeline than trying to exclude it via a Rule condition after the fact, which does not reduce licensed load at all since the event was already ingested and metered by the time a Rule evaluates it.
- **Route low-value verbose logs elsewhere.** A source whose full detail is valuable for compliance retention or forensic depth but not for real-time correlation is a candidate for dual-routing to a lower-cost, non-QRadar retention target, with only a filtered, high-signal subset actually forwarded into QRadar's licensed ingestion path.

PRODUCT VERSION NOTE *Where exactly in the pipeline QRadar's own EPS metering is calculated — at raw ingestion on the Event Collector (Part 2 §1.2), or after DSM parsing on the Event Processor (Part 2 §1.3) — determines whether a given filtering approach actually reduces licensed load or only reduces what gets parsed, indexed, and made searchable while the event still counts against the license at the point it was received. This book has no verified, version-current answer to state as settled fact for every release and packaging combination; confirm the actual metering point for your specific deployment against current IBM documentation before assuming a specific filtering technique buys back license headroom rather than only reducing storage and search-index load.*

Blind Spot *Every filtering technique in this section reduces licensed load by making an event never arrive, or never get parsed, at all — which is indistinguishable, from a Rule's or a Building Block's perspective, from the source going quiet for an unrelated reason (Part 18's log-source-stops-parsing failure mode). A **BB:**-prefixed condition that used to fire reliably against a now-filtered event category stops firing with no error anywhere, exactly the silent-failure shape DEH Part 27 §3.1 already documents for a broken DSM/QID mapping — except here the cause is a deliberate capacity decision instead of an accident, which makes it easier to lose track of which Rules were quietly depending on a category someone else filtered out for capacity reasons six months later. Any filtering decision made for capacity reasons belongs in the same change-management record Part 16 asks a team to keep for a rule change, cross-referenced against which deployed Rules and Building Blocks consumed the filtered category, so a later detection-coverage review doesn't mistake "we chose to stop collecting this" for "this stopped working."*

7. Growth forecasting as a recurring discipline

[PLATFORM ENGINEER]

Capacity planning done once, at initial deployment sizing, answers the question the deployment had on day one. It does not answer the question a deployment has eighteen months later, after three new business units, one acquisition's worth of newly onboarded infrastructure, and a

security program that has (correctly) pushed for wider telemetry coverage the whole time. Treating capacity planning as a recurring review — not a one-time sizing exercise — is the difference between a tier upgrade decided calmly ahead of need and a tier upgrade decided during a license-overage incident.

A workable recurring cadence tracks three inputs against current headroom on a fixed schedule (quarterly is a reasonable default for most SOC's): the deployment's actual trailing sustained EPS/FPM trend (measured per §5, trended over months, not read as a single point-in-time snapshot); the known onboarding pipeline — sources already committed to but not yet live, sized per §3–§4; and organic growth assumptions — headcount growth, infrastructure growth, or planned audit-policy changes that will increase load from *existing* sources without any new source being added at all. The output of that review is a single go/no-go decision made ahead of need: does the trend, plus the pipeline, plus organic growth, project past current headroom before the next scheduled review — and if so, is the answer a §6 load-reduction, a tier-upgrade budget request (§8), or an explicit, documented decision to accept a coverage gap rather than pay for the headroom.

flowchart TB

```
A["Candidate new Log Source\nor audit-policy change"] --> B["Baseline-measure\nEPS/FPM\ncontribution (Sec 3-4)"]
B --> C{"Fits within\ncurrent headroom?"}
C -->|"yes"| D["Onboard; monitor sustained\nrate via system-metric tile\nand AQL diagnostic (Sec 5)"]
C -->|"no"| E{"Scoping/filtering/routing\nreduces load enough\n(Sec 6)?"}
E -->|"yes"| F["Apply load reduction,\ndocument in change record\n(Part 16), then onboard"]
E -->|"no"| G["SOC Management decision:\ntier upgrade budget vs.\naccepted coverage gap (Sec 8)"]
D --> H["Feed into quarterly\ngrowth-forecast trendline"]
F --> H
G --> H
H -->|"projects past headroom\nbefore next review"| B
```

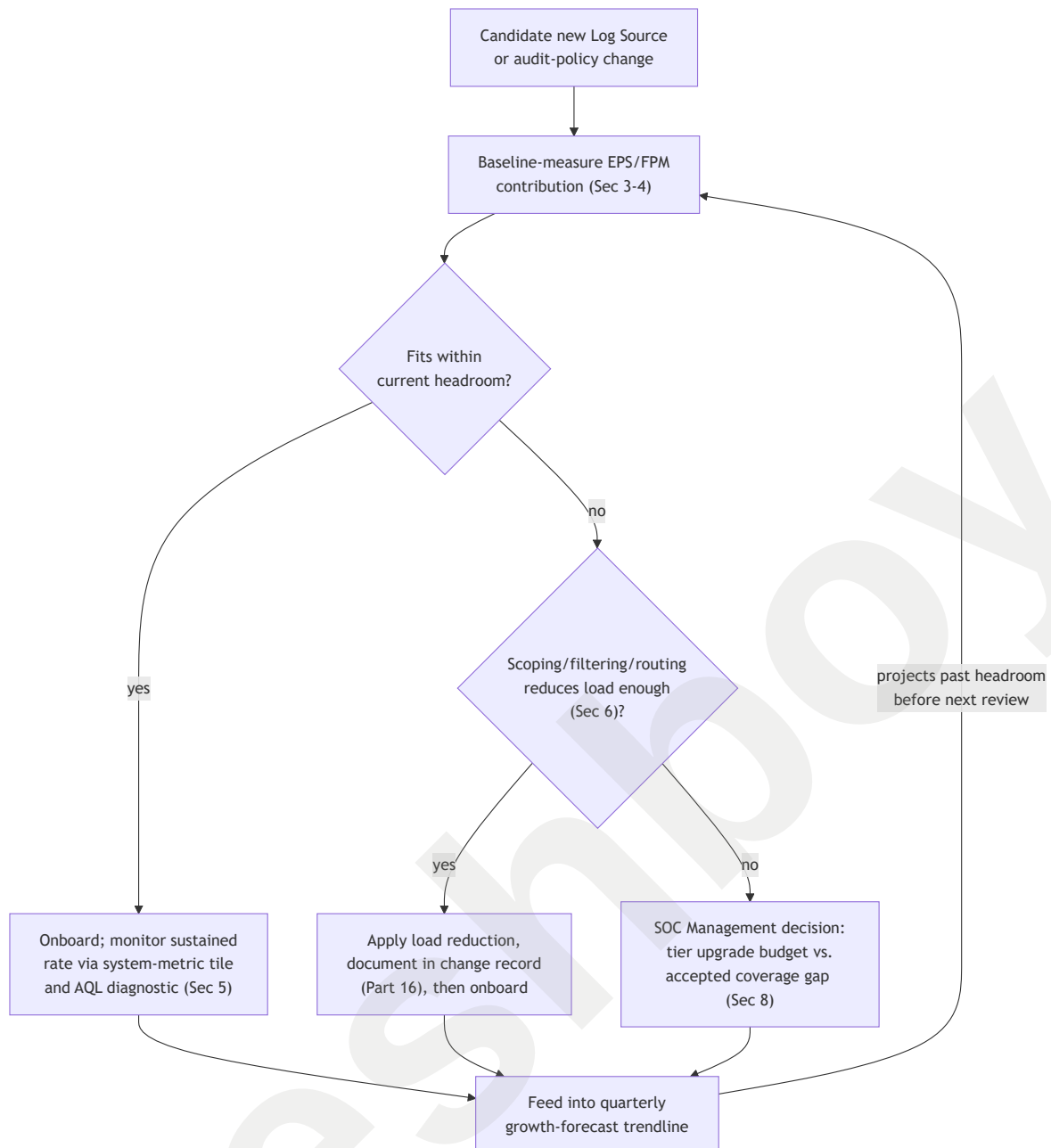


Figure 17.1 — Capacity planning as a closed loop, not a one-time sizing pass. *CONCEPTUAL.*

Illustrates the decision sequence this part builds section by section — baseline measurement, a headroom check, a load-reduction branch, and a SOC Management budget or accepted-gap decision — all feeding back into a recurring trendline review rather than terminating at "onboarded." Not a reproduction of any IBM capacity-planning workflow diagram; this book's own synthesis of the recurring discipline §7 argues for, illustrative only.

8. Budgeting the tier: the conversation this part has been building toward

[SOC MANAGEMENT]

Every section above produces inputs to one recurring budget conversation: does the current license tier's headroom cover the deployment's actual and projected sustained load, and if not, which of the available responses — tier upgrade, load reduction, or an explicitly accepted gap — is

the right call for this specific shortfall. Table 17.2 frames that decision as a comparison rather than a default, because "just upgrade the tier" is not always the answer a budget conversation should reach for first, and "just filter more aggressively" is not always safe to reach for either.

Table 17.2 — Responses to a projected capacity shortfall, compared. Supports deciding which lever to pull first when a growth-forecast review (§7) projects sustained load past current headroom.

Response	What it actually buys	What it costs	Best fit
Tier upgrade	Headroom for the full projected load, including future organic growth beyond this specific shortfall	Recurring licensing cost, scaled to the new tier	A shortfall driven by broad, ongoing organic growth (more hosts, more business units) rather than one identifiable noisy source
Source-side filtering/scoping (§6)	Load reduction with no new recurring cost, if the filtered category has genuinely low detection value	Coverage risk if the filtered category turns out to matter later (Blind Spot, §6); ongoing maintenance to keep the filter's rationale documented	A shortfall traceable to one specific over-verbose source or policy setting where the value of the removed category is genuinely low or already covered elsewhere
Dual-routing to non-QRadar retention (§6)	Preserves full-fidelity retention for compliance/forensics while reducing QRadar's licensed ingestion specifically	Additional storage/retention infrastructure and a second system to maintain	A source whose value is mostly retention/forensic depth rather than real-time correlation
Explicitly accepted coverage gap	No cost, immediate	A named, documented Visibility Debt (DEH TERMINOLOGY.md ; Part 13 §4's Blind Spot on the same concept) that has to be tracked, not quietly forgotten	A shortfall the budget genuinely cannot absorb this cycle, where the alternative to naming the gap honestly is discovering it during an incident review instead

SOC Management View *The fourth row in Table 17.2 is the option every team is tempted to reach for silently rather than name explicitly, because it has no immediate cost — and it is exactly the option Part 13 §4's Blind Spot warns looks identical, from inside QRadar's own dashboards, to a deliberately well-tuned deployment with nothing missing. A capacity shortfall absorbed by quietly under-provisioning one source's onboarding, with no record of the decision, is Visibility Debt with a specific, nameable cause (a licensing budget constraint) that this part's own Table 17.1–17.2 sequence gives a team the vocabulary to document instead of losing. Bring the accepted-gap option to the same budget conversation as the tier-upgrade option, explicitly, rather than letting a shortfall resolve itself by default into whichever source's onboarding got quietly deprioritized — the review-cost and staffing consequences of a QRadar practice (this book's Part 22 capstone) assume a team that can point to why a gap exists, not one that discovers a gap it forgot it had chosen.*

Cross-references: DEH Part 27 §1.1–§1.2 (AQL fundamentals and translation functions, applied to §5's diagnostic query rather than re-taught), §1.3 (the single-paragraph EPS/FPM licensing flag this part expands in full), §3.1 (the silent DSM/QID mapping failure whose shape §6's Blind Spot reapplies to

capacity-driven filtering); DEH Part 23 §5 (backend-specific constraints on cross-platform analytic design, of which licensed EPS/FPM capacity is this platform's own instance); DEH [TERMINOLOGY.md](#) (Visibility Debt, invoked in §8); this book's Part 2 (Event Collector/Event Processor roles and where in the pipeline load is metered, §6), Part 4 (Log Source onboarding, §3–§4), Part 7 (flow architecture and FPM, §1/§3), Part 9 (Building Block governance, §6), Part 12 (Ariel search performance — the adjacent but distinct resource constraint named in §1), Part 13 §1.1/§4 (system-metric dashboard tiles and the Visibility Debt framing reused in §5/§8), Part 16 (change management for filtering decisions, §6), Part 18 (troubleshooting a source that stops parsing, §6), Part 22 (staffing and budget capstone referenced in §8).

neshboy

Part 18 — QRadar-Specific Troubleshooting: Log Sources, DSMs, and Offense Flooding

Why this part exists

Every other part in this book describes how something in QRadar is supposed to work — how a Log Source gets onboarded (Part 4), how a Building Block library stays governable at scale (Part 9), how a Reference Set's type and TTL should be chosen (Part 10), how a noisy Offense gets tuned rather than suppressed (Part 15). This part is what happens after one of those things has already gone wrong in production, and the person who has to figure out why is not the person who built it. DEH Part 27 §3.1 names this book's central operational hazard in one sentence, about one worked example: "an unmapped or partially-mapped event lands under a generic QID with no custom properties, and the query fails silently." DEH Part 27 §2.3's Blind Spot names a second, structurally related hazard: a chained Rule whose first stage silently stops firing takes its second stage down with it, with no error surfacing anywhere. Neither of those sentences was written as a troubleshooting guide — they were written to prove a design point about QRadar's correlation model. This part turns both into a repeatable diagnostic practice, and adds a third failure mode neither DEH part had reason to cover: what to do when the queue that was supposed to be quiet suddenly isn't.

This part is a playbook for three named failure modes — a Log Source that stops parsing, a DSM/QID mismatch surfacing after a vendor agent upgrade, and an offense-flooding incident — each walked the same way: **symptom, likely cause set, verification step, fix**. None of the three is exotic. All three share the same trap: QRadar's platform-health indicators (a Log Source showing green, an event count that looks nonzero, an Offense that exists) answer a narrower question than the one a troubleshooter actually needs answered, and mistaking the narrow question for the broad one is how a real platform fault gets misdiagnosed as a detection-logic problem, or vice versa. This part does not re-teach AQL syntax — every diagnostic query below leans on DEH Part 27 §1's function set (`QIDNAME()`, `CATEGORYNAME()`, `LOGSOURCETYPENAME()`) without re-explaining it — and it does not re-litigate Sigma-first-vs-native authoring, which DEH Part 23 §5 settles generically. It also does not re-derive the DSM onboarding lifecycle (Part 4), the Reference Set type taxonomy and TTL discipline (Part 10), or the tuning-intake workflow (Part 15); this part points at each as the thing that prevents its own failure modes from recurring, not restates it.

1. A troubleshooting mindset: symptom, likely cause set, verification, fix

[CONCEPT] The single mistake this part exists to prevent is treating a platform status indicator as an answer instead of as one data point. "The Log Source shows events arriving," "the Rule exists and is enabled," and "an Offense was created" are all true statements that can coexist with a completely broken detection — Part 4 §4 already demonstrates this for the narrower DSM/QID case, and the same shape recurs at every layer this part covers. A troubleshooter who stops at the first green indicator has confirmed the pipe is connected, not that anything useful is flowing through it.

The four-step method below is deliberately mechanical, because the alternative — jumping straight from symptom to a guessed fix — is how a real platform fault gets mistaken for a detection-logic problem, or the reverse: a Rule Engineer who assumes their Rule Test logic is fine because "it worked last month" spends a day rewriting Building Block conditions that were never the actual problem, while a DSM extension silently dropping a field after an agent upgrade sits untouched.

1. **Symptom.** State precisely what was observed, not what it's assumed to mean. "The Offense count for this Rule dropped to zero" is a symptom. "The Rule stopped working" is already a half-formed conclusion.
2. **Likely cause set.** Enumerate every plausible cause for that exact symptom before investigating any single one — this book's own Tables 18.1–18.3 exist to make that enumeration something a troubleshooter can look up under time pressure rather than reconstruct from memory during an incident.
3. **Verification step.** For each candidate cause, a specific check — an AQL search, a Reference Set membership review, a Log Source status field — that confirms or rules it out. This step is where most troubleshooting time should actually go; skipping straight to a fix based on the most familiar cause, rather than the verified one, is how a real fix gets applied to the wrong problem.
4. **Fix.** The remediation for the verified cause, plus — critically — the part of this book that keeps it from recurring. A fix with no linked prevention step is a fix this part considers incomplete.

PRODUCT VERSION NOTE *This part refers generically to "System Notifications," "Log Activity," and "the Offenses tab" as the console surfaces a troubleshooter checks first. The exact navigation path, the specific wording QRadar uses for a Log Source status or coverage-gap indicator, and whether a given notification appears as a dashboard item, an email alert, or only on manual review, have all varied across QRadar releases and delivery models (on-prem vs. cloud/SaaS). Verify the current notification surface and wording in your own deployment before building an incident runbook that assumes a specific screen name or alert path — the diagnostic logic in this part does not depend on any of those specifics holding still.*

2. Failure mode 1 — A log source that stops parsing

[PLATFORM ENGINEER] "Stopped parsing" covers two symptoms that look similar from a distance but have almost entirely disjoint cause sets, and separating them is the first branch a troubleshooter has to take correctly:

- **The Log Source stops receiving events at all** — a collection-layer failure. Nothing reaches Ariel from this source, full stop.
- **The Log Source keeps receiving events, but they stop resolving to a meaningful QID (QRadar's internal event-taxonomy identifier) or custom-property set** — a parsing-layer failure. Bytes are arriving; the DSM stops making sense of them (or never did, for a subset of event types).

Table 18.1 separates the two, because confusing them wastes the first, most valuable diagnostic minutes of an incident — a platform engineer who spends an hour re-checking network connectivity for a source that is receiving data fine but parsing none of it is investigating the wrong layer entirely.

Table 18.1 — Failure mode 1: a log source that stops parsing. Supports deciding which layer (collection vs. parsing) actually failed before investigating either.

Symptom	Likely cause set	Verification step	Fix
Event count for this Log Source drops to zero	Network path broken (firewall rule change, routing change); source device stopped forwarding (config rollback, agent service stopped); collector/Event Collector itself down or over capacity	Check the Log Source's own last-event-received indicator; confirm from the source device's own logs or agent status that it is still attempting to forward; check collector health independently of this one source	Restore the network path or agent service; if the collector itself is saturated, this is a capacity question — Part 17
Event count stays nonzero, but the QID distribution shifts toward a generic/unmatched category	DSM signature stopped matching this source's payload shape — see Failure Mode 2 (§3) for the vendor-upgrade variant specifically	AQL search grouping by <code>QIDNAME(qid)</code> over a window spanning the suspected change point (query below); compare the generic-QID share before and after	Re-validate and, if needed, update the DSM or DSM extension per Part 4 §4–§5's checklist
Event count nonzero, QIDs look normal, but a specific custom property a downstream Building Block depends on is empty	Extraction pattern broke for that one field only, while the rest of the DSM's mapping kept working — the exact shape of Part 4 §5's Rule Autopsy	AQL search selecting the specific custom property, scoped to the affected QID, over a representative sample; confirm null rate	Fix the extraction pattern in the DSM extension; re-run Part 4's Validation Test for every Building Block depending on that property, not just the one that surfaced the gap
Log Source shows "active" but a scheduled report or dashboard depending on it looks stale	Report/dashboard scheduling or caching issue, unrelated to the Log Source itself — a false lead if investigated as a parsing failure	Confirm via a fresh ad hoc AQL search against <code>events</code> for this source that current data is actually present, independent of the report	Part 13 owns dashboard/report-specific troubleshooting; don't spend Log Source triage time here once this check clears the source itself

```
-- QRadar AQL — SELECT/FROM/WHERE syntax reads close to standard SQL but AQL is not SQL:
-- no general-purpose JOIN, and LAST n HOURS is AQL-specific time-window shorthand, not
-- a function call. QIDNAME()/LOGSOURCENAME() resolve internal integer IDs to text per
-- DEH Part 27 §1.2 — this part relies on both without re-explaining them.
SELECT QIDNAME(qid) AS "Event Name", COUNT(*) AS "Occurrences"
FROM events
WHERE LOGSOURCENAME(logsourceid) = 'suspect-source-01'
GROUP BY QIDNAME(qid)
LAST 24 HOURS
```

This query's limitation: it shows the QID distribution for one window, not a before/after comparison. Run it twice — once bounded to a window before the suspected change point (an explicit `START / STOP` range in place of `LAST 24 HOURS`) and once after — and compare the generic-QID row's share of total

events between the two. A generic-QID share that jumps from a small background rate to the majority of this source's traffic is the parsing-layer failure confirmed; a generic-QID share that stays flat while total event count drops to zero is the collection-layer failure instead, and this query alone won't distinguish that case from "the source went quiet" — check the Log Source's own last-event-received indicator for that branch.

Rule Autopsy

The rule: A syslog-TCP Log Source forwarding a perimeter firewall's traffic, feeding several threshold-based Rules keyed on `CATEGORYNAME(category)` matching a broad "denied/blocked" category — none of the LSASS-specific worked example this book carries from DEH Part 27; a plainer, higher-volume case that's more representative of what actually floods a real troubleshooting queue.

Why it shipped: The syslog-TCP connection used a certificate for transport encryption with a two-year validity window set at onboarding. Nobody added the expiry to a maintenance calendar, because the onboarding checklist (Part 4 Table 4.2) was followed in full at the time — the certificate was valid then, and nothing in that checklist re-checks a credential's future expiry as a distinct step.

How it failed: The certificate expired at 2 a.m. on a weekend. The firewall's syslog-TCP client failed the TLS handshake on every subsequent attempt and, per its own configuration, silently dropped forwarding rather than alerting on its own side. QRadar's Log Source showed no events starting at the expiry timestamp — a clean, detectable collection-layer failure by Table 18.1's own definition — but nobody was watching that Log Source's status over the weekend, and the threshold Rules downstream had nothing to evaluate. Every Rule fed by this source went quiet exactly the way a genuinely quiet firewall would: no errors, no Offenses, nothing to investigate.

The fix: Certificate/credential expiry dates for every certificate-based Log Source got added to the onboarding log Part 4 Table 4.2 already requires, with a renewal reminder scheduled well ahead of expiry — and a standing monitoring check for "zero events from a normally-active Log Source longer than its expected quiet period" was added, so a silent collection-layer failure surfaces as its own alert rather than only as an absence of Offenses nobody was specifically watching for.

PRODUCT VERSION NOTE *Whether QRadar surfaces a "Log Source stopped sending events" condition as a proactive System Notification, a dashboard item, or only as something a platform engineer discovers by manually reviewing the Log Source list, and how configurable that detection window is, has varied across releases and is not something this book can state as a fixed console behavior without lab access to verify (STYLE-GUIDE.md §0/§9). Confirm what your own deployment surfaces automatically, and what it doesn't, before assuming a silent collection failure like the one above would have been caught without the standing check the fix above describes.*

3. Failure mode 2 — A DSM/QID mismatch after a vendor agent upgrade

[PLATFORM ENGINEER] This is the narrower, sharper case Table 18.1's third row gestures at. It's typically discovered not by a platform engineer reviewing Log Source health, but by a Rule Engineer or analyst noticing a specific Rule has gone quiet, weeks after the actual cause — a vendor's routine agent, firmware, or software upgrade — already happened. Part 4 §5's Rule Autopsy walks one concrete instance in full narrative detail (a Sysmon agent upgrade changing `GrantedAccess`'s string format, silently emptying the custom property `BB:LSASS-Access-Candidate` depends on); this section doesn't re-tell that story, it generalizes the diagnostic technique to any vendor.

The defining shape of this failure mode: the Log Source keeps showing healthy, active status throughout — Table 18.1's collection layer never breaks. The QID itself often keeps resolving correctly too, because a vendor upgrade frequently changes a field's *format* without changing the log line's structure enough to break the DSM's signature match. What breaks is narrower and harder to notice: a custom property silently changes format, silently stops populating, or — worse — starts populating with a *plausible-looking but wrong* value a Rule Test's literal comparison no longer matches.

Table 18.2 — Failure mode 2: a DSM/QID mismatch after a vendor agent upgrade. Supports isolating which specific field broke, not just confirming that "something" did.

Symptom	Likely cause set	Verification step	Fix
A previously reliable Rule's Offense count drops to zero or near-zero, with no change to the Rule, Building Block, or Reference Set itself	Vendor agent/firmware/software upgrade changed a custom property's extraction format (case, padding, encoding) without changing the QID	AQL search selecting the specific custom property the Rule Test compares, over a window spanning the suspected upgrade date, checking for a null-rate or format shift (query below)	Update the DSM extension's extraction pattern to match the new format; re-run Part 4's Validation Test for every Building Block depending on that property
Same symptom, but the property is still populated — just with a value that no longer matches the Rule Test's literal comparison	Upgrade changed the <i>value</i> a field reports (e.g., a different access-mask encoding, a renamed category string) without changing its presence or format detectably	Compare the distinct values observed for that property before and after the suspected date, not just whether it's populated	Update the Rule Test's or Building Block's comparison value set to include the new value, after confirming it's semantically equivalent — do not just widen the match blindly
A Rule's Offense count doesn't drop but changes character — different <code>sourceip</code> /asset distribution, different time-of-day pattern	Vendor upgrade changed which events reach this QID at all (broadened or narrowed the DSM's own matching for that event type)	Compare QID-level event volume and its top contributing hosts before/after the suspected date	Confirm with the vendor's own release notes whether logging behavior intentionally changed; adjust the Building Block scope if the new behavior is legitimate and expected
Multiple, seemingly unrelated Rules go quiet around the same date	The broken property is a foundational one referenced by several Building Blocks — Part 9's dependency-graph problem surfacing as a troubleshooting symptom rather than a governance one	Check the Building Block dependency graph (Part 9 §1) for every affected Rule; confirm they share the same upstream custom property	Fix the shared extraction pattern once; re-validate every dependent Building Block, not just the one that was noticed first

```
-- QRadar AQL – SELECT/FROM/WHERE syntax, not standard SQL: no general-purpose JOIN, and
-- LAST n HOURS/an explicit START-STOP range are AQL's own time-window mechanisms. Custom
-- property names in double quotes follow DEH Part 27 §1.2's convention exactly. The QID
-- display string and property names below are DET-27-01's illustrative example (DEH Part
-- 27 §3.1-§3.2) carried forward for continuity, not a claim that any deployment's DSM
-- resolves them identically – verify both against your own Log Activity field list first.
SELECT "Granted Access", COUNT(*) AS "Occurrences"
FROM events
WHERE QIDNAME(qid) = 'Process accessed'
  AND "Target Process Name" ILIKE '%lsass.exe%'
  AND devicetime BETWEEN '2026-08-01 00:00' AND '2026-08-31 23:59'
GROUP BY "Granted Access"
```

Run this once for a window safely before the suspected vendor-upgrade date and once for a window safely after, and compare the distinct-value list, not just row counts. A property that reports one consistent value before the change and a *different* consistent value after — rather than simply going null — is the harder variant of this failure mode: the Rule Test's comparison silently stops matching, with the field still populated and the analytic still logically well-formed, which is exactly why comparing distinct values (not just presence) matters more here than in Failure Mode 1.

Validation Test Setup: A Rule or Building Block suspected of having gone quiet due to a vendor agent/firmware upgrade, the specific custom property it depends on identified, and the DSM extension's extraction pattern updated to match the new format. **Action:** Generate a fresh known-positive event on the upgraded source system — the same discipline Part 4 §6 requires at initial onboarding, applied here as re-validation rather than first validation — and confirm the resulting event in Log Activity. **Expected result:** The event resolves to the same QID as before the upgrade; the target custom property is populated in the corrected format; an ad hoc AQL search scoped to this Log Source and QID over a window including only post-fix events returns the property populated and matching the Rule Test's comparison value across more than one sample event; and the affected Rule produces a new Offense (or updates an existing one) from a subsequent live, real event — not only from the deliberately generated test event, which confirms the fix reaches production traffic and not only the test case that prompted it.

4. Failure mode 3 — An offense-flooding incident

[RULE ENGINEER] Where the first two failure modes present as *silence* — a Rule, Building Block, or Log Source that should be producing something and isn't — this one is the opposite: a queue that was manageable an hour ago and now isn't. Part 15 owns the full noisy-offense tuning workflow as an ongoing discipline; this section is narrower and more urgent — what to check in the first hour of an active flood, before there's time for Part 15's full intake process, and specifically how to tell whether the flood is a real security event, a platform misconfiguration, or both at once.

[ANALYST] The distinction matters most to whoever is working the queue directly during the flood: escalating every flooding Offense as a live incident when the actual cause is a stale Reference Set burns response capacity that a real campaign would need, and the reverse — writing off a genuine campaign as "probably just noisy config" — delays escalation on something that needed it immediately. The verification steps below exist to settle that question with evidence before either instinct wins by default.

The single most useful diagnostic move here is one most teams under flood pressure skip, because it feels like it wastes time that should go to the Offenses themselves: **compare the raw matching-event volume behind the flooding Rule against the Offense count it's producing, over the same window.** These two numbers answering differently is the fastest signal for which cause-set branch to investigate:

- **Matching-event volume spiked, and Offense count spiked proportionally.** The underlying activity itself increased — a real campaign, a misconfigured client hammering a service and triggering a legitimate threshold, or a new noisy Log Source (Part 17) whose volume the deployment wasn't sized for. The Rule is doing exactly what it was built to do; the input changed.
- **Matching-event volume stayed flat, but Offense count spiked.** Something in the *correlation logic itself* changed what counts as a match, independent of the world it's watching — a Reference Set that stopped excluding what it used to exclude, a Building Block edit that widened scope, or a threshold Rule Test's window or count value changed. This is a configuration-layer

flood, not a real-world one, and the fix is a rollback or a targeted tuning change, not an incident response to whatever the Offenses claim happened.

Table 18.3 breaks the cause set down further, with this volume-comparison check as the first verification step for nearly every row.

Table 18.3 — Failure mode 3: an offense-flooding incident. Supports separating a real activity surge from a configuration-layer flood before committing incident-response time to either.

Symptom	Likely cause set	Verification step	Fix
One Rule's Offense count spikes sharply	Reference Set drift — an exclusion list (e.g., <code>RS-Allowlisted-Scanner-IPs</code>) went stale after a legitimate source's address changed, and traffic that used to be excluded now matches	Check the Reference Set's last-updated timestamp and current membership against the source it's meant to allowlist; compare matching-event volume vs. Offense count as above	Update the Reference Set's membership; per Part 10, add or shorten a TTL and an owner if this Reference Set had neither
Same symptom	A Building Block feeding this Rule was edited (narrowed an exclusion, widened a match condition) for an unrelated reason and the change had a broader blast radius than intended — Part 9's dependency-graph problem again	Check the Building Block's recent change per whatever revision mechanism your deployment provides (see PRODUCT VERSION NOTE below); identify every other Rule that also depends on it	Roll back the specific change if it wasn't intentional; if intentional, confirm every dependent Rule was meant to be affected, not just the one that triggered the investigation
Same symptom	A threshold Rule Test's count or window was changed (deliberately or by an incorrect edit) to a value that fires far more easily	Compare the Rule's current threshold/window configuration against its documented intended value (Part 16's change record, if one exists)	Restore the intended threshold; if no change record exists, this incident is itself the argument for Part 16's change-management discipline
Multiple, unrelated Rules spike at once	A newly onboarded Log Source is pushing far more volume, or a different event mix, than estimated at onboarding — a real surge, not a configuration fault	Matching-event volume check (above) confirms a real spike; cross-check against Part 17's EPS/FPM budget for whether the deployment itself is now under licensing or processing pressure	Part 17 owns the capacity response; short-term, prioritize triage on the Rules with the clearest signal-to-noise, not an equal pass across every flooding Rule
Multiple, unrelated Rules spike at once, and the pattern looks credible as one campaign	A genuine, active security event	Pull contributing events across the flooding Offenses and look for a shared indicator (source IP range, account, process) consistent across Rules rather than an artifact of one broken Building Block	This is not a troubleshooting fix — escalate through the organization's incident-response process; do not spend further time on Tables 18.1–18.2 once a real campaign is confirmed

Noisy Offense Trap *The instinct under flood pressure is to silence the loudest Rule first — disable it, or bolt on a broad, hastily written exclusion — because that's the fastest way to make the queue stop growing. If the actual cause is a stale Reference Set or an over-widened Building Block (Table 18.3's first two rows), disabling the Rule fixes nothing; it converts a false-positive flood into a silent detection gap with the same root cause unaddressed, waiting to flood again once re-enabled or to stay disabled long enough that nobody remembers why. Per DEH [TERMINOLOGY.md](#)'s Tuning-vs-Suppression distinction (carried forward by Part 15), the volume-comparison check exists precisely so the fix targets the verified cause — restore the Reference Set, revert the Building Block change, restore the threshold — rather than reaching for whatever blunt instrument is fastest.*

Blind Spot *An Offense queue that has just spiked, viewed from the Offenses tab alone, looks identical whether the cause is a real active campaign or a Reference Set that silently went stale three weeks ago and finally got exercised by ordinary traffic today. Both produce "many new Offenses, seemingly correlated in time." Magnitude scoring — computed from severity, relevance, and credibility per Part 3 — doesn't resolve this either — a configuration-layer flood produces high-magnitude Offenses just as easily as a real campaign, because that scoring evaluates the fired Rule's own severity, the asset's relevance, and the event's credibility, not whether the Rule's match condition is currently doing what its author intended. The matching-event-volume-vs-Offense-count comparison above is the specific check that closes this gap; defaulting to "assume it's real" or "assume it's a misconfiguration" under time pressure is a coin flip with real cost either way wrong — lost response time on a real campaign, or burned incident-response hours chasing an attacker who isn't there.*

PRODUCT VERSION NOTE *Whether your deployment exposes a built-in revision history for a Rule or Building Block (who changed what, and when) directly in the console, versus requiring an external change log maintained by the team itself, has varied across QRadar releases and is not a capability this book can assert as universally present (Part 9's front matter flags the same uncertainty for governance purposes; this is the troubleshooting-time consequence of that same gap). If your deployment has no reliable built-in revision history, Part 16's change-ticket discipline is not optional process overhead — during an offense-flooding incident, it may be the only way to answer "what changed, and when" at all.*

5. A triage flowchart across all three failure modes

[PLATFORM ENGINEER] Figure 18.1 states this part's branching logic as a single diagram — which of the three tables to open first, based on the symptom actually observed, before any cause is investigated. Appendix A3 extends each of the three branches below into its own detailed decision tree; this figure is

the top-level router between them, not a replacement for either the appendix or Tables 18.1–18.3's own detail.

flowchart TD

A["Something's wrong – state the\nsymptom precisely, not a conclusion"] --> B{"What was actually observed?"}

B -->| "A known-active Log Source shows\nfewer or zero events" | C["Failure Mode 1:\nLog source stopped parsing\n(Table 18.1)"]

B -->| "Events still arriving, but a Rule/\nBuilding Block that used to fire\nhas gone quiet or looks wrong" | D["Failure Mode 2:\nDSM/QID mismatch after a\nvendor agent upgrade\n(Table 18.2)"]

B -->| "Offense count or queue\nvolume spiked sharply" | E["Failure Mode 3:\nOffense-flooding incident\n(Table 18.3)"]

C --> F{"Events stopped\nentirely, or just\nstopped resolving\nto a real QID?"}

F -->| "Stopped entirely" | G["Collection-layer fix:\nnetwork/agent/collector\n(Part 4 Table 4.1)"]

F -->| "Generic QID / empty\ncustom property" | H["Parsing-layer fix:\nDSM/extension update\n(Part 4 §4-§5)"]

D --> I["Compare custom-property\nvalue distribution\nbefore/after suspected\nupgrade date"]

I --> J["Update DSM extraction\npattern or Rule Test\ncomparison value;\nre-run Validation Test"]

E --> K{"Matching-event volume\nvs. Offense count:\ndid both spike, or\njust the Offense count?"}

K -->| "Both spiked\nproportionally" | L["Real surge –\ncampaign or capacity\n(Part 15 intake / Part 17)"]

K -->| "Only Offense\ncount spiked" | M["Configuration-layer flood –\nReference Set drift (Part 10)\nor Building Block change (Part 9)"]

G --> N["Document root cause\nand prevention step"]

H --> N

J --> N

L --> N

M --> N

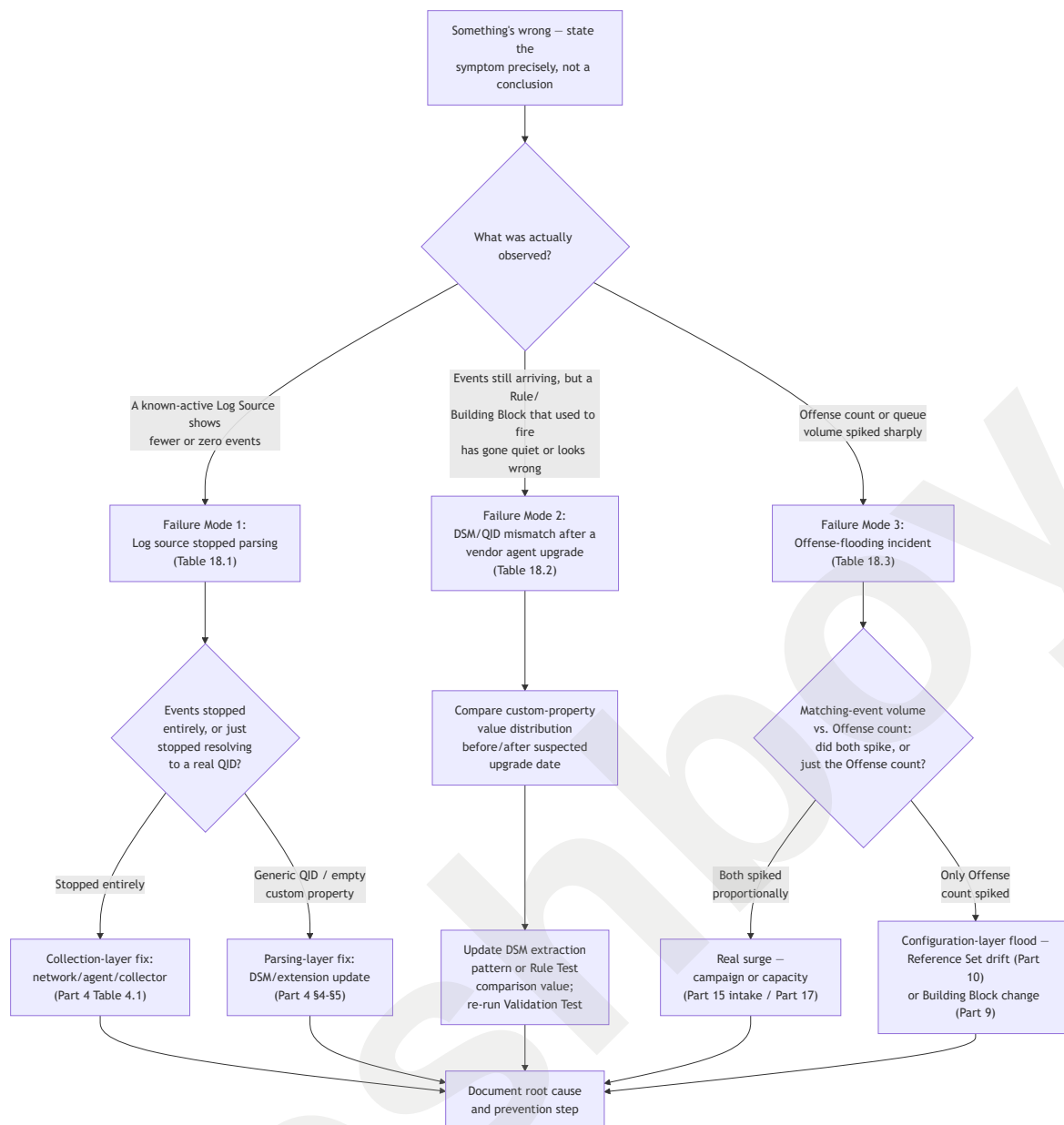


Figure 18.1 (FIG-18-01) — Top-level troubleshooting router across this part's three failure modes. *CONCEPTUAL.* Illustrates the first branch decision for each failure mode — collection vs. parsing layer for Failure Mode 1, and the matching-event-volume-vs-Offense-count comparison for Failure Mode 3 — as a single entry point a platform engineer or on-call analyst can follow under time pressure; not a reproduction of any QRadar console screen, and not a substitute for Tables 18.1–18.3's full cause sets or Appendix A3's per-mode decision trees. This book has no lab access to produce a captured console diagnostic view (STYLE-GUIDE.md §9), so this diagram is drawn as this book's own workflow, not sourced from IBM.

6. Telling a real incident from a platform fault, under time pressure

[SOC MANAGEMENT]

The volume-comparison check in §4 is a technical diagnostic, but the decision it feeds is often a management one as much as a technical one: whether to escalate a flooding queue as a security incident, treat it as a platform-maintenance ticket, or — the genuinely hard case — both,

because a real campaign and a stale Reference Set can coincide. A SOC manager fielding a "why do we have four hundred new Offenses" question from leadership mid-flood needs an answer grounded in Table 18.3's verification step, not a guess extrapolated from how alarming the Offense count looks, because the count alone — as the Blind Spot above states plainly — cannot distinguish the two causes on its own.

The practical discipline: whoever owns the on-call rotation should have Table 18.3's volume-comparison query ready before the first flood happens, not improvised during one, and status updates during a flood should frame around what's been verified ("matching-event volume is flat; this looks like a stale Reference Set, not a live campaign — confirming now") rather than the raw Offense count, which by itself supports either story equally well. Reporting a bare offense count as a proxy for severity during a flood repeats, at incident speed, the same coverage-count ambiguity Part 13 already warns against for routine dashboard reporting.

7. Closing the loop: what prevents recurrence

[PLATFORM ENGINEER] None of the three failure modes in this part are closed by the fix column in Tables 18.1–18.3 alone — each fix is paired with a part of this book whose entire job is making the same failure less likely to recur, and skipping that pairing is how a team ends up re-diagnosing the same incident under a different name a year later:

- **Failure Mode 1 (log source stops parsing)** is prevented by Part 4's onboarding checklist and its requirement to *re-run* known-positive validation after any change touching the source, not just at initial onboarding. The Rule Autopsy in §2 is a variant of the exact gap Part 4 §5 names: a validated mapping nobody re-validated after something changed source-side.
- **Failure Mode 2 (DSM/QID mismatch after a vendor upgrade)** is the same prevention target, sharpened to the trigger of a vendor-side agent, firmware, or software update. Part 4's onboarding log — Log Source Type, DSM/extension version, validation date, owner — is the record that should flag this source as due for re-validation the moment its vendor ships an update, if that log is actually maintained and checked against vendor release notes.
- **Failure Mode 3 (offense flooding)** is prevented by different parts depending on which branch of §4's volume comparison it lands in: Part 10's Reference Set TTL/ownership discipline for the configuration-layer branch, Part 15's tuning-intake workflow for the fuller diagnosis-to-re-validation cycle once the immediate flood is under control, and Part 17's capacity-planning discipline for the real-surge branch, when the surge is a Log Source the deployment was never sized for.

A troubleshooting playbook that stops at "fixed" without naming which of these disciplines was the actual gap has fixed the incident and left the recurrence risk exactly where it was.

Cross-references: DEH Part 27 §1.2 (`QIDNAME()` / `LOGSOURCEATYPENAME()` and the custom-property display-name convention this part's diagnostic queries rely on without re-teaching); DEH Part 27 §3.1 (the DSM/QID silent-failure mode this part's Failure Modes 1 and 2 operationalize into a diagnostic playbook); DEH Part 27 §2.3 (the chained-Rule Blind Spot underlying why a single broken field can

silently take down more than one Rule); DEH Part 23 §5 (Sigma-first-vs-native authoring tradeoff, not re-argued here). Within this book: Part 4 (DSM/QID onboarding checklist and the Rule Autopsy this part's §2–§3 extend); Part 9 (Building Block dependency graphs, relevant to both Failure Mode 2's multi-Rule variant and Failure Mode 3's configuration-layer branch); Part 10 (Reference Set TTL/ownership discipline that prevents Failure Mode 3's drift cause); Part 13 (dashboard/report-specific troubleshooting and the bare-count reporting risk §6 extends into an incident-communication context); Part 15 (the full noisy-offense tuning-intake workflow this part's §4 hands off to after first-hour triage); Part 16 (change-ticket discipline as the substitute for a missing built-in revision history, flagged in §4's PRODUCT VERSION NOTE); Part 17 (EPS/FPM capacity planning for Failure Mode 3's real-surge branch); Appendix A3 (per-failure-mode decision trees extending Figure 18.1's top-level router).

Part 19 — Apps, Extensions, and the App Framework

Why this part exists

[CONCEPT] Part 2 §1.6 named the App Host as one of six deployment component types and deferred the detail: apps run there (or on the Console, in a smaller deployment) so "a resource-hungry or poorly-behaved app doesn't compete with the Console's own management and UI workload." Part 13 built on the same framework from the other direction, treating Pulse as one of the surfaces a dashboard-and-reporting reader actually looks at. Neither part had room to answer the questions that matter once a SOC is actually deciding whether to install something: what is an "app" structurally, what can it see and touch once it's running, who vetted it before it went into production, and what happens to the rest of the deployment if it misbehaves. This part answers those questions.

DEH Part 27 does not cover the App Framework at all — its job was proving a point about AQL and the Custom Rules Engine (CRE), not surveying QRadar's extension ecosystem. Where this part shows an app querying Ariel or calling QRadar's REST API, it uses the exact AQL syntax and function set DEH Part 27 §1 and §1.2 already teach, and it does not re-explain `SELECT / FROM / WHERE / QIDNAME()` from scratch. Where this part touches whether an app's own bundled content should be authored Sigma-first or natively, it defers to DEH Part 23 §5's tradeoff rather than re-arguing it — an app vendor's packaging choice is a fact of that specific app, not a decision this book's reader makes for their own rule set.

This part covers, in order: what the App Framework actually is structurally (§1), where an app physically runs and why that placement decision matters (§2), how an app gets from IBM's marketplace to a running container in your deployment (§3), a survey of the app families a working QRadar SOC actually encounters (§4), what an installed app can reach once it's running (§5), and the governance risk a third-party app introduces against a production deployment (§6) — the scope item Part 22's staffing capstone assumes this part has already covered when it budgets review time against it.

1. What the App Framework actually is: isolation, not just packaging

[PLATFORM ENGINEER] An "app," in QRadar's own vocabulary, is a packaged, independently deployable unit of functionality — a UI panel, a background data-processing job, or both — that runs inside its own isolated runtime rather than as code merged into the Console's own process space. That isolation is the architectural point, not a footnote: it is what makes it possible to install a third party's code inside a production security platform at all without that code having unrestricted access to the Console's own filesystem, configuration, or the credentials the platform itself runs on. Each app declares, in a manifest packaged with it, what it needs — a defined slice of CPU and memory, and a set of permission scopes for which QRadar objects it's allowed to read or write (§5) — and the framework is responsible for enforcing that ceiling and that scope, not trusting the app's own code to self-limit.

This is the same resource-governance principle Part 2 Table 2.2 named without unpacking it: "App Framework app competing with Console UI responsiveness" is a real, observed failure mode precisely *because* an app's resource ceiling is enforceable, not because it never gets set. A misbehaving app that ignores its own declared footprint doesn't take down the CRE or Ariel search on a properly isolated App

Host — it degrades its own runtime, and the framework (or an administrator watching resource utilization) has a boundary to act against. An app running directly on the Console in an All-in-One or small deployment has a narrower version of the same protection: isolated from the Console's core processes, but sharing the same physical or virtual appliance's total CPU, memory, and disk I/O budget, which is exactly the tradeoff §2 covers.

PRODUCT VERSION NOTE *The specific containerization or process-isolation technology underlying the App Framework, the exact mechanism by which a resource ceiling is configured and enforced (a fixed allocation set at install time versus an adjustable quota an administrator can tune afterward), and the console menu path for viewing or changing an installed app's resource allocation have all been the kind of implementation detail that shifts across QRadar release cycles without changing the architectural principle above. Verify the current mechanism and menu path against your own deployment's Admin console and IBM's current documentation before treating any specific UI description as current.*

2. Where apps run: Console vs. dedicated App Host

[PLATFORM ENGINEER] Part 2 §1.6 already stated the decision in outline: smaller or All-in-One deployments run apps directly on the Console; a deployment large enough to justify the added component runs them on a dedicated App Host instead. What that section didn't have room for is the actual factors that should drive the choice for a specific deployment, rather than treating "add an App Host" as a blanket best practice regardless of scale.

Table 19.1 — Factors in the Console-vs-App-Host placement decision. Supports deciding whether a given deployment's app workload justifies the added component and its own maintenance/licensing overhead, rather than defaulting either way.

Factor	Favors Console-hosted apps	Favors dedicated App Host
Number and footprint of installed apps	One or two lightweight apps (a single dashboard-widget app)	Several apps, or any app with a known heavy background-processing job (a behavioral-analytics app scoring every asset nightly)
Console UI responsiveness complaints	None reported	Analysts report slow Offense-list or search-page rendering coinciding with app activity
Deployment size/topology	All-in-One or small distributed deployment already resource-constrained	Distributed deployment with headroom to add a component, or Console already near capacity from its own management workload
App vetting maturity (§6)	Only vetted, low-permission-scope apps installed	Any third-party or high-permission-scope app, where blast-radius containment matters most
Licensing/appliance budget	No budget for an additional component	Budget available, and the isolation is worth the added cost

A deployment does not have to choose once and never revisit it — the signal in Part 2 Table 2.2 ("App Framework app competing with Console UI responsiveness → App Host") is exactly the trigger for

moving from the left column to the right one as an app footprint grows, not a decision made permanently at initial deployment sizing.

3. The App Exchange: from IBM's marketplace to a running container

[PLATFORM ENGINEER] IBM operates a marketplace — commonly referred to as the App Exchange — through which both IBM-authored apps and third-party or partner-authored apps are published, discovered, and installed into a QRadar deployment. An app listed there ships as a package: the app's own code, a manifest declaring its required resource allocation and the specific permission scopes it needs (read access to Ariel search results, write access to create or update Offenses, read or write access to a named Reference Set — the same object categories this book's Parts 3 and 10, and DEH Part 27 §2.2, already cover), and version metadata stating which QRadar release(s) it's compatible with. An administrator installs a package either directly from the App Exchange listing inside the Console, or by uploading a package file obtained separately — the second path exists specifically for air-gapped or otherwise internet-restricted deployments that cannot reach the Exchange directly.

Installation itself does not imply review. The framework enforces the resource ceiling and permission scopes an app's manifest declares (§1), but it does not evaluate whether those declared permissions are *appropriate* for what the app actually claims to do, whether the app's code does anything beyond its stated purpose with the access it's granted, or whether the vendor will keep maintaining it against QRadar's own future releases. That evaluation is a governance function this book's reader has to build, not a gate the framework provides for free — the exact gap §6 covers in depth.

Figure 19.1 puts the install-time governance decision and the runtime data-access surface on one diagram, so the rest of this part can refer back to a single picture instead of re-describing the pipeline in every section.

flowchart TB

```
subgraph Exchange["IBM Security App Exchange"]
```

```
    PKG["App package:\ncode + manifest\n(resource request,\npermission scopes,\nversion compatibility)"]
```

```
end
```

```
PKG -->|"admin initiates install"| Review{"Governance review\n(Section 6):\napproved?"}
```

```
Review -->|"no"| Rejected["Not installed"]
```

```
Review -->|"yes"| Placement{"Console or dedicated\nApp Host?\n(Section 2, Part 2 Section 1.6)"}
```

```
Placement -->|"Console"| ConsoleRun["App container\nruns on Console"]
```

```
Placement -->|"App Host"| HostRun["App container\nruns on App Host"]
```

```
ConsoleRun --> Runtime["Isolated app runtime\n(framework-enforced\nCPU/memory ceiling,\ngranted permission scopes)"]
```

```
HostRun --> Runtime
```

```
Runtime -->|"calls"| API["QRadar REST API\n(Section 5)"]
```

```
API --> Ariel["Ariel events/flows\n(AQL search -\nDEH Part 27 Section 1)"]
```

```
API --> Offenses["Offense index\n(Part 3)"]
```

```
API --> RefData["Reference Sets/Maps\n(Part 10)"]
```

```
Runtime --> UI["App panel embedded\nin Console UI\n(e.g. Pulse widget, Part 13)"]
```

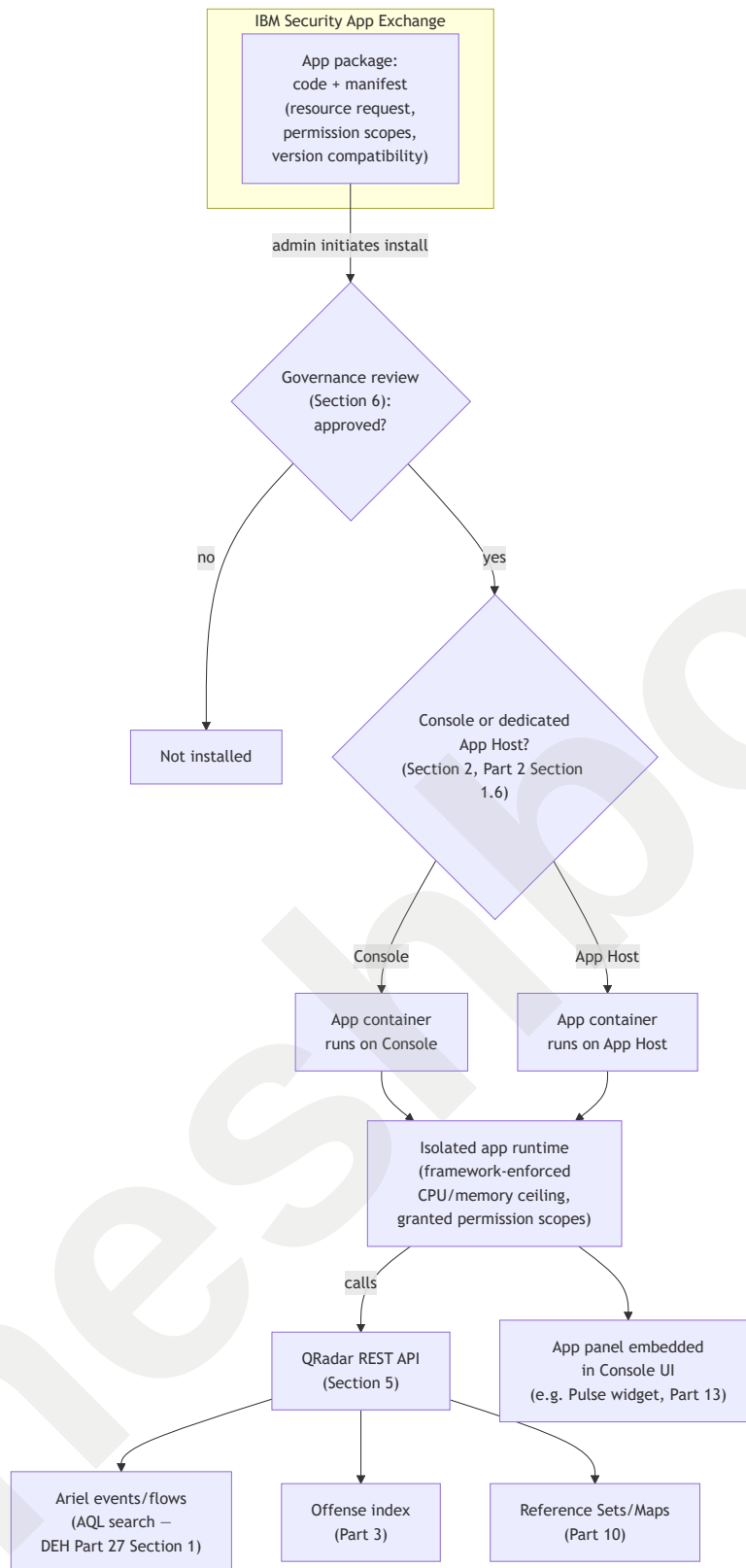


Figure 19.1 (FIG-19-01) — An app's path from the App Exchange to a running container, and what it can reach once running. *CONCEPTUAL*. Illustrates the governance-review and placement decisions this part argues are load-bearing (Sections 2 and 6), and the REST-API-mediated data-access surface Section 5 covers in depth — not a reproduction of any single IBM architecture diagram, and not a claim about a specific release's exact install-wizard steps or menu wording.

PRODUCT VERSION NOTE *The App Exchange's own name, URL, catalog browsing experience, and whether it is branded as part of "QRadar," a broader IBM Security Suite umbrella, or a successor product name, have shifted across IBM's own release and business-packaging history in ways this book's author cannot verify as current from outside a live deployment. Treat "App Exchange" in this part as the generic concept — a marketplace for QRadar apps — and verify the current name, URL, and any air-gapped-install procedure against IBM's own current documentation before relying on this part's description as a literal, current walkthrough.*

4. A survey of common apps

[CONCEPT] This section stays at survey depth on purpose — a full working guide to any one app's own configuration is out of this book's scope, which is about the platform these apps run on top of, not any single app's feature set. The point of this survey is knowing what each app family is for well enough to decide whether your SOC needs it, and which of this book's other parts its output actually feeds.

4.1 Use Case Manager

[RULE ENGINEER] A use-case-and-coverage-mapping app maps a deployment's actual Rules and Building Blocks against a framework of attack techniques — most commonly MITRE ATT&CK — to surface which techniques have deployed detection content and which don't. This is directly downstream of the **BB:/R:**-prefixed object library Part 9 covers: the app reads the same Rule and Building Block metadata that library governance already has to track, and presents it as a coverage map instead of a flat list. It does not create detection content on its own — a technique showing "no coverage" in the map is a prioritization signal for a rule engineer to act on, not a gap the app closes by itself. Treat its coverage claim with the same discipline Part 13 §4 applies to a bare offense-count dashboard metric: "we have a Rule tagged against this technique" is a narrower claim than "we would actually detect this technique's real-world variations," and the app's map only speaks to the first one.

PRODUCT VERSION NOTE *Whether a use-case-and-coverage-mapping app of this kind currently ships under the name "Use Case Manager," as a distinct installable App Exchange app at all, or has been renamed or folded into a different first-party offering, is the same kind of commercial-packaging fact §4.3's note flags for the behavioral-analytics app family rather than asserts here. Verify current availability, name, and the specific technique-framework version (e.g., which MITRE ATT&CK release) it maps against, against IBM's own current documentation before assuming this app exists, under this name, in the tier a reader is licensed for.*

4.2 Pulse

[PLATFORM ENGINEER] Pulse is a dashboarding and visualization app that Part 13 already covers from the consumption side — what a Pulse widget looks like to a SOC manager or executive reader, and how

it's backed by the same AQL-driven or Offense-index-driven data sources any Dashboard tab item uses (Part 13 §1.1). This part's addition is the App Framework side of that same fact: Pulse is not special-cased infrastructure baked into the Console — it is an app, subject to the same install, resource-allocation, and placement decisions (§§1–2) as any other, which matters directly if a deployment's executive-reporting workload is heavy enough to justify moving it to a dedicated App Host rather than leaving it competing with Console UI responsiveness.

4.3 User Behavior Analytics

[RULE ENGINEER] A behavioral-analytics app of this kind builds risk or anomaly scores for users and assets from baseline behavior, typically using the same underlying mechanisms Part 11 covers at full depth — Reference Sets/Maps holding accumulated state, and Rules (including Behavioral rule types) evaluating that state against a baseline rather than a fixed threshold. Its practical output is usually its own risk-score dashboard and/or its own Offense-generating Rules layered on top of the correlation content a team already has. That second property is exactly why §6's governance discussion applies to this app family with extra force: an app that deploys its own Rules and Building Blocks as part of its own installation is adding to the **BB: / R:** library Part 9 governs, whether or not the team that installed it thought of it that way.

PRODUCT VERSION NOTE *Whether a behavioral-analytics app of this kind currently ships as a distinct installable App Exchange app, is bundled into a broader QRadar packaging tier, or has been renamed, retired, or folded into a different product family entirely, is exactly the kind of commercial-packaging fact this book's §11 convention flags rather than asserts. Verify current availability, name, and packaging against IBM's own current documentation before assuming this app family exists, under this name, in the specific tier a reader is licensed for.*

4.4 Third-party and partner apps

[SOC MANAGEMENT] Beyond IBM's own apps, the App Exchange hosts partner- and community-authored apps covering threat-intelligence enrichment, ticketing-system integration, case-management handoff, and dozens of narrower integrations — a long tail no survey section can enumerate usefully. The governance posture in §6 applies to every one of them, and applies *more* strongly the further a given app is from IBM's own first-party maintenance commitment: a community-maintained integration with a small install base is more likely to lag a QRadar upgrade, go unmaintained after its original author moves on, or have had less security review than a flagship first-party app.

Table 19.2 — App families surveyed, and what each one's output feeds. Supports deciding which of this book's other parts a given app's evaluation should route through, rather than treating "install the app" as the end of the decision.

App family	Primary function	Feeds into	Version sensitivity
Use-case/coverage-mapping (e.g. Use Case Manager)	Maps deployed Rules/BBs to an attack-technique framework	Part 9 (BB library governance), Part 22 (staffing/coverage review)	Catalog naming and technique-framework version — flag per §11
Dashboarding/visualization (e.g. Pulse)	Renders AQL- and Offense-index-backed widgets	Part 13 (dashboards/reporting)	Widget catalog and default layout — flag per §11
Behavioral analytics (e.g. UBA-type app)	Baseline-based risk/anomaly scoring, often deploys its own Rules/BBs	Part 10 (reference data), Part 11 (behavioral rules), Part 9 (BB governance)	Product name, packaging tier, continued availability — flag per §11
Third-party/partner integrations	Threat-intel enrichment, ticketing, case handoff, niche integrations	Varies by integration; governance in §6 applies to all	Maintenance currency and update cadence — verify per app, not assumed

5. What an installed app can reach: AQL, the REST API, and the objects this book already governs

[PLATFORM ENGINEER] An app does not query Ariel storage directly, and it does not read or write a Rule, Building Block, or Reference Set through some app-specific back door — it goes through the same QRadar REST API surface a script, an external SOAR platform, or an analyst's own tooling would use, constrained to whatever permission scopes its manifest declared and the administrator approved at install time (§3). This matters for exactly one reason worth stating plainly: **an app's data access inherits every version-dependent property-name and QID (QRadar's internal event-taxonomy identifier) mapping risk this book and DEH Part 27 §3.1 already name for a human-written AQL search.** If an app's bundled dashboard widget queries a custom property your DSM (Device Support Module — the log-source parser that assigns a QID and extracts fields, DEH Part 27 §1.1) configuration never populated, or references a QID display string your environment's DSM assigns differently, that widget fails exactly the way an ad hoc analyst search fails under the same conditions — silently, with an empty or misleading result and no error surfaced to explain why.

A widget backed by an app's own saved search uses the identical AQL syntax and function set DEH Part 27 §1 and §1.2 teach in full — there is no separate "app query language":

```
-- QRadar AQL — the same SELECT/FROM/WHERE syntax and function set DEH Part 27 Section 1 and
-- Section 1.2 teach in full (QIDNAME(), LOGSOURCETYPENAME(), custom properties in double quotes).
-- Shown here only to make concrete that an installed app's dashboard tile queries Ariel through
-- this same surface, not a distinct app-specific query interface — see the framing paragraph above.
SELECT QIDNAME(qid) AS "Event Name", COUNT(*) AS "Occurrences"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) = 'Microsoft Windows Security Event Log'
GROUP BY QIDNAME(qid)
LAST 24 HOURS
```

This query's main limitation is the same one Part 13 §1.1 already names for any saved-search-backed dashboard tile: it is only as current and as complete as the DSM mapping behind the Log Source type it filters on, and an app vendor has no more visibility into your specific DSM configuration than any other consumer of the same AQL surface does.

The REST API itself is the mechanism an app's backend actually calls to submit that search or to read/write an Offense or Reference Set:

```
# Illustrative: an installed app's backend submitting an Ariel search via QRadar's REST API,
# the same endpoint pattern available to any authenticated caller with search permission --
# not a special app-only interface. Header name, endpoint path, and versioning scheme are
# flagged below as version-sensitive; do not treat this as a literal, current API call.
curl -k -H "SEC: <app-service-token>" \
  -X POST "https://<console-host>/api/ariel/searches" \
  --data-urlencode 'query_expression=SELECT QIDNAME(qid) AS "Event Name", COUNT(*) AS "Occurrences"
FROM events LAST 24 HOURS'
```

```
{
  "search_id": "b6e6b1b0-1c2d-4e3f-illustrative",
  "status": "WAIT"
}
```

PRODUCT VERSION NOTE *The REST API's exact authentication header name and mechanism, endpoint path and versioning scheme, and the specific permission-scope granularity an app manifest can request (a whole-category grant like "Offense read/write" versus a narrower per-object grant) have evolved across QRadar releases and are exactly the kind of literal API detail this part flags rather than asserts as current. Verify the current authentication mechanism, endpoint versioning, and permission-scope model against your own deployment's REST API documentation before an app's declared permission request is evaluated against it.*

The `SEC`-header authentication model and the create-search/poll-status/retrieve-results shape shown above are not this book's own guess at the REST API's surface: IBM's own public `qradar-mcp` reference implementation documents the same `SEC` / `QRadarCSRF` header pair (or an authorized service token passed as `SEC`) and the same Ariel-search lifecycle — submit a query, poll its status, then retrieve results — as the mechanism it wraps for programmatic callers (IBM, "qradar-mcp," GitHub, 2026: <https://github.com/IBM/qradar-mcp>). That confirms the shape of the pattern; it does not override the version note above about trusting any single header name or endpoint path as permanently current.

6. Governance risk: installing a third-party app against a production deployment

[SOC MANAGEMENT] Everything in §§1–5 establishes that the App Framework's isolation is real and enforceable. It does not follow that installation is safe by default — isolation contains an app's resource

footprint and constrains it to its declared permission scopes; it does not evaluate whether those scopes are appropriate, or audit what the app's code actually does with the access it's granted once running.

6.1 Permission scope is a grant, not a review

[RULE ENGINEER] An app that requests write access to Rules, Building Blocks, or Reference Sets is requesting the ability to change this book's Section D objects — the exact `BB:/R:/RS-`-prefixed library Part 9 and Part 10 govern — and the framework will grant that request as declared if an administrator approves the install, with no independent check that the app's actual behavior matches its stated purpose.

Rule Autopsy *The rule:* A behavioral-analytics app (§4.3) was installed specifically for its risk-scoring dashboard. As part of its own bundled content, it silently deployed several of its own Building Blocks and a handful of Rules with Offense-creating Responses, because that's how its baseline-comparison feature is implemented under the hood — not because anyone reviewing the install request understood that outcome. **Why it shipped:** The install approval focused on the dashboard feature the team wanted and the resource footprint in the manifest; nobody read far enough into the permission-scope list to notice "create/update Rule" and "create/update Building Block" sitting alongside "read Ariel search results." **How it failed:** Three weeks post-install, the Offense queue showed a new cluster of Offenses from Rules nobody on the rule-engineering team had authored, named after the app's own internal convention rather than this book's `R:/BB:` naming standard (§4, [STYLE-GUIDE.md](#)) — invisible to Part 9's governance review because that review assumes every Rule in the library came through the team's own authoring process. **The fix:** Treat "does this app's install request include write access to Rule, Building Block, or Reference Set objects" as a standing question in the pre-install checklist (§7), not an assumption to skip past because the app's advertised feature is something else entirely — and if the answer is yes, fold whatever it deploys into the same Part 9 governance inventory as any human-authored object, on the same review cadence.

6.2 The framework's isolation stops at resource and permission scope — it does not audit data handling

[PLATFORM ENGINEER] The framework enforces the CPU/memory ceiling and the permission scopes an app declares (§1). It does not, by itself, tell you what the app's own code does with the data it's granted access to read — whether it retains a copy outside the deployment, calls an external vendor endpoint for cloud-based analysis, or logs sensitive field values somewhere outside QRadar's own audit trail.

Blind Spot *The App Framework's resource and permission-scope enforcement answers "can this app touch more than it declared" and "is this app consuming more resources than its ceiling allows" — both real, enforced boundaries. It does not answer "what does this app do with the data it was correctly granted access to read." A third-party app with a legitimately granted "read Ariel search results" permission that also happens to forward a copy of matched events to an external vendor endpoint for cloud-side processing is operating entirely inside its declared scope from the framework's point of view — the framework has no mechanism to flag that as different from an app that keeps everything local. That evaluation is a data-handling and vendor-trust question a security/procurement review has to ask before install, not a gap the framework will surface after the fact.*

6.3 Review cost is a staffing line, not a one-time gate

[SOC MANAGEMENT] Part 9's closing SOC MANAGEMENT framing named the review-cost consequence of QRadar's own multi-object model for human-authored content; the App Framework adds a parallel review-cost line for anything installed rather than authored. Vetting an app's manifest, tracing what it actually deploys once installed (§6.1), and re-validating that after every app *and* every QRadar platform update, is ongoing work, not a single approval decision made once at install time.

SOC Management View *Budgeting a QRadar practice's headcount for rule review (Part 9, Part 22) without a corresponding line for app-vetting review understates the real review burden the moment a team installs anything beyond IBM's own first-party apps. A third-party app that quietly deploys its own Rules (§6.1) is exactly as much a review-cost item as a human-authored Rule, and an app that stops receiving vendor updates after a QRadar version bump is a maintenance liability nobody budgeted for if the original install decision was framed as "add a dashboard," not "take on an ongoing integration to maintain." Size the app-governance line explicitly, the same way Part 22 sizes rule-review headcount against the*
BB: / RS- / R: library.

7. A pre-install review checklist

[SOC MANAGEMENT] The checklist below collects §6's specific findings into the form a governance process actually uses — a gate to run every app installation request through, not a one-time exercise.

Table 19.3 — Pre-install review checklist for a QRadar App Framework app. Supports a consistent go/no-go decision regardless of which team member handles a given install request.

Review question	Why it matters	Owning role
What permission scopes does the manifest declare, and does every one map to a feature the team actually needs?	An unused write scope is pure risk with no offsetting benefit (§6.1)	Platform Engineer
Does the app deploy its own Rules, Building Blocks, or Reference Sets as part of installation?	If yes, it joins Part 9's governance inventory on the same review cadence as any human-authored object (§6.1)	Rule Engineer
Where does the app send data it's granted access to, and does any of it leave the deployment?	The framework does not audit data handling — this is a manual review question (§6.2)	SOC Management / Security Review
Is the app first-party (IBM), partner-maintained, or community-maintained, and what is its update cadence?	Maintenance currency risk scales with distance from first-party support (§4.4)	SOC Management
Does the deployment's current placement (Console vs. App Host, §2) have headroom for this app's declared resource request?	Avoids reproducing Part 2 Table 2.2's "app competing with Console UI responsiveness" signal after the fact	Platform Engineer
Is there a rollback plan (disable/uninstall) tested before this app is approved for production?	An app that deploys its own Rules doesn't necessarily clean them up on uninstall — verify before relying on uninstall as a safety net	Platform Engineer

Platform Reality *An app's install-time resource request is a declaration, not a guarantee of steady-state behavior — a behavioral-analytics app's nightly baseline-recomputation job, or a dashboarding app's widget refresh under a sudden spike in concurrent analyst viewers, can spike well above its typical footprint at a predictable or unpredictable interval. Watch an app's actual resource utilization for at least one full cycle of its heaviest workload (a nightly batch job, a peak-shift dashboard-viewing window) before treating its manifest's declared footprint as the number that matters for capacity planning, the same discipline Part 17 already applies to EPS/FPM licensing headroom.*

Cross-references: DEH Part 27 §1 and §1.2 (AQL syntax and the `QIDNAME()` / `LOGSOURCETYPE_NAME()` / custom-property function set this part's §5 queries reuse without re-teaching) and §2 (the Custom Rules Engine objects — Rule, Building Block, Reference Set — an app can deploy or reference, per §6.1); DEH Part 23 §5 (Sigma-first-vs-native authoring, referenced rather than re-argued for an app vendor's own bundled content). Within this book: Part 2 §1.6 (the App Host component role this part's §2 builds out in full); Part 3 (Offenses, Magnitude, Credibility, and Relevance — the object an app-deployed Rule creates); Part 9 (Building Block governance — the inventory an app's own bundled Rules/Building Blocks join per §6.1); Part 10 (Reference Data Collections — the object type a behavioral-analytics app typically reads or writes); Part 11 (Thresholds, Anomaly and Behavioral Rules — the native mechanism a UBA-type app's scoring often builds on); Part 13 (Dashboards, Pulse, and Executive Reporting — the consumption-side view of §4.2's Pulse survey); Part 17 (Licensing, EPS/FPM Management, and Capacity Planning — the capacity discipline §7's Platform Reality callout extends to app resource footprints); Part 22 (Staffing and Running a QRadar Practice — the headcount line §6.3 argues an app-vetting practice needs).

neshbooy

Part 20 — Asset Model, Network Hierarchy, and Vulnerability Data

Why this part exists

[CONCEPT] This book's Part 3 states magnitude as a function of three inputs — severity, relevance, credibility — the same architecture DEH Part 27 §6 already establishes when it describes a QRadar Offense's `magnitude` score as computed "from severity, relevance, and credibility rather than a single fixed severity value." Severity is a value a Rule or Building Block author sets at authoring time (Part 8, Part 9). Credibility is a score the Custom Rules Engine (CRE) derives from how many distinct sources agree an event happened and how reliable the reporting Log Source has historically been. Relevance is different in kind from both: it asks whether the asset actually involved in an event — the specific host at that IP, on that network segment, with those known vulnerabilities — is a host this environment actually cares about. Answering that question is not a property of the Rule that fired or the Log Source that reported it. It is a property of three things QRadar maintains about the network itself, continuously and mostly automatically: the asset database, the Network Hierarchy, and whatever vulnerability data has been imported against a given asset.

This part is about that data, not about magnitude's arithmetic — Part 3 owns the scoring architecture and states plainly that any specific numeric weighting between severity, relevance, and credibility is unpublished and version-variable rather than something this book guesses at. What follows here is the plumbing that feeds Relevance specifically: how QRadar decides what "known" even means for a given IP (§1–§2), how the Network Hierarchy gives that IP a place in your address space that a Rule Test or a dashboard can reason about (§3), how vulnerability data gets attached to an asset and what that attachment can and can't tell you (§4), and the failure mode that gives this part its second half: an asset model nobody re-validates after the infrastructure it describes has already changed (§5–§7). It does not re-teach AQL syntax — DEH Part 27 §1 owns that, and §6 below reuses it rather than re-explaining it — and it does not re-argue the query-language-portability tradeoff DEH Part 23 covers generically; where this part's asset- and vulnerability-aware search patterns touch that tradeoff, it says so and points back rather than re-deriving it.

1. What "the asset model" actually is

[CONCEPT] QRadar maintains a persistent record — an Asset Profile — for every IP address it has observed with enough corroborating detail to be worth tracking, built up over time from multiple, independent sources rather than from any single authoritative feed:

- **Passive discovery from events and flows.** Every event and flow the CRE evaluates in real time (DEH Part 27 §1.1's `events` / `flows` tables) carries a source and/or destination IP, and QRadar updates the corresponding Asset Profile's observed attributes — open ports and services inferred from flow data, an operating-system guess where a DSM's parsed fields support one, a hostname where DNS or an identity event supplied one — as a background process, not as a Rule Response an engineer configures.

- **Identity correlation.** Authentication and identity-provider events (a domain logon, a VPN session, a DHCP lease) let QRadar associate a username, and sometimes a MAC address, with an IP for the window that association was actually true — important because in a DHCP environment the same IP can legitimately belong to a different host by the next lease cycle (§5 returns to exactly this).
- **Vulnerability-scanner import.** A vulnerability assessment feed attaches a list of discovered vulnerabilities to the Asset Profile matching the scanned IP or hostname, covered in full in §4.
- **Manual assignment.** An admin can set fields by hand on a specific Asset Profile — most importantly for this part, an importance/weight value (§3) — for a host the automated sources alone would not otherwise mark as significant.

The practical consequence for everything downstream: an Asset Profile is a *merged, continuously updated inference*, not a static inventory record someone typed in once. That is a strength when the underlying network is stable — the model tracks reality without anyone maintaining it by hand — and it is the exact mechanism that turns into a liability once the network changes faster than the merge logic's assumptions hold, which §5 names directly.

PRODUCT VERSION NOTE Which specific event/flow fields feed automated OS and service fingerprinting, how aggressively QRadar merges a new observation into an existing Asset Profile versus creating a new one, and the exact Admin-console screen name for reviewing and editing a single Asset Profile by hand, have all seen incremental changes across QRadar release cycles. Treat this section's description as the stable shape of the mechanism — multiple independent sources merged into one profile per IP — and verify the exact field list and merge behavior against your own deployed release before writing a runbook that depends on a specific field being populated.

1.1 Asset Profile fields relevant to Relevance, at a glance

[PLATFORM ENGINEER] Table 20.1 supports a narrower question than "what does an Asset Profile contain" — it names only the fields that actually feed a Rule Test's asset-aware conditions or Relevance itself, since the full profile carries more descriptive detail (hostname history, observed ports) that matters for investigation (Part 14) but does not itself move a magnitude score.

Table 20.1 — Asset Profile fields that feed Relevance and asset-aware Rule Tests. Supports deciding which field a stale-data audit (§7) needs to check first for a given Offense.

Field	Populated by	Feeds	Staleness risk
IP / MAC association	Passive discovery, DHCP/identity events	Which Asset Profile an event's source/destination IP resolves to at all	High in DHCP environments — see §5
Importance / weight	Manual admin assignment (§3)	Relevance directly	High — set once at onboarding, rarely revisited
Vulnerability list	Scanner import (§4)	Relevance, and any Rule Test checking asset vulnerability	Moderate to high, tied to scan cadence
Network location	Resolved via Network Hierarchy (§2) against the asset's IP	Relevance (via network weight), local-vs-remote Rule Tests, dashboards grouped by segment	Moderate — tied to Network Hierarchy re-sync after IPAM changes
Observed OS / services	Passive fingerprinting from flow/event data	Investigative context (Part 14); not a direct Relevance input on its own	Low to moderate, self-correcting as new traffic arrives

2. Network Hierarchy: giving an address space a name a Rule can reason about

[PLATFORM ENGINEER] DEH Part 27 §1.1 introduces this concept in one clause, in service of a different point: AQL's `NETWORKNAME()` function "resolve[s] a raw IP into the network-hierarchy segment your admins have named it (Admin > Network Hierarchy), rather than a separately queryable network-hierarchy table." This part takes that clause as its starting fact and builds outward from it, because the Network Hierarchy it references is not just a naming convenience for a search result column — it is the second major input to Relevance, alongside asset weight, and the object that lets a Rule Test express "traffic between two of our own network segments" versus "traffic from the internet."

The Network Hierarchy is an admin-configured tree of named objects, each defined by one or more CIDR ranges — a segment for a data center subnet, a segment for a branch-office VLAN, a segment for guest wifi, nested under broader groupings as the environment's address space calls for. Every IP QRadar observes resolves to exactly one Network Hierarchy leaf (or to "unknown"/remote networks outside anything defined), and that resolution feeds three distinct things:

1. **Display naming** — the same `NETWORKNAME()` resolution DEH Part 27 §1.1 already names for AQL search output and dashboard grouping.
2. **Local-vs-remote classification** — Rule Tests and Building Blocks (Part 8, Part 9) that distinguish "an internal host talking to another internal host" from "an internal host talking to something outside every defined segment" depend entirely on the Network Hierarchy being complete; an address range nobody has defined resolves as remote/unknown by default, with real correlation consequences (§5).
3. **Network weight, contributing to Relevance** — a Network Hierarchy object can itself carry an importance value, separate from any individual Asset Profile's weight, so that traffic touching a segment defined as high-value (a data center subnet holding domain controllers, for instance) can contribute to Relevance even for an IP whose own Asset Profile has never been individually reviewed.

PRODUCT VERSION NOTE *The exact console path (`Admin > Network Hierarchy`, per DEH Part 27 §1.1's own citation), the specific weight/importance scale a Network Hierarchy object supports, and whether that weight is expressed as a numeric slider, a low/medium/high category, or something else entirely, are UI specifics this book cannot verify against a live console — no lab access, per this book's stated evidence constraint (STYLE-GUIDE.md §0/§9). Verify the current editor and scale against your own deployment before treating a specific number or control as a literal on-screen fact.*

2.1 The hierarchy is a manually maintained object, not a discovery result

[PLATFORM ENGINEER] This is the detail easiest to lose sight of once the hierarchy has existed for a few years: unlike an Asset Profile's passive enrichment (§1), the Network Hierarchy itself does not automatically discover new subnets from observed traffic. A new VLAN, a new cloud VPC's CIDR block, or a re-numbered branch office is invisible to the Network Hierarchy — and therefore to local-vs-remote Rule Tests and to network-weight-driven Relevance — until an admin adds it by hand. An environment that provisions infrastructure faster than someone remembers to update `Admin > Network Hierarchy` accumulates a growing blind spot in exactly the two things §2's list above says the hierarchy is for.

Blind Spot *Traffic to or from an address range that has never been added to the Network Hierarchy resolves as an unnamed remote/unknown segment for every purpose the hierarchy serves — `NETWORKNAME()` returns nothing useful for it, a Rule Test built on "traffic between two internal segments" does not recognize it as internal, and any network-weight contribution to Relevance for that range is simply absent, not merely low. A newly provisioned data-center subnet holding a fresh batch of production database hosts looks, to every Network-Hierarchy-dependent Rule and to Relevance itself, exactly like traffic from an unrecognized external network — until someone notices and adds the range. The Offense volume this produces is not obviously wrong in either direction: it can under-fire (a local-only Rule Test never matches traffic QRadar doesn't recognize as local) or it can over-fire (a "communication with an unknown/external network" Rule Test fires routinely against a segment that is, in fact, entirely internal), and neither failure mode announces itself as a Network Hierarchy gap in the Offense record — it just looks like a Rule behaving oddly against a specific IP range.*

3. Asset importance and network weight: the two hand-set inputs to Relevance

[PLATFORM ENGINEER] Both mechanisms named above — an Asset Profile's manually assigned importance (§1) and a Network Hierarchy object's manually assigned weight (§2) — are, as far as this book can verify without a live deployment, admin-set values rather than anything QRadar derives on its own. That is the operationally important fact, independent of the exact numeric scale either control

uses: **Relevance's two most direct levers are decisions a human made once, at some point, about which hosts and which segments matter**, not a live measurement of anything. A domain controller's Asset Profile carries whatever importance value someone assigned it during onboarding; a guest-wifi segment carries whatever network weight someone assigned the Network Hierarchy object representing it. Neither value updates itself when the domain controller is decommissioned or the guest-wifi segment is repurposed as a contractor VLAN with access to production systems.

This is also where this part's Part 3 dependency is tightest: Part 3 states the three magnitude inputs as architecture and explicitly declines to assert the numeric weighting between them, consistent with DEH Part 27 §6's treatment of magnitude as a computed score without a published formula. This part inherits that same discipline rather than reverse-engineering a weighting scheme from observed Offense behavior — the honest claim available here is that asset importance and network weight are real, configurable inputs to Relevance, not what fraction of a magnitude point either one is worth.

PRODUCT VERSION NOTE *Whether asset importance and Network Hierarchy weight combine additively, whether one can override the other, and whether a given release exposes both as numeric sliders on the same 0–10-style scale or as differently-shaped controls, is exactly the kind of internal weighting detail Part 3 flags as unpublished and version-variable. This part does not guess at that mechanism; if your own investigation of a live deployment's behavior suggests a specific combination rule, treat it as an observation about your version, not a platform-wide fact to build automation against.*

3.1 A worked failure: the weight that outlived the server

[PLATFORM ENGINEER]

Rule Autopsy The configuration: An Asset Profile for 10.4.12.31, assigned the highest importance value the console offers, set three years ago when that address hosted the organization's primary domain controller. **Why it shipped:** At onboarding, the SOC team correctly identified the domain controller as a high-value target and weighted its Asset Profile accordingly — a reasonable, deliberate decision, not an oversight, and exactly the kind of manual input §3 says Relevance depends on. **How it failed:** The domain controller was migrated to a new address during a data-center consolidation eighteen months later. The old address, 10.4.12.31, was reassigned by DHCP to a rotating pool of contractor laptops. Nobody's decommissioning checklist included revisiting that address's Asset Profile importance, because the address itself was never the thing anyone thought of as "the domain controller" — the hostname and the service were. Every Offense touching 10.4.12.31 from that point forward inherited a Relevance boost intended for a domain controller and applied instead to whichever laptop happened to hold that lease that week, ranking ordinary contractor-laptop noise artificially high in the queue for eighteen months before a queue-fatigue review (Part 15) traced the pattern back to the stale weight. **The fix:** Asset importance review became a required line item on the infrastructure-decommissioning and IP-reassignment checklist, not just the vulnerability-scan and Reference Set cleanup items Part 10 and Part 18 already name for a retired host — and the Asset Profile for a reassigned high-importance address is explicitly reset (not left at its prior value "just in case") the moment the reassignment is known, with a new importance value assigned deliberately once the address's new role is understood, rather than inherited from whatever address it happens to be.

4. Vulnerability data: what "this asset is vulnerable" actually means to QRadar

[PLATFORM ENGINEER] The third input this part covers is vulnerability data attached to an Asset Profile — commonly discussed under the umbrella term VIS (Vulnerability Information Service) in QRadar's own documentation and community material, though this book cannot verify the current branding, licensing scope, or exact ingestion mechanism against a live console (see the PRODUCT VERSION NOTE immediately below). Conceptually, the mechanism is stable regardless of its current name: a vulnerability-assessment source — historically a native scanner add-on, and more commonly today a third-party scanner (Qualys, Tenable/Nessus, Rapid7, or others) feeding results in through a supported integration — reports a list of discovered vulnerabilities per scanned IP or hostname, and QRadar attaches that list to the matching Asset Profile.

That attachment feeds two genuinely different things, and conflating them is a real risk this section exists to head off:

1. **Relevance.** An asset with known, unpatched vulnerabilities is a more relevant target for an Offense involving it than an asset with a clean scan history, all else equal — the same intuition behind asset importance in §3, but derived from scan data instead of a manual assignment.
2. **A dedicated Rule Test category.** Independent of Relevance, a Rule Test checking whether the source or destination asset in a matched event carries a specific known vulnerability (or any

vulnerability at all) lets a Rule fire — or a Building Block narrow its candidate match — specifically on exploit-relevant context, e.g., "flag this exposure attempt only against a host we know is vulnerable to it," the same intent as an EDR-reported CVE match feeding a correlation decision.

PRODUCT VERSION NOTE *The current name, packaging, and licensing model for vulnerability-scanner integration in QRadar — whether it ships as a built-in capability, a separately licensed add-on, or purely through third-party scanner integrations configured against the platform — has changed more than once across QRadar's release history and business-unit ownership changes, exactly the category of packaging claim STYLE-GUIDE.md §11 flags as aging fastest. This part describes the mechanism (scan results attach to an Asset Profile and feed Relevance and a dedicated Rule Test category) as the stable claim; do not treat any specific product name, licensing tier, or supported-scanner list in this section as current without verifying it against your own deployment's admin console and current IBM documentation.*

PRODUCT VERSION NOTE *The exact Rule Wizard wording for a vulnerability-aware Rule Test (this part's own paraphrase above — "when the source or destination asset... carries a specific known vulnerability" — is illustrative phrasing, not a transcript) has shifted in wording and grouping across releases, the same drift risk DEH Part 27 §2.1 already flags for the Rule Test catalog generally. Verify the current category name and exact condition options in your own deployment's Rule Wizard before quoting a literal string from this section.*

4.1 The coverage gap a clean vulnerability record actually means

[THREAT HUNTER]

Blind Spot *A vulnerability-aware Rule Test or a Relevance boost derived from scan data can only see what a scanner has actually scanned. An Asset Profile with no vulnerability entries reads, on the surface, identically for two very different hosts: one genuinely patched and clean, and one simply never reached by a scan — an ephemeral container that existed for six minutes between two scan windows, a segment excluded from the scanner's scope by a firewall rule nobody revisited, a newly provisioned cloud instance that spun up after the last scheduled scan completed. Neither QRadar's console nor a Rule Test built against vulnerability data distinguishes "verified clean" from "never checked," and an analyst or a Rule Response treating an empty vulnerability list as reassurance is trusting a scan-coverage gap as if it were a security finding. Before treating a host's clean vulnerability record as evidence of anything, confirm the scanner's scope and last-scan timestamp actually covered that specific host recently enough to mean something — the same discipline this book's Part 13 §4 Blind Spot already applies to a clean-looking dashboard tile, applied here to a clean-looking Asset Profile field instead.*

[THREAT HUNTER]

A hunt built around this gap starts from the same AQL fundamentals DEH Part 27 §1 teaches, extended with whatever asset/vulnerability-lookup fields your deployment exposes in a search's context — a search surface DEH Part 23 §3's general point about backend-specific enrichment applies to directly: an AQL query that joins event data against asset and vulnerability context this way is doing something a KQL or SPL query against the same event stream would express with a different join or lookup construct entirely, not a portable pattern across backends.

```
-- QRadar AQL, not standard SQL – this query relies on QRadar's own asset-context
-- resolution for a matched IP, which has no direct KQL/SPL equivalent syntax (DEH
-- Part 23 §3's backend-specific-enrichment point applies here). NETWORKNAME() is
-- confirmed AQL syntax per DEH Part 27 §1.1; any additional asset/vulnerability
-- lookup field name is illustrative only – this book has not verified an exact
-- function/field name against a live console (see the PRODUCT VERSION NOTE above)
-- and you should confirm the current equivalent in your own Log Activity > Advanced
-- Search field list before reusing this shape.
SELECT sourceip, NETWORKNAME(sourceip) AS "Source Network",
       destinationip, QIDNAME(qid) AS "Event Name"
FROM events
WHERE NETWORKNAME(sourceip) = 'DMZ-Web-Tier'
LAST 7 DAYS
```

Hunter's Note This query's actual hunting value is not the rows it returns from `events` — it's the cross-check you run manually afterward against the asset-management console for the same `sourceip` list: does each address's Asset Profile show a recent vulnerability scan at all? A source-network filter like `'DMZ-Web-Tier'` narrows you to a segment; it says nothing about whether that segment's hosts were actually in scope for the last completed scan. Pair every asset- or vulnerability-aware hunt with a direct look at scan coverage and timestamps, not just the Rule Test or search result that assumes the data behind it is current.

5. The staleness problem, named directly

[PLATFORM ENGINEER]

Every mechanism in §1–§4 shares one structural weakness: each is accurate at the moment it was last updated and silently stops being accurate the moment the infrastructure it describes changes without a corresponding update. Nothing about QRadar's own architecture forces that update to happen. Four concrete change events drive this in practice:

- **DHCP churn.** The domain-controller example in §3.1 is the general case, not an edge case: any environment where IP addresses are reassigned faster than Asset Profile importance is reviewed will eventually apply a stale weight to the wrong host, in either direction — over-weighting a now-unimportant address, or under-weighting a newly critical one that inherited a low-importance address's leftover profile.
- **IPAM and Network Hierarchy drift.** §2.1's Blind Spot is the direct consequence of new address space outrunning a manually maintained hierarchy — a growth pattern, not a one-time gap, in

any environment provisioning infrastructure on a faster cycle than someone remembers to touch Admin > Network Hierarchy.

- **Decommissioning without cleanup.** A retired host's Asset Profile, Network Hierarchy membership, and any Reference Set entry referencing it by IP or hostname (Part 10's drift-risk framing applies identically here) all persist by default. QRadar does not know a host is gone until something tells it — a scanner no longer finding it, or an admin removing the profile by hand — and until then, that stale profile's weight and vulnerability history keep influencing Relevance for whatever now holds that address.
- **Ephemeral and cloud-native infrastructure.** A container or autoscaled instance that lives for minutes to hours defeats the asset model's core assumption — that an IP-to-host mapping is stable enough to be worth persisting and weighting — before any manual review cycle could plausibly catch up to it. §4.1's scan-coverage gap and this staleness problem compound directly in this environment: an ephemeral host is both unlikely to be individually weighted and unlikely to be scanned before it disappears.

Platform Reality *None of these four drivers is a QRadar defect — they are the ordinary cost of a model that trades manual maintenance for automated correlation. A live vulnerability scan feed, a maintained Network Hierarchy, and a periodically reviewed set of asset-importance values are inputs a team has to keep current the same way a firewall rule base or a Reference Set (Part 10) needs periodic review, not a one-time setup task. The failure mode in practice is not that any single mechanism breaks — it is that all four keep running exactly as designed, producing a magnitude score that reads as confident and specific right up until someone asks how recently the asset model behind it was actually checked against the infrastructure it claims to describe.*

6. Analyst impact: trusting Relevance without trusting it blindly

[ANALYST] Part 14's triage workflow already treats magnitude as a queue-ranking signal, not a verdict — this part gives that caution a specific, checkable form for any Offense where Relevance looks like it's doing unusual work. Before treating a surprisingly high or surprisingly low magnitude as a signal about the underlying event, an analyst working a queue that draws on the asset model above has a fast, three-question check available:

1. **Does the involved asset's importance value match what this host actually is right now** — not what its Asset Profile description says it was set up as, per §3.1's worked failure?
2. **Is the involved IP's Network Hierarchy resolution current** — does it land in a named, correctly weighted segment, or does it silently fall out to "unknown/remote" the way §2.1's Blind Spot describes?
3. **If vulnerability data is contributing to this Offense's Relevance or matched a vulnerability-aware Rule Test, when was this asset actually last scanned** — a recent, real result, or a stale entry from a scan that predates a rebuild or a redeploy, per §4's coverage-gap warning?

None of these three checks require re-opening this part's whole architecture during a live triage — each is a single lookup against the Asset Profile or Network Hierarchy entry already linked from the Offense record. The discipline worth building into a team's triage habit is treating "this Offense's magnitude looks wrong for reasons that don't track back to the Rule logic itself" as a prompt to check the asset model before assuming the Rule (Part 8, Part 9) is the thing that needs tuning — a Noisy Offense Trap that traces back to a stale asset weight has a different fix than one that traces back to an actual Rule Test problem, and tuning the Rule instead of correcting the asset data leaves the underlying staleness in place to distort the next Offense too.

7. Keeping the model honest: a re-validation cadence

[SOC MANAGEMENT] The technical fix for every failure mode in §5 is straightforward in isolation — update the Asset Profile, add the missing Network Hierarchy range, re-scan the segment. The organizational problem is that none of those fixes has an obvious owner by default. Network Hierarchy and IPAM changes are usually a network-engineering decision; vulnerability scan scope and cadence usually belong to a vulnerability-management or infrastructure-security function; asset decommissioning checklists belong to whoever runs that lifecycle process — and a SOC team consuming Relevance as a triage input often has write access to none of the three, while depending on all three being current.

SOC Management View *Treat asset-model currency as a named cross-team dependency, not a QRadar administration task the SOC can fully own or fully delegate. A minimum viable cadence: a Network Hierarchy review triggered by IPAM change tickets rather than a fixed calendar (since address-space changes, not the calendar, are what actually invalidates the hierarchy); a quarterly sample audit of high-importance Asset Profiles against current infrastructure inventory, checking specifically for the §3.1 failure pattern of a weight that outlived its host; and a documented answer to "what is our vulnerability scan's actual coverage and cadence for the segments feeding Relevance" that the SOC can point to rather than assume. None of this requires QRadar functionality beyond what §1–§4 already describe — the gap is process ownership, not platform capability, and naming an owner for each of the three inputs is cheaper than discovering the gap the way §3.1's worked failure did, eighteen months and a queue-fatigue review later.*

Validation Test Setup: A test host provisioned on a network range recently added (or about to be added) to the Network Hierarchy, with a known, deliberately assigned Asset Profile importance value distinct from that segment's default. **Action:** Generate a single event or flow involving the test host that would match an existing Rule contributing to an Offense (any low-noise Rule already in production is sufficient — the specific Rule doesn't matter for this test), then open the resulting Offense. **Expected result:** The Offense's linked Asset Profile reflects the test host's actual current importance value and correct Network Hierarchy segment, not a stale or default value — confirming the asset-model pipeline in §1–§3 is resolving new or recently-changed infrastructure correctly, rather than merely confirming that magnitude renders a number at all. Run this test after any Network Hierarchy change or bulk asset-importance review, the same way DEH Part 27 §3.3's own Detection Test validates a Rule after a DSM change, applied here to the asset model instead of the Rule logic.

flowchart TB

```

A["Events / flows observed\nin real time"] -->|"passive discovery"| P["Asset Profile\n(per IP)"]
ID["Identity / auth events\n(username, MAC, DHCP lease)"] -->|"correlation"| P
VS["Vulnerability scanner\nimport (4)"] -->|"attaches CVE list"| P
MAN["Manual admin\nimportance assignment (3)"] -->|"asset weight"| P
NH["Network Hierarchy\n(2) – CIDR tree,\nper-segment weight"] -->|"resolves IP to\nnamed segment"| P
P --> REL["Relevance input"]
SEV["Severity\n(Rule/BB-authored,\nPart 8/9)"] --> MAG
CRED["Credibility\n(CRE-derived,\nPart 3)"] --> MAG
REL --> MAG["Magnitude\n(Part 3 / DEH Part 27 §6)"]
MAG --> OFF["Offense ranking\nin the queue (Part 14)"]
STALE["No automatic re-validation\ntrigger on infra change (5)"] -.->|"silently distorts"| P

```

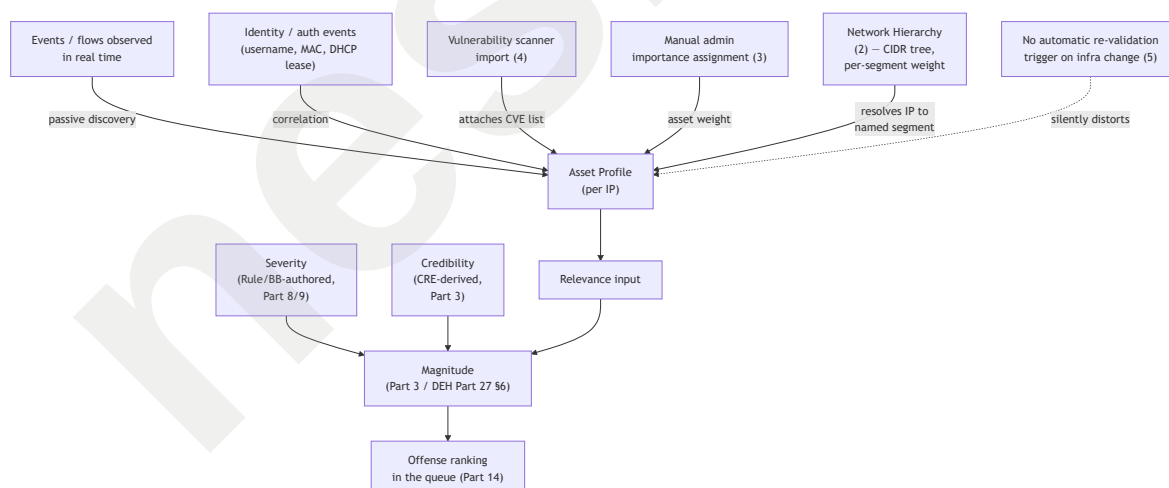


Figure 20.1 (FIG-20-01) — How Relevance's inputs reach magnitude, and where staleness enters unseen. *CONCEPTUAL.* Illustrates the four independent paths (passive discovery, identity correlation, vulnerability import, manual assignment) and the Network Hierarchy's parallel contribution that together populate an Asset Profile feeding Relevance, converging with Severity and Credibility into the magnitude score DEH Part 27 §6 and this book's Part 3 already establish as architecture — and the

dashed path showing that nothing in this diagram forces a re-check when the underlying infrastructure changes, the mechanism §5 names directly. Not a reproduction of any IBM architecture diagram, and not a claim about magnitude's internal weighting formula, which Part 3 states as unpublished/version-variable rather than guessed at here.

Cross-references: DEH Part 27 §1.1 (`NETWORKNAME()` and the Ariel `events / flows` model this part's AQL example builds on), §2.1 (Rule Test catalog wording drift, the same risk this part flags for a vulnerability-aware Rule Test), §6 (Offense magnitude as severity/relevance/credibility, the architecture this part's Relevance inputs feed); DEH Part 23 §3 (backend-specific query enrichment, the general point this part's asset/vulnerability-lookup AQL example instantiates); this book's Part 3 (magnitude, credibility, relevance — the scoring architecture this part supplies data for), Part 8 and Part 9 (Rule/Building Block authoring — where Severity is set, and where a vulnerability-aware Rule Test is built), Part 10 (Reference Data drift risk, the same pattern this part names for a decommissioned host's stale profile), Part 13 §4 (the coverage-trap discipline this part's §4.1 Blind Spot applies to a clean vulnerability record), Part 14 (Offense triage workflow this part's §6 extends), Part 15 (tuning workflow — distinguishing a Rule problem from an asset-model problem), Part 18 (troubleshooting — decommissioning cleanup checklist referenced in §5).

Part 21 — Multi-Tenancy, RBAC, and MSSP Operations

Why this part exists

[CONCEPT] DEH Part 27's closing paragraph names a review-cost consequence of QRadar's own object model: reviewing "one detection" here means reviewing a Building Block, a Reference Set's maintained membership, and a Rule's chained logic as three separately-changeable artifacts, not one query file. That paragraph frames the problem for a single-tenant shop — one deployment, one customer, one review burden. This part covers what happens to that same three-object review burden the moment one QRadar Console serves more than one customer, business unit, or otherwise legally- or operationally-separated audience from a shared pool of infrastructure: an MSSP running twelve client environments off one deployment, or a large enterprise segregating a regulated subsidiary's data from its parent's SOC.

QRadar gives you three constructs to make that separation real rather than aspirational — Domains, Security Profiles and User Roles, and Tenants — and this part's job is to explain how each one actually works, how they interact with the Rules, Building Blocks, Reference Sets, and Offenses every earlier Section D and Section B part already built, and, most importantly, where that interaction is incomplete: which objects genuinely isolate per customer and which ones remain shared platform-wide whether you intended that or not. This part does not re-teach AQL (DEH Part 27 §1 owns that, referenced below only where a domain-scoped search needs it) and does not re-argue Sigma-first-vs-native authoring (DEH Part 23 §5). It assumes this book's own Part 3 (Offense scoring), Part 9 (Building Block governance), and Part 10 (Reference Data types) and builds the MSSP-specific version of each.

1. Two constructs, two different jobs, one shared name problem

[CONCEPT] "Multi-tenancy" in QRadar conversation covers two genuinely different mechanisms that are easy to conflate because they're often deployed together and because QRadar's own vocabulary for the second one has shifted:

- **Domain** — a data-tagging construct. QRadar stamps every incoming event and flow with a Domain identifier, derived from which Log Source, Log Source Group, vulnerability scanner, or network-CIDR association produced it, and that Domain identifier travels with the record through Ariel and into any Offense the record contributes to. A Domain is what makes "this event belongs to Customer A, not Customer B" a fact QRadar can enforce downstream, rather than a convention analysts have to remember on their own.
- **Tenant** — a packaging and licensing construct built on top of Domain isolation, used specifically for MSSP-style operation: a Tenant groups one or more Domains under a customer-facing identity with its own licensed EPS/Flows Per Minute (FPM) allocation (Part 17) and its own administrative boundary, so an MSSP can report per-customer consumption and, in principle, let a customer's own delegated administrator manage a slice of configuration without touching another customer's.

Security Profiles and User Roles are the RBAC layer that actually enforces who sees what and who can do what — covered in full in §3 — and they are the mechanism that turns a Domain tag into an actual visibility boundary for a specific analyst. A Domain with no Security Profile built against it is metadata sitting on an event, not an access control.

PRODUCT VERSION NOTE *Whether "Tenant" appears in your own console as a first-class object distinct from Domain, or whether your deployment's licensing tier exposes Tenant Management at all, has varied across QRadar release cycles and packaging changes — this book's style guide (§11) flags commercial-packaging claims as exactly the category that ages fastest and least gracefully, and this is one of them. Some deployments run genuine multi-customer MSSP segregation using Domains and Security Profiles alone, with no separately licensed Tenant construct in play at all. Verify which of the two constructs your specific deployment actually has enabled before designing an MSSP operating model around a Tenant Management capability you haven't confirmed is present.*

2. Domains: how data gets tagged at the source

[PLATFORM ENGINEER] A Domain is defined by a set of inclusion rules an administrator configures — typically some combination of specific Log Sources, Log Source Groups, vulnerability scanners, and network-CIDR ranges assigned to it. When an event or flow arrives, QRadar evaluates it against every configured Domain's inclusion rules and stamps the matching Domain's identifier onto the normalized record, alongside the QID (QRadar's internal event-taxonomy identifier — Part 4) and custom properties the DSM already extracted. An event that matches no configured Domain lands in the deployment's default Domain — effectively "unscoped," which matters the moment §3's Security Profiles start restricting visibility by Domain, because an analyst restricted to Customer A's Domain will not see an event QRadar filed under the default Domain even if that event actually originated from Customer A's infrastructure through a Log Source nobody remembered to assign.

This tagging happens once, at ingestion, on the Event/Flow Processor pipeline Part 2 already describes — it is not something a Rule, a saved search, or an analyst chooses per-query. Everything downstream (Ariel storage, Custom Rules Engine — universally abbreviated CRE — evaluation, Offense indexing) inherits whatever Domain the record was stamped with at that point, which is exactly why getting the inclusion-rule assignment right during onboarding matters as much as getting the DSM/QID mapping right (Part 4 §1's onboarding checklist gains a Domain-assignment line item the moment more than one Domain exists in a deployment).

PRODUCT VERSION NOTE *The exact console location for configuring Domains (an Admin-tab section, historically named along the lines of "Domain Management"), the precise set of inclusion-rule types available (Log Source, Log Source Group, scanner, CIDR, and whether additional types have been added in later releases), and whether a given release allows overlapping Domain definitions or enforces a strict partition, are all specifics this book cannot confirm against a live console (STYLE-GUIDE.md §0/§9 — no lab access). Verify the current inclusion-rule catalog and overlap behavior in your own deployment before assuming this section's description is a literal, current transcript.*

2.1 Domain assignment and Offense indexing

[PLATFORM ENGINEER] Because an Offense is built from contributing events (Part 3), and every contributing event carries a Domain tag, an Offense itself is stamped with the Domain of the events that created it. This is the mechanical basis for §3's per-customer Offense queue segregation: an analyst whose Security Profile restricts them to Domain `Customer-A` sees only Offenses whose contributing events were tagged `Customer-A`, in the same Offenses tab every other analyst uses, filtered transparently rather than through a separate console. A single Rule firing against events from two different Domains produces two separate Offenses, one per Domain, rather than one Offense straddling both — Domain segregation applies at the Offense-indexing layer, not just at the display layer, which is the detail that makes it a real isolation boundary rather than a filtered view over commingled data.

3. Security Profiles and User Roles: the two independent axes of RBAC

[PLATFORM ENGINEER] QRadar splits access control into two independent decisions, and conflating them is the single most common RBAC design mistake in a multi-tenant deployment:

- **User Role** answers "what can this person do" — which tabs they can open (Offenses, Log Activity, Admin), which administrative capabilities they hold (can they edit a Rule, deploy a Building Block change, manage other users), and which apps they can access. A User Role has no awareness of Domains or customer boundaries at all; it is a pure capability grant.
- **Security Profile** answers "what data can this person see" — which Networks (from the Network Hierarchy, Part 20), which Log Source Groups, and which Domains are visible to a user holding this profile. A Security Profile has no awareness of what a user is allowed to *do*; it is a pure visibility grant.

A user's actual, effective access is the combination of both, assigned together when the account is created: a Tier-1 analyst User Role (view Offenses, add notes, cannot edit Rules) paired with a Security Profile scoped to `Customer-A`'s Domain only produces an account that can triage Customer A's Offenses and nothing else's — the same User Role paired with an unrestricted Security Profile produces an account with identical capabilities but visibility across every customer in the deployment. Table 21.1 makes that intersection concrete.

Table 21.1 — RBAC and tenancy objects at a glance. Supports deciding which construct actually needs to change when an access request comes in — "give this analyst access to Customer B" is a Security Profile change; "let this analyst edit Rules" is a User Role change; conflating the two produces either an over-broad grant or a support ticket that goes to the wrong admin.

Object	Answers	Set by	Typical grain	Version note
Domain	Which customer/business unit does this data belong to	Log Source, Log Source Group, scanner, or CIDR inclusion rules, at ingestion	Deployment-wide, configured once per data source	Inclusion-rule catalog and console location have shifted across releases — verify current set
Security Profile	What data can this user see (Networks, Log Source Groups, Domains)	Admin, assigned per user or user group	Per user/group	Whether a Security Profile can restrict to multiple named Domains individually, or only to broader groupings, is version-dependent — verify before designing a fine-grained profile
User Role	What actions can this user perform	Admin, assigned per user or user group	Per user/group	Exact capability list and default role templates have expanded across releases
Tenant	Commercial/licensing grouping of one or more Domains, with its own EPS/FPM allocation	Admin (where the Tenant Management capability is licensed and enabled — §1)	Per customer, MSSP context	Availability and packaging tier have varied; do not assume presence — see §1's PRODUCT VERSION NOTE

PRODUCT VERSION NOTE *Whether Security Profile visibility restrictions and User Role capability grants are exposed as fully independent assignment steps in your specific release's user-management UI, or bundled into a combined "role template" workflow that still resolves to the same two-axis model underneath, is a console-workflow detail this book will not assert as a literal transcript. The two-axis architecture (visibility separate from capability) is the stable claim here; the exact screen layout for assigning both is not.*

[SOC MANAGEMENT] The practical governance discipline this two-axis model demands: every access request should name both axes explicitly — "this person needs Tier-1 Analyst capability, scoped to Customer A and Customer C's Domains" — rather than "give them access like Priya has," which silently copies whatever combination of Role and Security Profile Priya happens to hold today, including any Domain scope that was never re-reviewed since Priya's own onboarding. A copied-access request is how one analyst's originally-correct, narrowly-scoped account becomes the template for a dozen accounts with visibility nobody individually approved.

4. What actually isolates per Domain — and what doesn't

[RULE ENGINEER] This is the section that matters most for anyone who has already built Rules, Building Blocks, and Reference Sets under this book's Section D and now needs to reason about what happens

to those same objects once a second customer's Domain enters the deployment. The honest answer is that QRadar's three correlation objects do not all inherit Domain awareness the same way, and assuming they do is the most consequential mistake an MSSP-facing rule engineer can make.

- **Offenses are Domain-scoped by construction** (§2.1) — this isolation is real and automatic, inherited from the events that built the Offense.
- **Rules and Building Blocks are shared, deployment-wide objects by default.** A Rule you build lives in one catalog, evaluated against every incoming event regardless of which Domain tagged it, unless you deliberately add a Rule Test that restricts evaluation to specific Domain(s) — a scoping condition, not a default behavior. `R: Suspicious LSASS Access – Unapproved Process` (DEH Part 27 §3.3), built once, evaluates against Customer A's and Customer B's Sysmon telemetry identically unless someone explicitly narrows it. For most detections this is exactly what you want — one well-tuned Rule, maintained once, protecting every customer identically. It is also exactly the design decision that makes an allowlist change made *for* one customer a change that silently affects every other customer sharing that Rule, which brings us to the object that makes this concrete.
- **Reference Sets, Maps, and Tables are global membership stores, not Domain-partitioned data.** `RS-Allowlisted-LSASS-Tools` (Part 10) has exactly one membership list across the entire deployment. Adding Customer A's legitimate backup-agent path to that Reference Set, because Customer A's environment happens to run a backup tool that trips `BB:LSASS-Access-Candidate`, makes that same path invisible to `R: Suspicious LSASS Access – Unapproved Process` for Customer B too — even though Customer B may never have deployed that backup agent, and even though Customer B's own analysts and administrators have no visibility into `RS-Allowlisted-LSASS-Tools`'s membership at all if their Security Profile scopes them away from the Admin tab. This is not a bug in any one Reference Set; it is the architecture's default behavior, and §5 below is what happens when nobody accounts for it.

PRODUCT VERSION NOTE *Whether a given QRadar release offers any native mechanism to scope Reference Set membership per Domain — a Domain-qualified reference-data key, or an equivalent partitioning feature — rather than treating reference data as a single global namespace, is not a claim this book makes either way; it has not been reliably verifiable against current IBM documentation at the time of writing (see this book's front-matter constraint). If your deployment has such a mechanism, it changes §5's risk materially — confirm its presence and actually use it before assuming the global-namespace behavior described here is your only option.*

Figure 21.1 lays out the resulting asymmetry: one clean isolation boundary (Offenses, via Domain tagging) sitting on top of one shared authoring layer (Rules, Building Blocks, Reference Sets) that has no equivalent boundary unless someone builds one deliberately.

flowchart TB

```

subgraph Ingest["Ingestion – tagged per Domain (2)"]
    LSA["Log Sources: Customer A"] -->|"Domain = Customer-A"| EVA["Tagged events"]
    LSB["Log Sources: Customer B"] -->|"Domain = Customer-B"| EVB["Tagged events"]
end

subgraph Shared["Shared authoring catalog – NOT Domain-partitioned by default (4)"]
    BB["BB:LSASS-Access-Candidate\n(one object, all Domains)"]
    RS["RS-Allowlisted-LSASS-Tools\n(one global membership list)"]
    R["R: Suspicious LSASS Access –\nUnapproved Process\n(one object, all Domains)"]
    BB --> R
    RS -->|"membership check,\nglobal namespace"| R
end

EVA --> BB
EVB --> BB
R -->|"Offense stamped\nDomain = Customer-A"| OA["Customer A's\nOffense queue"]
R -->|"Offense stamped\nDomain = Customer-B"| OB["Customer B's\nOffense queue"]

SPA["Security Profile:\nSP-Customer-A-Analysts"] -->|"visibility scoped to\nDomain Customer-A\nonly"| OA
SPB["Security Profile:\nSP-Customer-B-Analysts"] -->|"visibility scoped to\nDomain Customer-B\nonly"| OB

```

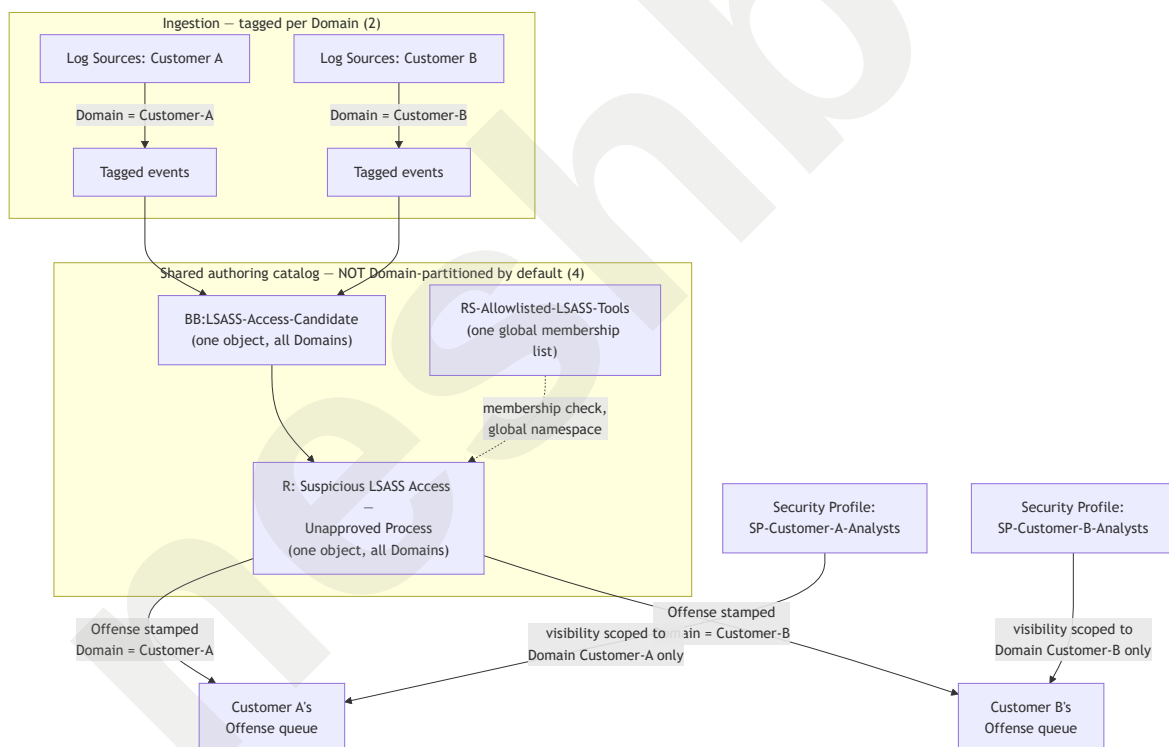


Figure 21.1 (FIG-21-01) — Domain-scoped Offense visibility sitting on top of a shared, un-partitioned correlation catalog. *CONCEPTUAL*. Illustrates the structural asymmetry this section argues from prose: Offense visibility genuinely separates Customer A from Customer B through Domain tagging and Security Profile scoping, while the Building Block, Reference Set, and Rule that produced those Offenses remain single, shared, deployment-wide objects with no equivalent partition unless an administrator deliberately adds Domain-scoping Rule Tests or an equivalent control. Not a reproduction

of any IBM architecture diagram, and not a claim about a specific release's reference-data partitioning capability (see the PRODUCT VERSION NOTE above).

5. A worked failure: the allowlist that crossed a customer boundary

[RULE ENGINEER] §4's global-namespace behavior stops being an abstract risk the first time a real tuning change, made correctly for one customer, reaches a Rule shared with another. The following walks that failure end to end.

Rule Autopsy *The rule:* `R: Suspicious LSASS Access – Unapproved Process`, chained to `BB:LSASS-Access-Candidate` and `RS-ALLOWListed-LSASS-Tools`, deployed once and evaluated across both Domain `Customer-A` and Domain `Customer-B` in a shared MSSP console — exactly DEH Part 27 §3.3's worked example, now carrying two customers' worth of traffic instead of one. **Why it shipped:** Customer A onboarded a legitimate backup agent that legitimately opens a handle to `lsass.exe` with memory-read rights as part of its own credential-vaulting feature. Customer A's assigned analyst, working a real Noisy Offense Trap (Part 15's tuning-intake process, correctly followed for a single-tenant deployment) added the backup agent's binary path to `RS-ALLOWListed-LSASS-Tools` and closed the ticket. Every step taken was the right tuning action for a single-tenant shop. **How it failed:** `RS-ALLOWListed-LSASS-Tools` has no Domain qualifier (§4) — the entry Customer A's analyst added suppressed `BB:LSASS-Access-Candidate` matches for that exact binary path across every Domain in the deployment, including Customer B, who does not run that backup product at all but happens to share a directory-naming convention with a subset of Customer A's legitimate paths through nothing more than coincidence. Months later, an actual credential-dumping tool renamed to match one of the allowlisted paths on a Customer B host produced zero Offense — not because the Rule failed to evaluate, but because the allowlist entry a different customer's analyst added, for a reason entirely unrelated to Customer B, silently suppressed it. No error, no warning, no cross-tenant audit trail pointed at the cause; the first sign of the problem was a post-incident review asking why a known-bad tool never fired anything. **The fix:** Treat any shared Reference Set consulted by a Rule that evaluates across more than one Domain as a cross-tenant-risk object requiring its own governance tier beyond Part 9's ordinary Building Block/Reference Set change process: either partition the allowlist logic explicitly — a separate, Domain-suffixed Reference Set per customer (`RS-ALLOWListed-LSASS-Tools-CustomerA`, `RS-ALLOWListed-LSASS-Tools-CustomerB`) checked by Domain-scoped Rule Tests, at the cost of one more object per customer to maintain — or, where the underlying tool and path genuinely are common across every customer's environment, keep one shared Reference Set but require any addition to be reviewed against every customer's threat model, not just the requesting customer's, before it ships. Either fix is real work; the failure mode this Rule Autopsy documents is what happens when neither is done and a single-tenant tuning habit is applied unchanged to a multi-tenant Rule.

Blind Spot *Nothing in QRadar's own architecture stops a shared Reference Set, Building Block, or Rule from spanning every Domain in a deployment, and nothing surfaces a warning when an edit intended for one customer's tuning need is about to affect another's detection surface — the same "no error, no exception, quietly matches nothing" failure shape DEH Part 27 §3.1 documents for a broken DSM/QID mapping, recurring here one layer up, across a tenant boundary instead of within one deployment. An MSSP that assumes Domain tagging alone provides tenant isolation for its correlation logic — not just its Offense queues — has a detection gap it cannot see from inside QRadar's own console, discoverable only by the kind of cross-customer audit §6 below has to make a named, recurring task rather than an incident-driven afterthought.*

6. The MSSP-specific multiplication of the three-object review burden

[SOC MANAGEMENT] DEH Part 27's closing paragraph frames the base cost: reviewing one QRadar detection means reviewing a Building Block, a Reference Set's maintained membership, and a Rule's chained logic as three separately-changeable artifacts. Part 9 §5 of this book already extends that cost across a **BB:** library's own dependency graph for a single tenant. An MSSP running N customer Domains off one shared correlation catalog inherits both costs multiplied by a factor that has nothing to do with the number of customers and everything to do with how many of them share a given object:

- A Rule, Building Block, or Reference Set used by only one customer's Domain-scoped Rule carries that customer's review cost alone — no different from a single-tenant shop.
- A Rule, Building Block, or Reference Set shared across multiple customers' Domains — the common case for anything not deliberately partitioned, per §4 — carries a review cost proportional to every customer whose detection surface that object touches, exactly as §5's Rule Autopsy demonstrates for one allowlist entry affecting two customers who never coordinated with each other.

SOC Management View Budget a named review step, run before any change to a shared Reference Set, Building Block, or un-scoped Rule ships, that explicitly enumerates every customer Domain the object is known to affect — not just the customer whose ticket requested the change — and requires a second reviewer's sign-off against that full list, the same two-person discipline Part 9 §4 already requires for a high-fan-in Building Block, extended here across a customer boundary rather than just a rule-dependency boundary. A team that skips this step is not avoiding the cost; it is deferring it to the day one customer's legitimate tuning request quietly reopens or closes a detection gap for a customer who never asked for the change and was never told it happened. Where the customer contract or a regulatory boundary (a segregated subsidiary, a government client with its own audit requirements) requires provable non-interference between tenants, the Domain-suffixed partitioning approach in §5's fix is not an optional hardening step — it is the only version of this architecture that can actually support the compliance claim, and should be priced into the MSSP's onboarding cost for that customer rather than discovered as rework later.

[SOC MANAGEMENT] This same multiplication applies to dashboards and scheduled reports (Part 13). A shared dashboard tile or report template built against un-scoped Ariel data, rather than a Domain-filtered saved search, risks the same cross-tenant exposure this section names for correlation objects — a management-facing report meant for Customer A's stakeholders that inadvertently aggregates Customer B's event volume into a "total" figure is a contractual and, depending on the customer relationship, a legal problem, not just an inaccurate chart. Part 13 §7 flags this and defers the Domain-scoping detail to this part: any dashboard, Pulse widget, or report distributed outside the MSSP's own internal team must have its backing search explicitly filtered to the intended Domain(s), verified per customer before first distribution, not assumed correct because the dashboard "looks like" it's scoped to one tenant's tab.

7. Verifying isolation actually holds

[PLATFORM ENGINEER] Because Domain/Security-Profile isolation is enforced by configuration rather than by physically separate infrastructure, "we configured it correctly" and "it actually isolates" are different claims, and only the second one is worth anything to a customer relying on it. The isolation boundary that matters most — can Customer A's analyst account see anything belonging to Customer B — is testable directly, the same discipline this book applies to every other silent-failure surface (a Rule that stopped matching, a dashboard tile that renders zero for the wrong reason) rather than trusted on the strength of the configuration screen looking correct.

Validation Test Setup: Two Domains configured (*Customer-A*, *Customer-B*) with Log Sources correctly assigned to each per §2; two test analyst accounts, each holding an identical Tier-1 Analyst User Role but Security Profiles scoped respectively to *Customer-A* only and *Customer-B* only; at least one Offense already open in each Domain. **Action:** Log in as the *Customer-A*-scoped analyst account and attempt to locate, by any means available in the console (Offenses tab, Log Activity search, an Advanced Search against *events* naming *Customer-B*'s known Log Source or asset), any data tagged Domain *Customer-B*. Repeat symmetrically for the *Customer-B*-scoped account against *Customer-A*'s data. **Expected result:** Neither account can locate, list, or search into the other Domain's Offenses or events through any console surface — not merely that the default Offenses-tab view happens not to show them, since a default view filtered by convenience is not the same guarantee as a Security Profile that structurally excludes the data from every search surface. Any surface where the restricted account can see even a count or a summary figure derived from the other Domain's data (a global "total events today" system tile, for instance) is a finding to document and fix, not a false alarm — §6's cross-tenant exposure risk applies to summary numbers exactly as it does to raw events.

8. A governance checklist for a multi-tenant deployment

[SOC MANAGEMENT] Table 21.2 collects this part's recommendations into a form a platform owner can run against a new customer onboarding or a periodic MSSP-wide audit, rather than holding the whole part in mind at once.

Table 21.2 — Multi-tenancy and RBAC governance checklist. Supports an onboarding or audit decision: is this customer's data, and this customer's detection surface, actually isolated the way the contract or compliance boundary requires.

Check	Confirms	Owning section
Every Log Source, Log Source Group, scanner, and CIDR range assigned to the correct Domain, with none left in the default/unscoped Domain	No customer's data lands unintentionally outside its intended isolation boundary	§2
Security Profile and User Role assigned as two explicit, independently-justified grants per account, not copied from a peer's existing access	Visibility and capability both match the actual access decision made, not an inherited default	§3
Isolation validated by direct test (Validation Test above), not assumed from configuration screens alone	The boundary that matters — cross-Domain visibility — actually holds under test	§7
Every shared Rule, Building Block, and Reference Set consulted by a multi-Domain Rule inventoried, with affected customer list current	A change proposed for one customer can be checked against every customer actually affected before it ships	§4, §5, §6
High-risk shared Reference Sets (allowlists, exclusion lists) evaluated for Domain-suffixed partitioning, especially where a contract or compliance boundary requires provable non-interference	Cross-tenant leakage risk (§5) is closed structurally where it matters, not left to review discipline alone	§5, §6
Dashboard tiles, Pulse widgets, and report templates distributed to any customer-facing audience confirmed to filter by that customer's Domain before first distribution	No aggregate figure or chart inadvertently exposes another customer's volume or Offense data	§6, Part 13 §7
Second-reviewer sign-off required for any change to a Rule, Building Block, or Reference Set known to span more than one customer Domain	The MSSP-specific review-cost multiplication (§6) is budgeted and enforced, not assumed absorbed by ordinary Part 9/Part 16 change control	§6

Cross-references: DEH Part 27 §2 (Custom Rules Engine, Building Blocks, Rule Tests), §3.3 (BB:LSASS-Access-Candidate, RS-Allowlisted-LSASS-Tools, and R: Suspicious LSASS Access – Unapproved Process worked example, reused in this part's §4–§5), §6 (Offense as Alert/Case hybrid; the closing SOC MANAGEMENT paragraph naming the three-object review burden this part multiplies across customers); DEH Part 23 §5 (Sigma-first vs. native authoring, not re-argued here); this book's Part 3 (Offense magnitude/credibility/relevance and Offense indexing, extended by §2.1's Domain-tagging mechanism), Part 9 (Building Block governance and change control, extended by §5–§6's cross-tenant review burden), Part 10 (Reference Data types, extended by §4's global-namespace limitation), Part 13 §7 (dashboard/report sharing across roles, resolved here for Domain-based tenant separation), Part 15 (Noisy-Offense tuning workflow, the single-tenant process §5's Rule Autopsy shows applied unsafely across a tenant boundary), Part 17 (EPS/FPM capacity, the metric a Tenant's licensing allocation is built on), Part 20 (Network Hierarchy, the Networks a Security Profile scopes), and Part 22 (staffing and TCO capstone, where this part's review-cost multiplication becomes a headcount line item).

Part 22 — Staffing and Running a QRadar Practice

Why this part exists

[CONCEPT] Every part before this one built or operated one piece of a QRadar practice: the deployment architecture (Part 2), the offense-scoring model (Part 3), log source and DSM onboarding (Parts 4–7), the Custom Rules Engine's authoring surface — Rule Wizard, Building Blocks, reference data (Parts 8–11) — Ariel and dashboards (Parts 12–13), the day-to-day queue (Parts 14–16), platform health (Parts 17–19), and the asset model and multi-tenancy layer (Parts 20–21). None of those parts answered the question a SOC manager or a budget owner actually opens this book to ask: how many people, in which roles, does running this platform actually take, and what does it cost beyond the license line item everyone already expects to see?

This part is the capstone that answers that question, and it does so by building directly on a claim DEH Part 27 makes only once, in its own closing paragraph, and does not have room to develop: that reviewing "one detection" in QRadar means reviewing a **BB:**-prefixed Building Block, an **RS:**-prefixed Reference Set's maintained membership, and an **R:**-prefixed Rule's chained logic as three separately-changeable artifacts, not one file — a review-cost shape that does not exist for a team running a single Sigma rule or a single KQL scheduled query. DEH Part 23 (Query Language Strategy) owns the cross-platform version of the hiring-pool and query-language-choice question generally; this part is the QRadar-specific instance of it for a practice that has already chosen this platform, narrowed to the staffing consequence of *that* choice rather than a re-argument of the choice itself.

What follows treats staffing as an architectural consequence, not a generic industry benchmark borrowed from a different platform's headcount model. §1 names the roles this book's own tag taxonomy already implies and what each actually touches. §2 turns Part 9's Building Block fan-in problem and Part 27's three-object review burden into a concrete review-cost driver. §3 does the same for analyst-side queue staffing, building on Part 14's disposition discipline and Part 15's tuning-intake volume. §4 covers platform engineering staffing — onboarding backlog, capacity planning, App Framework maintenance. §5 is the MSSP-vs-in-house question, built on Part 21's Domain/tenant model. §6 assembles license, App Framework, and staffing cost into one total-cost-of-ownership frame. §7 gives a worksheet. §8 closes on the review cadence that keeps a staffing model from decaying the moment the person who built it moves on.

1. What a QRadar practice actually staffs against

[CONCEPT] This book's six content tags are not just a reading aid — four of them name real staffing roles, and the fifth and sixth (**[CONCEPT]** and **[THREAT HUNTER]**) name work that a practice's existing roles absorb rather than hire a dedicated headcount line for at most shop sizes. **[SOC MANAGEMENT]** is the role asking the question this part answers. The other three — **[ANALYST]**, **[RULE ENGINEER]**, **[PLATFORM ENGINEER]** — are the roles whose combined headcount, review cadence, and coverage model this part actually budgets.

Table 22.1 states what each role owns, which parts of this book cover that ownership in depth, and which named QRadar objects that role's work centers on — the anchor a budget conversation needs before it can turn "we need more people" into "we need one more of *which* role, doing *what*."

Table 22.1 — QRadar practice roles, ownership, and primary objects. Supports deciding which role a new hire, contractor, or reassignment actually fills before writing a job requisition around a generic "SOC engineer" title that hides which of these three very different jobs is actually understaffed.

Role	Owns	Primary objects touched	Owning parts
Analyst	Offense triage, disposition, reason-coded closure, escalation	Offense queue (magnitude/credibility/relevance)	Part 3, Part 14
Rule Engineer	Rule Wizard authoring, BB: design and governance, RS- /reference-data type selection and maintenance, change management	R: Rules, BB: Building Blocks, RS- Reference Sets/Maps/Tables	Part 8, Part 9, Part 10, Part 16
Platform Engineer	Deployment topology, DSM/log source onboarding, custom-property governance, Ariel performance, licensing/capacity, App Framework, troubleshooting	Log Sources, DSMs, Ariel indexes, App Host, license/EPS budget	Part 2, Parts 4–7, Part 12, Part 17, Part 18, Part 19
SOC Management	Headcount, cadence, MSSP-vs-in-house structure, TCO, cross-role escalation boundaries	None directly — budgets and governs the other three roles' work	Part 22 (this part)

[SOC MANAGEMENT] A practice below a certain size does not staff four separate people for these four roles — a small team's single senior hire routinely covers Rule Engineer and Platform Engineer work together, with SOC Management folded into a lead-analyst or team-lead title rather than a dedicated line. What matters for this part's purpose is not that every role gets its own headcount, but that every role's *work* is accounted for by someone, explicitly, rather than assumed to happen as a side effect of the Analyst role's queue time — the single most common staffing failure this part exists to name is a shop that budgeted Analyst FTEs to cover queue volume and never separately budgeted the Rule Engineer hours that keep that same queue from flooding in the first place (Part 15's tuning-intake backlog is exactly what accumulates when this gap goes unbudgeted).

2. The three-object review burden as a headcount driver

[RULE ENGINEER] DEH Part 27's closing paragraph names the structural fact this section turns into a staffing number: a QRadar detection is not one file. It is a Rule that chains one or more Building Block matches against one or more Reference Set membership checks, and each of those three object types is separately versioned, separately permissioned, and separately capable of drifting out of sync with the other two (a Rule referencing a Building Block by name after the Building Block was renamed, per Part 9's own Rule Autopsy pattern). A team that budgets Rule Engineer review time the way a KQL or SPL shop budgets query review time — one reviewer-hour per detection, roughly — is budgeting for a review unit that does not exist in this platform.

Part 9 §1 already showed why the multiplier is not even constant per detection: a Building Block's Rule Test can itself reference another Building Block, so a **BB:** library of any real size is a dependency graph, not a flat list, and a foundational Building Block referenced by forty Rules turns one proposed edit into a forty-Rule regression check, not a one-Rule review. Table 22.2 makes the review-cost driver concrete as something a headcount model can actually multiply against, rather than leaving "it's more work" as an unquantified intuition.

Table 22.2 — Review-cost multipliers by change type. Supports estimating Rule Engineer review hours for a proposed change before it is scheduled, using the fan-in concept from Part 9 §1 and the three-object decomposition from DEH Part 27 §2–§4.

Change type	What has to be re-verified	Review-cost driver	Owning section
New Rule, new dedicated Building Block, new dedicated Reference Set	One Rule's logic, one Building Block's match condition, one Reference Set's initial population	Fixed, one-time — closest to a single-detection review cost in a KQL/SPL shop	DEH Part 27 §3.3
Edit to a low-fan-in Building Block (feeds one or two Rules)	Those one or two Rules only	Small multiplier over a single-detection review	Part 9 §1
Edit to a high-fan-in Building Block (feeds many Rules directly or through nesting)	Every dependent Rule and every second-order Building Block referencing it	Multiplier scales with fan-in count — a forty-Rule dependency means a forty-Rule regression check	Part 9 §1, §3
Reference Set population change (allowlist addition/removal)	Every Rule that checks membership against that specific set, for both false-negative and false-positive direction	Usually small, but silent if skipped — Part 9's Reference Set drift risk	Part 9 §1, Part 10
Building Block or Reference Set rename	Every dependent object by name, since QRadar's console-native workflow gives no automatic rename-propagation guaranteed across versions	Full dependency-export re-check, same cost class as a high-fan-in edit	Part 9 §3.1

[SOC MANAGEMENT] The practical staffing consequence: a shop that has grown its **BB:** library past a few dozen custom entries (Part 9's own threshold for when informal team memory stops working) needs to budget Rule Engineer review hours against the *dependency graph's* size, not against the *count of Rules* alone — two shops with the same 150 active Rules can have wildly different review costs depending on whether those Rules sit on a flat, low-reuse Building Block layer or a deeply nested, high-fan-in one. Part 9 §5's quarterly audit cadence (orphan detection, near-duplicate detection, fan-in-outlier detection) is not optional governance hygiene from a staffing perspective — it is the mechanism that keeps this multiplier from growing silently until a review that used to take an afternoon takes a week, with nobody having budgeted the week.

PRODUCT VERSION NOTE *Whether a given QRadar release exposes a built-in Building Block dependency-visualization or impact-analysis view — showing which Rules and Building Blocks reference a given object before you edit it — versus requiring a manual configuration export and offline diff (Part 9 §3's stated workaround), is a version-dependent capability this book cannot confirm current for any specific release without lab access (STYLE-GUIDE.md §0/§9). Verify what your own console's Admin tab actually offers before assuming either the manual workaround or a native dependency view is what your reviewers will be using; the review-cost multiplier in Table 22.2 holds either way, but the reviewer-hours-per-change estimate changes materially depending on which tooling is actually available.*

3. Analyst-side staffing: queue volume, coverage, and disposition cost

[ANALYST] The Analyst role's headcount is driven by three inputs this book has already covered separately and this section combines: offense volume (a function of Rule/Building Block/Reference Set tuning maturity — Part 15), average time-to-disposition per offense (a function of Part 14's triage workflow and the reason-code discipline Part 14 §3 requires), and shift-coverage requirements if the practice runs anything close to 24/7 monitoring.

The coverage math is the part most staffing models understate. A single continuously-staffed seat — one analyst on shift at all times, every day of the year — is not three people working three eight-hour shifts. Vacation, sick leave, training time, and turnover mean the commonly cited industry rule of thumb for one truly continuous 24/7/365 seat is closer to four to five full-time analysts, not three; this is a general SOC-staffing fact independent of QRadar specifically, and this part states it plainly because it is the single most common staffing-model error a first-time SOC budget makes, QRadar or otherwise. A practice running business-hours-only coverage avoids this multiplier entirely, at the cost of an unmonitored window Part 3's offense-lifecycle discussion already flags as a real detection gap, not a budget-neutral tradeoff.

[SOC MANAGEMENT] Part 14 §3's reason-code discipline — no disposition closes without a reason code — costs seconds per offense, and Part 14's own closing paragraph already names this as a real, budgetable cost rather than a free process improvement. At queue volumes common in a maturing QRadar deployment (high before tuning matures, per Part 15's own framing), those seconds multiply into real analyst-hours per shift. Budgeting Analyst headcount without also budgeting the Part 15 tuning-intake pipeline that reduces offense volume over time is how a practice ends up permanently overstaffed on Analyst headcount to compensate for a Rule Engineer function that was never separately funded — the same gap §1 names generically, made concrete here as a specific, recurring line item.

SOC Management View *An illustrative — not sourced, not universal — starting worksheet for a single-tenant, moderate-volume QRadar deployment: roughly 150–300 offenses per day pre-tuning, falling toward 40–80 per day once Part 15's tuning workflow has had two to three quarters to mature against a stable Building Block library; an average disposition time (including reason-code entry) in the low single-digit minutes for a routine, non-escalated offense. Against that volume, a 24/7 single-analyst-seat coverage model needs roughly four to five Analyst FTEs before counting any shift lead or escalation-tier separation; a business-hours-only model needs one to two. These numbers describe one illustrative shop, not a formula — the actual multiplier for any real deployment depends on offense volume, disposition complexity, and tuning maturity specific to that environment, and should be re-derived from that environment's own queue data (Part 14's disposition records are exactly the input) rather than copied from this box.*

4. Platform engineering staffing: onboarding, capacity, and the maintenance tail

[PLATFORM ENGINEER] The Platform Engineer role's staffing driver is less cyclical than the Analyst role's queue and less dependency-graph-shaped than the Rule Engineer role's review burden — it is dominated by backlog and maintenance debt. Three sources of that debt, each already covered in depth elsewhere in this book and combined here into a staffing input:

- **Log source onboarding backlog** (Parts 4–5). Every new log source is a one-time onboarding cost (protocol configuration, DSM mapping validation per Part 4's checklist) plus, for anything without native DSM coverage, an ongoing maintenance cost for a hand-built DSM Editor extension that has to survive the source's own format changes (Part 5). A practice growing its log source count faster than its Platform Engineer headcount accumulates exactly the kind of onboarding backlog Part 4 warns produces a silently-unmapped or partially-mapped source — a staffing shortfall that shows up as a detection gap, not a visible queue, which makes it easy to underbudget until Part 18's troubleshooting playbook gets invoked.
- **Custom property and Ariel performance debt** (Parts 6, 12). A custom property extracted by regex without performance review, or a saved search pattern that scans rather than uses an indexed quick-filter property, is a maintenance cost that compounds with data volume — Part 12's tuning discipline is not a one-time setup task, it is ongoing work that needs budgeted hours as retention and EPS both grow.
- **Licensing, capacity, and App Framework footprint** (Parts 17, 19). Forecasting EPS/FPM growth against the licensed tier, planning for a new noisy log source's actual sustained rate before it goes live, and monitoring App Host resource contention as more apps get installed are all Platform Engineer work with a real time cost that scales with deployment size, not with offense volume.

Platform Reality *A common under-staffing pattern this section names directly: a practice hires exactly enough Platform Engineer capacity to complete the initial onboarding wave — the log sources present at go-live — and does not separately budget for the steady-state rate of new log source requests that follows in any organization whose infrastructure keeps changing. The onboarding backlog that results does not announce itself as a staffing shortfall; it shows up as "we've been meaning to onboard that source for two quarters," a sentence that, if said about a source relevant to an active investigation, is a coverage gap wearing a scheduling-delay disguise.*

5. MSSP vs. in-house: the review-cost problem changes shape, not size

[SOC MANAGEMENT] Part 21 covers the Domain/tenant model's mechanics — how a shared QRadar deployment segments Rules, Offenses, and Reference Sets across multiple client organizations or business units under a shared security profile structure. This section covers the staffing consequence: an MSSP operating a shared deployment across many tenant Domains does not avoid the three-object review burden §2 describes — it changes its shape.

A single in-house shop's Building Block fan-in problem (Part 9 §1) stays within one organization: a bad edit to a high-fan-in **BB:** affects that one shop's own Rules. An MSSP that maintains a shared "core" Building Block library — common threat patterns reused across every tenant Domain, with tenant-specific Reference Sets holding each client's own allowlist and tenant-specific Rules chaining the shared Building Block against that tenant's own Reference Set — has effectively multiplied the fan-in problem across paying customers instead of across one org's own Rule set. Figure 22.2 extends Part 9's own dependency-graph diagram to show exactly this shape.

flowchart TB

```

subgraph Core["Shared core library (MSSP provider side)"]
    BBcore["BB:Core-Credential-Access-Candidate\n(shared across every tenant Domain)"]
end
subgraph T1["Tenant Domain A"]
    RS1["RS-TenantA-Allowlisted-Tools"]
    R1["R: TenantA Suspicious\nCredential Access"]
end
subgraph T2["Tenant Domain B"]
    RS2["RS-TenantB-Allowlisted-Tools"]
    R2["R: TenantB Suspicious\nCredential Access"]
end
subgraph T3["Tenant Domain C ..."]
    RS3["RS-TenantC-Allowlisted-Tools"]
    R3["R: TenantC Suspicious\nCredential Access"]
end
BBcore --> R1
BBcore --> R2
BBcore --> R3
RS1 --> R1
RS2 --> R2
RS3 --> R3

```

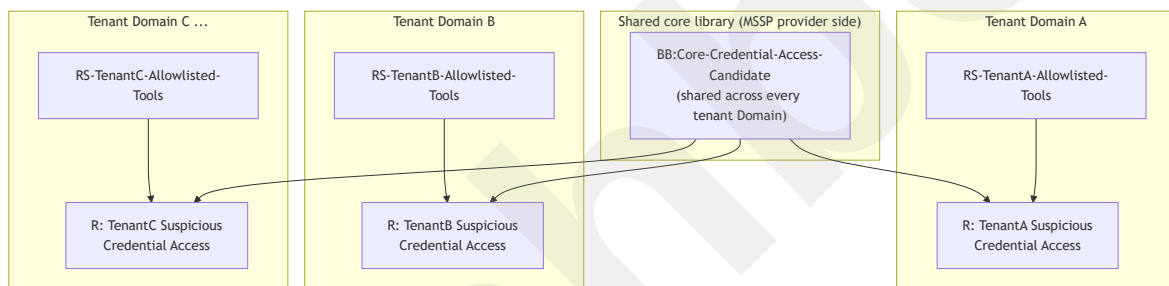


Figure 22.2 (FIG-22-02) — A shared core Building Block's fan-in across tenant Domains.

CONCEPTUAL. Extends Part 9's Figure 9.1 dependency-graph concept from a single organization's Rule set to an MSSP's multi-tenant deployment: one shared **BB**: edit's blast radius now spans every tenant Domain referencing it, not just one team's own Rules, which is exactly why an MSSP's core-library review has to be treated as higher-stakes than the same edit in a single-tenant shop, per Part 21's Domain-separation model. Not a capture of any real multi-tenant deployment's configuration.

[SOC MANAGEMENT] Table 22.3 lays out the tradeoff a practice actually faces, rather than treating "MSSP vs. in-house" as a binary business-model choice made once and forgotten — it is a staffing-shape decision with recurring consequences this book's own review-burden framing already justifies naming explicitly.

Table 22.3 — MSSP-shared vs. in-house staffing tradeoffs. Supports deciding, or re-evaluating, which model fits a given practice's growth pattern before defaulting to whichever model the practice happened to start with.

Dimension	MSSP shared-core model	In-house single-tenant model
Marginal Rule Engineer cost per new client/business unit	Low — new tenant mostly consumes existing shared BB : library, needs only its own RS - allowlist and thin tenant-specific Rules	Not applicable — one org, cost doesn't marginalize this way
Blast radius of a shared Building Block edit	Every tenant Domain referencing it — Figure 22.2	Contained to one org's own Rules — Part 9 §1's original framing
Review stakes per high-fan-in change	Higher — a regression affects multiple paying customers simultaneously	Lower — affects one org's own queue
Analyst staffing elasticity	Better — shared queue-management tooling and cross-tenant shift coverage amortize the 24/7 coverage multiplier (§3) across more offense volume per analyst-hour	Worse — the full §3 coverage multiplier applies to one org's own volume alone
Customization ceiling	Lower per-tenant — deep tenant-specific Rule Wizard logic competes with shared-library consistency	Higher — no cross-tenant consistency constraint
Licensing/capacity planning (Part 17)	Aggregated across tenants, requiring careful per-tenant EPS accounting to avoid one noisy tenant consuming shared capacity	Scoped to one org's own growth forecast

PRODUCT VERSION NOTE *The specific Domain/tenant security-profile features that determine how strictly a shared Building Block, Reference Set, or Rule can be scoped or hidden from a given tenant's own view — and whether a given release lets an MSSP delegate limited, tenant-scoped Rule authoring to a client without exposing the shared core library — are RBAC/Domain capabilities that have evolved across QRadar releases and are covered in depth by Part 21. Verify the current Domain/tenant feature set against your specific deployment before committing to a shared-core staffing model that assumes a level of tenant isolation your release may not actually provide.*

6. Total cost of ownership: license, App Framework, and staffing together

[SOC MANAGEMENT] A budget conversation that stops at the license line item is the most common TCO mistake this part sees reason to name directly: the license tier (committed EPS/FPM capacity, per Part 17) is real cost, but it is not the majority of what running a QRadar practice actually costs over a multi-year horizon once staffing (§§2–4) and App Framework add-ons (Part 19 — Pulse, UBA, Use Case Manager, or any third-party App Exchange app a practice has installed) are counted alongside it.

Table 22.4 names the categories a complete TCO conversation needs to include, without attaching invented dollar figures to any of them — this book has no authoritative, current pricing to cite, and inventing a number here would be exactly the kind of unverifiable specific claim \$0's honesty constraint exists to prevent.

Table 22.4 — TCO cost categories for a QRadar practice. Supports building a complete budget conversation rather than one anchored on the license line alone; no dollar figures are given because none are sourced or current enough to state as fact.

Cost category	Primary driver	Owning part(s)	Version sensitivity
License tier (EPS/FPM)	Sustained ingest rate, committed capacity, growth headroom	Part 17	High — tier structure and overage handling change across release cycles
On-prem infrastructure or cloud/SaaS delivery	Deployment topology choice, hardware/cloud spend	Part 2, Part 17	High — packaging/delivery model and vendor-side ownership have shifted historically
App Framework add-ons	Which apps are installed (Pulse, UBA, Use Case Manager, third-party App Exchange apps) and their App Host resource footprint	Part 19	High — bundling, separate-install requirements, and licensing for individual apps vary by release and packaging tier
Rule Engineer staffing	Active Rule/Building Block/Reference Set count and dependency-graph fan-in (\$2)	Part 8, Part 9, Part 10	Low — the review-cost architecture itself is stable
Platform Engineer staffing	Log source count and growth rate, DSM extension maintenance debt, Ariel performance tuning cadence	Parts 4–7, 12, 18	Low — the maintenance-tail shape is stable; specific tooling availability is version-sensitive (\$2's PRODUCT VERSION NOTE)
Analyst staffing	Offense volume, disposition complexity, coverage-hours requirement	Part 3, Part 14, Part 15	Low — coverage math is general SOC staffing, not QRadar-specific
Onboarding/training cost for new Rule Engineer or Platform Engineer hires	Rule Wizard catalog depth (Part 8), BB: library taxonomy (Part 9), reference-data type taxonomy (Part 10) — genuinely deep, platform-specific knowledge that does not transfer from a KQL/SPL background without ramp time	Part 8, Part 9, Part 10	Low — the ramp-time cost is a consequence of platform depth, not a specific version's UI

PRODUCT VERSION NOTE QRadar's own commercial packaging — on-prem vs. cloud/SaaS delivery, which business unit sells and supports which piece, and the license-tier structure itself — has changed more than once across release cycles and business decisions this book does not have current visibility into at time of writing (STYLE-GUIDE.md §11's packaging/ownership trigger). Treat every cell in Table 22.4's "License tier" and "On-prem infrastructure or cloud/SaaS delivery" rows as a category to re-price against IBM's current commercial terms at budget time, not as a stable line item that can be estimated once and reused across budget cycles.

7. A minimal staffing worksheet

[SOC MANAGEMENT] Table 22.5 collects this part's role and coverage math into a single starting worksheet — explicitly a starting point for a small-to-moderate single-tenant deployment to adapt, not a formula that produces a correct headcount number for every practice regardless of scale, tuning maturity, or MSSP structure.

Table 22.5 — Illustrative minimum staffing worksheet, single-tenant, moderate volume. Supports a first-pass headcount conversation; every "Minimum FTE" figure is this book's own illustrative estimate against the volume assumptions in §3's SOC Management View box, not a benchmark sourced from any specific real deployment or vendor guidance.

Role	Minimum FTE (business-hours-only)	Minimum FTE (24/7 coverage)	Primary scaling factor
Analyst	1–2	4–5	Offense volume and disposition complexity (§3) — falls as Part 15 tuning matures
Rule Engineer	0.5 (often combined with Platform Engineer at this scale)	1	BB: / RS- library size and dependency-graph fan-in (§2) — rises with library age and Rule count
Platform Engineer	0.5–1	1	Log source onboarding rate, App Framework footprint, licensed EPS growth (§4)
SOC Management	0.25–0.5 (often a lead-analyst or lead-engineer title, not a dedicated line)	0.25–0.5	Cadence-governance overhead (§8) — rises with MSSP tenant count (§5)

[SOC MANAGEMENT] The single most important use of Table 22.5 is not the numbers themselves — it is the exercise of assigning every one of §1's four roles a nonzero number before deciding any of them can be absorbed into another. A practice that zeroes out the Rule Engineer row because "the analysts can handle small tuning changes themselves" has not eliminated that cost; it has moved it, unbudgeted, into Analyst hours that were sized for triage, not for the dependency-graph review §2 describes — and moved it specifically into the role with the least visibility into Building Block fan-in, since that visibility is exactly what the Rule Engineer role's own tooling and practice (Part 9 §3) is built to provide.

8. Keeping the model current: cadence, not a one-time hire

[SOC MANAGEMENT] A staffing model built once at go-live and never revisited decays for the same reason Part 9 §5's governance cadence and Part 16's change-management discipline both exist: the inputs driving §2's review-cost multiplier, §3's offense volume, and §4's onboarding backlog all move continuously, not once. Part 9 §5's own recommendation — a named Building Block library owner running a quarterly audit, a rolling deprecation policy, an ownership review — is the Rule Engineer-side instance of a broader discipline this part asks a practice to run for its whole staffing model, not just its BB: library:

- Re-derive §3's offense-volume and disposition-time inputs from Part 14's own disposition records each quarter, not from the assumption that made sense at go-live.
- Re-run Part 9 §3's dependency-graph audit before every headcount conversation that touches Rule Engineer staffing, so the review-cost multiplier in Table 22.2 is measured against the library's

current fan-in shape, not its shape a year ago.

- Re-price Table 22.4's license and App Framework rows against IBM's current commercial terms at each renewal, given §6's PRODUCT VERSION NOTE — a TCO model that was accurate at signing is not guaranteed to still be accurate at the next renewal cycle.
- Treat any MSSP tenant-count growth as a trigger to re-check Figure 22.2's fan-in shape specifically, since onboarding tenant number twenty against a shared core library that was only ever reviewed at tenant number five is exactly the kind of unmeasured blast-radius growth §5 warns about.

What Would Change My Mind This part treats the three-object review burden (§2) as the dominant Rule Engineer staffing driver, ahead of raw Rule count. If a practice running a large, deeply nested **BB:** library found — through actual measured review-hour data across multiple change cycles — that review time tracked flat Rule count just as well as it tracked dependency-graph fan-in, that would be a real argument for simplifying Table 22.2's model back to a per-Rule estimate rather than requiring a fan-in audit before every headcount conversation. This part has not seen that data; it states the fan-in-driven model as the more defensible default because Part 9's own dependency-graph argument is structural, not because any real measured comparison between the two models has been run against a production **BB:** library this book has access to.

Cross-references: DEH Part 27 §2 (Building Blocks and Rule Tests), §2.2 (Reference Sets), §3.3 (three-object decomposition worked example), and its closing SOC MANAGEMENT paragraph (the three-object review-cost consequence this whole part is built on); DEH Part 23 (Query Language Strategy — the cross-platform hiring-pool question this part narrows to a QRadar-specific staffing model, not re-argued here); DEH **TERMINOLOGY.md** (Disposition, Triage, Tuning/Suppression — inherited unchanged); this book's Part 3 (magnitude/credibility/relevance and offense-lifecycle volume drivers), Part 8 (Rule Wizard catalog), Part 9 (Building Block governance, dependency-graph fan-in, and the quarterly audit cadence this part's §8 extends), Part 10 (reference-data type taxonomy), Part 14 (triage workflow and disposition-record data this part's §3 draws from), Part 15 (tuning-intake workflow and its effect on offense volume), Part 16 (rule change management without a detection-as-code pipeline), Part 17 (licensing, EPS/FPM, and capacity planning), Part 18 (troubleshooting playbook triggered by onboarding-backlog gaps), Part 19 (App Framework footprint and add-on cost), Part 20 (asset model staleness as a relevance-weighting risk), and Part 21 (Domain/tenant/RBAC model this part's §5 builds its MSSP staffing analysis on).